

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Департамент программной инженерии

СОГЛАСОВАНО

Руководитель департамента программной
инженерии, доцент

_____ / С. А. Лебедев /
«_____» _____ 2026 г.

УТВЕРЖДАЮ

Академический руководитель
образовательной программы «Программная
инженерия» старший преподаватель
департамента программной инженерии

_____ / Н.А. Павлочев /
«_____» _____ 2026 г.

Подп. и дата	
Инв. № дубл.	
Взаим. инв. №	
Подп. и дата	
Инв. № подл.	

Веб-приложение для обработки результатов соревнований по программированию

Текст программы

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.05.05-01 12 01-1-ЛУ

Исполнитель:

студент группы БПИ233

_____ / Е.А. Анисевич /
«_____» _____ 2026 г.

УТВЕРЖДЁН

RU.17701729.05.05-01 12 01-1-ЛУ

Веб-приложение для обработки результатов соревнований по программированию

Текст программы

RU.17701729.05.05-01 12 01-1-ЛУ

Листов: 605

Инв. № подл.	Подп. и дата	Взаим. инв. №	Инв. № дубл.	Подп. и дата

Содержание

1	ТЕКСТ ПРОГРАММЫ (Backend)	10
1.1	__init__.py	10
1.2	settings.py	10
1.3	alembic\env.py	12
1.4	alembic\versions\240bb27e187a_.py	15
1.5	alembic\versions\c1d2e3f4a5b6_drop_technical_data_from_ai_sessions.py . . .	23
1.6	alembic\versions\f9e1d2c3b4a5_add_uploaded_to_ai_generated_files.py . . .	24
1.7	api__init__.py	25
1.8	api\health.py	26
1.9	api\crypt__init__.py	26
1.10	api\crypt\crypt_password.py	27
1.11	api\crypt\jwt_token.py	27
1.12	api\pydantic_schemas__init__.py	29
1.13	api\pydantic_schemas\user__init__.py	30
1.14	api\pydantic_schemas\user\ai_task.py	30
1.15	api\pydantic_schemas\user\auth.py	33
1.16	api\pydantic_schemas\user\codeforces.py	35
1.17	api\pydantic_schemas\user\yandex.py	37
1.18	api\user__init__.py	38
1.19	api\user\contests.py	39
1.20	api\user\merge_submissions.py	48
1.21	api\user\merge_table.py	48
1.22	api\user\auth__init__.py	51
1.23	api\user\auth\base_auth.py	52
1.24	api\user\auth\change_password.py	52
1.25	api\user\auth\location.py	54
1.26	api\user\auth\login.py	54
1.27	api\user\auth\logout.py	58
1.28	api\user\auth\profile.py	60

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.29	api\user\auth\refresh.py	62
1.30	api\user\auth\tokens.py	64
1.31	api\user\codeforces__init__.py	65
1.32	api\user\codeforces\base_codeforces.py	66
1.33	api\user\codeforces\create_signature.py	66
1.34	api\user\codeforces\format.py	66
1.35	api\user\codeforces\get_response.py	72
1.36	api\user\codeforces\auth__init__.py	73
1.37	api\user\codeforces\auth\login.py	74
1.38	api\user\codeforces\auth\logout.py	77
1.39	api\user\codeforces\standings__init__.py	78
1.40	api\user\codeforces\standings\standings.py	78
1.41	api\user\codeforces\submissions__init__.py	80
1.42	api\user\codeforces\submissions\submissions.py	80
1.43	api\user\gpt__init__.py	82
1.44	api\user\gpt\ai_tasks.py	83
1.45	api\user\gpt\base_gpt.py	98
1.46	api\user\gpt\services__init__.py	98
1.47	api\user\gpt\services\ai_file_helpers.py	98
1.48	api\user\gpt\services\ai_service.py	102
1.49	api\user\gpt\services\session_store.py	110
1.50	api\user\gpt\services\upload_orchestrator.py	111
1.51	api\user\plagiarism__init__.py	123
1.52	api\user\plagiarism\base_plagiarism.py	123
1.53	api\user\polygon__init__.py	134
1.54	api\user\polygon\base_polygon.py	135
1.55	api\user\polygon\create_problem.py	136
1.56	api\user\polygon\create_signature.py	138
1.57	api\user\polygon\get_response.py	138
1.58	api\user\polygon\problem_info.py	140
1.59	api\user\polygon\problem_script.py	141
1.60	api\user\polygon\auth__init__.py	143

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.61	api\user\polygon\auth\login.py	143
1.62	api\user\polygon\auth\logout.py	144
1.63	api\user\polygon\commit__init__.py	145
1.64	api\user\polygon\commit\build_package.py	145
1.65	api\user\polygon\commit\commit_problem.py	147
1.66	api\user\polygon\commit\get_packages.py	149
1.67	api\user\polygon\files__init__.py	150
1.68	api\user\polygon\files\save_statement.py	150
1.69	api\user\polygon\files\save_test.py	153
1.70	api\user\polygon\files\gen__init__.py	155
1.71	api\user\polygon\files\gen\add_source_file.py	155
1.72	api\user\polygon\files\gen\set_checker.py	157
1.73	api\user\polygon\files\gen\set_generator.py	158
1.74	api\user\polygon\files\gen\set_interactor.py	159
1.75	api\user\polygon\files\gen\set_script.py	160
1.76	api\user\polygon\files\gen\set_solution.py	162
1.77	api\user\polygon\files\gen\set_validator.py	164
1.78	api\user\polygon\settings__init__.py	165
1.79	api\user\polygon\settings\enable_groups.py	165
1.80	api\user\polygon\settings\enable_points.py	167
1.81	api\user\polygon\settings\set_tags.py	169
1.82	api\user\polygon\settings\set_test_group.py	170
1.83	api\user\polygon\settings\update_info.py	172
1.84	api\user\yandex__init__.py	174
1.85	api\user\yandex\base_yandex.py	174
1.86	api\user\yandex\format.py	175
1.87	api\user\yandex\auth__init__.py	181
1.88	api\user\yandex\auth\access_expire.py	181
1.89	api\user\yandex\auth\login.py	181
1.90	api\user\yandex\auth\logout.py	184
1.91	api\user\yandex\standings__init__.py	185
1.92	api\user\yandex\standings\standings.py	185

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.93	api\{\}user\{\}yandex\{\}submissions\{\}__init__.py	188
1.94	api\{\}user\{\}yandex\{\}submissions\{\}submissions.py	188
1.95	app\{\}__init__.py	191
1.96	app\{\}database.py	191
1.97	app\{\}logging_config.py	192
1.98	app\{\}server.py	195
1.99	app\{\}middlewares\{\}log_middleware.py	200
1.100	logs\{\}__init__.py	201
1.101	logs\{\}logs.py	201
1.102	models\{\}__init__.py	202
1.103	models\{\}base.py	203
1.104	models\{\}ai\{\}__init__.py	203
1.105	models\{\}ai\{\}ai_generated_file.py	204
1.106	models\{\}ai\{\}ai_session.py	204
1.107	models\{\}contest\{\}__init__.py	206
1.108	models\{\}contest\{\}contest.py	206
1.109	models\{\}contest\{\}contest_participant.py	207
1.110	models\{\}contest\{\}task.py	209
1.111	models\{\}contest\{\}task_result.py	210
1.112	models\{\}participant\{\}__init__.py	211
1.113	models\{\}participant\{\}participant.py	211
1.114	models\{\}plagiarism\{\}__init__.py	212
1.115	models\{\}plagiarism\{\}pair_of_banned_submissions.py	212
1.116	models\{\}plagiarism\{\}plagiarism_report.py	214
1.117	models\{\}submissions\{\}__init__.py	215
1.118	models\{\}submissions\{\}plagiarism_report.py	215
1.119	models\{\}submissions\{\}submission.py	216
1.120	models\{\}user\{\}__init__.py	218
1.121	models\{\}user\{\}refresh_token.py	218
1.122	models\{\}user\{\}role.py	219
1.123	models\{\}user\{\}user.py	220
1.124	plagiarism\{\}main.cpp	221

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.125	plagiarism\ast\ast_analyzer.cpp	223
1.126	plagiarism\ast\ast_analyzer.h	238
1.127	plagiarism\ast\ast_features.h	238
1.128	plagiarism\ast\ast_hashing.cpp	239
1.129	plagiarism\ast\ast_hashing.h	243
1.130	plagiarism\ast\ast_tree.h	244
1.131	plagiarism\ast\call_graph.cpp	246
1.132	plagiarism\ast\call_graph.h	250
1.133	plagiarism\cmake-build-debug\CMakeFiles\4.1.2\CompilerIdCXX\CMakeCXXCompilerId.cxx	
1.134	plagiarism\models\metrics.h	281
1.135	plagiarism\models\programming_language.h	281
1.136	plagiarism\models\similar_submission_pair.h	281
1.137	plagiarism\models\similarity_result.h	282
1.138	plagiarism\models\submission.h	283
1.139	plagiarism\models\submission_data.h	283
1.140	plagiarism\models\submission_features.h	284
1.141	plagiarism\models\token.h	285
1.142	plagiarism\models\token_features.h	285
1.143	plagiarism\models\winnowing_features.h	286
1.144	plagiarism\outdated_metrics\levenshtein_similarity.cpp	287
1.145	plagiarism\outdated_metrics\levenshtein_similarity.h	288
1.146	plagiarism\preprocessing\comments.cpp	289
1.147	plagiarism\preprocessing\comments.h	292
1.148	plagiarism\preprocessing\cpp_preprocessor.cpp	293
1.149	plagiarism\preprocessing\cpp_preprocessor.h	298
1.150	plagiarism\preprocessing\line_endings.cpp	298
1.151	plagiarism\preprocessing\line_endings.h	299
1.152	plagiarism\preprocessing\lines.cpp	300
1.153	plagiarism\preprocessing\lines.h	302
1.154	plagiarism\preprocessing\llvm_ir_pipeline.cpp	303
1.155	plagiarism\preprocessing\llvm_ir_pipeline.h	306
1.156	plagiarism\preprocessing\normalize.cpp	306

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.157	plagiarism\{}\preprocessing\{}\normalize.h	307
1.158	plagiarism\{}\preprocessing\{}\string_normalizer.cpp	307
1.159	plagiarism\{}\preprocessing\{}\string_normalizer.h	310
1.160	plagiarism\{}\similarity\{}\bindings.cpp	311
1.161	plagiarism\{}\similarity\{}\compare_submissions.cpp	313
1.162	plagiarism\{}\similarity\{}\compare_submissions.h	316
1.163	plagiarism\{}\similarity\{}\lsh.cpp	317
1.164	plagiarism\{}\similarity\{}\lsh.h	320
1.165	plagiarism\{}\similarity\{}\metrics.cpp	321
1.166	plagiarism\{}\similarity\{}\metrics.h	329
1.167	plagiarism\{}\similarity\{}\similarity_params.h	332
1.168	plagiarism\{}\similarity\{}\winnowing.cpp	332
1.169	plagiarism\{}\similarity\{}\winnowing.h	335
1.170	plagiarism\{}\submission_builder\{}\submission_builder.cpp	336
1.171	plagiarism\{}\submission_builder\{}\submission_builder.h	337
1.172	plagiarism\{}\submission_builder\{}\submission_stages.cpp	337
1.173	plagiarism\{}\submission_builder\{}\submission_stages.h	340
1.174	plagiarism\{}\submission_loader\{}\submission_loader.cpp	341
1.175	plagiarism\{}\submission_loader\{}\submission_loader.h	346
1.176	plagiarism\{}\tokenizer\{}\normalize_tokens.cpp	347
1.177	plagiarism\{}\tokenizer\{}\normalize_tokens.h	350
1.178	plagiarism\{}\tokenizer\{}\token_features_builder.cpp	350
1.179	plagiarism\{}\tokenizer\{}\token_features_builder.h	352
1.180	plagiarism\{}\tokenizer\{}\tokenizer.cpp	352
1.181	plagiarism\{}\tokenizer\{}\tokenizer.h	356
1.182	tests\{}__init__.py	356
1.183	tests\{}\api_tests\{}__init__.py	356
1.184	tests\{}\api_tests\{}\test_base.py	356
1.185	tests\{}\api_tests\{}\test_health.py	357
1.186	tests\{}\api_tests\{}\user\{}\test_change_password.py	357
1.187	tests\{}\api_tests\{}\user\{}\test_login.py	377
1.188	tests\{}\api_tests\{}\user\{}\test_logout.py	384

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.189	tests\{}\api_tests\{}\user\{}\test_logout_all.py	389
1.190	tests\{}\api_tests\{}\user\{}\test_refresh.py	397
1.191	tests\{}\api_tests\{}\user\{}\test_register.py	404
1.192	tests\{}\api_tests\{}\user\{}\test_unlink_codeforces.py	412
1.193	tests\{}\api_tests\{}\user\{}\test_unlink_yandex.py	414
1.194	tests\{}\api_tests\{}\yandex\{}\test_login.py	416
2	ТЕКСТ ПРОГРАММЫ (Frontend)	416
2.1	App.tsx	416
2.2	index.css	419
2.3	main.tsx	419
2.4	api\{}\instance.ts	420
2.5	components\{}\AuthInitializer.tsx	422
2.6	components\{}\Pagination.tsx	423
2.7	components\{}\ThemeInitializer.tsx	428
2.8	components\{}\ThemeToggle.tsx	428
2.9	components\{}\layout\{}\ContestLayout.tsx	430
2.10	components\{}\layout\{}\MainLayout.tsx	432
2.11	pages\{}\AITaskSession.tsx	441
2.12	pages\{}\AITasksList.tsx	478
2.13	pages\{}\CodeforcesCallback.tsx	490
2.14	pages\{}\ContestAnalytics.tsx	491
2.15	pages\{}\ContestOverview.tsx	496
2.16	pages\{}\ContestSubmissions.tsx	500
2.17	pages\{}\ContestViewPage.tsx	505
2.18	pages\{}\ContestsPage.tsx	513
2.19	pages\{}\DashboardHome.tsx	520
2.20	pages\{}\ImportSubmissions.tsx	522
2.21	pages\{}\Landing.tsx	530
2.22	pages\{}\LoadContestPage.tsx	534
2.23	pages\{}\LoginPage.tsx	541
2.24	pages\{}\NotFound.tsx	546

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2.25	pages\{}\PlagiarismComparation.tsx	547
2.26	pages\{}\PlagiarismReport.tsx	550
2.27	pages\{}\PlagiarismSetup.tsx	559
2.28	pages\{}\Profile.tsx	563
2.29	pages\{}\RegisterPage.tsx	586
2.30	pages\{}\SubmissionSource.tsx	593
2.31	pages\{}\YandexCallback.tsx	598
2.32	store\{}\useAuthStore.ts	599
2.33	store\{}\useThemeStore.ts	601

ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ	603
---	------------

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ	604
-----------------------------------	------------

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1. ТЕКСТ ПРОГРАММЫ (Backend)

1.1. __init__.py

Листинг 1: __init__.py

1.2. settings.py

Листинг 2: settings.py

```
from pathlib import Path

from pydantic_settings import BaseSettings, SettingsConfigDict
from sqlalchemy import URL

class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env", env_file_encoding="utf-8", extra="ignore"
    )

    PROJECT_ROOT: Path = Path(__file__).parent.parent

    # General settings
    DEFAULT_PAGE_SIZE: int = 100

    # CORS settings
    CORS_ORIGINS: str = "http://localhost:5173"

    @property
    def cors_origins(self) -> list[str]:
        return [o.strip() for o in self.CORS_ORIGINS.split(",") if o.strip()]
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
# JWT settings
ALGORITHM: str = "HS256"
SECRET_KEY: str
EXPIRES_ACCESS: int = 30
EXPIRES_REFRESH: int = 43200

# Yandex OAuth2
YANDEX_HOST: str = "https://api.contest.yandex.net/api/public/v2"
YANDEX_CLIENT_ID: str
YANDEX_CLIENT_SECRET: str

# Codeforces settings
CODEFORCES_HOST: str = "https://codeforces.com/api"
CF_CLIENT_ID: str
CF_CLIENT_SECRET: str

# Polygon settings
POLYGON_HOST: str = "https://polygon.codeforces.com/api"

# OpenAI settings
OPENAI_HOST: str = "https://openrouter.ai/api/v1"
OPENAI_API_KEY: str

# Database settings
POSTGRES_DB: str
POSTGRES_USER: str
POSTGRES_PASSWORD: str
POSTGRES_HOST: str = "postgres"
POSTGRES_PORT: int = 5432

@property
def database_url(self) -> str:
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
return URL.create(
    drivename="postgresql+asyncpg",
    username=self.POSTGRES_USER,
    password=self.POSTGRES_PASSWORD,
    host=self.POSTGRES_HOST,
    port=self.POSTGRES_PORT,
    database=self.POSTGRES_DB,
).render_as_string(hide_password=False)

settings = Settings()
```

1.3. alembic\{ }env.py

Листинг 3: alembic\{ }env.py

```
import asyncio
from logging.config import fileConfig

from sqlalchemy import pool
from sqlalchemy.ext.asyncio import async_engine_from_config

from alembic import context
from models import * # noqa: F403
from settings import settings

# this is the Alembic Config object, which provides
# access to the values within the .ini file in use.
config = context.config

config.set_main_option("sqlalchemy.url", settings.database_url.replace("%", "%%"))

# Interpret the config file for Python logging.
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# This line sets up loggers basically.
if config.config_file_name is not None:
    fileConfig(config.config_file_name)

# add your model's MetaData object here
# for 'autogenerate' support
# from myapp import mymodel
# target_metadata = mymodel.Base.metadata
target_metadata = Base.metadata # noqa: F405

# other values from the config, defined by the needs of env.py,
# can be acquired:
# my_important_option = config.get_main_option("my_important_option")
# ... etc.

def run_migrations_offline() -> None:
    """Run migrations in 'offline' mode.

    This configures the context with just a URL
    and not an Engine, though an Engine is acceptable
    here as well. By skipping the Engine creation
    we don't even need a DBAPI to be available.

    Calls to context.execute() here emit the given string to the
    script output.

    """
    url = config.get_main_option("sqlalchemy.url")
    context.configure(
        url=url,

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        target_metadata=target_metadata,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
    )

    with context.begin_transaction():
        context.run_migrations()

def do_run_migrations(connection):
    context.configure(connection=connection, target_metadata=target_metadata)
    with context.begin_transaction():
        context.run_migrations()

async def run_migrations_online() -> None:
    """Run migrations in 'online' mode.

    In this scenario we need to create an Engine
    and associate a connection with the context.

    """

    connectable = async_engine_from_config(
        config.get_section(config.config_ini_section, {}),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )

    async with connectable.connect() as connection:
        await connection.run_sync(do_run_migrations)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        await connectable.dispose()

if context.is_offline_mode():
    run_migrations_offline()
else:
    asyncio.run(run_migrations_online())
```

1.4. alembic\{}\versions\{}\240bb27e187a_.py

Листинг 4: alembic\{}\versions\{}\240bb27e187a_.py

```
"""

Revision ID: 240bb27e187a
Revises:
Create Date: 2026-04-26 10:32:19.900584

"""

from typing import Sequence, Union

import sqlalchemy as sa

from alembic import op

# revision identifiers, used by Alembic.
revision: str = "240bb27e187a"
down_revision: Union[str, Sequence[str], None] = None
branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
def upgrade() -> None:
    """Upgrade schema."""
    # ### commands auto generated by Alembic - please adjust! ###
    op.create_table(
        "roles",
        sa.Column("id", sa.Integer(), nullable=False),
        sa.Column("name", sa.String(length=100), nullable=False),
        sa.PrimaryKeyConstraint("id"),
    )
    op.create_table(
        "users",
        sa.Column("id", sa.Integer(), nullable=False),
        sa.Column("role_id", sa.Integer(), nullable=False),
        sa.Column("login", sa.String(length=50), nullable=False),
        sa.Column("password", sa.String(length=120), nullable=False),
        sa.Column("email", sa.String(length=100), nullable=False),
        sa.Column("codeforces_api_key", sa.String(length=100), nullable=True),
        sa.Column("codeforces_api_secret", sa.String(length=100), nullable=True),
        sa.Column("polygon_api_key", sa.String(length=100), nullable=True),
        sa.Column("polygon_api_secret", sa.String(length=100), nullable=True),
        sa.Column("yandex_access_token", sa.String(length=255), nullable=True),
        sa.ForeignKeyConstraint(
            ["role_id"],
            ["roles.id"],
        ),
        sa.PrimaryKeyConstraint("id"),
        sa.UniqueConstraint("login"),
    )
    op.create_table(
        "ai_sessions",
        sa.Column("id", sa.String(), nullable=False),
        sa.Column("user_id", sa.Integer(), nullable=False),
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

sa.Column("model", sa.String(), nullable=False),
sa.Column("system_prompt", sa.Text(), nullable=False),
sa.Column("history", sa.JSON(), nullable=False),
sa.Column("statement", sa.JSON(), nullable=True),
sa.Column("technical_data", sa.JSON(), nullable=True),
sa.Column("progress", sa.JSON(), nullable=True),
sa.Column("stage", sa.String(), nullable=False),
sa.Column("polygon_problem_id", sa.Integer(), nullable=True),
sa.Column("package_id", sa.Integer(), nullable=True),
sa.Column("upload_errors", sa.JSON(), nullable=True),
sa.Column("ai_fix_attempts", sa.JSON(), nullable=False),
sa.Column("created_at", sa.DateTime(), nullable=False),
sa.Column("updated_at", sa.DateTime(), nullable=False),
sa.ForeignKeyConstraint(
    ["user_id"],
    ["users.id"],
),
sa.PrimaryKeyConstraint("id"),
)
op.create_index(op.f("ix_ai_sessions_id"), "ai_sessions", ["id"], unique=False)
op.create_table(
    "contests",
    sa.Column("id", sa.Integer(), nullable=False),
    sa.Column("user_id", sa.Integer(), nullable=False),
    sa.Column("platform", sa.String(length=100), nullable=False),
    sa.Column("external_id", sa.Integer(), nullable=False),
    sa.Column("name", sa.String(length=256), nullable=False),
    sa.Column("type", sa.String(length=5), nullable=False),
    sa.Column("unofficial", sa.Boolean(), nullable=False),
    sa.Column("start_time", sa.DateTime(), nullable=True),
    sa.Column("duration", sa.Interval(), nullable=True),
    sa.ForeignKeyConstraint(

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        ["user_id"],
        ["users.id"],
    ),
    sa.PrimaryKeyConstraint("id"),
)
op.create_table(
    "participants",
    sa.Column("id", sa.Integer(), nullable=False),
    sa.Column("user_id", sa.Integer(), nullable=False),
    sa.Column("login", sa.String(length=255), nullable=False),
    sa.Column("name", sa.String(length=255), nullable=True),
    sa.Column("rating", sa.Float(), nullable=True),
    sa.ForeignKeyConstraint(
        ["user_id"],
        ["users.id"],
    ),
    sa.PrimaryKeyConstraint("id"),
)
op.create_table(
    "refresh_tokens",
    sa.Column("id", sa.UUID(), nullable=False),
    sa.Column("user_id", sa.Integer(), nullable=False),
    sa.Column("refresh_hash", sa.String(length=256), nullable=False),
    sa.Column("user_agent", sa.String(length=255), nullable=True),
    sa.Column("last_seen", sa.DateTime(), nullable=False),
    sa.Column("created_at", sa.DateTime(), nullable=False),
    sa.Column("expires_in", sa.DateTime(), nullable=False),
    sa.ForeignKeyConstraint(
        ["user_id"],
        ["users.id"],
    ),
    sa.PrimaryKeyConstraint("id"),
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
sa.UniqueConstraint("refresh_hash"),
)
op.create_table(
    "ai_generated_files",
    sa.Column("id", sa.Integer(), nullable=False),
    sa.Column("session_id", sa.String(), nullable=False),
    sa.Column("filename", sa.String(), nullable=False),
    sa.Column("content", sa.Text(), nullable=False),
    sa.Column("file_type", sa.String(), nullable=False),
    sa.ForeignKeyConstraint(
        ["session_id"],
        ["ai_sessions.id"],
    ),
    sa.PrimaryKeyConstraint("id"),
)
op.create_table(
    "contest_participants",
    sa.Column("id", sa.Integer(), nullable=False),
    sa.Column("contest_id", sa.Integer(), nullable=False),
    sa.Column("participant_id", sa.Integer(), nullable=False),
    sa.Column("login", sa.String(length=255), nullable=False),
    sa.Column("name", sa.String(length=255), nullable=True),
    sa.Column("score", sa.Float(), nullable=True),
    sa.ForeignKeyConstraint(
        ["contest_id"],
        ["contests.id"],
    ),
    sa.ForeignKeyConstraint(
        ["participant_id"],
        ["participants.id"],
    ),
    sa.PrimaryKeyConstraint("id"),
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
)  
op.create_table(  
    "plagiarism_reports",  
    sa.Column("id", sa.Integer(), nullable=False),  
    sa.Column("contest_id", sa.Integer(), nullable=False),  
    sa.Column("status", sa.String(), nullable=False),  
    sa.Column("created_at", sa.DateTime(), nullable=False),  
    sa.Column("threshold", sa.Float(), nullable=False),  
    sa.Column("only_ok", sa.Boolean(), nullable=False),  
    sa.ForeignKeyConstraint(  
        ["contest_id"],  
        ["contests.id"],  
    ),  
    sa.PrimaryKeyConstraint("id"),  
)  
op.create_table(  
    "tasks",  
    sa.Column("id", sa.String(), nullable=False),  
    sa.Column("contest_id", sa.Integer(), nullable=False),  
    sa.Column("short_name", sa.String(length=20), nullable=True),  
    sa.Column("full_name", sa.String(length=100), nullable=False),  
    sa.Column("max_score", sa.Float(), nullable=True),  
    sa.ForeignKeyConstraint(  
        ["contest_id"],  
        ["contests.id"],  
    ),  
    sa.PrimaryKeyConstraint("id"),  
)  
op.create_table(  
    "task_results",  
    sa.Column("id", sa.Integer(), nullable=False),  
    sa.Column("contest_participant_id", sa.Integer(), nullable=False),
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

sa.Column("task_id", sa.String(), nullable=False),
sa.Column("score", sa.Float(), nullable=True),
sa.Column("tries_count", sa.Integer(), nullable=True),
sa.Column("verdict", sa.String(length=50), nullable=False),
sa.Column("last_success_time", sa.DateTime(), nullable=True),
sa.Column("banned", sa.Boolean(), nullable=False),
sa.ForeignKeyConstraint(
    ["contest_participant_id"],
    ["contest_participants.id"],
),
sa.ForeignKeyConstraint(
    ["task_id"],
    ["tasks.id"],
),
sa.PrimaryKeyConstraint("id"),
)
op.create_table(
    "submissions",
    sa.Column("id", sa.String(), nullable=False),
    sa.Column("contest_id", sa.Integer(), nullable=False),
    sa.Column("task_result_id", sa.Integer(), nullable=False),
    sa.Column("participant_login", sa.String(length=50), nullable=False),
    sa.Column("task_name", sa.String(length=50), nullable=False),
    sa.Column("send_time", sa.DateTime(), nullable=False),
    sa.Column("language", sa.String(length=50), nullable=False),
    sa.Column("score", sa.Float(), nullable=True),
    sa.Column("verdict", sa.String(length=50), nullable=False),
    sa.Column("run_time", sa.Interval(), nullable=False),
    sa.Column("memory_bytes", sa.Integer(), nullable=False),
    sa.Column("banned", sa.Boolean(), nullable=False),
    sa.Column("source", sa.String(), nullable=True),
    sa.ForeignKeyConstraint(

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        ["contest_id"],
        ["contests.id"],
    ),
    sa.ForeignKeyConstraint(
        ["task_result_id"],
        ["task_results.id"],
    ),
    sa.PrimaryKeyConstraint("id"),
)
op.create_table(
    "pair_of_banned_submissions",
    sa.Column("id", sa.Integer(), autoincrement=True, nullable=False),
    sa.Column("contest_id", sa.Integer(), nullable=False),
    sa.Column("report_id", sa.Integer(), nullable=False),
    sa.Column("first_submission_id", sa.String(), nullable=False),
    sa.Column("second_submission_id", sa.String(), nullable=False),
    sa.Column("percentage", sa.Float(), nullable=False),
    sa.ForeignKeyConstraint(
        ["contest_id"],
        ["contests.id"],
    ),
    sa.ForeignKeyConstraint(
        ["first_submission_id"],
        ["submissions.id"],
    ),
    sa.ForeignKeyConstraint(
        ["report_id"],
        ["plagiarism_reports.id"],
    ),
    sa.ForeignKeyConstraint(
        ["second_submission_id"],
        ["submissions.id"],

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    ),
    sa.PrimaryKeyConstraint("id"),
)
# ### end Alembic commands ###

def downgrade() -> None:
    """Downgrade schema."""

    # ### commands auto generated by Alembic - please adjust! ###
    op.drop_table("pair_of_banned_submissions")
    op.drop_table("submissions")
    op.drop_table("task_results")
    op.drop_table("tasks")
    op.drop_table("plagiarism_reports")
    op.drop_table("contest_participants")
    op.drop_table("ai_generated_files")
    op.drop_table("refresh_tokens")
    op.drop_table("participants")
    op.drop_table("contests")
    op.drop_index(op.f("ix_ai_sessions_id"), table_name="ai_sessions")
    op.drop_table("ai_sessions")
    op.drop_table("users")
    op.drop_table("roles")

    # ### end Alembic commands ###

```

1.5. alembic\versions\c1d2e3f4a5b6_drop_technical_data_from_ai_sessions.py

Листинг 5: alembic\versions\c1d2e3f4a5b6_drop_technical_data_from_ai_sessions.py

```

"""Drop technical_data column from ai_sessions

```

Revision ID: c1d2e3f4a5b6

Revises: f9e1d2c3b4a5

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Create Date: 2026-05-01 00:01:00.000000

"""

```
from typing import Sequence, Union
```

```
import sqlalchemy as sa
```

```
from alembic import op
```

```
revision: str = "c1d2e3f4a5b6"
```

```
down_revision: Union[str, Sequence[str], None] = "f9e1d2c3b4a5"
```

```
branch_labels: Union[str, Sequence[str], None] = None
```

```
depends_on: Union[str, Sequence[str], None] = None
```

```
def upgrade() -> None:
```

```
    op.drop_column("ai_sessions", "technical_data")
```

```
def downgrade() -> None:
```

```
    op.add_column(
```

```
        "ai_sessions",
```

```
        sa.Column("technical_data", sa.JSON(), nullable=True),
```

```
    )
```

1.6. alembic\versions\{f9e1d2c3b4a5}_add_uploaded_to_ai_generated_

Листинг 6: alembic\versions\{f9e1d2c3b4a5}_add_uploaded_to_ai_generated_files.py

```
"""Add uploaded column to ai_generated_files
```

Revision ID: f9e1d2c3b4a5

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Revises: 240bb27e187a

Create Date: 2026-05-01 00:00:00.000000

"""

from typing import Sequence, Union

import sqlalchemy as sa

from alembic import op

revision: str = "f9e1d2c3b4a5"

down_revision: Union[str, Sequence[str], None] = "240bb27e187a"

branch_labels: Union[str, Sequence[str], None] = None

depends_on: Union[str, Sequence[str], None] = None

def upgrade() -> None:

op.add_column(

"ai_generated_files",

sa.Column("uploaded", sa.Boolean(), nullable=False, server_default="false")

↪ ,

)

def downgrade() -> None:

op.drop_column("ai_generated_files", "uploaded")

1.7. api\{}__init__.py

Листинг 7: api\{}__init__.py

from api.health import router as health_router

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
__all__ = ["health_router"]
```

1.8. api\health.py

Листинг 8: api\health.py

```
from fastapi import APIRouter

router = APIRouter()

@router.get("/health")
async def health():
    return {"status": "ok"}
```

1.9. api\crypt__init__.py

Листинг 9: api\crypt__init__.py

```
from api.crypt.crypt_password import hash_password, verify_password
from api.crypt.jwt_token import (create_token, get_current_payload,
                                get_current_user, verify_token)

__all__ = [
    "create_token",
    "get_current_user",
    "get_current_payload",
    "hash_password",
    "verify_password",
    "verify_token",
]
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.10. api\{}crypt\{}crypt_password.py

Листинг 10: api\{}crypt\{}crypt_password.py

```
import hashlib

from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password):
    return pwd_context.hash(password)

def verify_password(plain_password, hashed_password):
    return pwd_context.verify(plain_password, hashed_password)

def hash_token(token: str):
    return hashlib.sha256(token.encode()).hexdigest()
```

1.11. api\{}crypt\{}jwt_token.py

Листинг 11: api\{}crypt\{}jwt_token.py

```
from datetime import datetime, timedelta

import jwt

from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer

from settings import settings
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

def create_token(data: dict, created_at: datetime, expires_delta: int):
    to_encode = data.copy()

    expire = created_at + timedelta(minutes=expires_delta)
    to_encode.update({"exp": int(expire.timestamp())})

    encoded_jwt = jwt.encode(
        to_encode, settings.SECRET_KEY, algorithm=settings.ALGORITHM
    )

    return encoded_jwt


def verify_token(token: str):
    try:
        payload = jwt.decode(
            token, settings.SECRET_KEY, algorithms=[settings.ALGORITHM]
        )
        return payload
    except jwt.ExpiredSignatureError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED, detail="Token has expired."
        )
    except jwt.InvalidTokenError as e:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED, detail=f"Invalid token. {e}"
        )


security = HTTPBearer()

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

def get_current_user(credentials: HTTPAuthorizationCredentials = Depends(security)
    ↪ ):
    token = credentials.credentials

    payload = verify_token(token)

    user_id = payload.get("user_id")
    if not user_id:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="User_ID_not_found_in_token",
        )

    return user_id

def get_current_payload(credentials: HTTPAuthorizationCredentials = Depends(
    ↪ security)):
    token = credentials.credentials

    payload = verify_token(token)

    return payload

```

1.12. api\{}\pydantic_schemas\{}__init__.py

Листинг 12: api\{}\pydantic_schemas\{}__init__.py

```

from api.pydantic_schemas.user.auth import (Authorization, ChangePassword,
                                              Token, UserLogin, UserRegister)
from api.pydantic_schemas.user.codeforces import (LinkCodeforces, Standings,
                                                    Submissions)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

from api.pydantic_schemas.user.yandex import Standings as YandexStandings
from api.pydantic_schemas.user.yandex import Submissions as YandexSubmissions

__all__ = [
    "Authorization",
    "ChangePassword",
    "LinkCodeforces",
    "Standings",
    "Submissions",
    "Token",
    "UserLogin",
    "UserRegister",
    "YandexStandings",
    "YandexSubmissions",
]

```

1.13. api\pydantic_schemas\user__init__.py

Листинг 13: api\pydantic_schemas\user__init__.py

1.14. api\pydantic_schemas\user\ai_task.py

Листинг 14: api\pydantic_schemas\user\ai_task.py

```

from typing import Any, Dict, List, Optional

from pydantic import BaseModel

class AIStatementRequest(BaseModel):
    idea: Optional[str] = ""
    model: str

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

user_prompt: Optional[str] = ""
history: Optional[List[Dict]] = []

```

```

class UpdateSessionSettingsRequest(BaseModel):
    model: Optional[str] = None
    system_prompt: Optional[str] = None

```

```

class AIStatementResponse(BaseModel):
    statement: Dict[str, Any]
    session_id: str
    stage: str

```

```

class RefineRequest(BaseModel):
    session_id: str
    feedback: str

```

```

class ApproveStatementRequest(BaseModel):
    """ """

    session_id: str

```

```

class GenerateFilesResponse(BaseModel):
    session_id: str
    technical_data: Dict[str, Any]
    stage: str

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

class RefineFileRequest(BaseModel):
    """ """

    session_id: str
    file_key: str # 'validator', 'generator', etc.
    feedback: str #

```

```

class ApproveFilesRequest(BaseModel):
    """ """

    session_id: str

```

```

class ManualFixRequest(BaseModel):
    """ """

    session_id: str
    file_key: str
    new_content: str

```

```

class UploadProgressResponse(BaseModel):
    status: str
    stage: str
    current_step: Optional[str] = None
    error: Optional[str] = None
    retries: Optional[int] = None
    technical_data: Optional[Dict] = None
    upload_errors: Optional[Dict] = None
    polygon_problem_id: Optional[int] = None

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.15. api\{}\pydantic_schemas\{}\user\{}\auth.py

Листинг 15: api\{}\pydantic_schemas\{}\user\{}\auth.py

```

import re

from pydantic import BaseModel, EmailStr, field_validator

class UserRegister(BaseModel):
    login: str
    password: str
    email: EmailStr

    @field_validator("login")
    @classmethod
    def validate_login(cls, value: str):
        if len(value) < 5:
            raise ValueError("Login_length_must_contain_at_least_5_characters")
        if len(value) > 30:
            raise ValueError("Login_length_must_contain_at_most_30_characters")

        value_without_underscore = value.replace("_", "")
        if not value_without_underscore.isalnum():
            raise ValueError("Login_must_contain_only_alphanumeric_characters")

        return value

    @field_validator("password")
    @classmethod
    def validate_password(cls, value: str):
        if len(value) < 8:
            raise ValueError("Password_length_must_contain_at_least_8_characters")

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

if len(value) > 30:
    raise ValueError("Password length must contain at most 30 characters")
if not re.search("[a-z]", value):
    raise ValueError("Password must contain lowercase letters")
if not re.search("[A-Z]", value):
    raise ValueError("Password must contain uppercase letters")
if not re.search("[0-9]", value):
    raise ValueError("Password must contain numbers")
if not re.search(r"[.,<>_?!@#$$%^&*()]", value):
    raise ValueError("Password must contain special characters")

return value

```

```
class UserLogin(BaseModel):
```

```

    login: str
    password: str

```

```
class Token(BaseModel):
```

```

    access_token: str
    refresh_token: str
    token_type: str

```

```
class Authorization(BaseModel):
```

```
    Bearer: str
```

```
class ChangePassword(BaseModel):
```

```

    old_password: str
    new_password: str

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

confirm_password: str

@field_validator("new_password")
@classmethod
def validate_password(cls, value: str):
    if len(value) < 8:
        raise ValueError("Password length must contain at least 8 characters")
    if len(value) > 30:
        raise ValueError("Password length must contain at most 30 characters")
    if not re.search("[a-z]", value):
        raise ValueError("Password must contain lowercase letters")
    if not re.search("[A-Z]", value):
        raise ValueError("Password must contain uppercase letters")
    if not re.search("[0-9]", value):
        raise ValueError("Password must contain numbers")
    if not re.search(r"[.,<>_?!@#$$%^&*()]", value):
        raise ValueError("Password must contain special characters")

    return value

```

1.16. api\pydantic_schemas\user\codeforces.py

Листинг 16: api\pydantic_schemas\user\codeforces.py

```

from pydantic import BaseModel, field_validator

```

```

class LinkCodeforces(BaseModel):

```

```

    api_key: str

```

```

    api_secret: str

```

```

class Standings(BaseModel):

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

contest_id: int
as_manager: bool
from_pos: int
count: int
show_unofficial: bool

@field_validator("from_pos")
@classmethod
def from_pos_validator(cls, value: int):
    if value < 1:
        raise ValueError("From_position_must_be_positive")
    return value

@field_validator("count")
@classmethod
def count_validator(cls, value: int):
    if value < 1:
        raise ValueError("Count_must_be_positive")
    return value

```

```

class Submissions(BaseModel):
    contest_id: int
    as_manager: bool
    from_pos: int
    count: int
    include_source: bool

    @field_validator("from_pos")
    @classmethod
    def from_pos_validator(cls, value: int):
        if value < 1:

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        raise ValueError("From_position_must_be_positive")
    return value

@field_validator("count")
@classmethod
def count_validator(cls, value: int):
    if value < 1:
        raise ValueError("Count_must_be_positive")
    return value

```

1.17. api\pydantic_schemas\user\yandex.py

Листинг 17: api\pydantic_schemas\user\yandex.py

```

from pydantic import BaseModel, field_validator

class Standings(BaseModel):
    contest_id: int
    as_manager: bool
    from_pos: int
    count: int
    show_unofficial: bool

    @field_validator("count")
    @classmethod
    def count_validator(cls, value: int):
        if value < 1:
            raise ValueError("Count_must_be_positive")
        return value

class Submissions(BaseModel):

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

contest_id: int
from_pos: int
count: int

@field_validator("from_pos")
@classmethod
def from_pos_validator(cls, value: int):
    if value < 1:
        raise ValueError("From_position_must_be_positive")
    return value

@field_validator("count")
@classmethod
def count_validator(cls, value: int):
    if value < 1:
        raise ValueError("Count_must_be_positive")
    return value

```

1.18. api\user__init__.py

Листинг 18: api\user__init__.py

```

from api.user.contests import (contest_router, get_contest_table,
                               get_user_contests)

__all__ = [
    "contest_router",
    "get_user_contests",
    "get_contest_table",
]

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.19. api\{}\user\{}\contests.py

Листинг 19: api\{}\user\{}\contests.py

```

from base64 import b64decode
from math import ceil

from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.params import Query
from sqlalchemy import func, select
from sqlalchemy.ext.asyncio import AsyncSession

from api.crypt import get_current_user
from app.database import get_db
from models import Contest, ContestParticipant, Submission, Task, TaskResult

contest_router = APIRouter()

@contest_router.get("/list")
async def get_user_contests(
    user_id: int = Depends(get_current_user), db: AsyncSession = Depends(get_db)
):
    contests = await db.execute(
        select(Contest).filter_by(user_id=user_id).order_by(Contest.id.desc())
    )
    contests = contests.scalars().all()

    return [
        {
            "id": contest.id,
            "name": contest.name,
            "platform": contest.platform,

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

        "external_id": contest.external_id,
        "type": contest.type,
        "unofficial": contest.unofficial,
        "start_time": (
            contest.start_time.isoformat() if contest.start_time else None
        ),
        "duration": contest.duration if contest.duration else None,
    }
    for contest in contests
]
```

```
@contest_router.get("/{contest_id}/overview")
```

```
async def get_contest_overview(
```

```
    contest_id: int,
```

```
    user_id: int = Depends(get_current_user),
```

```
    db: AsyncSession = Depends(get_db),
```

```
):
```

```
    contest = await db.execute(select(Contest).filter_by(id=contest_id))
```

```
    contest = contest.scalars().first()
```

```
    if not contest:
```

```
        raise HTTPException(status_code=404, detail="Contest not found")
```

```
    tasks_count = await db.execute(
```

```
        select(func.count(Task.id)).filter_by(contest_id=contest_id)
```

```
    )
```

```
    participants_count = await db.execute(
```

```
        select(func.count(ContestParticipant.id)).filter_by(contest_id=contest_id)
```

```
    )
```

```
    submissions_count = await db.execute(
```

```
        select(func.count(Submission.id)).filter_by(contest_id=contest_id)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

)

```

return {
    "id": contest.id,
    "name": contest.name,
    "external_id": contest.external_id, # ID Codeforces
    "start_time": contest.start_time,
    "type": contest.platform, # 'cf' 'yandex'
    "stats": {
        "tasks": tasks_count.scalar(),
        "participants": participants_count.scalar(),
        "submissions": submissions_count.scalar(),
    },
}

```

```
@contest_router.get("/{contest_id}/table")
```

```
async def get_contest_table(
```

```

    contest_id: int,
    page: int = Query(1, ge=1, description=""),
    per_page: int = Query(50, ge=10, le=200, description=""),
    search: str = Query("", description=""),
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),

```

```
):
```

```

    contest = await db.execute(select(Contest).filter_by(id=contest_id))
    contest = contest.scalars().first()

```

```
if not contest:
```

```
    raise HTTPException(
```

```
        status_code=status.HTTP_404_NOT_FOUND, detail="Contest not found"
```

```
)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

tasks = await db.execute(
    select(Task).filter_by(contest_id=contest.id).order_by(Task.short_name.asc
        ↪ ())
)
tasks = tasks.scalars().all()

participants_query = select(ContestParticipant).filter_by(contest_id=contest.id
    ↪ )
count_query = select(func.count(ContestParticipant.id)).filter_by(
    contest_id=contest.id
)

if search.strip():
    search_filter = f"%{search.strip().lower()}%"
    participants_query = participants_query.filter(
        (func.lower(ContestParticipant.name).like(search_filter))
        | (func.lower(ContestParticipant.login).like(search_filter))
    )
    count_query = count_query.filter(
        (func.lower(ContestParticipant.name).like(search_filter))
        | (func.lower(ContestParticipant.login).like(search_filter))
    )

total = await db.execute(count_query)
total = total.scalar()
total_pages = ceil(total / per_page) if total > 0 else 1

offset = (page - 1) * per_page
participants_query = (
    participants_query.order_by(ContestParticipant.score.desc())
    .offset(offset)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        .limit(per_page)
    )

    participants = await db.execute(participants_query)
    participants = participants.scalars().all()

    participant_ids = [participant.id for participant in participants]

    all_results = await db.execute(
        select(TaskResult).filter(
            TaskResult.contest_participant_id.in_(participant_ids)
        )
    )
    all_results = all_results.scalars().all()

    results_by_participant: dict[int, dict[int, TaskResult]] = {}
    for result in all_results:
        if result.contest_participant_id not in results_by_participant:
            results_by_participant[result.contest_participant_id] = {}
        results_by_participant[result.contest_participant_id][result.task_id] =
            ↪ result

    rows = []
    for participant in participants:
        results_map = results_by_participant.get(participant.id, {})

        row_results = []
        for task in tasks:
            result = results_map.get(task.id)
            row_results.append(
                {
                    "score": result.score if result else 0,

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        "verdict": result.verdict if result else "NULL",
        "tries": result.tries_count if result else 0,
        "time": (
            result.last_success_time.isoformat()
            if result and result.last_success_time
            else None
        ),
    }
)

rows.append(
    {
        "id": participant.id,
        "name": participant.name,
        "login": participant.login,
        "total_score": participant.score,
        "results": row_results,
    }
)

return {
    "contest_name": contest.name,
    "contest_type": contest.type,
    "tasks": [
        {"short_name": t.short_name, "full_name": t.full_name} for t in tasks
    ],
    "rows": rows,
    "pagination": {
        "page": page,
        "per_page": per_page,
        "total": total,
        "total_pages": total_pages,
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    },
}
```

```
@contest_router.get("/{contest_id}/submissions_list")
```

```
async def get_contest_submissions_headers(
```

```
    contest_id: int,
```

```
    page: int = Query(1, ge=1, description=""),
```

```
    per_page: int = Query(50, ge=10, le=200, description=""),
```

```
    search: str = Query("", description=",,,ID"),
```

```
    user_id: int = Depends(get_current_user),
```

```
    db: AsyncSession = Depends(get_db),
```

```
):
```

```
    submissions_query = select(Submission).filter_by(contest_id=contest_id)
```

```
    count_query = select(func.count(Submission.id)).filter_by(contest_id=contest_id
```

```
        ↪ )
```

```
    if search.strip():
```

```
        search_filter = f"%{search.strip().lower()}%"
```

```
        search_condition = (
```

```
            (func.lower(Submission.participant_login).like(search_filter))
```

```
            | (func.lower(Submission.task_name).like(search_filter))
```

```
            | (func.cast(Submission.id, func.text()).like(search_filter))
```

```
        )
```

```
        submissions_query = submissions_query.filter(search_condition)
```

```
        count_query = count_query.filter(search_condition)
```

```
    total = await db.execute(count_query)
```

```
    total = total.scalar()
```

```
    total_pages = ceil(total / per_page) if total > 0 else 1
```

```
    offset = (page - 1) * per_page
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

submissions_query = (
    submissions_query.order_by(Submission.send_time.desc())
    .offset(offset)
    .limit(per_page)
)

submissions = await db.execute(submissions_query)
submissions = submissions.scalars().all()

return {
    "items": [
        {
            "id": submission.id,
            "participant_login": submission.participant_login,
            "task_name": submission.task_name,
            "send_time": submission.send_time,
            "language": submission.language,
            "score": submission.score,
            "verdict": submission.verdict,
        }
        for submission in submissions
    ],
    "pagination": {
        "page": page,
        "per_page": per_page,
        "total": total,
        "total_pages": total_pages,
    },
}

```

@contest_router.get("/submissions/{submission_id}/source")

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

async def get_submission_source(
    submission_id: str,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    submission = await db.execute(select(Submission).filter_by(id=submission_id))
    submission = submission.scalars().first()

    return {
        "id": submission.id,
        "participant_login": submission.participant_login,
        "task_name": submission.task_name,
        "send_time": submission.send_time,
        "language": submission.language,
        "score": submission.score,
        "verdict": submission.verdict,
        "run_time": str(submission.run_time),
        "memory_bytes": str(submission.memory_bytes),
        "banned": submission.banned,
        "source": b64decode(submission.source).decode("utf-8"),
    }

```

```
@contest_router.delete("/{contest_id}")
```

```

async def delete_contest(
    contest_id: int,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    result = await db.execute(select(Contest).filter_by(id=contest_id, user_id=
        ↪ user_id))

    contest = result.scalars().first()

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

if not contest:
    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND, detail="Contest_not_found"
    )

await db.delete(contest)
await db.commit()

return {"message": "Contest_deleted_successfully"}

```

1.20. api\{}user\{}merge_submissions.py

Листинг 20: api\{}user\{}merge_submissions.py

```

from fastapi import HTTPException, status

async def merge_submissions(submissions, db):
    try:
        for submission in submissions:
            await db.merge(submission)
        await db.commit()
    except Exception as e:
        await db.rollback()
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR, detail=str(e)
        )

```

1.21. api\{}user\{}merge_table.py

Листинг 21: api\{}user\{}merge_table.py

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

from fastapi import HTTPException, status
from sqlalchemy import select

from models import Contest, ContestParticipant, Participant, TaskResult

async def merge_table(contest, tasks, rows, user_id, db):
    try:
        contest_copy = await db.execute(
            select(Contest).filter_by(external_id=contest.external_id, user_id=
                ↪ user_id)
        )
        contest_copy = contest_copy.scalars().first()

        if contest_copy:
            contest.id = contest_copy.id

        for participant, results in rows:
            global_participant = await db.execute(
                select(Participant).filter_by(login=participant.login)
            )
            global_participant = global_participant.scalars().first()

            if not global_participant:
                global_participant = Participant(
                    user_id=user_id,
                    login=participant.login,
                    name=participant.name,
                )
                db.add(global_participant)
                await db.flush()

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

participant.participant_id = global_participant.id

participant_copy = await db.execute(
    select(ContestParticipant).filter_by(
        contest_id=contest.id, participant_id=global_participant.id
    )
)

participant_copy = participant_copy.scalars().first()

if participant_copy:
    participant.id = participant_copy.id

participant.contest = contest

for result in results:
    result.contest_participant = participant

    if hasattr(participant, "id") and participant.id:
        row_copy = await db.execute(
            select(TaskResult).filter_by(
                contest_participant_id=participant.id,
                task_id=result.task_id,
            )
        )
        row_copy = row_copy.scalars().first()
        if row_copy:
            result.id = row_copy.id

await db.merge(contest)
await db.commit()

return {

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        "message": "Standings updated successfully",
        "contest_name": contest.name,
    }

except Exception as e:
    await db.rollback()
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR, detail=str(e)
    )

```

1.22. api\user\auth__init__.py

Листинг 22: api\user\auth__init__.py

```

from api.user.auth.base_auth import router as auth_router
from api.user.auth.change_password import change_password
from api.user.auth.location import get_location
from api.user.auth.login import login, register
from api.user.auth.logout import logout, logout_all
from api.user.auth.profile import get_me, get_sessions
from api.user.auth.refresh import refresh
from api.user.auth.tokens import get_tokens

__all__ = [
    "auth_router",
    "get_location",
    "get_tokens",
    "register",
    "login",
    "logout",
    "logout_all",
    "change_password",
    "get_me",

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    "get_sessions",
    "refresh",
]

```

1.23. api\{}user\{}auth\{}base_auth.py

Листинг 23: api\{}user\{}auth\{}base_auth.py

```

from fastapi import APIRouter

router = APIRouter()

```

1.24. api\{}user\{}auth\{}change_password.py

Листинг 24: api\{}user\{}auth\{}change_password.py

```

import logging

from fastapi import Depends, HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from api.crypt import get_current_user, hash_password, verify_password
from api.pydantic_schemas import ChangePassword
from api.user.auth import auth_router
from app.database import get_db
from models import User

logger = logging.getLogger(__name__)

@auth_router.post("/change_password")
async def change_password(

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

payload: ChangePassword,
user_id: int = Depends(get_current_user),
db: AsyncSession = Depends(get_db),
) -> dict:

user = await db.execute(select(User).filter_by(id=user_id))
user = user.scalars().first()

if not user:
    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND, detail="User not found."
    )

if not verify_password(payload.old_password, user.password):
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST, detail="Invalid password."
    )

if payload.old_password == payload.new_password:
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
        detail="New password can't be the same.",
    )

if payload.new_password != payload.confirm_password:
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
        detail="New password and confirmation do not match.",
    )

user.password = hash_password(payload.new_password)
await db.commit()

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
logger.info(f"Password changed: user_id={user_id}")
return {"message": "Successfully changed password."}
```

1.25. api\user\auth\location.py

Листинг 25: api\user\auth\location.py

```
import httpx

async def get_location(ip: str) -> str:
    if ip in ("127.0.0.1", "localhost") or ip.startswith("172."):
        return "_"

    try:
        async with httpx.AsyncClient(timeout=1.0) as client:
            response = await client.get(
                f"https://ip-api.com/json/{ip}?fields=status,country,city"
            )
            data = response.json()
            if data.get("status") == "success":
                return f"{data.get('city')}, {data.get('country')}"
    except Exception:
        pass
    return "_"
```

1.26. api\user\auth\login.py

Листинг 26: api\user\auth\login.py

```
import logging
import uuid
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

from fastapi import Depends, HTTPException, Request, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from api.crypt import hash_password, verify_password
from api.crypt.crypt_password import hash_token
from api.pydantic_schemas import Token, UserLogin, UserRegister
from api.user.auth.base_auth import router as auth_router
from api.user.auth.tokens import get_tokens
from api.user.auth.location import get_location
from app.database import get_db
from models import RefreshToken, Role, User

logger = logging.getLogger(__name__)

@auth_router.post("/register", response_model=Token)
async def register(
    payload: UserRegister, request: Request, db: AsyncSession = Depends(get_db)
):
    user = await db.execute(select(User).filter_by(login=payload.login))
    user = user.scalars().first()

    if user:
        logger.warning(f"Registration failed: login '{payload.login}' already exists")
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="User with such login already exists.",
        )

    role = await db.execute(select(Role).filter_by(name="Admin"))

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

role = role.scalars().first()

if not role:
    logger.error("Registration failed: system role 'Admin' not found in DB")
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail='Default system role "Admin" not found. Contact developer.',
    )

hashed_password = hash_password(payload.password)

new_user = User(
    login=payload.login,
    password=hashed_password,
    email=payload.email,
    role_id=role.id,
)

db.add(new_user)
await db.commit()
await db.refresh(new_user)

logger.info(f"New user registered: '{payload.login}' (id={new_user.id})")

return await login(
    UserLogin(login=payload.login, password=payload.password),
    request=request,
    db=db,
)

```

```
@auth_router.post("/login", response_model=Token)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

async def login(
    payload: UserLogin, request: Request, db: AsyncSession = Depends(get_db)
) -> Token:
    client_ip = request.client.host if request.client else "unknown"

    user = await db.execute(select(User).filter_by(login=payload.login))
    user = user.scalars().first()

    if not user:
        logger.warning(f"Login failed: unknown login '{payload.login}' from {
            ↪ client_ip}")
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid login or password.",
        )

    if not verify_password(payload.password, user.password):
        logger.warning(f"Login failed: wrong password for '{payload.login}' from {
            ↪ client_ip}")
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid login or password.",
        )

    user_id: int = user.id
    session_id = uuid.uuid4()

    location = await get_location(request.client.host)
    user_agent = f'{location}|{request.headers.get("user-agent", "Unknown")}'

    access_token, refresh_token, created_at, expires_in = get_tokens(
        user_id, str(session_id)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

)

refresh_hash = hash_token(refresh_token)

refresh_token_sub = RefreshToken(
    id=session_id,
    refresh_hash=refresh_hash,
    user_id=user_id,
    user_agent=user_agent,
    created_at=created_at,
    expires_in=expires_in,
)

db.add(refresh_token_sub)
await db.commit()

logger.info(f"Login successful: user_id={user_id} ('{payload.login}') from {
    ↪ client_ip}")

return Token(
    access_token=access_token, refresh_token=refresh_token, token_type="Bearer"
)

```

1.27. api\{}user\{}auth\{}logout.py

Листинг 27: api\{}user\{}auth\{}logout.py

```

import logging

from fastapi import Depends, HTTPException, status
from sqlalchemy import delete, select
from sqlalchemy.ext.asyncio import AsyncSession

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

from api.crypt import get_current_user
from api.crypt.crypt_password import hash_token
from api.pydantic_schemas import Token
from api.user.auth import auth_router
from app.database import get_db
from models import RefreshToken

logger = logging.getLogger(__name__)

@auth_router.post("/logout")
async def logout(payload: Token, db: AsyncSession = Depends(get_db)) -> dict:
    refresh_hash = hash_token(payload.refresh_token)

    db_token = await db.execute(
        select(RefreshToken).filter_by(refresh_hash=refresh_hash)
    )
    db_token = db_token.scalars().first()

    if not db_token:
        logger.warning("Logout failed: invalid refresh token")
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid refresh token.
            ↪ "
        )

    user_id = db_token.user_id
    await db.delete(db_token)
    await db.commit()

    logger.info(f"Logout: user_id={user_id}")
    return {"message": "Successfully logged out."}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

@auth_router.post("/logout_all")
async def logout_all(
    user_id: int = Depends(get_current_user), db: AsyncSession = Depends(get_db)
) -> dict:
    await db.execute(delete(RefreshToken).where(RefreshToken.user_id == user_id))
    await db.commit()

    logger.info(f"Logout_all_sessions: user_id={user_id}")
    return {"message": "Successfully_logged_all_devices_out."}

```

1.28. api\{}user\{}auth\{}profile.py

ЛІСТІНГ 28: api\{}user\{}auth\{}profile.py

```

from fastapi import HTTPException, status
from fastapi.params import Depends
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from api.crypt import get_current_payload, get_current_user
from api.user.auth import auth_router
from app.database import get_db
from models import RefreshToken, User

@auth_router.get("/me")
async def get_me(
    user_id: int = Depends(get_current_user), db: AsyncSession = Depends(get_db)
):
    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

```

Ізм.	Ліст	№ докум.	Підп.	Дата
RU.17701729.05.05-01 12 01-1				
Інв. № подл.	Підп. и дата	Взам. инв. №	Інв. № дубл.	Підп. и дата

```

if not user:
    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND, detail="User not found."
    )

return {
    "id": user.id,
    "login": user.login,
    "email": user.email,
    "role_id": user.role_id,
    "is_yandex_linked": bool(user.yandex_access_token),
    "is_codeforces_linked": bool(user.codeforces_api_key),
    "is_polygon_linked": bool(user.polygon_api_key),
}

```

```
@auth_router.get("/sessions")
```

```
async def get_sessions(
```

```
    payload: dict = Depends(get_current_payload), db: AsyncSession = Depends(get_db
    ↪ )
```

```
):
```

```
    user_id = payload.get("user_id")
```

```
    current_sid = payload.get("sid")
```

```
    sessions = await db.execute(
```

```
        select(RefreshToken)
```

```
        .filter_by(user_id=user_id)
```

```
        .order_by(RefreshToken.created_at.desc())
```

```
    )
```

```
    sessions = sessions.scalars().all()
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

return [
    {
        "id": session.id,
        "user_agent": session.user_agent,
        "is_current": str(session.id) == current_sid,
        "last_seen": session.last_seen.isoformat() + "Z",
        "created_at": session.created_at.isoformat() + "Z",
        "expires_in": session.expires_in.isoformat() + "Z",
    }
    for session in sessions
]

```

1.29. api\{}\user\{}\auth\{}\refresh.py

Листинг 29: api\{}\user\{}\auth\{}\refresh.py

```

import logging
import uuid

from fastapi import Depends, HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from api.crypt.crypt_password import hash_token
from api.pydantic_schemas import Token
from api.user.auth.base_auth import router as auth_router
from api.user.auth.tokens import get_tokens
from app.database import get_db
from models import RefreshToken

logger = logging.getLogger(__name__)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

@auth_router.post("/refresh", response_model=Token)
async def refresh(payload: Token, db: AsyncSession = Depends(get_db)):
    old_refresh_hash = hash_token(payload.refresh_token)

    db_token = await db.execute(
        select(RefreshToken).filter_by(refresh_hash=old_refresh_hash)
    )
    db_token = db_token.scalars().first()

    if not db_token:
        logger.warning("Token_refresh_failed: invalid_refresh_token")
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid_refresh_token.
            ↪ "
        )

    new_sid = uuid.uuid4()
    user_id = db_token.user_id

    access_token, refresh_token, created_at, expires_in = get_tokens(
        user_id, str(new_sid)
    )

    new_refresh_hash = hash_token(refresh_token)

    db_token.id = new_sid
    db_token.refresh_hash = new_refresh_hash
    db_token.created_at = created_at
    db_token.expires_in = expires_in

    await db.commit()

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

logger.debug(f"Token refreshed: user_id={user_id}")

return Token(
    access_token=access_token, refresh_token=refresh_token, token_type="Bearer"
)

```

1.30. api\user\auth\tokens.py

Листинг 30: api\user\auth\tokens.py

```

from datetime import datetime, timedelta, timezone
from typing import Tuple

from api.crypt import create_token
from settings import settings

def get_tokens(user_id: int, session_id: str) -> Tuple[str, str, datetime,
    ↪ datetime]:
    created_at = datetime.now(timezone.utc)
    expires_in = created_at + timedelta(minutes=settings.EXPIRES_REFRESH)

    data = {
        "user_id": user_id,
        "sid": session_id,
    }

    access_token = create_token(
        data=data, created_at=created_at, expires_delta=settings.EXPIRES_ACCESS
    )

    refresh_token = create_token(
        data=data, created_at=created_at, expires_delta=settings.EXPIRES_REFRESH

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

)

return (
    access_token,
    refresh_token,
    created_at.replace(tzinfo=None),
    expires_in.replace(tzinfo=None),
)

```

1.31. api\{user}\{codeforces}\{__init__.py

Листинг 31: api\{user}\{codeforces}\{__init__.py

```

from api.user.codeforces.auth.login import link_codeforces
from api.user.codeforces.auth.logout import unlink_codeforces
from api.user.codeforces.base_codeforces import router as codeforces_router
from api.user.codeforces.format import (format_codeforces_standings,
                                         format_codeforces_submissions)
# from api.user.codeforces.auth.login import get_codeforces_auth_url,
#     ↪ codeforces_callback
from api.user.codeforces standings standings import codeforces_standings
from api.user.codeforces submissions submissions import codeforces_submissions

__all__ = [
    "link_codeforces",
    "unlink_codeforces",
    "codeforces_router",
    "format_codeforces_standings",
    "format_codeforces_submissions",
    "get_response",
    "codeforces_standings",
    "codeforces_submissions",
]

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.32. api\user\codeforces\base_codeforces.py

Листинг 32: api\user\codeforces\base_codeforces.py

```
from fastapi import APIRouter

router = APIRouter()
```

1.33. api\user\codeforces\create_signature.py

Листинг 33: api\user\codeforces\create_signature.py

```
from hashlib import sha512
from random import randrange

async def create_signature(
    method_name: str, params: dict[str, str], secret: str
) -> str:
    sorted_params = sorted(params.items())
    random_string = f"{randrange(1000000):06d}"

    signature_string = f'{random_string}/{method_name}?{"&".join([f"{k}={v}" for k,
        ↪ v in sorted_params])}#{secret}'

    hash_signature = sha512(signature_string.encode()).hexdigest()

    return f"{random_string}{hash_signature}"
```

1.34. api\user\codeforces\format.py

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Листинг 34: api\{user}\{codeforces}\{format.py

```

from datetime import datetime, timedelta
from typing import Dict, List, Tuple

from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from models import Contest, ContestParticipant, Submission, Task, TaskResult

def format_codeforces_standings(
    standings: Dict[str, dict], user_id: int, unofficial: bool
) -> Tuple[Contest, List[Task], List[ContestParticipant]]:
    contest = standings["contest"]
    problems = standings["problems"]
    rows = standings["rows"]

    start_time = None
    if "startTimeSeconds" in contest:
        start_time = datetime.fromtimestamp(contest["startTimeSeconds"])

    formatted_contest = Contest(
        external_id=contest["id"],
        user_id=user_id,
        platform="cf",
        name=contest["name"],
        type=contest["type"] if contest["type"] != "CF" else "ICPC",
        unofficial=unofficial,
        start_time=start_time,
        duration=timedelta(seconds=contest["durationSeconds"]),
    )

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

best_scores = [0.0] * len(problems)
for row in rows:
    for idx, problem in enumerate(row["problemResults"]):
        points = float(problem.get("points", 0))
        if points > best_scores[idx]:
            best_scores[idx] = points

formatted_problems: List[Task] = []
for idx, problem in enumerate(problems):
    max_score = problem.get("points", best_scores[idx])
    formatted_problems.append(
        Task(
            id=f'cf_{user_id}_{contest["id"]}_{problem["index"]}',
            contest=formatted_contest,
            short_name=problem["index"],
            full_name=problem["name"],
            max_score=max_score,
        )
    )

merged_participants = {}
merged_results = {}

for row in rows:
    party = row["party"]
    login = (
        party["members"][0]["handle"]
        if "teamId" not in party
        else f'team_{party["teamId"]}'
    )
    name = party.get("teamName") or party["members"][0].get("name")

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

if login not in merged_participants:
    merged_participants[login] = ContestParticipant(
        login=login, name=name, score=0.0
    )
    merged_results[login] = {}

for idx, problem in enumerate(row["problemResults"]):
    task_id = formatted_problems[idx].id
    points = float(problem.get("points", 0))
    tries = int(problem.get("rejectedAttemptCount", 0))

    success_time = None
    if points > 0 and start_time and "bestSubmissionTimeSeconds" in problem:
        success_time = start_time + timedelta(
            seconds=problem["bestSubmissionTimeSeconds"]
        )

    if task_id in merged_results[login]:
        old_res = merged_results[login][task_id]

        if points > old_res.score:
            old_res.score = points
            old_res.last_success_time = success_time
        elif points == old_res.score and success_time:
            if (
                not old_res.last_success_time
                or success_time < old_res.last_success_time
            ):
                old_res.last_success_time = success_time

        old_res.tries_count += tries
    else:

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

merged_results[login][task_id] = TaskResult(
    task_id=task_id,
    score=points,
    tries_count=tries,
    last_success_time=success_time,
    verdict="NULL",
    banned=False,
)

formatted_rows = []
for login, contest_participant in merged_participants.items():
    total_score = 0.0
    task_results_list = []

    for problem in formatted_problems:
        res = merged_results[login].get(problem.id)
        if not res:
            continue

        if res.score > 0:
            res.verdict = "OK" if res.score == problem.max_score else "PARTIAL"
        elif res.tries_count > 0:
            res.verdict = "WA"
        else:
            res.verdict = "NULL"

        total_score += res.score
        task_results_list.append(res)

    contest_participant.score = total_score
    formatted_rows.append((contest_participant, task_results_list))

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
return formatted_contest, formatted_problems, formatted_rows
```

```
async def format_codeforces_submissions(
    submissions: List[dict], user_id: int, contest_id: int, db: AsyncSession
) -> List[Submission]:
    formatted_submissions = []
    for submission in submissions:
        problem = submission["problem"]
        problem = await db.execute(
            select(Task).filter_by(full_name=problem["name"], contest_id=contest_id)
        )
        problem = problem.scalars().first()

        party = submission["author"]
        participant_login = (
            party["members"][0]["handle"]
            if "teamId" not in party
            else f'team_{party["teamId"]}'
        )

        participant = await db.execute(
            select(ContestParticipant).filter_by(
                contest_id=contest_id, login=participant_login
            )
        )
        participant = participant.scalars().first()

        if not participant:
            continue

        task_result = await db.execute(
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

        select(TaskResult).filter_by(
            task_id=problem.id, contest_participant_id=participant.id
        )
    )
    task_result = task_result.scalars().first()

    if not task_result:
        continue

    formatted_submission = Submission(
        id=f'cf_{user_id}_{contest_id}_{submission["id"]}',
        contest_id=contest_id,
        task_result_id=task_result.id,
        participant_login=participant_login,
        task_name=problem.full_name,
        send_time=datetime.fromtimestamp(submission["creationTimeSeconds"]),
        language=submission["programmingLanguage"],
        score=float(submission.get("points", 0)),
        verdict=submission["verdict"],
        run_time=timedelta(milliseconds=submission["timeConsumedMillis"]),
        memory_bytes=int(submission["memoryConsumedBytes"]),
        source=submission["sourceBase64"],
    )

    formatted_submissions.append(formatted_submission)

return formatted_submissions

```

1.35. api\{}user\{}codeforces\{}get_response.py

Листинг 35: api\{}user\{}codeforces\{}get_response.py

```
import json
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

from aiohttp import ClientSession
from fastapi import HTTPException, status

async def get_response(client: ClientSession, url, params):
    async with client.post(url, data=params) as response:
        response_text = await response.text()

    try:
        result = json.loads(response_text)
    except json.JSONDecodeError:
        if response.status != 200:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=f"HTTP_{response.status}: {response_text[:200]}",
            )
        else:
            return {"message": response_text}

    if result["status"] == "OK":
        try:
            return result["result"]
        except KeyError:
            return {"message": response_text}

    else:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST, detail=result["comment"]
        )

```

1.36. api\user\codeforces\auth__init__.py

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Листинг 36: api\user\codeforces\auth__init__.py

1.37. api\user\codeforces\auth\login.py

Листинг 37: api\user\codeforces\auth\login.py

```
import logging

from fastapi import Depends
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from api.crypt import get_current_user
from api.pydantic_schemas import LinkCodeforces
from api.user.codeforces.base_codeforces import router as codeforces_router
from app.database import get_db
from models import User

logger = logging.getLogger(__name__)

@codeforces_router.post("/link")
async def link_codeforces(
    payload: LinkCodeforces,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    # CF API
    user.codeforces_api_key = payload.api_key
    user.codeforces_api_secret = payload.api_secret
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
await db.commit()
```

```
logger.info(f"Codeforces_API_key_linked: user_id={user_id}")
```

```
return {"message": "Codeforces_account_successfully_linked"}
```

```
# def generate_pkce():
```

```
# verifier = secrets.token_urlsafe(32)
```

```
# challenge = base64.urlsafe_b64encode(
```

```
# hashlib.sha256(verifier.encode()).digest()
```

```
# ).decode().rstrip("=")
```

```
# return verifier, challenge
```

```
#
```

```
#
```

```
# class CodeforcesCallbackRequest(BaseModel):
```

```
# code: str
```

```
# code_verifier: str
```

```
#
```

```
#
```

```
# @codeforces_router.get('/auth_url')
```

```
# async def get_codeforces_auth_url():
```

```
# verifier, challenge = generate_pkce()
```

```
# state = secrets.token_urlsafe(16)
```

```
#
```

```
# url = (
```

```
# f'https://codeforces.com/oauth/authorize?'
```

```
# f'response_type=code&'
```

```
# f'client_id={settings.CF_CLIENT_ID}&'
```

```
# f'redirect_uri={"http://localhost:5173/codeforces/callback"}&'
```

```
# f'scope=openid&'
```

```
# f'state={state}&'
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# f'code_challenge={challenge}&'
# f'code_challenge_method=S256'
# )
# return {'url': url, 'code_verifier': verifier}
#
#
# @codeforces_router.post('/callback')
# async def codeforces_callback(
#     payload: CodeforcesCallbackRequest,
#     user_id: int = Depends(get_current_user),
#     db: AsyncSession = Depends(get_db)
# ):
#     async with httpx.AsyncClient() as client:
#         response = await client.post(
#             'https://codeforces.com/oauth/token',
#             data={
#                 'grant_type': 'authorization_code',
#                 'code': payload.code,
#                 'client_id': settings.CF_CLIENT_ID,
#                 'client_secret': settings.CF_CLIENT_SECRET,
#                 'redirect_uri': "http://localhost:5173/codeforces/callback",
#                 'code_verifier': payload.code_verifier,
#             }
#         )
#
#         if response.status_code != 200:
#             raise HTTPException(
#                 status_code=status.HTTP_400_BAD_REQUEST,
#                 detail='Failed to login with Codeforces API',
#             )
#
#         data = response.json()

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# token = data.get('access_token')
#
# print(response.text)
#
# user = await db.execute(select(User).filter_by(id=user_id))
# user = user.scalars().first()
#
# user.codeforces_access_token = token
# await db.commit()
#
# return {'message': 'Codeforces account successfully linked'}

```

1.38. api\{}user\{}codeforces\{}auth\{}logout.py

Листинг 38: api\{}user\{}codeforces\{}auth\{}logout.py

```

from fastapi import Depends
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from api.crypt import get_current_user
from api.user.codeforces.base_codeforces import router as codeforces_router
from app.database import get_db
from models import User

@codeforces_router.post("/unlink")
async def unlink_codeforces(
    user_id: int = Depends(get_current_user), db: AsyncSession = Depends(get_db)
):
    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

user.codeforces_api_key = None

user.codeforces_api_secret = None

# user.codeforces_access_token = None

await db.commit()


return {"message": "Codeforces_account_successfully_unlinked"}

```

1.39. api\user\codeforces\standings__init__.py

Листинг 39: api\user\codeforces\standings__init__.py

1.40. api\user\codeforces\standings\standings.py

Листинг 40: api\user\codeforces\standings\standings.py

```

from time import time


from aiohttp import ClientSession
from fastapi import Depends, HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL


from api.crypt import get_current_user
from api.pydantic_schemas import Standings
from api.user.codeforces import codeforces_router, create_signature
from api.user.codeforces.format import format_codeforces_standings
from api.user.codeforces.get_response import get_response
from api.user.merge_table import merge_table
from app.database import get_db
from models import User
from settings import settings

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

@codeforces_router.post("/standings")
async def codeforces_standings(
    standings: Standings,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    method_name = "contest_standings"

    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.codeforces_api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Codeforces API is not configured",
        )

    api_key = str(user.codeforces_api_key)
    api_secret = str(user.codeforces_api_secret)
    current_time_unix = int(time())

    params = {
        "apiKey": api_key,
        "time": str(current_time_unix),
        "contestId": standings.contest_id,
        "asManager": standings.as_manager,
        "from": standings.from_pos,
        "count": standings.count,
        "showUnofficial": standings.show_unofficial,
    }

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

signature = await create_signature(method_name, params, api_secret)
params["apiSig"] = signature

url = URL(settings.CODEFORCES_HOST) / method_name

async with ClientSession() as client:
    response = await get_response(client, url, params)

contest, tasks, rows = format_codeforces_standings(
    response, user_id, standings.show_unofficial
)

return await merge_table(contest, tasks, rows, user_id, db)

```

1.41. api\{}user\{}codeforces\{}submissions\{}__init__.py

Листинг 41: api\{}user\{}codeforces\{}submissions\{}__init__.py

1.42. api\{}user\{}codeforces\{}submissions\{}submissions.py

Листинг 42: api\{}user\{}codeforces\{}submissions\{}submissions.py

```

from time import time

from aiohttp import ClientSession
from fastapi import Depends, HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.crypt import get_current_user

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

from api.pydantic_schemas import Submissions
from api.user.codeforces import codeforces_router, create_signature
from api.user.codeforces.format import format_codeforces_submissions
from api.user.codeforces.get_response import get_response
from api.user.merge_submissions import merge_submissions
from app.database import get_db
from models import Contest, User
from settings import settings

```

```

@codeforces_router.post("/submissions")
async def codeforces_submissions(
    submissions: Submissions,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    method_name = "contest.status"

    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.codeforces_api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Codeforces API is not configured",
        )

    api_key = str(user.codeforces_api_key)
    api_secret = str(user.codeforces_api_secret)
    current_time_unix = int(time())

    params = {

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    "apiKey": api_key,
    "time": str(current_time_unix),
    "contestId": submissions.contest_id,
    "asManager": submissions.as_manager,
    "from": submissions.from_pos,
    "count": submissions.count,
    "includeSources": "true", # TODO
}

signature = await create_signature(method_name, params, api_secret)
params["apiSig"] = signature

url = URL(settings.CODEFORCES_HOST) / method_name

async with ClientSession() as client:
    response = await get_response(client, url, params)

contest = await db.execute(
    select(Contest).filter_by(external_id=submissions.contest_id)
)
contest = contest.scalars().first()

submissions_result = await format_codeforces_submissions(
    response, user_id, contest.id, db
)

return await merge_submissions(submissions_result, db)

```

1.43. api\{}user\{}gpt\{}__init__.py

Листинг 43: api\{}user\{}gpt\{}__init__.py

```
from api.user.gpt.base_gpt import gpt_router
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

from . import ai_tasks

__all__ = ["ai_tasks", "gpt_router"]

```

1.44. api\{}user\{}gpt\{}ai_tasks.py

Листинг 44: api\{}user\{}gpt\{}ai_tasks.py

```

# api/user/gpt/router.py

import json
import uuid
from datetime import datetime, timezone

from fastapi import BackgroundTasks, Depends, HTTPException
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from api.crypt import get_current_user
from api.pydantic_schemas.user.ai_task import (AIStatementRequest,
                                                AIStatementResponse,
                                                ApproveFilesRequest,
                                                ApproveStatementRequest,
                                                ManualFixRequest,
                                                RefineFileRequest,
                                                RefineRequest,
                                                UpdateSessionSettingsRequest)

from api.user.gpt import gpt_router
from api.user.gpt.services.ai_file_helpers import (get_all_file_contents,
                                                    get_session_files,
                                                    upsert_ai_file,
                                                    upsert_all_ai_files)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

from api.user.gpt.services.ai_service import TaskAIService
from api.user.gpt.services.upload_orchestrator import (
    retry_upload_after_manual_fix, run_upload_pipeline)
from app.database import get_db
from models.ai.ai_session import AISession, PipelineStage

#

def now_utc() -> datetime:
    return datetime.now(timezone.utc).replace(tzinfo=None)

def touch(session: AISession) -> None:
    session.updated_at = now_utc()

async def get_session_or_404(
    session_id: str,
    user_id: int,
    db: AsyncSession,
) -> AISession:
    session = await db.get(AISession, session_id)
    if not session:
        raise HTTPException(404, " ")
    if session.user_id != user_id:
        raise HTTPException(403, " ")
    return session

#

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

@gpt_router.get("/sessions")
async def list_sessions(
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    result = await db.execute(
        select(AISession)
        .where(AISession.user_id == user_id)
        .order_by(AISession.updated_at.desc())
    )
    sessions = result.scalars().all()

    return [
        {
            "session_id": s.id,
            "stage": s.stage,
            "name": (
                s.statement.get("name", "□") if s.statement else ""
            ),
            "model": s.model,
            "polygon_problem_id": s.polygon_problem_id,
            "created_at": s.created_at.isoformat() if s.created_at else None,
            "updated_at": s.updated_at.isoformat() if s.updated_at else None,
        }
        for s in sessions
    ]

```

#

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

@gpt_router.post("/create-session")
async def create_session(
    request: AIStatementRequest,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    ts = now_utc()
    idea = (request.idea or "").strip()

    if not idea:
        session = AISession(
            id=str(uuid.uuid4()),
            user_id=user_id,
            model=request.model,
            system_prompt=request.user_prompt or "",
            statement=None,
            history=[],
            stage=PipelineStage.STATEMENT,
            progress={"status": "idle"},
            created_at=ts,
            updated_at=ts,
        )
        db.add(session)
        await db.commit()
        return {"session_id": session.id, "statement": None, "stage": PipelineStage
            ↪ .STATEMENT}

    ai = TaskAIService()
    statement_data = await ai.generate_statement(
        user_idea=idea,
        model=request.model,
        user_prompt=request.user_prompt,

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        history=request.history or [],
    )

    session = AISession(
        id=str(uuid.uuid4()),
        user_id=user_id,
        model=request.model,
        system_prompt=request.user_prompt or "",
        statement=statement_data,
        history=request.history or [],
        stage=PipelineStage.STATEMENT,
        progress={"status": "idle"},
        created_at=ts,
        updated_at=ts,
    )
    db.add(session)
    await db.commit()

    return AIStatementResponse(
        statement=statement_data,
        session_id=session.id,
        stage=PipelineStage.STATEMENT,
    )
```

#

```
@gpt_router.delete("/session/{session_id}")
```

```
async def delete_session(
```

```
    session_id: str,
```

```
    user_id: int = Depends(get_current_user),
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

    db: AsyncSession = Depends(get_db),
):
    session = await get_session_or_404(session_id, user_id, db)
    await db.delete(session)
    await db.commit()
    return {"status": "deleted"}

# 1:

@gpt_router.post("/refine-statement", response_model=AIStatementResponse)
async def refine_statement(
    request: RefineRequest,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    session = await get_session_or_404(request.session_id, user_id, db)

    if session.stage != PipelineStage.STATEMENT:
        raise HTTPException(
            400, f"⚠️ {session.stage}"
        )

    history = list(session.history or [])
    if session.statement:
        history.append(
            {
                "role": "assistant",
                "content": json.dumps(session.statement, ensure_ascii=False),
            }
        )

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
history.append({"role": "user", "content": request.feedback})
```

```
ai = TaskAIService()
```

```
new_statement = await ai.generate_statement(
```

```
    user_idea=request.feedback,
```

```
    model=session.model,
```

```
    user_prompt=session.system_prompt,
```

```
    history=history,
```

```
)
```

```
session.history = history
```

```
session.statement = new_statement
```

```
touch(session)
```

```
await db.commit()
```

```
return AIStatementResponse(
```

```
    statement=new_statement,
```

```
    session_id=session.id,
```

```
    stage=PipelineStage.STATEMENT,
```

```
)
```

```
# 2:
```

```
@gpt_router.post("/approve-statement")
```

```
async def approve_statement(
```

```
    request: ApproveStatementRequest,
```

```
    user_id: int = Depends(get_current_user),
```

```
    db: AsyncSession = Depends(get_db),
```

```
):
```

```
    session = await get_session_or_404(request.session_id, user_id, db)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

if session.stage not in (PipelineStage.STATEMENT, PipelineStage.FAILED):
    raise HTTPException(400, f"␣␣␣␣␣␣'{session.stage}'")

session.stage = PipelineStage.FILES_REVIEW
session.progress = {
    "status": "generating_files",
    "current_step": "␣␣...",
}
touch(session)
await db.commit()

try:
    ai = TaskAIService()
    tech_data = await ai.generate_technical_stuff(session.statement, session.
        ↪ model)

    await upsert_all_ai_files(db, session.id, tech_data, uploaded=False)

    session.progress = {
        "status": "files_ready",
        "current_step": "␣␣␣",
    }
    touch(session)
    await db.commit()

    return {
        "session_id": session.id,
        "stage": session.stage,
        "technical_data": tech_data,
    }

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

except Exception as e:
    session.stage = PipelineStage.FAILED
    session.progress = {
        "status": "failed",
        "error": str(e),
        "current_step": "░░",
    }
    touch(session)
    await db.commit()
    raise HTTPException(500, f"░░:░{e}")

```

3:

```

@gpt_router.post("/refine-file")
async def refine_file(
    request: RefineFileRequest,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    """ """
    session = await get_session_or_404(request.session_id, user_id, db)

    session_files = await get_session_files(db, session.id)
    file_obj = session_files.get(request.file_key)
    if file_obj is None:
        raise HTTPException(400, f"░' {request.file_key}'░░░░")

    ai = TaskAIService()
    new_code = await ai.refine_file(
        file_key=request.file_key,

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        current_code=file_obj.content,
        feedback=request.feedback,
        statement=session.statement,
        model=session.model,
    )

```

```

await upsert_ai_file(db, session.id, request.file_key, new_code, uploaded=False
    ↪ )

```

```

touch(session)

```

```

await db.commit()

```

```

return {
    "session_id": session.id,
    "file_key": request.file_key,
    "new_code": new_code,
    "stage": session.stage,
}

```

```

@gpt_router.post("/manual-fix-file")

```

```

async def manual_fix_file(

```

```

    request: ManualFixRequest,

```

```

    user_id: int = Depends(get_current_user),

```

```

    db: AsyncSession = Depends(get_db),

```

```

):

```

```

    """

```

```

    session = await get_session_or_404(request.session_id, user_id, db)

```

```

    await upsert_ai_file(db, session.id, request.file_key, request.new_content,
        ↪ uploaded=False)

```

```

    upload_errors = dict(session.upload_errors or {})

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

upload_errors.pop(request.file_key, None)
session.upload_errors = upload_errors

touch(session)
await db.commit()

return {
    "session_id": session.id,
    "file_key": request.file_key,
    "stage": session.stage,
    "remaining_errors": list(upload_errors.keys()),
}

```

4:

```

@gpt_router.post("/approve-files")
async def approve_files(
    request: ApproveFilesRequest,
    background_tasks: BackgroundTasks,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    """    Polygon    """
    session = await get_session_or_404(request.session_id, user_id, db)

    if session.stage not in (
        PipelineStage.FILES_REVIEW,
        PipelineStage.FAILED,
        PipelineStage.FIXING_ERRORS,
    ):

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        raise HTTPException(
            400, f"UUUUU'{session.stage}'"
        )

    session_files = await get_session_files(db, session.id)
    if not session_files:
        raise HTTPException(400, "UUUU")

    session.stage = PipelineStage.UPLOADING
    session.progress = {
        "status": "uploading",
        "current_step": "UUUPolygon...",
    }
    touch(session)
    await db.commit()

    background_tasks.add_task(run_upload_pipeline, request.session_id)
    return {"status": "upload_started", "session_id": request.session_id}

```

5: /

```

@gpt_router.post("/retry-after-manual-fix")
async def retry_after_manual_fix_endpoint(
    request: ApproveFilesRequest,
    background_tasks: BackgroundTasks,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    """
    ( DONE)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Polygon.

"""

```
session = await get_session_or_404(request.session_id, user_id, db)
```

```
if session.stage not in (
    PipelineStage.FIXING_ERRORS,
    PipelineStage.FAILED,
    PipelineStage.DONE,
):
    raise HTTPException(
        400, f"{}'{session.stage}'"
    )
```

```
session.stage = PipelineStage.UPLOADING
```

```
session.progress = {
    "status": "uploading",
    "current_step": "Polygon...",
}
```

```
session.upload_errors = {}
```

```
touch(session)
```

```
await db.commit()
```

```
background_tasks.add_task(retry_upload_after_manual_fix, request.session_id)
```

```
return {"status": "retry_started", "session_id": request.session_id}
```

#

```
@gpt_router.get("/upload-progress/{session_id}")
```

```
async def get_upload_progress(
```

```
    session_id: str,
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    session = await get_session_or_404(session_id, user_id, db)
    progress = session.progress or {}

    return {
        "status": progress.get("status", "idle"),
        "stage": session.stage,
        "current_step": progress.get("current_step"),
        "error": progress.get("error"),
        "retries": progress.get("retries"),
        "upload_errors": session.upload_errors or {},
        "polygon_problem_id": session.polygon_problem_id,
        "technical_data": await get_all_file_contents(db, session_id),
    }

# ( )

@gpt_router.get("/session/{session_id}")
async def get_session(
    session_id: str,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    session = await get_session_or_404(session_id, user_id, db)

    return {
        "session_id": session.id,
        "stage": session.stage,

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

"statement": session.statement,
"technical_data": await get_all_file_contents(db, session_id),
"history": session.history or [],
"model": session.model,
"system_prompt": session.system_prompt,
"progress": session.progress or {"status": "idle"},
"upload_errors": session.upload_errors or {},
"polygon_problem_id": session.polygon_problem_id,
}

```

#

```
@gpt_router.patch("/session/{session_id}/settings")
```

```
async def update_session_settings(
```

```
    session_id: str,
```

```
    request: UpdateSessionSettingsRequest,
```

```
    user_id: int = Depends(get_current_user),
```

```
    db: AsyncSession = Depends(get_db),
```

```
):
```

```
    session = await get_session_or_404(session_id, user_id, db)
```

```
    if request.model is not None:
```

```
        session.model = request.model
```

```
    if request.system_prompt is not None:
```

```
        session.system_prompt = request.system_prompt
```

```
    touch(session)
```

```
    await db.commit()
```

```
    return {"status": "ok", "model": session.model, "system_prompt": session.
```

```
        ↪ system_prompt}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.45. api\user\gpt\base_gpt.py

Листинг 45: api\user\gpt\base_gpt.py

```
from fastapi import APIRouter

gpt_router = APIRouter()
```

1.46. api\user\gpt\services__init__.py

Листинг 46: api\user\gpt\services__init__.py

1.47. api\user\gpt\services\ai_file_helpers.py

Листинг 47: api\user\gpt\services\ai_file_helpers.py

```
from sqlalchemy import select, update
from sqlalchemy.ext.asyncio import AsyncSession

from models.ai.ai_generated_file import AIGeneratedFile

FILE_NAME_MAP: dict[str, str] = {
    "validator": "validator.cpp",
    "generator": "generator.cpp",
    "checker": "checker.cpp",
    "solution_cpp": "solution.cpp",
    "solution_py": "solution.py",
    "wa_sol": "wa.cpp",
    "tl_sol": "tl.cpp",
    "re_sol": "re.cpp",
    "ml_sol": "ml.cpp",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    "script": "script.txt",
}

async def upsert_ai_file(
    db: AsyncSession,
    session_id: str,
    file_type: str,
    content: str,
    *,
    uploaded: bool = False,
) -> None:

    filename = FILE_NAME_MAP.get(file_type, file_type)
    result = await db.execute(
        select(AIGeneratedFile).where(
            AIGeneratedFile.session_id == session_id,
            AIGeneratedFile.file_type == file_type,
        )
    )

    existing = result.scalar_one_or_none()
    if existing:
        existing.content = content
        existing.filename = filename
        existing.uploaded = uploaded
    else:
        db.add(AIGeneratedFile(
            session_id=session_id,
            filename=filename,
            content=content,
            file_type=file_type,
            uploaded=uploaded,
        ))

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
async def upsert_all_ai_files(
    db: AsyncSession,
    session_id: str,
    tech_data: dict,
    *,
    uploaded: bool = False,
) -> None:
    for file_type, content in tech_data.items():
        if content:
            await upsert_ai_file(db, session_id, file_type, content, uploaded=
                ↪ uploaded)

async def mark_uploaded(
    db: AsyncSession,
    session_id: str,
    file_type: str,
) -> None:
    await db.execute(
        update(AIGeneratedFile)
        .where(
            AIGeneratedFile.session_id == session_id,
            AIGeneratedFile.file_type == file_type,
        )
        .values(uploaded=True)
    )

async def get_session_files(
    db: AsyncSession,
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    session_id: str,
) -> dict[str, AIGeneratedFile]:
    """Returns {file_type: AIGeneratedFile} for all files in the session."""
    result = await db.execute(
        select(AIGeneratedFile).where(AIGeneratedFile.session_id == session_id)
    )
    return {f.file_type: f for f in result.scalars().all()}

async def get_all_file_contents(
    db: AsyncSession,
    session_id: str,
) -> dict[str, str]:
    """Returns {file_type: content} for all files regardless of upload status."""
    files = await get_session_files(db, session_id)
    return {k: v.content for k, v in files.items()}

async def get_uploaded_file_contents(
    db: AsyncSession,
    session_id: str,
) -> dict[str, str]:
    """Returns {file_type: content} only for files confirmed uploaded to Polygon.
    ↪ """
    result = await db.execute(
        select(AIGeneratedFile).where(
            AIGeneratedFile.session_id == session_id,
            AIGeneratedFile.uploaded.is_(True),
        )
    )
    return {f.file_type: f.content for f in result.scalars().all()}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.48. api\{}user\{}gpt\{}services\{}ai_service.py

Листинг 48: api\{}user\{}gpt\{}services\{}ai_service.py

```
import json
import logging
import re
from typing import Dict, List

import httpx
from fastapi import HTTPException

from settings import settings

logger = logging.getLogger(__name__)

MAX_IDEA_CHARS = 12_000

class TaskAIService:
    def __init__(self):
        self.api_key = settings.OPENAI_API_KEY
        base_url = settings.OPENAI_HOST.rstrip("/")
        self.url = (
            f"{base_url}/chat/completions"
            if not base_url.endswith("/chat/completions")
            else base_url
        )

        self.headers = {
            "Authorization": f"Bearer_{self.api_key}",
            "Content-Type": "application/json",
            "HTTP-Referer": "https://judge-system.com",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        "X-Title": "Judge_System_AI",
    }

    async def _base_ask(
        self, model: str, messages: List[Dict], json_mode: bool = True
    ) -> Dict:
        payload = {
            "model": model,
            "messages": messages,
        }
        if json_mode:
            payload["response_format"] = {"type": "json_object"}

        async with httpx.AsyncClient(timeout=120.0) as client:
            try:
                response = await client.post(
                    self.url, headers=self.headers, json=payload
                )
                if response.status_code != 200:
                    logger.error(
                        f"AI_API_Error: {response.status_code} - {response.text}"
                    )
                    raise HTTPException(
                        status_code=500, detail=f"AI_Service_error: {response.text}"
                    )

                result = response.json()
                content = result["choices"][0]["message"]["content"]

                if json_mode:
                    content = content.strip()
                    content = re.sub(
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

        r"~'('(?:(?:json)?\n?|' '$", "", content, flags=re.MULTILINE
    ).strip()
    try:
        return json.loads(content)
    except json.JSONDecodeError:
        match = re.search(r"\{.*\}", content, re.DOTALL)
        if match:
            return json.loads(match.group())
        raise
    else:
        return {"text": content}
except Exception as e:
    logger.error(f"AI_Service_Exception: {str(e)}")
    raise

async def generate_statement(
    self, user_idea: str, model: str, user_prompt: str, history: List[Dict]
) -> Dict:
    if len(user_idea) > MAX_IDEA_CHARS:
        raise HTTPException(
            status_code=400,
            detail=(
                f"_{len(user_idea)}_"
                f"_{MAX_IDEA_CHARS}_"
            ),
        )

    system_prompt = (
        "_____"
        ":_____,_____"
        "_____,_____"
        "_LaTeX____JSON:_"
    )

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        "{name, _legend, _input, _output, _notes, _tutorial}."
    )

    final_prompt = user_prompt or system_prompt

    messages = [{"role": "system", "content": final_prompt}]
    messages.extend(history)
    messages.append({"role": "user", "content": user_idea})

    return await self._base_ask(model, messages)

async def generate_technical_stuff(
    self, approved_statement: Dict, model: str
) -> Dict:
    system_prompt = (
        "testlib.h"
        "
        https://codeforces.com/blog/entry/18289(testlib),
        https://codeforces.com/blog/entry/18291(),
        https://codeforces.com/blog/entry/18426(),
        https://codeforces.com/blog/entry/18431(),
        Polygon.
        _:_validator.cpp,_generator.cpp,_checker.cpp,
        solution.cpp(Main),_:
        WA(WrongAnswer),_TL(TimeLimitExceeded),_RE(RuntimeError),_ML(
            MemoryLimit).
        _JSON:_{validator,_generator,_checker,_solution_py,_solution_cpp,
        "wa_sol,_tl_sol,_re_sol,_ml_sol,_script}."
        scriptPolygon:
        "Freemarker_TemplateEngine_is_an_engine,_created_to_process_text_files.
        _> _"
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

"It process only parts of text files, that are surrounded with special
    ↪ Freemarker directives."

"Everything else is copied directly to the output."

"So if the file does not contain Freemarker at all,"

"that the result of the processing such file is an exact copy of this
    ↪ file."

"Input:"

"gen_100_100_1_>_1"
"gen_100_100_2_>_2"
"gen_100_100_3_>_3"
"gen_100_100_4_>_4"

"Output:"

"gen_100_100_1_>_1"
"gen_100_100_2_>_2"
"gen_100_100_3_>_3"
"gen_100_100_4_>_4"

"Variables are created automatically during their first assignment. To
    ↪ assign a value to the variable #assign directive is used."

"<#assign i=_10/>"

"To evaluate some expression with variables you should enclose it in ${}
    ↪ symbols."

"<#assign i=_10/>"

"gen_${i}_${i*2+_1}_${i*_10}_>_"$

"Input:"

"<#assign i=_2/>"

"<#assign j=_3/>"

"gen_${i}_${j}_${i+_j}_${i*_j+_i+_j+_2}_>_"$

"<#assign i2=_i*i/>"

"<#assign j2=_j*j/>"

"gen_${i2}_${j2}_>_"$

"Output:"

"gen_2_3_5_13_>_"$

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
"gen_4_9_>_"$
"To iterate through the range of integers, #list directive is used."
"<#list 10..20 as iter></#list>"
"For example:"
"<#list 10..20 as iter>"
"gen_100_200_${iter*iter+_10}_>_"$
"</#list>"
"Of course, loops can be nested. And all variables, that were declared
    ↪ previously, can be used inside."
"Input:"
"<#assign maxN=_1/>"
"<#list 1..5 as power>"
"<#assign maxN=_maxN*_10>"
"<#list 1..5 as testNumber>"
"generatorFile_${maxN}_abacaba_${testNumber}_>_"$
"</#list>"
"</#list>"
"Output:"
"generatorFile_10_abacaba_1_>_"$
"generatorFile_10_abacaba_2_>_"$
"generatorFile_10_abacaba_3_>_"$
"generatorFile_10_abacaba_4_>_"$
"generatorFile_10_abacaba_5_>_"$
"generatorFile_100_abacaba_1_>_"$
"generatorFile_100_abacaba_2_>_"$
"generatorFile_100_abacaba_3_>_"$
"generatorFile_100_abacaba_4_>_"$
"generatorFile_100_abacaba_5_>_"$
"generatorFile_1000_abacaba_1_>_"$
"generatorFile_1000_abacaba_2_>_"$
"generatorFile_1000_abacaba_3_>_"$
"generatorFile_1000_abacaba_4_>_"$
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
"generatorFile_1000_abacaba_5_>"
"generatorFile_10000_abacaba_1_>"
"generatorFile_10000_abacaba_2_>"
"generatorFile_10000_abacaba_3_>"
"generatorFile_10000_abacaba_4_>"
"generatorFile_10000_abacaba_5_>"
"generatorFile_100000_abacaba_1_>"
"generatorFile_100000_abacaba_2_>"
"generatorFile_100000_abacaba_3_>"
"generatorFile_100000_abacaba_4_>"
"generatorFile_100000_abacaba_5_>"
"For more detailed tutorial please see Freemarker Documentation"

"generator.cpp, generator."
", "
" , "
" , "
", , "
" , "
", , , "
" ( ) ."
" , "
", n, "
" ."
" , "
" , "
" ."

)

user_prompt = (
    f"Problem Statement: \n{json.dumps(approved_statement, ensure_ascii=False
    ↵ )}"
)

messages = [
    {"role": "system", "content": system_prompt},
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        {"role": "user", "content": user_prompt},
    ]
    return await self._base_ask(model, messages)

async def fix_code(
    self, error: str, component: str, code: str, statement: Dict, model: str
) -> str:
    system_prompt = f"Fix the {component} code for Polygon. Error: {error}.
        ↳ Return only the corrected code, no explanations."
    user_prompt = f"Statement:\n{json.dumps(statement, ensure_ascii=False)}\n\
        ↳ nBroken code:\n{code}"
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ]
    result = await self._base_ask(model, messages, json_mode=False)
    return result["text"].strip()

async def refine_file(
    self,
    file_key: str,
    current_code: str,
    feedback: str,
    statement: Dict,
    model: str,
) -> str:
    system_prompt = (
        f"Polygon."
        f"'{file_key}'."
        f"markdown."
    )
    user_prompt = (

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
f"_: \n{json.dumps(statement, ensure_ascii=False)}\n\n"
f"_{file_key}): \n{current_code}\n\n"
f"_:_{feedback}"
)
messages = [
    {"role": "system", "content": system_prompt},
    {"role": "user", "content": user_prompt},
]
result = await self._base_ask(model, messages, json_mode=False)
return result["text"].strip()
```

1.49. api\{}user\{}gpt\{}services\{}session_store.py

Листинг 49: api\{}user\{}gpt\{}services\{}session_store.py

```
import uuid
from typing import Any, Dict

class SessionStore:
    _sessions: Dict[str, Dict[str, Any]] = {}

    @classmethod
    def create(cls, data: Dict[str, Any]) -> str:
        session_id = str(uuid.uuid4())
        cls._sessions[session_id] = data
        return session_id

    @classmethod
    def get(cls, session_id: str) -> Dict[str, Any]:
        return cls._sessions.get(session_id)

    @classmethod
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
def update(cls, session_id: str, data: Dict[str, Any]):  
    if session_id in cls._sessions:  
        cls._sessions[session_id].update(data)  
  
@classmethod  
def delete(cls, session_id: str):  
    cls._sessions.pop(session_id, None)
```

1.50. api\user\gpt\services\upload_orchestrator.py

Листинг 50: api\user\gpt\services\upload_orchestrator.py

```
import asyncio  
import logging  
import re  
import time  
import traceback  
from typing import Dict  
  
from sqlalchemy import update  
from sqlalchemy.ext.asyncio import AsyncSession  
  
from api.user.gpt.services.ai_file_helpers import (  
    get_session_files, mark_uploaded, upsert_ai_file)  
from api.user.gpt.services.ai_service import TaskAIService  
from api.user.polygon.commit.commit_problem import commit  
from api.user.polygon.commit.get_packages import get_packages  
from api.user.polygon.create_problem import create_problem  
from api.user.polygon.files.gen.set_checker import set_checker  
from api.user.polygon.files.gen.set_generator import set_generator  
from api.user.polygon.files.gen.set_script import set_script  
from api.user.polygon.files.gen.set_solution import set_solution  
from api.user.polygon.files.gen.set_validator import set_validator
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
from api.user.polygon.files.save_statement import save_statement
from app.database import Session
from models.ai.ai_session import AISession, PipelineStage

logger = logging.getLogger(__name__)

MAX_RETRIES = 3
PACKAGE_POLL_INTERVAL = 10
PACKAGE_POLL_TIMEOUT = 600

STEP_MAP = [
    ("validator", set_validator, "validator", "validator.cpp", []),
    ("generator", set_generator, "generator", "generator.cpp", []),
    ("checker", set_checker, "checker", "checker.cpp", []),
    ("sol_main", set_solution, "solution_cpp", "solution.cpp", ["MA"]),
    ("sol_wa", set_solution, "wa_sol", "wa.cpp", ["WA"]),
    ("sol_tl", set_solution, "tl_sol", "tl.cpp", ["TL"]),
    ("sol_re", set_solution, "re_sol", "re.cpp", ["RE"]),
    ("sol_ml", set_solution, "ml_sol", "ml.cpp", ["ML"]),
]

async def _update_session(session_id: str, data: Dict, db: AsyncSession):
    await db.execute(update(AISession).where(AISession.id == session_id).values(**
        ↪ data))
    await db.commit()

def _make_polygon_name(model: str, statement: Dict, session_id: str) -> str:
    model_short = re.sub(r"[^a-z0-9]", "", model.split("/")[-1].lower())[:8]
    suffix = session_id[:4]
    timestamp = str(int(time.time()))[-5:]
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
return f"{model_short}-task-{suffix}-{timestamp}"

async def _upload_steps(
    session_id: str,
    problem_id: int,
    user_id: int,
    statement: Dict,
    model: str,
    progress: Dict,
    db: AsyncSession,
) -> dict:
    """
        Polygon.
        ,   AIGeneratedFile.uploaded == True.
        upload_errors ,   .
    """

    ai = TaskAIService()
    session_files = await get_session_files(db, session_id)
    upload_errors = {}

    for step_name, upload_func, data_key, file_name, extra_args in STEP_MAP:
        file_obj = session_files.get(data_key)
        if not file_obj:
            continue

        # ,
        if file_obj.uploaded:
            logger.info(f"[{session_id}] Skipping already-uploaded {file_name}")
            continue

        code = file_obj.content
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

progress["current_step"] = f"_{file_name}..."
await _update_session(session_id, {"progress": progress}, db)

success = False
last_error = None

for attempt in range(1, MAX_RETRIES + 1):
    try:
        args = [problem_id, file_name, code, *extra_args, user_id, db]
        await upload_func(*args)
        success = True
        break
    except Exception as e:
        last_error = str(e)
        logger.warning(f"[{session_id}]_{step_name}_{attempt}_{attempt}_failed
            ↳ :_{e}")
        progress["retries"] = attempt
        progress["current_step"] = (
            f"_{file_name},_"
            f"({attempt}/{MAX_RETRIES})..."
        )
        await _update_session(session_id, {"progress": progress}, db)

    if attempt < MAX_RETRIES:
        try:
            code = await ai.fix_code(last_error, step_name, code,
                ↳ statement, model)
            await upsert_ai_file(db, session_id, data_key, code, uploaded
                ↳ =False)
            await db.commit()
        except Exception as fix_err:

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        logger.warning(f"[{session_id}] AI fix failed for {step_name}
        ↪ ): {fix_err}")

if success:
    logger.info(f"[{session_id}] Uploaded {file_name}")
    await mark_uploaded(db, session_id, data_key)
    await db.commit()
else:
    logger.error(
        f"[{session_id}] Failed to upload {file_name} after {MAX_RETRIES}
        ↪ attempts: {last_error}"
    )
    upload_errors[data_key] = {
        "file_name": file_name,
        "error": last_error,
        "needs_manual_fix": True,
    }

#
script_obj = session_files.get("script")
if script_obj and not script_obj.uploaded:
    progress["current_step"] = "...
    await _update_session(session_id, {"progress": progress}, db)
    await set_script(problem_id, "tests", script_obj.content, user_id, db)
    await mark_uploaded(db, session_id, "script")
    await db.commit()

return upload_errors

async def run_upload_pipeline(session_id: str):
    """
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

UPLOADING: Polygon ( ), ,
, .
"""

async with Session() as db:
    res = await db.get(AISession, session_id)
    if not res:
        return

    user_id = res.user_id
    statement = res.statement
    model = res.model

    progress = {
        "status": "uploading",
        "current_step": "Polygon...",
        "retries": 0,
        "error": None,
    }
    await _update_session(
        session_id, {"progress": progress, "stage": PipelineStage.UPLOADING}, db
    )

    try:
        # 1.
        if res.polygon_problem_id:
            problem_id = res.polygon_problem_id
            logger.info(f"[{session_id}] Reusing existing Polygon problem {
                ↪ problem_id}")
        else:
            polygon_name = _make_polygon_name(model, statement, session_id)
            logger.info(f"[{session_id}] Creating Polygon problem '{polygon_name
                ↪ }'")

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
problem_id = await create_problem(name=polygon_name, user_id=user_id
    ↪ , db=db)

logger.info(f"[{session_id}]_Problem_created:_polygon_id={problem_id
    ↪ }")

await _update_session(session_id, {"polygon_problem_id": problem_id
    ↪ }, db)

# 2.

progress["current_step"] = "_..."

await _update_session(session_id, {"progress": progress}, db)

await save_statement(
    problem_id=problem_id,
    lang="russian",
    name=statement["name"],
    legend=statement["legend"],
    input_legend=statement["input"],
    output_legend=statement["output"],
    notes=statement.get("notes"),
    tutorial=statement.get("tutorial"),
    scoring="",
    interaction="",
    user_id=user_id,
    db=db,
)

# 3.

upload_errors = await _upload_steps(
    session_id, problem_id, user_id, statement, model, progress, db
)

if upload_errors:
    progress["status"] = "waiting_manual_fix"
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
progress["current_step"] = "░░░░░.░░░░."
await _update_session(
    session_id,
    {
        "progress": progress,
        "upload_errors": upload_errors,
        "stage": PipelineStage.FIXING_ERRORS,
    },
    db,
)
return

# 4.
progress["current_step"] = "░...░"
await _update_session(session_id, {"progress": progress}, db)
await commit(problem_id, user_id, db)

# 5.
await _build_and_poll_package(session_id, problem_id, user_id, progress,
    ↪ db)

except Exception as e:
    logger.exception(f"[{session_id}] Upload pipeline failed: {e}")
    progress["status"] = "failed"
    progress["error"] = str(e)
    progress["traceback"] = traceback.format_exc()
    await _update_session(
        session_id, {"progress": progress, "stage": PipelineStage.FAILED},
        ↪ db
    )
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
async def retry_upload_after_manual_fix(session_id: str):
    """
        (    DONE).
        uploaded=False
        .    Polygon    .
    """
    async with Session() as db:
        res = await db.get(AISession, session_id)
        if not res:
            return

        statement = res.statement
        user_id = res.user_id
        model = res.model
        problem_id = res.polygon_problem_id

        progress = {
            "status": "uploading",
            "current_step": "□□□...",
            "retries": 0,
            "error": None,
        }
        await _update_session(
            session_id, {"progress": progress, "stage": PipelineStage.UPLOADING}, db
        )

    try:
        upload_errors = await _upload_steps(
            session_id, problem_id, user_id, statement, model, progress, db
        )

        if upload_errors:
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
progress["status"] = "waiting_manual_fix"
progress["current_step"] = "□□□□"
await _update_session(
    session_id,
    {
        "progress": progress,
        "upload_errors": upload_errors,
        "stage": PipelineStage.FIXING_ERRORS,
    },
    db,
)
return

await _update_session(session_id, {"upload_errors": {}}, db)

progress["current_step"] = "□..."
await _update_session(session_id, {"progress": progress}, db)
await commit(problem_id, user_id, db)

await _build_and_poll_package(session_id, problem_id, user_id, progress,
    ↪ db)

except Exception as e:
    logger.exception(f"[{session_id}] □Retry □pipeline □failed: □{e}")
    progress["status"] = "failed"
    progress["error"] = str(e)
    await _update_session(
        session_id, {"progress": progress, "stage": PipelineStage.FAILED},
        ↪ db
    )
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
async def _build_and_poll_package(
    session_id: str,
    problem_id: int,
    user_id: int,
    progress: Dict,
    db: AsyncSession,
):
    from api.user.polygon.commit.build_package import \
        build_package as polygon_build_package

    progress["current_step"] = "□□..."
    await _update_session(
        session_id, {"progress": progress, "stage": PipelineStage.BUILDING_PACKAGE
        ↪ }, db
    )

    await polygon_build_package(problem_id=problem_id, user_id=user_id, db=db)

    elapsed = 0
    while elapsed < PACKAGE_POLL_TIMEOUT:
        await asyncio.sleep(PACKAGE_POLL_INTERVAL)
        elapsed += PACKAGE_POLL_INTERVAL

        packages = await get_packages(problem_id, user_id, db)
        if not packages:
            continue

        latest = packages[-1]
        state = latest.get("state", "PENDING")
        package_id = latest.get("id")

        progress["current_step"] = f"□:□{state}..."
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

await _update_session(session_id, {"progress": progress}, db)

if state == "READY":
    logger.info(
        f"[{session_id}]_Package_built:_polygon_id={problem_id}_package_id={
            ↪ package_id}"
    )
    progress["status"] = "done"
    progress["current_step"] = (
        f"!_Polygon_ID:_{problem_id},_Package:_{package_id}"
    )
    await _update_session(
        session_id,
        {
            "progress": progress,
            "stage": PipelineStage.DONE,
            "package_id": package_id,
        },
        db,
    )
    return

elif state == "FAILED":
    error_comment = latest.get("comment", "___")
    logger.error(f"[{session_id}]_Package_build_failed:_{error_comment}")
    progress["status"] = "waiting_manual_fix"
    progress["current_step"] = "___"
    progress["error"] = error_comment
    await _update_session(
        session_id,
        {
            "progress": progress,

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        "stage": PipelineStage.FIXING_ERRORS,
        "upload_errors": {
            "package": {
                "error": error_comment,
                "needs_manual_fix": True,
            }
        },
    },
    db,
)

return

logger.error(f"[{session_id}]_Package_build_timed_out_after_{
    ↪ PACKAGE_POLL_TIMEOUT}s")
progress["status"] = "failed"
progress["current_step"] = ""
await _update_session(
    session_id, {"progress": progress, "stage": PipelineStage.FAILED}, db
)
```

1.51. api\user\plagiarism__init__.py

Листинг 51: api\user\plagiarism__init__.py

```
from api.user.plagiarism.base_plagiarism import router as plagiarism_router

__all__ = ["plagiarism_router"]
```

1.52. api\user\plagiarism\base_plagiarism.py

Листинг 52: api\user\plagiarism\base_plagiarism.py

```
import asyncio
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

import logging
from base64 import b64decode
from math import ceil

import plagiarism_cpp
from fastapi import APIRouter, Depends, HTTPException, Query, status
from pydantic import BaseModel
from sqlalchemy import func, select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import joinedload

from api.crypt import get_current_user
from app.database import Session, get_db
from models import Contest, Submission
from models.plagiarism.pair_of_banned_submissions import \
    PairOfBannedSubmissions
from models.plagiarism.plagiarism_report import PlagiarismReport

router = APIRouter()
logger = logging.getLogger(__name__)

class PlagiarismCheckBody(BaseModel):
    threshold: float
    onlyOk: bool = False

@router.get("/contests/{contest_id}/reports")
async def get_contest_reports(
    contest_id: int,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
):
    contest = await db.execute(
        select(Contest).filter_by(id=contest_id, user_id=user_id)
    )
    contest = contest.scalars().first()

    if not contest:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="Contest not found"
        )

    reports = await db.execute(
        select(PlagiarismReport)
        .filter_by(contest_id=contest_id)
        .order_by(PlagiarismReport.id.desc())
    )
    reports = reports.scalars().all()

    return [
        {
            "id": report.id,
            "contest_id": report.contest_id,
            "status": report.status,
            "threshold": report.threshold,
            "only_ok": report.only_ok,
            "created_at": report.created_at,
            "pairs_count": len(report.pairs),
        }
        for report in reports
    ]
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

@router.get("/reports/{report_id}")
async def get_report(
    report_id: int,
    page: int = Query(1, ge=1),
    per_page: int = Query(20, ge=10, le=100),
    task_name: str = Query(None),
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    report = await db.execute(select(PlagiarismReport).filter_by(id=report_id))
    report = report.scalars().first()

    if not report:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="Report_not_found"
        )

    contest = await db.execute(
        select(Contest).filter_by(id=report.contest_id, user_id=user_id)
    )
    contest = contest.scalars().first()

    if not contest:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="Contest_not_found"
        )

    if report.status != "completed":
        return {
            "id": report.id,
            "contest_id": report.contest_id,
            "status": report.status,

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    "pairs": [],
    "pagination": {
        "page": page,
        "per_page": per_page,
        "total": 0,
        "total_pages": 1,
    },
}

tasks = (
    select(Submission.task_name)
    .distinct()
    .join(
        PairOfBannedSubmissions,
        PairOfBannedSubmissions.first_submission_id == Submission.id,
    )
    .filter(PairOfBannedSubmissions.report_id == report_id)
)

tasks = await db.execute(tasks)
available_tasks = [t for t in tasks.scalars().all() if t]

query = (
    select(PairOfBannedSubmissions)
    .options(
        joinedload(PairOfBannedSubmissions.first_submission),
        joinedload(PairOfBannedSubmissions.second_submission),
    )
    .filter_by(report_id=report_id)
)

if task_name:
    query = query.join(

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

        Submission, PairOfBannedSubmissions.first_submission_id == Submission.id
    ).filter(Submission.task_name == task_name)

total_query = select(func.count()).select_from(query.subquery())
total = await db.execute(total_query)
total = total.scalar() or 0

pairs_result = await db.execute(
    query.order_by(PairOfBannedSubmissions.percentage.desc())
        .offset((page - 1) * per_page)
        .limit(per_page)
)
pairs = pairs_result.scalars().all()

return {
    "id": report.id,
    "status": report.status,
    "tasks": available_tasks,
    "pairs": [
        {
            "id": pair.id,
            "user1": pair.first_submission.participant_login,
            "user2": pair.second_submission.participant_login,
            "task_name": pair.first_submission.task_name,
            "percent": round(pair.percentage, 2),
        }
        for pair in pairs
    ],
    "pagination": {
        "page": page,
        "total": total,
        "total_pages": ceil(total / per_page) if total > 0 else 1,
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    },  
}
```

```
@router.get("/pairs/{pair_id}")  
async def get_pair(  
    pair_id: int,  
    user_id: int = Depends(get_current_user),  
    db: AsyncSession = Depends(get_db),  
):  
    pair = await db.execute(  
        select(PairOfBannedSubmissions)  
        .options(  
            joinedload(PairOfBannedSubmissions.first_submission),  
            joinedload(PairOfBannedSubmissions.second_submission),  
        )  
        .filter_by(id=pair_id)  
    )  
    pair = pair.scalars().first()  
  
    if not pair:  
        raise HTTPException(  
            status_code=status.HTTP_404_NOT_FOUND, detail="Pair not found"  
        )  
  
    contest = await db.execute(  
        select(Contest).filter_by(id=pair.contest_id, user_id=user_id)  
    )  
    contest = contest.scalars().first()  
  
    if not contest:  
        raise HTTPException(  

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        status_code=status.HTTP_404_NOT_FOUND, detail="Contest_not_found"
    )

return {
    "id": pair.id,
    "percent": round(pair.percentage, 2),
    "user1": pair.first_submission.participant_login,
    "user2": pair.second_submission.participant_login,
    "sub1_id": str(pair.first_submission_id),
    "sub2_id": str(pair.second_submission_id),
    "code1": pair.first_submission.source or "",
    "code2": pair.second_submission.source or "",
}

@router.post("/contests/{contest_id}/check")
async def run_plagiarism_check(
    contest_id: int,
    body: PlagiarismCheckBody,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    contest = await db.execute(
        select(Contest).filter_by(id=contest_id, user_id=user_id)
    )
    contest = contest.scalars().first()

    if not contest:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="Contest_not_found"
        )
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

report = PlagiarismReport(
    contest_id=contest_id,
    status="processing",
    threshold=body.threshold,
    only_ok=body.onlyOk,
)
db.add(report)
await db.commit()
await db.refresh(report)

logger.info(
    f"Plagiarism_check_started: report_id={report.id} contest_id={contest_id}"
    f"threshold={body.threshold} only_ok={body.onlyOk} user_id={user_id}"
)

asyncio.create_task(
    process_plagiarism_report(
        report_id=report.id,
        contest_id=contest_id,
        threshold=body.threshold,
        only_ok=body.onlyOk,
    )
)

return {
    "reportId": report.id,
    "status": "processing",
}

```

```

async def process_plagiarism_report(
    report_id: int,

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

contest_id: int,
threshold: float,
only_ok: bool,
):
    async with Session() as db:
        try:
            query = select(Submission).filter_by(contest_id=contest_id)

            if only_ok:
                query = query.filter_by(verdict="OK")

            submissions_result = await db.execute(query)
            submissions = submissions_result.scalars().all()

            py_submissions = []

            for submission in submissions:
                if not submission.source:
                    continue

                py_sub = plagiarism_cpp.Submission()
                py_sub.id = str(submission.id)
                py_sub.language = (
                    plagiarism_cpp.ProgrammingLanguage.Cpp
                ) # TODO: support more languages
                decoded_code = b64decode(submission.source).decode("utf-8")
                py_sub.rawCode = decoded_code
                py_sub.participant = submission.participant_login or ""
                py_sub.problem = submission.task_name or ""

                py_submissions.append(py_sub)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

logger.info(
    f"[report={report_id}]_Comparing_{len(py_submissions)}_submissions_"
    f"(contest={contest_id},_threshold={threshold})"
)

pairs = plagiarism_cpp.compute_similarity_pairs(py_submissions,
    ↪ threshold)

logger.info(f"[report={report_id}]_Found_{len(pairs)}_suspicious_pairs")

for pair in pairs:
    db.add(
        PairOfBannedSubmissions(
            contest_id=contest_id,
            report_id=report_id,
            first_submission_id=str(pair.first_submission_id),
            second_submission_id=str(pair.second_submission_id),
            percentage=pair.plagiarism_percent,
        )
    )

report = await db.get(PlagiarismReport, report_id)
if report:
    report.status = "completed"

await db.commit()
logger.info(f"[report={report_id}]_Plagiarism_check_completed")

except Exception:
    logger.exception(f"[report={report_id}]_Plagiarism_check_failed")
    report = await db.get(PlagiarismReport, report_id)
    if report:

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        report.status = "failed"

        await db.commit()

    raise

```

1.53. api\{}user\{}polygon\{}__init__.py

Листинг 53: api\{}user\{}polygon\{}__init__.py

```

from api.user.polygon.auth.login import link_polygon
from api.user.polygon.auth.logout import unlink_polygon
from api.user.polygon.base_polygon import polygon_router
from api.user.polygon.commit.build_package import build_package
from api.user.polygon.commit.commit_problem import commit
from api.user.polygon.create_problem import create_problem
from api.user.polygon.create_signature import create_signature
from api.user.polygon.files.gen.add_source_file import add_source
from api.user.polygon.files.gen.set_checker import set_checker
from api.user.polygon.files.gen.set_generator import set_generator
from api.user.polygon.files.gen.set_interactor import set_interactor
from api.user.polygon.files.gen.set_script import set_script
from api.user.polygon.files.gen.set_solution import set_solution
from api.user.polygon.files.gen.set_validator import set_validator
from api.user.polygon.files.save_statement import save_statement
from api.user.polygon.files.save_test import save_test
from api.user.polygon.get_response import get_response
from api.user.polygon.problem_info import problem_info
from api.user.polygon.problem_script import problem_script
from api.user.polygon.settings.enable_groups import enable_groups
from api.user.polygon.settings.enable_points import enable_points
from api.user.polygon.settings.set_tags import set_tags
from api.user.polygon.settings.set_test_group import set_test_group
from api.user.polygon.settings.update_info import update_info

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
__all__ = [
    "link_polygon",
    "unlink_polygon",
    "polygon_router",
    "build_package",
    "commit",
    "create_problem",
    "enable_groups",
    "enable_points",
    "problem_info",
    "problem_script",
    "save_statement",
    "save_test",
    "set_checker",
    "set_generator",
    "set_interactor",
    "set_script",
    "set_solution",
    "set_validator",
    "set_tags",
    "set_test_group",
    "update_info",
    "add_source",
    "create_signature",
    "get_response",
]
```

1.54. api\{}user\{}polygon\{}base_polygon.py

Листинг 54: api\{}user\{}polygon\{}base_polygon.py

```
from fastapi import APIRouter
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
polygon_router = APIRouter()
```

1.55. api\{}\user\{}\polygon\{}\create_problem.py

Листинг 55: api\{}\user\{}\polygon\{}\create_problem.py

```
from time import time

from aiohttp import ClientSession
from fastapi import HTTPException, status
from pydantic import BaseModel
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response, PolygonAPIError
from models import User
from settings import settings


class CreateProblemResponse(BaseModel):
    problemId: int


async def create_problem(name: str, user_id: int, db: AsyncSession):
    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.polygon_api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Polygon API is not configured",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
)

method_name = "problem.create"

current_time_unix = int(time())

params = {
    "apiKey": user.polygon_api_key,
    "time": current_time_unix,
    "name": name,
}

signature = create_signature(method_name, params, user.polygon_api_secret)

params["apiSig"] = signature

url = URL(settings.POLYGON_HOST) / method_name

async with ClientSession() as session:
    try:
        response = await get_response(session, url, params)

        if response["id"] == "":
            raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail=
                ↳ "Failed to create problem")

        return response["id"]
    except PolygonAPIError as e:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail=str(
            ↳ e))
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.56. api\{}user\{}polygon\{}create_signature.py

Листинг 56: api\{}user\{}polygon\{}create_signature.py

```

from hashlib import sha512
from random import randrange

def create_signature(method_name: str, params: dict[str, str], secret: str) -> str
    ↪ :
    sorted_params = sorted(params.items())
    random_string = f"{randrange(1000000):06d}"

    signature_string = f'{{random_string}}/{{method_name}}?{{"&".join([f"{{k}}={{v}}" for {{k,
    ↪ {{v}} in {{sorted_params}}])}}#{{secret}}'

    hash_signature = sha512(signature_string.encode()).hexdigest()

    return f"{{random_string}}{{hash_signature}}"

```

1.57. api\{}user\{}polygon\{}get_response.py

Листинг 57: api\{}user\{}polygon\{}get_response.py

```

import json
import logging

from aiohttp import ClientSession

logger = logging.getLogger(__name__)

class PolygonAPIError(Exception):
    def __init__(

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

self,
message: str,
*,
http_status: int | None = None,
method: str | None = None,
raw_response: str | None = None,
):
    super().__init__(message)
    self.message = message
    self.http_status = http_status
    self.raw_response = raw_response

def __str__(self):
    return self.message

```

```

async def get_response(client: ClientSession, url, params):
    async with client.post(url, data=params) as response:
        response_text = await response.text()

    try:
        result = json.loads(response_text)
    except json.JSONDecodeError:
        if response.status != 200:
            raise PolygonAPIError(
                f"HTTP_{response.status}: {response_text[:300]}",
                http_status=response.status,
                raw_response=response_text,
            )
        else:
            return {"message": response_text}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
if result["status"] == "OK":
    try:
        return result["result"]
    except KeyError:
        return {"message": response_text}

comment = result.get("comment", "Unknown_Polygon_error")
raise PolygonAPIError(
    comment,
    http_status=response.status,
    raw_response=response_text,
)
```

1.58. api\{}user\{}polygon\{}problem_info.py

Листинг 58: api\{}user\{}polygon\{}problem_info.py

```
from time import time

from aiohttp import ClientSession
from fastapi import HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from models import User
from settings import settings

async def problem_info(problem_id: int, user_id: int, db: AsyncSession):
    user = await db.execute(select(User).filter_by(id=user_id))
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
user = user.scalars().first()

if not user.polygon_api_key:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Polygon API is not configured",
    )

method_name = "problem.info"

current_time_unix = int(time())

params = {
    "apiKey": user.polygon_api_key,
    "time": current_time_unix,
    "problemId": problem_id,
}

signature = create_signature(method_name, params, user.polygon_api_secret)
params["apiSig"] = signature

url = URL(settings.POLYGON_HOST) / method_name

async with ClientSession() as session:
    return await get_response(session, url, params)
```

1.59. api\user\polygon\problem_script.py

Листинг 59: api\user\polygon\problem_script.py

```
from time import time
```

```
from aiohttp import ClientSession
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

from fastapi import HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from models import User
from settings import settings

async def problem_script(problem_id: int, testset: str, user_id: int, db:
    ↪ AsyncSession):
    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.polygon_api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Polygon API is not configured",
        )

    method_name = "problem.script"

    current_time_unix = int(time())

    params = {
        "apiKey": user.polygon_api_key,
        "time": current_time_unix,
        "problemId": problem_id,
        "testset": testset,
    }

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

signature = create_signature(method_name, params, user.polygon_api_secret)
params["apiSig"] = signature

url = URL(settings.POLYGON_HOST) / method_name

async with ClientSession() as session:
    return await get_response(session, url, params)

```

1.60. api\user\polygon\auth__init__.py

Листинг 60: api\user\polygon\auth__init__.py

1.61. api\user\polygon\auth\login.py

Листинг 61: api\user\polygon\auth\login.py

```

import logging

from fastapi import Depends
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from api.crypt import get_current_user
from api.pydantic_schemas import LinkCodeforces
from api.user.polygon.base_polygon import polygon_router
from app.database import get_db
from models import User

logger = logging.getLogger(__name__)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

@polygon_router.post("/link")
async def link_polygon(
    payload: LinkCodeforces,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    # Polygon API
    user.polygon_api_key = payload.api_key
    user.polygon_api_secret = payload.api_secret

    await db.commit()

    logger.info(f"Polygon API key linked: user_id={user_id}")
    return {"message": "Polygon account successfully linked"}

```

1.62. api\user\polygon\auth\logout.py

Листинг 62: api\user\polygon\auth\logout.py

```

from fastapi import Depends
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from api.crypt import get_current_user
from api.user.polygon.base_polygon import polygon_router
from app.database import get_db
from models import User

@polygon_router.post("/unlink")

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

async def unlink_polygon(
    user_id: int = Depends(get_current_user), db: AsyncSession = Depends(get_db)
):
    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    user.polygon_api_key = None
    user.polygon_api_secret = None
    await db.commit()

    return {"message": "Polygon account successfully unlinked"}

```

1.63. api\user\polygon\commit__init__.py

Листинг 63: api\user\polygon\commit__init__.py

1.64. api\user\polygon\commit\build_package.py

Листинг 64: api\user\polygon\commit\build_package.py

```

from time import time

from aiohttp import ClientSession
from fastapi import HTTPException, status
from pydantic import BaseModel
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from models import User

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

from settings import settings

class BuildPackageResponse(BaseModel):
    detail: str

async def build_package(problem_id: int, user_id: int, db: AsyncSession):
    method_name = "problem.buildPackage"

    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.polygon_api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Polygon API is not configured",
        )

    current_time_unix = int(time())

    params = {
        "apiKey": user.polygon_api_key,
        "time": str(current_time_unix),
        "problemId": str(problem_id),
        "full": "true",
        "verify": "true",
    }

    signature = create_signature(method_name, params, user.polygon_api_secret)

    params["apiSig"] = signature

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

url = URL(settings.POLYGON_HOST) / method_name

async with ClientSession() as session:
    await get_response(session, url, params)

return {"detail": "___"}

```

1.65. api\{}user\{}polygon\{}commit\{}commit_problem.py

Листинг 65: api\{}user\{}polygon\{}commit\{}commit_problem.py

```

from time import time

from aiohttp import ClientSession
from fastapi import HTTPException, status
from pydantic import BaseModel
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from models import User
from settings import settings

class CommitResponse(BaseModel):
    detail: str

async def commit(problem_id: int, user_id: int, db: AsyncSession):
    method_name = "problem.commitChanges"

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
user = await db.execute(select(User).filter_by(id=user_id))
user = user.scalars().first()

if not user.polygon_api_key:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Polygon_API_is_not_configured",
    )

current_time_unix = int(time())

params = {
    "apiKey": user.polygon_api_key,
    "time": str(current_time_unix),
    "problemId": str(problem_id),
    "minorChanges": "true",
    "message": "gpt-generated-task",
}

signature = create_signature(method_name, params, user.polygon_api_secret)

params["apiSig"] = signature

url = URL(settings.POLYGON_HOST) / method_name

async with ClientSession() as session:
    response = await get_response(session, url, params)
    if response["conflictOccurred"]:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="Conflict_occurred_while_committing_changes_to_the_problem.",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    )

    if not response["committed"]:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN, detail=response["message"]
        )

    return {"detail": response["message"]}

```

1.66. api\{}user\{}polygon\{}commit\{}get_packages.py

Листинг 66: api\{}user\{}polygon\{}commit\{}get_packages.py

```

from time import time

from aiohttp import ClientSession
from fastapi import HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from models import User
from settings import settings

async def get_packages(problem_id: int, user_id: int, db: AsyncSession):
    method_name = "problem.packages"

    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.polygon_api_key:

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
raise HTTPException(
    status_code=status.HTTP_401_UNAUTHORIZED,
    detail="Polygon API is not configured",
)

current_time_unix = int(time())

params = {
    "apiKey": user.polygon_api_key,
    "time": str(current_time_unix),
    "problemId": str(problem_id),
}

signature = create_signature(method_name, params, user.polygon_api_secret)
params["apiSig"] = signature

url = URL(settings.POLYGON_HOST) / method_name

async with ClientSession() as session:
    return await get_response(session, url, params)
```

1.67. api\user\polygon\files__init__.py

Листинг 67: api\user\polygon\files__init__.py

1.68. api\user\polygon\files\save_statement.py

Листинг 68: api\user\polygon\files\save_statement.py

```
from time import time
from typing import Optional
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

from aiohttp import ClientSession
from fastapi import HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from models import User
from settings import settings

async def save_statement(
    problem_id: int,
    lang: str,
    name: str,
    legend: str,
    input_legend: str,
    output_legend: str,
    scoring: Optional[str],
    interaction: Optional[str],
    notes: str | None,
    tutorial: str | None,
    user_id: int,
    db: AsyncSession,
):

    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.polygon_api_key:
        raise HTTPException(

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Polygon API is not configured",
    )

method_name = "problem.saveStatement"

current_time_unix = int(time())

params = {
    "apiKey": user.polygon_api_key,
    "time": current_time_unix,
    "problemId": problem_id,
    "lang": lang,
    "encoding": "utf-8",
    "name": name,
    "legend": legend,
    "input": input_legend,
    "output": output_legend,
}

if scoring:
    params["scoring"] = scoring
if interaction:
    params["interaction"] = interaction
if notes:
    params["notes"] = notes
if tutorial:
    params["tutorial"] = tutorial

signature = create_signature(method_name, params, user.polygon_api_secret)
params["apiSig"] = signature
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
url = URL(settings.POLYGON_HOST) / method_name

async with ClientSession() as session:
    return await get_response(session, url, params)
```

1.69. api\{}user\{}polygon\{}files\{}save_test.py

Листинг 69: api\{}user\{}polygon\{}files\{}save_test.py

```
from time import time

from aiohttp import ClientSession
from fastapi import HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from models import User
from settings import settings

async def save_test(
    problem_id: int,
    test_set: str,
    test_index: int,
    test_input: str,
    test_group: str,
    test_points: float,
    test_use_in_statements: bool,
    user_id: int,
    db: AsyncSession,
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

):

```
user = await db.execute(select(User).filter_by(id=user_id))
user = user.scalars().first()

if not user.polygon_api_key:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Polygon API is not configured",
    )

method_name = "problem.saveTest"

current_time_unix = int(time())

params = {
    "apiKey": user.polygon_api_key,
    "time": current_time_unix,
    "problemId": problem_id,
    "testset": test_set,
    "testIndex": test_index,
    "testInput": test_input,
}

if test_group:
    params["testGroup"] = test_group
if test_points:
    params["testPoints"] = test_points

if test_use_in_statements:
    params["testUseInStatements"] = "true"
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
signature = create_signature(method_name, params, user.polygon_api_secret)
params["apiSig"] = signature

url = URL(settings.POLYGON_HOST) / method_name

async with ClientSession() as client:
    return await get_response(client, url, params)
```

1.70. api\{}user\{}polygon\{}files\{}gen\{}__init__.py

Листинг 70: api\{}user\{}polygon\{}files\{}gen\{}__init__.py

1.71. api\{}user\{}polygon\{}files\{}gen\{}add_source_file.py

Листинг 71: api\{}user\{}polygon\{}files\{}gen\{}add_source_file.py

```
import logging
from time import time

from aiohttp import ClientSession
from fastapi import HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from models import User
from settings import settings

logger = logging.getLogger(__name__)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
async def add_source(
    problem_id: int, name: str, file: str, user_id: int, db: AsyncSession
):
    method_name = "problem.saveFile"

    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.polygon_api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Polygon API is not configured",
        )

    current_time_unix = int(time())

    params = {
        "apiKey": user.polygon_api_key,
        "time": str(current_time_unix),
        "type": "source",
        "problemId": str(problem_id),
        "name": name,
        "file": file,
    }

    signature = create_signature(method_name, params, user.polygon_api_secret)
    params["apiSig"] = signature

    url = URL(settings.POLYGON_HOST) / method_name

    async with ClientSession() as client:
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
return await get_response(client, url, params)
```

1.72. api\{}\user\{}\polygon\{}\files\{}\gen\{}\set_checker.py

Листинг 72: api\{}\user\{}\polygon\{}\files\{}\gen\{}\set_checker.py

```
from time import time

from aiohttp import ClientSession
from fastapi import HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from api.user.polygon.files.gen.add_source_file import add_source
from models import User
from settings import settings

async def set_checker(
    problem_id: int, name: str, checker_file: str, user_id: int, db: AsyncSession
):
    method_name = "problem.setChecker"

    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.polygon_api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Polygon API is not configured",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
)

await add_source(problem_id, name, checker_file, user_id, db)

current_time_unix = int(time())

params = {
    "apiKey": user.polygon_api_key,
    "time": str(current_time_unix),
    "problemId": str(problem_id),
    "checker": name,
}

signature = create_signature(method_name, params, user.polygon_api_secret)
params["apiSig"] = signature

url = URL(settings.POLYGON_HOST) / method_name

async with ClientSession() as client:
    return await get_response(client, url, params)
```

1.73. api\user\polygon\files\gen\set_generator.py

Листинг 73: api\user\polygon\files\gen\set_generator.py

```
from sqlalchemy.ext.asyncio import AsyncSession

from api.user.polygon.files.gen.add_source_file import add_source

async def set_generator(
    problem_id: int, name: str, generator_file: str, user_id: int, db: AsyncSession
):
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
return await add_source(problem_id, name, generator_file, user_id, db)
```

1.74. api\{}user\{}polygon\{}files\{}gen\{}set_interactor.py

Листинг 74: api\{}user\{}polygon\{}files\{}gen\{}set_interactor.py

```
from time import time

from aiohttp import ClientSession
from fastapi import HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from api.user.polygon.files.gen.add_source_file import add_source
from models import User
from settings import settings

async def set_interactor(
    problem_id: int, name: str, interactor_file: str, user_id: int, db:
        ↳ AsyncSession
):
    method_name = "problem.setInteractor"

    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.polygon_api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
        detail="Polygon_API_is_not_configured",
    )

    await add_source(problem_id, name, interactor_file, user_id, db)

    current_time_unix = int(time())

    params = {
        "apiKey": user.polygon_api_key,
        "time": str(current_time_unix),
        "problemId": str(problem_id),
        "interactor": name,
    }

    signature = create_signature(method_name, params, user.polygon_api_secret)
    params["apiSig"] = signature

    url = URL(settings.POLYGON_HOST) / method_name

    async with ClientSession() as client:
        return await get_response(client, url, params)
```

1.75. api\{}user\{}polygon\{}files\{}gen\{}set_script.py

Листинг 75: api\{}user\{}polygon\{}files\{}gen\{}set_script.py

```
from time import time

from aiohttp import ClientSession
from fastapi import HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from models import User
from settings import settings

async def set_script(
    problem_id: int, testset: str, script_file: str, user_id: int, db: AsyncSession
):

    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.polygon_api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Polygon API is not configured",
        )

    method_name = "problem.saveScript"

    current_time_unix = int(time())

    params = {
        "apiKey": user.polygon_api_key,
        "time": str(current_time_unix),
        "problemId": str(problem_id),
        "testset": testset,
        "source": script_file,
    }
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
signature = create_signature(method_name, params, user.polygon_api_secret)
params["apiSig"] = signature

url = URL(settings.POLYGON_HOST) / method_name

async with ClientSession() as client:
    return await get_response(client, url, params)
```

1.76. api\{}user\{}polygon\{}files\{}gen\{}set_solution.py

Листинг 76: api\{}user\{}polygon\{}files\{}gen\{}set_solution.py

```
from time import time

from aiohttp import ClientSession
from fastapi import HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from models import User
from settings import settings

async def set_solution(
    problem_id: int,
    name: str,
    solution_file: str,
    tag: str,
    user_id: int,
    db: AsyncSession,
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

):

```
user = await db.execute(select(User).filter_by(id=user_id))
user = user.scalars().first()

if not user.polygon_api_key:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Polygon API is not configured",
    )

method_name = "problem.saveSolution"

current_time_unix = int(time())

if tag is None:
    tag = "MA"

params = {
    "apiKey": user.polygon_api_key,
    "time": str(current_time_unix),
    "problemId": str(problem_id),
    "name": name,
    "tag": tag,
    "file": solution_file,
}

signature = create_signature(method_name, params, user.polygon_api_secret)
params["apiSig"] = signature

url = URL(settings.POLYGON_HOST) / method_name
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
async with ClientSession() as client:  
    return await get_response(client, url, params)
```

1.77. api\{}user\{}polygon\{}files\{}gen\{}set_validator.py

Листинг 77: api\{}user\{}polygon\{}files\{}gen\{}set_validator.py

```
from time import time  
  
from aiohttp import ClientSession  
from fastapi import HTTPException, status  
from sqlalchemy import select  
from sqlalchemy.ext.asyncio import AsyncSession  
from yarl import URL  
  
from api.user.polygon.create_signature import create_signature  
from api.user.polygon.files.gen.add_source_file import add_source  
from api.user.polygon.get_response import get_response  
from models import User  
from settings import settings  
  
async def set_validator(  
    problem_id: int, name: str, validator_file: str, user_id: int, db: AsyncSession  
) :  
  
    user = await db.execute(select(User).filter_by(id=user_id))  
    user = user.scalars().first()  
  
    if not user.polygon_api_key:  
        raise HTTPException(  
            status_code=status.HTTP_401_UNAUTHORIZED,  
            detail="Polygon API is not configured",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
)

method_name = "problem.setValidator"

await add_source(problem_id, name, validator_file, user_id, db)

current_time_unix = int(time())

params = {
    "apiKey": user.polygon_api_key,
    "time": str(current_time_unix),
    "problemId": str(problem_id),
    "validator": name,
}

signature = create_signature(method_name, params, user.polygon_api_secret)
params["apiSig"] = signature

url = URL(settings.POLYGON_HOST) / method_name

async with ClientSession() as client:
    return await get_response(client, url, params)
```

1.78. api\user\polygon\settings__init__.py

Листинг 78: api\user\polygon\settings__init__.py

1.79. api\user\polygon\settings\enable_groups.py

Листинг 79: api\user\polygon\settings\enable_groups.py

```
from time import time
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
from aiohttp import ClientSession
from fastapi import HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from models import User
from settings import settings

async def enable_groups(
    problem_id: int, test_set: str, enable: bool, user_id: int, db: AsyncSession
):

    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.polygon_api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Polygon API is not configured",
        )

    method_name = "problem.enableGroups"

    current_time_unix = int(time())

    params = {
        "apiKey": user.polygon_api_key,
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
"time": current_time_unix,
"problemId": problem_id,
"testset": test_set,
}

if enable:
    params["enable"] = "true"

signature = create_signature(method_name, params, user.polygon_api_secret)
params["apiSig"] = signature

url = URL(settings.POLYGON_HOST) / method_name

async with ClientSession() as session:
    return await get_response(session, url, params)
```

1.80. api\{}user\{}polygon\{}settings\{}enable_points.py

Листинг 80: api\{}user\{}polygon\{}settings\{}enable_points.py

```
from time import time

from aiohttp import ClientSession
from fastapi import HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from models import User
from settings import settings
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
async def enable_points(problem_id: int, enable: bool, user_id: int, db:
    ↪ AsyncSession):
    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.polygon_api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Polygon API is not configured",
        )

    method_name = "problem.enablePoints"

    current_time_unix = int(time())

    params = {
        "apiKey": user.polygon_api_key,
        "time": current_time_unix,
        "problemId": problem_id,
    }

    if enable:
        params["enable"] = "true"

    signature = create_signature(method_name, params, user.polygon_api_secret)
    params["apiSig"] = signature

    url = URL(settings.POLYGON_HOST) / method_name

    async with ClientSession() as session:
        return await get_response(session, url, params)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.81. api\{}user\{}polygon\{}settings\{}set_tags.py

Листинг 81: api\{}user\{}polygon\{}settings\{}set_tags.py

```
from time import time

from aiohttp import ClientSession
from fastapi import HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from models import User
from settings import settings

async def set_tags(problem_id: int, tags: str, user_id: int, db: AsyncSession):
    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.polygon_api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Polygon_API_is_not_configured",
        )

    method_name = "problem.saveTags"

    current_time_unix = int(time())
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
params = {
    "apiKey": user.polygon_api_key,
    "time": current_time_unix,
    "problemId": problem_id,
    "tags": tags,
}

signature = create_signature(method_name, params, user.polygon_api_secret)
params["apiSig"] = signature

url = URL(settings.POLYGON_HOST) / method_name

async with ClientSession() as session:
    return await get_response(session, url, params)
```

1.82. api\user\polygon\settings\set_test_group.py

Листинг 82: api\user\polygon\settings\set_test_group.py

```
from time import time

from aiohttp import ClientSession
from fastapi import HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from models import User
from settings import settings
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

async def set_test_group(
    problem_id: int,
    test_set: str,
    test_group: str,
    test_indicies: str,
    user_id: int,
    db: AsyncSession,
):
    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.polygon_api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Polygon API is not configured",
        )

    method_name = "problem.setTestGroup"

    current_time_unix = int(time())

    params = {
        "apiKey": user.polygon_api_key,
        "time": current_time_unix,
        "problemId": problem_id,
        "testset": test_set,
        "testGroup": test_group,
        "testIndices": test_indicies,
    }

    signature = create_signature(method_name, params, user.polygon_api_secret)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

params["apiSig"] = signature

url = URL(settings.POLYGON_HOST) / method_name

async with ClientSession() as session:
    return await get_response(session, url, params)

```

1.83. api\{}user\{}polygon\{}settings\{}update_info.py

Листинг 83: api\{}user\{}polygon\{}settings\{}update_info.py

```

from time import time

from aiohttp import ClientSession
from fastapi import HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from yarl import URL

from api.user.polygon.create_signature import create_signature
from api.user.polygon.get_response import get_response
from models import User
from settings import settings

async def update_info(
    problem_id: int,
    input_file_name: str,
    output_file_name: str,
    interactive: bool,
    time_limit: int,
    memory_limit: int,
    user_id: int,

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

db: AsyncSession,
):
    method_name = "problem.updateInfo"

    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.polygon_api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Polygon API is not configured",
        )

    current_time_unix = int(time())

    params = {
        "apiKey": user.polygon_api_key,
        "time": current_time_unix,
        "problemId": problem_id,
    }

    if input_file_name:
        params["inputFile"] = input_file_name
    if output_file_name:
        params["outputFile"] = output_file_name
    if interactive:
        params["interactive"] = interactive
    if time_limit:
        params["timeLimit"] = time_limit
    if memory_limit:
        params["memoryLimit"] = memory_limit

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

signature = create_signature(method_name, params, user.polygon_api_secret)
params["apiSig"] = signature

url = URL(settings.POLYGON_HOST) / method_name

async with ClientSession() as session:
    return await get_response(session, url, params)

```

1.84. api\{}user\{}yandex\{}__init__.py

Листинг 84: api\{}user\{}yandex\{}__init__.py

```

from api.user.yandex.auth.login import get_yandex_auth_url, yandex_callback
from api.user.yandex.auth.logout import logout_yandex
from api.user.yandex.base_yandex import router as yandex_router
from api.user.yandex standings import yandex_standings
from api.user.yandex.submissions import yandex_submissions

__all__ = [
    "get_yandex_auth_url",
    "yandex_callback",
    "logout_yandex",
    "yandex_standings",
    "yandex_submissions",
    "yandex_router",
]

```

1.85. api\{}user\{}yandex\{}base_yandex.py

Листинг 85: api\{}user\{}yandex\{}base_yandex.py

```

from fastapi import APIRouter

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
router = APIRouter()
```

1.86. api\{}user\{}yandex\{}format.py

Листинг 86: api\{}user\{}yandex\{}format.py

```
import logging
from base64 import b64encode
from datetime import datetime, timedelta
from typing import Any, Dict, List, Tuple

from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from models import Contest, ContestParticipant, Submission, Task, TaskResult

logger = logging.getLogger(__name__)

def format_yandex_standings(
    contest_info: dict, standings: Dict[str, dict], user_id: int, unofficial: bool
) -> Tuple[Contest, List[Task], List[ContestParticipant]]:
    contest = contest_info
    problems = standings.get("titles", [])
    rows = standings.get("rows", [])

    start_time = None
    if "startTime" in contest:
        try:
            start_time = datetime.fromisoformat(
                contest["startTime"].replace("Z", "+00:00")
            ).replace(tzinfo=None)
        except (KeyError, TypeError, ValueError):
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
start_time = None

contest_type = "ICPC" if contest_info["standingsPlugin"] == "acm" else "IOI"
best_scores = [0.0] * len(problems)
for row in rows:
    for idx, problem in enumerate(row["problemResults"]):
        if problem["score"] == "":
            points = int(problem["status"] == "ACCEPTED")
        else:
            points = float(problem["score"])

        if points > best_scores[idx]:
            best_scores[idx] = points

logger.debug(f"Best_scores_per_problem: {best_scores}")

formatted_contest = Contest(
    external_id=contest["id"],
    user_id=user_id,
    platform="yandex",
    name=contest["name"],
    type=contest_type,
    unofficial=unofficial,
    start_time=start_time,
    duration=timedelta(seconds=contest.get("duration", 0)),
)

formatted_problems: List[Task] = []
for idx, problem in enumerate(problems):
    max_score = best_scores[idx]
    formatted_problems.append(
        Task(
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инов. № подл.	Подп. и дата	Взам. инв. №	Инов. № дубл.	Подп. и дата

```

        id=f'ya_{user_id}_{contest["id"]}_{problem["id"]}',
        contest=formatted_contest,
        short_name=problem["title"],
        full_name=problem["name"],
        max_score=max_score,
    )
)

merged_participants = {}
merged_results = {}

for row in rows:
    login = row["participantInfo"]["login"]
    name = row["participantInfo"]["name"]

    if login not in merged_participants:
        merged_participants[login] = ContestParticipant(
            login=login, name=name, score=0.0
        )
        merged_results[login] = {}

    for idx, problem in enumerate(row["problemResults"]):
        task_id = formatted_problems[idx].id
        if problem["score"] == "":
            points = int(problem["status"] == "ACCEPTED")
        else:
            points = float(problem["score"])
        tries = int(problem.get("submissionCount", 1)) - 1

        success_time = None

        if points > 0 and start_time and "submitDelay" in problem:

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        success_time = start_time + timedelta(seconds=problem["submitDelay"
        ↪ ])

if task_id in merged_results[login]:
    old_res = merged_results[login][task_id]

    if points > old_res.score:
        old_res.score = points
        old_res.last_success_time = success_time
    elif points == old_res.score and success_time:
        if (
            not old_res.last_success_time
            or success_time < old_res.last_success_time
        ):
            old_res.last_success_time = success_time

    old_res.tries_count += tries
else:
    merged_results[login][task_id] = TaskResult(
        task_id=task_id,
        score=points,
        tries_count=tries,
        last_success_time=success_time,
        verdict="NULL",
        banned=False,
    )

formatted_rows = []
for login, contest_participant in merged_participants.items():
    total_score = 0.0
    task_results_list = []

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

for problem in formatted_problems:
    res = merged_results[login].get(problem.id)
    if not res:
        continue

    if res.score > 0:
        res.verdict = "OK" if res.score == problem.max_score else "PARTIAL"
    elif res.tries_count > 0:
        res.verdict = "WA"
    else:
        res.verdict = "NULL"

    total_score += res.score
    task_results_list.append(res)

contest_participant.score = total_score
formatted_rows.append((contest_participant, task_results_list))

return formatted_contest, formatted_problems, formatted_rows

```

```

async def format_yandex_submissions(
    submissions: Tuple[Any], user_id: int, contest_id: int, db: AsyncSession
) -> List[Submission]:
    formatted_submissions = []
    for submission in submissions:
        logger.debug(f"Processing submission id={submission.get('id')}")

        problem = await db.execute(
            select(Task).filter_by(
                short_name=submission["problemAlias"], contest_id=contest_id
            )

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
)
problem = problem.scalars().first()

participant_name = submission["author"]

participant = await db.execute(
    select(ContestParticipant).filter_by(
        contest_id=contest_id, name=participant_name
    )
)
participant = participant.scalars().first()

if not participant:
    continue

task_result = await db.execute(
    select(TaskResult).filter_by(
        task_id=problem.id, contest_participant_id=participant.id
    )
)
task_result = task_result.scalars().first()

if not task_result:
    continue

if submission["score"] == "" or submission["score"] is None:
    points = int(submission["verdict"] == "ACCEPTED")
else:
    points = float(submission["score"])

formatted_submission = Submission(
    id=f'yandex_{user_id}_{contest_id}_{submission["id"]}',
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
contest_id=contest_id,
task_result_id=task_result.id,
participant_login=participant.login,
task_name=problem.full_name,
send_time=datetime.fromisoformat(
    submission["submissionTime"].replace("Z", "+00:00")
).replace(tzinfo=None),
language=submission["compiler"],
score=points,
verdict=submission["verdict"],
run_time=timedelta(milliseconds=submission["time"]),
memory_bytes=int(submission["memory"]),
source=b64encode(submission["source"].encode("utf-8")).decode("utf-8"),
)
formatted_submissions.append(formatted_submission)

return formatted_submissions
```

1.87. api\{}user\{}yandex\{}auth\{}__init__.py

Листинг 87: api\{}user\{}yandex\{}auth\{}__init__.py

1.88. api\{}user\{}yandex\{}auth\{}access_expire.py

Листинг 88: api\{}user\{}yandex\{}auth\{}access_expire.py

1.89. api\{}user\{}yandex\{}auth\{}login.py

Листинг 89: api\{}user\{}yandex\{}auth\{}login.py

```
import logging
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
import httpx
from fastapi import HTTPException
from fastapi.params import Depends
from pydantic import BaseModel
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from starlette import status

from api.crypt import get_current_user
from api.user.yandex.base_yandex import router as yandex_router
from app.database import get_db
from models import User
from settings import settings

logger = logging.getLogger(__name__)

@yandex_router.get("/auth_url")
async def get_yandex_auth_url():
    url = (
        f"https://oauth.yandex.ru/authorize?response_type=code"
        f"&client_id={settings.YANDEX_CLIENT_ID}"
    )

    return {"url": url}

class YandexCallbackRequest(BaseModel):
    code: str

@yandex_router.post("/callback")
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
async def yandex_callback(
    payload: YandexCallbackRequest,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    async with httpx.AsyncClient() as client:
        response = await client.post(
            "https://oauth.yandex.ru/token",
            data={
                "grant_type": "authorization_code",
                "code": payload.code,
                "client_id": settings.YANDEX_CLIENT_ID,
                "client_secret": settings.YANDEX_CLIENT_SECRET,
            },
        )

    if response.status_code != 200:
        logger.error(
            f"Yandex_OAuth_failed_for_user_id={user_id}:"
            f"status={response.status_code}body={response.text[:200]}"
        )
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Failed_to_login_with_Yandex_API",
        )

    data = response.json()
    token = data.get("access_token")

    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
user.yandex_access_token = token
await db.commit()

logger.info(f"Yandex_account_linked: user_id={user_id}")
return {"message": "Yandex_account_successfully_linked"}
```

1.90. api\user\yandex\auth\logout.py

Листинг 90: api\user\yandex\auth\logout.py

```
from fastapi.params import Depends
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from api.crypt import get_current_user
from api.user.yandex.base_yandex import router as yandex_router
from app.database import get_db
from models import User

@yandex_router.post("/logout")
async def logout_yandex(
    user_id: int = Depends(get_current_user), db: AsyncSession = Depends(get_db)
):
    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    user.yandex_access_token = None
    await db.commit()

    return {"message": "Yandex_account_logged_out"}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.91. api\user\yandex\standings__init__.py

Листинг 91: api\user\yandex\standings__init__.py

1.92. api\user\yandex\standings\standings.py

Листинг 92: api\user\yandex\standings\standings.py

```
import httpx
from fastapi import Depends, HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from api.crypt import get_current_user
from api.pydantic_schemas import YandexStandings
from api.user.merge_table import merge_table
from api.user.yandex import yandex_router
from api.user.yandex.format import format_yandex_standings
from app.database import get_db
from models import User
from settings import settings

@yandex_router.post("/standings")
async def yandex_standings(
    standings: YandexStandings,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    method_name = f"contests/{standings.contest_id}"

    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
if not user.yandex_access_token:
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Yandex_API_is_not_configured",
    )

headers = {
    "Authorization": f"OAuth_{user.yandex_access_token}",
}

async with httpx.AsyncClient() as client:
    contest_info = await client.get(
        f"{settings.YANDEX_HOST}/{method_name}", headers=headers
    )

    if contest_info.status_code == 400:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Contest_standings_are_not_available",
        )
    elif contest_info.status_code == 401:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Your_token_expired_or_invalid",
        )
    elif contest_info.status_code == 403:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You_do_not_have_permission_to_contest",
        )
    elif contest_info.status_code == 404:
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="Contest_not_found"
        )
    elif contest_info.status_code != 200:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Failed_to_fetch_standings",
        )

    contest_info = contest_info.json()
    contest_info["id"] = standings.contest_id

    params = {
        "forJudge": "true" if standings.as_manager else "false",
        "page": 1 + (standings.from_pos // standings.count),
        "pageSize": standings.count,
        "showExternal": "true" if standings.show_unofficial else "false",
        "showVirtual": "true" if standings.show_unofficial else "false",
    }

    standings_data = await client.get(
        f"{settings.YANDEX_HOST}/{method_name}/standings",
        params=params,
        headers=headers,
    )

    if standings_data.status_code != 200:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Failed_to_fetch_standings",
        )

    standings_data = standings_data.json()
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
contest, tasks, rows = format_yandex_standings(
    contest_info, standings_data, user_id, standings.show_unofficial
)

return await merge_table(contest, tasks, rows, user_id, db)
```

1.93. api\{}user\{}yandex\{}submissions\{}__init__.py

Листинг 93: api\{}user\{}yandex\{}submissions\{}__init__.py

1.94. api\{}user\{}yandex\{}submissions\{}submissions.py

Листинг 94: api\{}user\{}yandex\{}submissions\{}submissions.py

```
import asyncio

import httpx

from fastapi import Depends, HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from api.crypt import get_current_user
from api.pydantic_schemas import YandexSubmissions
from api.user.merge_submissions import merge_submissions
from api.user.yandex import yandex_router
from api.user.yandex.format import format_yandex_submissions
from app.database import get_db
from models import Contest, User
from settings import settings
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

@yandex_router.post("/submissions")
async def yandex_submissions(
    submissions: YandexSubmissions,
    user_id: int = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    method_name = f"contests/{submissions.contest_id}/submissions"

    user = await db.execute(select(User).filter_by(id=user_id))
    user = user.scalars().first()

    if not user.yandex_access_token:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Yandex_API_is_not_configured",
        )

    headers = {
        "Authorization": f"OAuth_{user.yandex_access_token}",
    }

    async with httpx.AsyncClient(timeout=30.0) as client:
        params = {
            "page": 1 + (submissions.from_pos // submissions.count),
            "pageSize": submissions.count,
        }

        submissions_info = await client.get(
            f"{settings.YANDEX_HOST}/{method_name}", headers=headers, params=params
        )

        if submissions_info.status_code == 400:

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Contest standings are not available",
        )
    elif submissions_info.status_code == 401:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Your token expired or invalid",
        )
    elif submissions_info.status_code == 403:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You do not have permission to contest",
        )
    elif submissions_info.status_code == 404:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND, detail="Contest not found"
        )
    elif submissions_info.status_code != 200:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Failed to fetch standings",
        )

    submissions_info = submissions_info.json()["submissions"]

    async def fetch_source(submission_info):
        source_url = (
            f'{settings.YANDEX_HOST}/{method_name}/{submission_info["id"]}/'
            '↪ source'
        )

    try:
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        submission_source = await client.get(source_url, headers=headers)
        submission_info["source"] = (
            submission_source.text
            if submission_source.status_code == 200
            else ""
        )
    except Exception:
        submission_info["source"] = ""
    return submission_info

tasks = [fetch_source(submission_info) for submission_info in
    ↪ submissions_info]
submissions_info = await asyncio.gather(*tasks)

contest = await db.execute(
    select(Contest).filter_by(external_id=submissions.contest_id)
)
contest = contest.scalars().first()

submissions_result = await format_yandex_submissions(
    submissions_info, user_id, contest.id, db
)

return await merge_submissions(submissions_result, db)
```

1.95. app\{}__init__.py

Листинг 95: app\{}__init__.py

1.96. app\{}\database.py

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Листинг 96: app\{}\database.py

```
from sqlalchemy.ext.asyncio import (AsyncSession, async_sessionmaker,
                                     create_async_engine)

from settings import settings

engine = create_async_engine(settings.database_url)
Session = async_sessionmaker(bind=engine, class_=AsyncSession, expire_on_commit=
    ↪ False)

async def get_db():
    async with Session() as session:
        yield session
```

1.97. app\{}\logging_config.py

Листинг 97: app\{}\logging_config.py

```
import logging
import sys
from logging.handlers import RotatingFileHandler

import colorlog

from settings import settings

DATE_FORMAT = "%Y-%m-%d_%H:%M:%S"
PLAIN_FORMAT = "%(asctime)s|%(levelname)-8s|%(name)s|%(message)s"
COLOR_FORMAT = "%(log_color)s%(asctime)s|%(levelname)-8s%(reset)s|%(cyan)s%(
    ↪ name)s%(reset)s|%(message)s"

LOG_DIR = settings.PROJECT_ROOT / "logs"
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
LOG_DIR.mkdir(exist_ok=True)

LOG_FILE = LOG_DIR / "app.log"
ERROR_LOG_FILE = LOG_DIR / "error.log"

def setup_logging(level: int = logging.INFO, log_to_file: bool = True,
    ↪ log_to_console: bool = True) -> None:
    root_logger = logging.getLogger()
    root_logger.setLevel(level)

    for handler in root_logger.handlers[:]:
        root_logger.removeHandler(handler)

    plain_formatter = logging.Formatter(PLAIN_FORMAT, DATE_FORMAT)

    if log_to_console:
        console = logging.StreamHandler(sys.stdout)
        console.setLevel(level)
        console.setFormatter(colorlog.ColoredFormatter(
            COLOR_FORMAT,
            datefmt=DATE_FORMAT,
            log_colors={
                "DEBUG": "cyan",
                "INFO": "green",
                "WARNING": "yellow",
                "ERROR": "red",
                "CRITICAL": "bold_red",
            },
        ))
        root_logger.addHandler(console)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
if log_to_file:
    file_handler = RotatingFileHandler(
        LOG_FILE, maxBytes=10 * 1024 * 1024, backupCount=5, encoding="utf-8"
    )
    file_handler.setLevel(level)
    file_handler.setFormatter(plain_formatter)
    root_logger.addHandler(file_handler)

    error_handler = RotatingFileHandler(
        ERROR_LOG_FILE, maxBytes=10 * 1024 * 1024, backupCount=5, encoding="utf
        ↪ -8"
    )
    error_handler.setLevel(logging.ERROR)
    error_handler.setFormatter(plain_formatter)
    root_logger.addHandler(error_handler)

# Route uvicorn startup/error messages through our root handlers
for name in ("uvicorn", "uvicorn.error"):
    uv_logger = logging.getLogger(name)
    uv_logger.handlers = []
    uv_logger.propagate = True

# Disable uvicorn's built-in access log HTTP requests are logged by
    ↪ LoggingMiddleware
uv_access = logging.getLogger("uvicorn.access")
uv_access.handlers = []
uv_access.propagate = False

def get_logger(name: str) -> logging.Logger:
    return logging.getLogger(name)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.98. app\{}\server.py

Листинг 98: app\{}\server.py

```
import asyncio
import logging
from contextlib import asynccontextmanager
from datetime import datetime, timezone

from fastapi import Depends, FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy import delete, select, text, update
from sqlalchemy.exc import OperationalError, SQLAlchemyError

from api import health_router
from api.crypt import get_current_user, verify_token
from api.user import contest_router
from api.user.auth import auth_router
from api.user.codeforces import codeforces_router
from api.user.gpt import gpt_router
from api.user.plagiarism import plagiarism_router
from api.user.polygon.base_polygon import polygon_router
from api.user.yandex import yandex_router
from app.database import Session, engine
from app.logging_config import get_logger, setup_logging
from app.middlewares.log_middleware import LoggingMiddleware
from models import RefreshToken, Role
from settings import settings

LOG_LEVEL = logging.INFO
LOG_TO_STDOUT = True
LOG_TO_FILE = True
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
setup_logging(LOG_LEVEL, LOG_TO_FILE, LOG_TO_STDOUT)
logger = get_logger(__name__)

async def cleanup_expired_tokens():
    while True:
        try:
            async with Session() as db:
                now = datetime.now(timezone.utc).replace(tzinfo=None)
                stmt = delete(RefreshToken).where(RefreshToken.expires_in < now)
                await db.execute(stmt)
                await db.commit()
        except Exception as e:
            logger.error(f"Token_cleanup_failed: {e}")

        await asyncio.sleep(86400)

@asynccontextmanager
async def lifespan(app: FastAPI):
    logger.info("=" * 60)
    logger.info("App_startup_complete.")
    logger.info("=" * 60)

    cleanup_task = asyncio.create_task(cleanup_expired_tokens())

    for _ in range(5):
        try:
            async with engine.connect() as conn:
                await conn.execute(text("SELECT 1"))
                await conn.commit()
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        async with Session() as db:
            result = await db.execute(select(Role).filter_by(name="User"))
            if not result.scalars().first():
                db.add(Role(name="User"))
                db.add(Role(name="Admin"))
                await db.commit()

        break
    except (OperationalError, Exception):
        await asyncio.sleep(2)

yield

cleanup_task.cancel()
await engine.dispose()

logger.info("=" * 60)
logger.info("App shutdown complete.")
logger.info("=" * 60)

app = FastAPI(lifespan=lifespan)

app.add_middleware(LoggingMiddleware)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.CORS_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
@app.middleware("http")
async def update_last_seen_middleware(request: Request, call_next):
    auth_header = request.headers.get("Authorization")
    if auth_header and auth_header.startswith("Bearer_"):
        token = auth_header.split("_")[1]
        try:
            payload = verify_token(token)
        except Exception:
            logger.exception("Failed_to_verify_bearer_token_in_middleware")
        else:
            sid = payload.get("sid")
            if sid:
                async with Session() as db:
                    try:
                        await db.execute(
                            update(RefreshToken)
                                .where(RefreshToken.id == sid)
                                .values(last_seen=datetime.now().replace(tzinfo=None))
                        )
                        await db.commit()
                    except SQLAlchemyError:
                        await db.rollback()
                        logger.exception(
                            "Failed_to_update_last_seen_for_token_in_middleware"
                        )

    return await call_next(request)

app.include_router(health_router, prefix="/api", tags=["health"])
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

app.include_router(auth_router, prefix="/api/auth", tags=["auth"])
app.include_router(
    contest_router,
    prefix="/api/contests",
    tags=["contest"],
    dependencies=[Depends(get_current_user)],
)
app.include_router(
    codeforces_router,
    prefix="/api/codeforces",
    tags=["codeforces"],
    dependencies=[Depends(get_current_user)],
)
app.include_router(
    polygon_router,
    prefix="/api/polygon",
    tags=["polygon"],
    dependencies=[Depends(get_current_user)],
)
app.include_router(
    gpt_router, prefix="/api/ai", tags=["gpt"], dependencies=[Depends(
        ↪ get_current_user)]
)
app.include_router(
    yandex_router,
    prefix="/api/yandex",
    tags=["yandex"],
    dependencies=[Depends(get_current_user)],
)
app.include_router(
    plagiarism_router,
    prefix="/api/analytics",

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
tags=["plagiarism"],  
dependencies=[Depends(get_current_user)],  
)
```

1.99. app\{}\middlewares\{}log_middleware.py

Листинг 99: app\{}\middlewares\{}log_middleware.py

```
import logging  
import time  
  
from fastapi import Request  
from starlette.middleware.base import BaseHTTPMiddleware  
  
logger = logging.getLogger("app.http")  
  
class LoggingMiddleware(BaseHTTPMiddleware):  
    async def dispatch(self, request: Request, call_next):  
        path = request.url.path  
        if path == "/api/health":  
            return await call_next(request)  
  
        method = request.method  
        client_ip = request.client.host if request.client else "unknown"  
        query_params = dict(request.query_params)  
  
        start_time = time.time()  
        try:  
            response = await call_next(request)  
        except Exception as e:  
            elapsed = time.time() - start_time  
            logger.exception(
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
f"{method}_{path}_{IP}_{client_ip}_{Error}_{e}_{Time}_{elapsed:.3f}s"
    ↪ f}s"
)
raise

elapsed = time.time() - start_time
params_str = f"_{Params}_{query_params}" if query_params else ""
msg = f"{method}_{path}_{IP}_{client_ip}_{params_str}_{Status}_{response.
    ↪ status_code}_{Time}_{elapsed:.3f}s"

if response.status_code >= 500:
    logger.error(msg)
elif response.status_code >= 400:
    logger.warning(msg)
else:
    logger.info(msg)

response.headers["X-Process-Time"] = str(elapsed)
return response
```

1.100. logs\{\}__init__.py

Листинг 100: logs\{\}__init__.py

1.101. logs\{\}logs.py

Листинг 101: logs\{\}logs.py

```
import logging
```

```
def log_middleware(func):
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
async def wrapper(*args, **kwargs):
    logging.info(f"Started_{func.__name__}_{args={args}}_{kwargs={kwargs}}")

    try:
        response = await func(*args, **kwargs)
        return response
    except Exception as e:
        logging.error(f"Exception_caught_in_{func.__name__}:_{e}")
        raise e
    finally:
        logging.info(f"Ended_{func.__name__}")

return wrapper
```

1.102. models\{}__init__.py

Листинг 102: models\{}__init__.py

```
from models.ai.ai_generated_file import AIGeneratedFile
from models.ai.ai_session import AISession
from models.base import Base
from models.contest.contest import Contest
from models.contest.contest_participant import ContestParticipant
from models.contest.task import Task
from models.contest.task_result import TaskResult
from models.participant.participant import Participant
from models.plagiarism.pair_of_banned_submissions import \
    PairOfBannedSubmissions
from models.plagiarism.plagiarism_report import PlagiarismReport
from models.submissions.submission import Submission
from models.user.refresh_token import RefreshToken
from models.user.role import Role
from models.user.user import User
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
__all__ = [  
    "Base",  
    "AISession",  
    "AIGeneratedFile",  
    "Contest",  
    "ContestParticipant",  
    "Task",  
    "TaskResult",  
    "PairOfBannedSubmissions",  
    "PlagiarismReport",  
    "Submission",  
    "Participant",  
    "RefreshToken",  
    "Role",  
    "User",  
]
```

1.103. models\{}\base.py

Листинг 103: models\{}\base.py

```
from sqlalchemy.orm import DeclarativeBase  
  
class Base(DeclarativeBase):  
    pass
```

1.104. models\{}\ai\{}__init__.py

Листинг 104: models\{}\ai\{}__init__.py

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.105. models\ai\ai_generated_file.py

Листинг 105: models\ai\ai_generated_file.py

```
from sqlalchemy import ForeignKey, Text
from sqlalchemy.orm import Mapped, mapped_column

from models.base import Base

class AIGeneratedFile(Base):
    __tablename__ = "ai_generated_files"
    id: Mapped[int] = mapped_column(primary_key=True)
    session_id: Mapped[str] = mapped_column(ForeignKey("ai_sessions.id"))
    filename: Mapped[str] = mapped_column()
    content: Mapped[str] = mapped_column(Text)
    file_type: Mapped[str] = mapped_column()
    # True = this content has been successfully uploaded to Polygon
    uploaded: Mapped[bool] = mapped_column(default=False)
```

1.106. models\ai\ai_session.py

Листинг 106: models\ai\ai_session.py

```
import enum
from datetime import datetime

from sqlalchemy import JSON, ForeignKey, Text
from sqlalchemy.orm import Mapped, mapped_column

from models.base import Base

class PipelineStage(str, enum.Enum):
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
STATEMENT = "statement"
FILES_REVIEW = "files_review"
UPLOADING = "uploading"
FIXING_ERRORS = "fixing_errors"
BUILDING_PACKAGE = "building_package"
DONE = "done"
FAILED = "failed"
```

```
class AISession(Base):
    __tablename__ = "ai_sessions"

    id: Mapped[str] = mapped_column(primary_key=True, index=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    model: Mapped[str] = mapped_column()
    system_prompt: Mapped[str] = mapped_column(Text)
    history: Mapped[list] = mapped_column(JSON, default=list)
    statement: Mapped[dict] = mapped_column(JSON, nullable=True)
    progress: Mapped[dict] = mapped_column(
        JSON, default=lambda: {"status": "idle"}, nullable=True
    )

    stage: Mapped[str] = mapped_column(default=PipelineStage.STATEMENT, nullable=
        ↪ False)

    polygon_problem_id: Mapped[int] = mapped_column(nullable=True)
    package_id: Mapped[int] = mapped_column(nullable=True)
    upload_errors: Mapped[dict] = mapped_column(JSON, nullable=True)
    ai_fix_attempts: Mapped[dict] = mapped_column(JSON, default=dict)

    created_at: Mapped[datetime] = mapped_column()
    updated_at: Mapped[datetime] = mapped_column()
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.107. models\{}contest\{}__init__.py

Листинг 107: models\{}contest\{}__init__.py

1.108. models\{}contest\{}contest.py

Листинг 108: models\{}contest\{}contest.py

```
from datetime import datetime, timedelta
from typing import TYPE_CHECKING

from sqlalchemy import ForeignKey, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from models.base import Base

if TYPE_CHECKING:
    from models.contest.contest_participant import ContestParticipant
    from models.contest.task import Task
    from models.plagiarism.pair_of_banned_submissions import \
        PairOfBannedSubmissions
    from models.plagiarism.plagiarism_report import PlagiarismReport
    from models.user.user import User

class Contest(Base):
    __tablename__ = "contests"

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))

    platform: Mapped[str] = mapped_column(String(100))
    external_id: Mapped[int] = mapped_column()
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

name: Mapped[str] = mapped_column(String(256))
type: Mapped[str] = mapped_column(String(5))

unofficial: Mapped[bool] = mapped_column()

start_time: Mapped[datetime | None] = mapped_column()
duration: Mapped[timedelta | None] = mapped_column()

user: Mapped["User"] = relationship(back_populates="contests")

contest_participants: Mapped[list["ContestParticipant"]] = relationship(
    back_populates="contest", cascade="all, delete-orphan"
)
tasks: Mapped[list["Task"]] = relationship(
    back_populates="contest", cascade="all, delete-orphan"
)
pairs_of_banned_submissions: Mapped[list["PairOfBannedSubmissions"]] =
    ↪ relationship(
        back_populates="contest", cascade="all, delete-orphan"
    )

plagiarism_reports: Mapped[list["PlagiarismReport"]] = relationship(
    back_populates="contest", cascade="all, delete-orphan"
)

```

1.109. models\{}contest\{}contest_participant.py

Листинг 109: models\{}contest\{}contest_participant.py

```
from typing import TYPE_CHECKING
```

```
from sqlalchemy import ForeignKey, String
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

from sqlalchemy.orm import Mapped, mapped_column, relationship

from models.base import Base

if TYPE_CHECKING:
    from models.contest.contest import Contest
    from models.contest.task_result import TaskResult
    from models.participant.participant import Participant

class ContestParticipant(Base):
    __tablename__ = "contest_participants"

    id: Mapped[int] = mapped_column(primary_key=True)

    contest_id: Mapped[int] = mapped_column(ForeignKey("contests.id"))
    participant_id: Mapped[int] = mapped_column(ForeignKey("participants.id"))

    login: Mapped[str] = mapped_column(String(255))
    name: Mapped[str | None] = mapped_column(String(255))

    score: Mapped[float | None] = mapped_column()

    contest: Mapped["Contest"] = relationship(back_populates="contest_participants"
        ↪ )

    participant: Mapped["Participant"] = relationship(
        back_populates="contest_participants"
    )

    tasks_results: Mapped[list["TaskResult"]] = relationship(
        back_populates="contest_participant",

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        cascade="all, delete-orphan",  
    )
```

1.110. models\{}contest\{}task.py

Листинг 110: models\{}contest\{}task.py

```
from typing import TYPE_CHECKING

from sqlalchemy import ForeignKey, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from models.base import Base

if TYPE_CHECKING:
    from models.contest.contest import Contest
    from models.contest.task_result import TaskResult

class Task(Base):
    __tablename__ = "tasks"

    id: Mapped[str] = mapped_column(primary_key=True)
    contest_id: Mapped[int] = mapped_column(ForeignKey("contests.id"))

    short_name: Mapped[str | None] = mapped_column(String(20))
    full_name: Mapped[str] = mapped_column(String(100))

    max_score: Mapped[float | None] = mapped_column()

    task_results: Mapped[list["TaskResult"]] = relationship(back_populates="task")

    contest: Mapped["Contest"] = relationship(back_populates="tasks")
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.111. models\{}contest\{}task_result.py

Листинг 111: models\{}contest\{}task_result.py

```
from datetime import datetime
from typing import TYPE_CHECKING

from sqlalchemy import ForeignKey, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from models.base import Base

if TYPE_CHECKING:
    from models.contest.contest_participant import ContestParticipant
    from models.contest.task import Task
    from models.submissions.submission import Submission

class TaskResult(Base):
    __tablename__ = "task_results"

    id: Mapped[int] = mapped_column(primary_key=True)
    contest_participant_id: Mapped[int] = mapped_column(
        ForeignKey("contest_participants.id")
    )
    task_id: Mapped[str] = mapped_column(ForeignKey("tasks.id"))

    score: Mapped[float | None] = mapped_column()
    tries_count: Mapped[int | None] = mapped_column()
    verdict: Mapped[str] = mapped_column(String(50))
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
last_success_time: Mapped[datetime | None] = mapped_column()

banned: Mapped[bool] = mapped_column(default=False)

task: Mapped["Task"] = relationship(back_populates="task_results")
contest_participant: Mapped["ContestParticipant"] = relationship(
    back_populates="tasks_results"
)

submissions: Mapped[list["Submission"]] = relationship(back_populates="
    ↪ task_result")
```

1.112. models\{}participant\{}__init__.py

Листинг 112: models\{}participant\{}__init__.py

1.113. models\{}participant\{}participant.py

Листинг 113: models\{}participant\{}participant.py

```
from typing import TYPE_CHECKING

from sqlalchemy import ForeignKey, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from models.base import Base

if TYPE_CHECKING:
    from models.contest.contest_participant import ContestParticipant
    from models.user.user import User
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

class Participant(Base):
    __tablename__ = "participants"

    id: Mapped[int] = mapped_column(primary_key=True)

    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))

    login: Mapped[str] = mapped_column(String(255))
    name: Mapped[str | None] = mapped_column(String(255))

    rating: Mapped[float | None] = mapped_column()

    user: Mapped["User"] = relationship(back_populates="participants")

    contest_participants: Mapped[list["ContestParticipant"]] = relationship(
        back_populates="participant"
    )

```

1.114. models\{}plagiarism\{}__init__.py

Листинг 114: models\{}plagiarism\{}__init__.py

1.115. models\{}plagiarism\{}pair_of_banned_submissions.py

Листинг 115: models\{}plagiarism\{}pair_of_banned_submissions.py

```

from typing import TYPE_CHECKING

from sqlalchemy import ForeignKey
from sqlalchemy.orm import Mapped, mapped_column, relationship

from models.base import Base

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

if TYPE_CHECKING:
    from models.contest.contest import Contest
    from models.plagiarism.plagiarism_report import PlagiarismReport
    from models.submissions.submission import Submission

class PairOfBannedSubmissions(Base):
    __tablename__ = "pair_of_banned_submissions"

    id: Mapped[int] = mapped_column(primary_key=True, autoincrement=True)

    contest_id: Mapped[int] = mapped_column(ForeignKey("contests.id"))
    report_id: Mapped[int] = mapped_column(ForeignKey("plagiarism_reports.id"))

    first_submission_id: Mapped[str] = mapped_column(ForeignKey("submissions.id"))
    second_submission_id: Mapped[str] = mapped_column(ForeignKey("submissions.id"))

    percentage: Mapped[float] = mapped_column()

    contest: Mapped["Contest"] = relationship(
        back_populates="pairs_of_banned_submissions"
    )
    report: Mapped["PlagiarismReport"] = relationship(back_populates="pairs")

    first_submission: Mapped["Submission"] = relationship(
        foreign_keys=[first_submission_id], back_populates="banned_as_first"
    )
    second_submission: Mapped["Submission"] = relationship(
        foreign_keys=[second_submission_id], back_populates="banned_as_second"
    )

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.116. models\{}\plagiarism\{}\plagiarism_report.py

Листинг 116: models\{}\plagiarism\{}\plagiarism_report.py

```

from datetime import datetime
from typing import TYPE_CHECKING

from sqlalchemy import ForeignKey
from sqlalchemy.orm import Mapped, mapped_column, relationship

from models.base import Base

if TYPE_CHECKING:
    from models.contest.contest import Contest
    from models.plagiarism.pair_of_banned_submissions import \
        PairOfBannedSubmissions

class PlagiarismReport(Base):
    __tablename__ = "plagiarism_reports"

    id: Mapped[int] = mapped_column(primary_key=True)

    contest_id: Mapped[int] = mapped_column(ForeignKey("contests.id"))

    status: Mapped[str] = mapped_column(
        default="processing"
    ) # "processing", "completed", "failed"

    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)

    threshold: Mapped[float] = mapped_column()
    only_ok: Mapped[bool] = mapped_column()

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

contest: Mapped["Contest"] = relationship(back_populates="plagiarism_reports")

pairs: Mapped[list["PairOfBannedSubmissions"]] = relationship(
    back_populates="report", cascade="all, delete-orphan", lazy="selectin"
)

```

1.117. models\{}\submissions\{}__init__.py

Листинг 117: models\{}\submissions\{}__init__.py

1.118. models\{}\submissions\{}\plagiarism_report.py

Листинг 118: models\{}\submissions\{}\plagiarism_report.py

```

from datetime import datetime
from typing import TYPE_CHECKING

from sqlalchemy import ForeignKey, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from models.base import Base

if TYPE_CHECKING:
    from models.plagiarism.pair_of_banned_submissions import \
        PairOfBannedSubmissions

class PlagiarismReport(Base):
    __tablename__ = "plagiarism_reports"

    id: Mapped[int] = mapped_column(primary_key=True, autoincrement=True)
    contest_id: Mapped[int] = mapped_column(ForeignKey("contests.id"))

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
status: Mapped[str] = mapped_column(String(32), default="processing")
threshold: Mapped[float] = mapped_column(default=0.8)
only_ok: Mapped[bool] = mapped_column(default=False)

created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
updated_at: Mapped[datetime] = mapped_column(
    default=datetime.utcnow, onupdate=datetime.utcnow
)

pairs: Mapped[list["PairOfBannedSubmissions"]] = relationship(
    back_populates="report", cascade="all, delete-orphan"
)
```

1.119. models\{}\submissions\{}\submission.py

Листинг 119: models\{}\submissions\{}\submission.py

```
from datetime import datetime, timedelta
from typing import TYPE_CHECKING

from sqlalchemy import ForeignKey, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from models.base import Base

if TYPE_CHECKING:
    from models.contest.task_result import TaskResult
    from models.plagiarism.pair_of_banned_submissions import \
        PairOfBannedSubmissions

class Submission(Base):
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
__tablename__ = "submissions"

id: Mapped[str] = mapped_column(primary_key=True)

contest_id: Mapped[int] = mapped_column(ForeignKey("contests.id"))
task_result_id: Mapped[int] = mapped_column(ForeignKey("task_results.id"))

participant_login: Mapped[str] = mapped_column(String(50))
task_name: Mapped[str] = mapped_column(String(50))

send_time: Mapped[datetime] = mapped_column()

language: Mapped[str] = mapped_column(String(50))

score: Mapped[float | None] = mapped_column()
verdict: Mapped[str] = mapped_column(String(50))

run_time: Mapped[timedelta] = mapped_column()
memory_bytes: Mapped[int] = mapped_column()

banned: Mapped[bool] = mapped_column(default=False)

source: Mapped[str | None] = mapped_column()

task_result: Mapped["TaskResult"] = relationship(back_populates="submissions")

banned_as_first: Mapped[list["PairOfBannedSubmissions"]] = relationship(
    primaryjoin="Submission.id==PairOfBannedSubmissions.first_submission_id",
    back_populates="first_submission",
)

banned_as_second: Mapped[list["PairOfBannedSubmissions"]] = relationship(
    primaryjoin="Submission.id==PairOfBannedSubmissions.second_submission_id",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
back_populates="second_submission",  
)
```

1.120. models\{user}\{__init__.py

Листинг 120: models\{user}\{__init__.py

1.121. models\{user}\{refresh_token.py

Листинг 121: models\{user}\{refresh_token.py

```
import uuid  
from datetime import datetime  
from typing import TYPE_CHECKING  
  
from sqlalchemy import ForeignKey, String  
from sqlalchemy.dialects.postgresql import UUID  
from sqlalchemy.orm import Mapped, mapped_column, relationship  
  
from models.base import Base  
  
if TYPE_CHECKING:  
    from models.user.user import User  
  
class RefreshToken(Base):  
    __tablename__ = "refresh_tokens"  
  
    id: Mapped[uuid.UUID] = mapped_column(  
        UUID(as_uuid=True), primary_key=True, default=uuid.uuid4  
    )  
  
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

refresh_hash: Mapped[str] = mapped_column(String(256), unique=True)

user_agent: Mapped[str | None] = mapped_column(String(255))
last_seen: Mapped[datetime] = mapped_column(
    default=datetime.now().replace(tzinfo=None),
    onupdate=datetime.now().replace(tzinfo=None),
)

created_at: Mapped[datetime] = mapped_column()
expires_in: Mapped[datetime] = mapped_column()

user: Mapped["User"] = relationship(back_populates="refresh_tokens")

```

1.122. models\user\role.py

Листинг 122: models\user\role.py

```

from typing import TYPE_CHECKING

from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from models.base import Base

if TYPE_CHECKING:
    from models.user.user import User

class Role(Base):
    __tablename__ = "roles"

    id: Mapped[int] = mapped_column(primary_key=True)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
name: Mapped[str] = mapped_column(String(100))
```

```
user: Mapped[list["User"]] = relationship(back_populates="role")
```

1.123. models\{}\user\{}\user.py

Листинг 123: models\{}\user\{}\user.py

```
from typing import TYPE_CHECKING

from sqlalchemy import ForeignKey, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from models.base import Base

if TYPE_CHECKING:
    from models.contest.contest import Contest
    from models.participant.participant import Participant
    from models.user.refresh_token import RefreshToken
    from models.user.role import Role

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)

    role_id: Mapped[int] = mapped_column(ForeignKey("roles.id"))

    login: Mapped[str] = mapped_column(String(50), unique=True)
    password: Mapped[str] = mapped_column(String(120))
    email: Mapped[str] = mapped_column(String(100))
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

codeforces_api_key: Mapped[str | None] = mapped_column(String(100))
codeforces_api_secret: Mapped[str | None] = mapped_column(String(100))

# codeforces_access_token: Mapped[str | None] = mapped_column(String(255))

polygon_api_key: Mapped[str | None] = mapped_column(String(100))
polygon_api_secret: Mapped[str | None] = mapped_column(String(100))

yandex_access_token: Mapped[str | None] = mapped_column(String(255))

refresh_tokens: Mapped[list["RefreshToken"]] = relationship(
    back_populates="user", cascade="all, delete-orphan"
)
contests: Mapped[list["Contest"]] = relationship(
    back_populates="user", cascade="all, delete-orphan"
)
participants: Mapped[list["Participant"]] = relationship(
    back_populates="user", cascade="all, delete-orphan"
)
role: Mapped["Role"] = relationship(back_populates="user")

```

1.124. plagiarism\{}main.cpp

Листинг 124: plagiarism\{}main.cpp

```

#include <chrono>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

#include "models/submission.h"

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

#include "similarity/compare_submissions.h"
#include "submission_loader/submission_loader.h"

int main(int argc, char* argv[]) {
    auto start = std::chrono::steady_clock::now();

    if (argc < 3 || std::string(argv[1]) != "--load-from") {
        std::cerr << "Usage:\n";
        std::cerr << "  " << argv[0] << "  --load-from <root_dir> <problem>\n";
        return 1;
    }

    const std::string root_dir = argv[2];

    std::vector<std::string> problems;
    for (int i = 3; i < argc; ++i) {
        problems.push_back(argv[i]);
    }

    std::vector<Submission> submissions;

    std::cout << "Loading submissions from: " << root_dir << "\n";
    if (!problems.empty()) {
        std::cout << "Problems: ";
        for (const auto& p : problems) std::cout << p << " ";
        std::cout << "\n";
        submissions = SubmissionLoader::load_problem_submissions(root_dir, problems
            ↪ );
    } else {
        submissions = SubmissionLoader::load_from_directory(root_dir);
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

std::cout << "Loaded_" << submissions.size() << "_submissions\n";

const std::vector<SimilarSubmissionPair> pairs =
    compute_similarity_pairs(submissions, 0.0);

std::cout << std::fixed << std::setprecision(2);
std::cout << "Found_pairs:" << pairs.size() << "\n";

for (const auto& p : pairs) {
    std::cout
        << p.first_submission_id
        << "_" << p.first_participant
        << "/" << (p.first_problem.empty() ? "[no_task]" : p.first_problem)
        << "/" << p.first_file_name << ")"
        << "_->_"
        << p.second_submission_id
        << "_" << p.second_participant
        << "/" << (p.second_problem.empty() ? "[no_task]" : p.second_problem)
        << "/" << p.second_file_name << ")"
        << "_=>_" << p.plagiarism_percent << "%\n";
}

auto finish = std::chrono::steady_clock::now();
auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(finish - start)
    ↪ .count();

std::cout << "Total_time:" << ms << "_ms\n";
return 0;
}

```

1.125. plagiarism\ast_ast_analyzer.cpp

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Листинг 125: plagiarism\{\}ast\{\}ast_analyzer.cpp

```

#include "ast_analyzer.h"

#include <clang-c/Index.h>

#include <algorithm>
#include <string>
#include <unordered_map>
#include <utility>

static std::string to_std_string(CXString cx_string) {
    const char* c_str = clang_getCString(cx_string);
    std::string result;
    if (c_str) {
        result = c_str;
    } else {
        result = "";
    }
    clang_disposeString(cx_string);
    return result;
}

static std::string get_kind_name(CXCursor cursor) {
    CXCursorKind kind = clang_getCursorKind(cursor);
    return to_std_string(clang_getCursorKindSpelling(kind));
}

static bool is_from_main_file(CXCursor cursor) {
    CXSourceLocation location = clang_getCursorLocation(cursor);
    return clang_Location_isFromMainFile(location) != 0;
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

static bool is_transparent_wrapper_kind(CXCursorKind kind) {
    if (kind == CXCursor_ParenExpr) return true;
    if (kind == CXCursor_UnexposedExpr) return true;
    return false;
}

static bool is_interesting_kind(CXCursorKind kind) {
    if (kind == CXCursor_FunctionDecl) return true;
    if (kind == CXCursor_IfStmt) return true;
    if (kind == CXCursor_ForStmt) return true;
    if (kind == CXCursor_WhileStmt) return true;
    if (kind == CXCursor_ReturnStmt) return true;
    if (kind == CXCursor_CallExpr) return true;
    return false;
}

static void count_interesting_kind(CXCursorKind kind, AstFeatures& features) {
    if (kind == CXCursor_FunctionDecl) {
        ++features.functions_cnt;
    } else if (kind == CXCursor_IfStmt) {
        ++features.ifs_cnt;
    } else if (kind == CXCursor_ForStmt) {
        ++features.fors_cnt;
    } else if (kind == CXCursor_WhileStmt) {
        ++features.whiles_cnt;
    } else if (kind == CXCursor_ReturnStmt) {
        ++features.returns_cnt;
    } else if (kind == CXCursor_CallExpr) {
        ++features.calls_cnt;
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

static AstNodeType convert_kind(CXCursorKind kind) {
    if (kind == CXCursor_TranslationUnit) return AstNodeType::kTranslationUnit;
    if (kind == CXCursor_FunctionDecl) return AstNodeType::kFunctionDecl;
    if (kind == CXCursor_ParmDecl) return AstNodeType::kParamDecl;
    if (kind == CXCursor_VarDecl) return AstNodeType::kVarDecl;
    if (kind == CXCursor_CompoundStmt) return AstNodeType::kCompoundStmt;
    if (kind == CXCursor_IfStmt) return AstNodeType::kIfStmt;
    if (kind == CXCursor_ForStmt) return AstNodeType::kForStmt;
    if (kind == CXCursor_WhileStmt) return AstNodeType::kWhileStmt;
    if (kind == CXCursor_ReturnStmt) return AstNodeType::kReturnStmt;
    if (kind == CXCursor_CallExpr) return AstNodeType::kCallExpr;
    if (kind == CXCursor_BinaryOperator) return AstNodeType::kBinaryOp;
    if (kind == CXCursor_UnaryOperator) return AstNodeType::kUnaryOp;
    if (kind == CXCursor_DeclRefExpr) return AstNodeType::kDeclRefExpr;
    if (kind == CXCursor_IntegerLiteral) return AstNodeType::kIntegerLiteral;
    if (kind == CXCursor_FloatingLiteral) return AstNodeType::kFloatLiteral;
    if (kind == CXCursor_StringLiteral) return AstNodeType::kStringLiteral;
    return AstNodeType::kUnknown;
}

```

```

static LineRange extract_line_range(CXCursor cursor) {
    LineRange line_range;

    CXSourceRange range = clang_getCursorExtent(cursor);
    CXSourceLocation begin = clang_getRangeStart(range);
    CXSourceLocation end = clang_getRangeEnd(range);

    unsigned begin_line = 0;
    unsigned end_line = 0;

    clang_getSpellingLocation(begin, nullptr, &begin_line, nullptr, nullptr);
    clang_getSpellingLocation(end, nullptr, &end_line, nullptr, nullptr);
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    line_range.start_line = static_cast<int>(begin_line);
    line_range.end_line = static_cast<int>(end_line);
    return line_range;
}

static std::unique_ptr<AstNode> make_node(CXCursor cursor, AstNode* parent, int
    ↪ depth) {
    auto node = std::make_unique<AstNode>();
    node->type = convert_kind(clang_getCursorKind(cursor));
    node->role = AstNodeRole::kNone;
    node->raw_name = to_std_string(clang_getCursorSpelling(cursor));
    node->normalized_value = node->raw_name;
    node->usr = to_std_string(clang_getCursorUSR(cursor));
    node->line_range = extract_line_range(cursor);
    node->parent = parent;
    node->depth = depth;
    return node;
}

struct normalization_state {
    std::unordered_map<std::string, std::vector<std::string>> bindings;
    std::vector<std::vector<std::string>> scope_changes;
    std::vector<int> next_arg_stack;
    std::vector<int> next_var_stack;
};

static bool opens_scope(CXCursorKind kind) {
    if (kind == CXCursor_TranslationUnit) return true;
    if (kind == CXCursor_FunctionDecl) return true;
    if (kind == CXCursor_CompoundStmt) return true;
    if (kind == CXCursor_ForStmt) return true;

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    if (kind == CXCursor_WhileStmt) return true;
    if (kind == CXCursor_IfStmt) return true;
    return false;
}

static void push_scope(normalization_state& state) {
    state.scope_changes.emplace_back();
}

static void pop_scope(normalization_state& state) {
    if (state.scope_changes.empty()) {
        return;
    }

    auto declared = std::move(state.scope_changes.back());
    state.scope_changes.pop_back();

    for (const std::string& raw_name : declared) {
        auto it = state.bindings.find(raw_name);
        if (it == state.bindings.end() || it->second.empty()) {
            continue;
        }
        it->second.pop_back();
        if (it->second.empty()) {
            state.bindings.erase(it);
        }
    }
}

static void push_function_counters(normalization_state& state) {
    state.next_arg_stack.push_back(0);
    state.next_var_stack.push_back(0);
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
}

```

```
static void pop_function_counters(normalization_state& state) {
    if (!state.next_arg_stack.empty()) {
        state.next_arg_stack.pop_back();
    }
    if (!state.next_var_stack.empty()) {
        state.next_var_stack.pop_back();
    }
}

```

```
static std::string lookup_normalized_name(const normalization_state& state, const
↳ std::string& raw_name) {
    auto it = state.bindings.find(raw_name);
    if (it == state.bindings.end() || it->second.empty()) {
        return "ID_GLOBAL";
    }
    return it->second.back();
}

```

```
static std::string declare_symbol(normalization_state& state, const std::string&
↳ raw_name, bool is_param) {
    if (state.scope_changes.empty()) {
        push_scope(state);
    }

    std::string label;
    if (is_param) {
        if (state.next_arg_stack.empty()) {
            state.next_arg_stack.push_back(0);
        }
        label = "ARG_" + std::to_string(state.next_arg_stack.back()++);
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    } else {
        if (state.next_var_stack.empty()) {
            state.next_var_stack.push_back(0);
        }
        label = "VAR_" + std::to_string(state.next_var_stack.back()++);
    }

    state.bindings[raw_name].push_back(label);
    state.scope_changes.back().push_back(raw_name);
    return label;
}

static void normalize_node_value(CXCursorKind kind, AstNode& node,
    ↪ normalization_state& state) {
    if (kind == CXCursor_IntegerLiteral || kind == CXCursor_FloatingLiteral) {
        node.normalized_value = "NUM";
        return;
    }
    if (kind == CXCursor_StringLiteral) {
        node.normalized_value = "STR";
        return;
    }
    if (kind == CXCursor_ParmDecl) {
        node.normalized_value = declare_symbol(state, node.raw_name, true);
        return;
    }
    if (kind == CXCursor_VarDecl) {
        node.normalized_value = declare_symbol(state, node.raw_name, false);
        return;
    }
    if (kind == CXCursor_DeclRefExpr) {
        node.normalized_value = lookup_normalized_name(state, node.raw_name);
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        return;
    }
    if (kind == CXCursor_FunctionDecl) {
        node.normalized_value = "FUNC";
        return;
    }
    if (kind == CXCursor_CallExpr) {
        node.normalized_value = "CALL";
        return;
    }

    node.normalized_value = node.raw_name;
}

static int fill_subtree_size(AstNode* node) {
    int size = 1;
    for (auto& child : node->children) {
        size += fill_subtree_size(child.get());
    }
    node->subtree_size = size;
    return size;
}

struct traverse_data {
    AstNode* parent = nullptr;
    AstFeatures* features = nullptr;
    normalization_state* normalization = nullptr;
    int depth = 0;
};

static void dfs_collect(CXCursor cursor, traverse_data& data);

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

static enum CXChildVisitResult for_kids(
    CXCursor cursor,
    CXCursor parent,
    CXClientData client_data
) {
    (void)parent;

    auto* data = static_cast<traverse_data*>(client_data);
    dfs_collect(cursor, *data);

    return CXChildVisit_Continue;
}

static void dfs_collect(CXCursor cursor, traverse_data& data) {
    CXCursorKind kind = clang_getCursorKind(cursor);

    if (kind != CXCursor_TranslationUnit && !is_from_main_file(cursor)) {
        return;
    }

    if (is_transparent_wrapper_kind(kind)) {
        clang_visitChildren(cursor, for_kids, &data);
        return;
    }

    if (data.features != nullptr) {
        data.features->max_depth = std::max(data.features->max_depth, data.depth);

        if (is_interesting_kind(kind)) {
            std::string kind_name = get_kind_name(cursor);

            count_interesting_kind(kind, *data.features);
        }
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        data.features->preorder_kinds.push_back(kind_name);
        ++data.features->kind_freq[kind_name];
    }
}

bool opened_scope = false;
bool opened_function = false;
if (data.normalization != nullptr && opens_scope(kind)) {
    push_scope(*data.normalization);
    opened_scope = true;
    if (kind == CXCursor_FunctionDecl) {
        push_function_counters(*data.normalization);
        opened_function = true;
    }
}

auto node = make_node(cursor, data.parent, data.depth);
if (data.normalization != nullptr) {
    normalize_node_value(kind, *node, *data.normalization);
}

AstNode* current = node.get();

if (data.parent != nullptr) {
    data.parent->children.push_back(std::move(node));
}

traverse_data child_data;
child_data.parent = current;
child_data.features = data.features;
child_data.normalization = data.normalization;
child_data.depth = data.depth + 1;

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

clang_visitChildren(cursor, for_kids, &child_data);

if (data.normalization != nullptr && opened_scope) {
    if (opened_function) {
        pop_function_counters(*data.normalization);
    }
    pop_scope(*data.normalization);
}
}

struct parsed_unit {
    CXIndex index = nullptr;
    CXTranslationUnit translation_unit = nullptr;
};

static parsed_unit parse_code_to_unit(const std::string& code) {
    parsed_unit parsed;

    parsed.index = clang_createIndex(0, 0);
    const char* file_name = "input.cpp";

    CXUnsavedFile unsaved_file{};
    unsaved_file.Filename = file_name;
    unsaved_file.Contents = code.c_str();
    unsaved_file.Length = static_cast<unsigned long>(code.size());

    const char* args[] = {
        "-x", "c++",
        "-std=c++20"
    };

    CXErrorCode error_code = clang_parseTranslationUnit2(

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    parsed.index,
    file_name,
    args, 3,
    &unsaved_file, 1,
    CXTranslationUnit_None,
    &parsed.translation_unit
);

if (error_code != CXError_Success || parsed.translation_unit == nullptr) {
    if (parsed.translation_unit != nullptr) {
        clang_disposeTranslationUnit(parsed.translation_unit);
        parsed.translation_unit = nullptr;
    }
    clang_disposeIndex(parsed.index);
    parsed.index = nullptr;
}

return parsed;
}

AstFeatures analyze_ast(const std::string& code) {
    AstFeatures features;

    parsed_unit parsed = parse_code_to_unit(code);
    if (parsed.translation_unit == nullptr) {
        return features;
    }

    features.parse_ok = true;

    CXCursor root_cursor = clang_getTranslationUnitCursor(parsed.translation_unit);

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

AstNode scratch_root;
normalization_state normalization;
traverse_data data;
data.parent = &scratch_root;
data.features = &features;
data.normalization = &normalization;
data.depth = 0;
dfs_collect(root_cursor, data);

clang_disposeTranslationUnit(parsed.translation_unit);
clang_disposeIndex(parsed.index);

return features;
}

AstTree build_ast_tree(const std::string& code) {
    AstTree tree;

    parsed_unit parsed = parse_code_to_unit(code);
    if (parsed.translation_unit == nullptr) {
        return tree;
    }

    tree.parse_ok = true;

    CXCursor root_cursor = clang_getTranslationUnitCursor(parsed.translation_unit);
    tree.root = make_node(root_cursor, nullptr, 0);

    traverse_data child_data;
    child_data.parent = tree.root.get();
    child_data.features = nullptr;
    normalization_state normalization;

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

child_data.normalization = &normalization;
child_data.depth = 1;
clang_visitChildren(root_cursor, for_kids, &child_data);

fill_subtree_size(tree.root.get());

clang_disposeTranslationUnit(parsed.translation_unit);
clang_disposeIndex(parsed.index);

return tree;
}

std::pair<AstFeatures, AstTree> analyze_and_build_ast(const std::string& code) {
    AstFeatures features;
    AstTree tree;

    parsed_unit parsed = parse_code_to_unit(code);
    if (parsed.translation_unit == nullptr) {
        return {features, std::move(tree)};
    }

    features.parse_ok = true;
    tree.parse_ok = true;

    CXCursor root_cursor = clang_getTranslationUnitCursor(parsed.translation_unit);
    tree.root = make_node(root_cursor, nullptr, 0);

    traverse_data child_data;
    child_data.parent = tree.root.get();
    child_data.features = &features;
    normalization_state normalization;
    child_data.normalization = &normalization;

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    child_data.depth = 1;

    clang_visitChildren(root_cursor, for_kids, &child_data);

    fill_subtree_size(tree.root.get());

    clang_disposeTranslationUnit(parsed.translation_unit);
    clang_disposeIndex(parsed.index);

    return {features, std::move(tree)};
}

```

1.126. plagiarism\ast_ast_analyzer.h

Листинг 126: plagiarism\ast_ast_analyzer.h

```

#pragma once

#include <string>
#include "ast_features.h"
#include "ast_tree.h"

AstFeatures analyze_ast(const std::string& code);
AstTree build_ast_tree(const std::string& code);
std::pair<AstFeatures, AstTree> analyze_and_build_ast(const std::string& code);

```

1.127. plagiarism\ast_ast_features.h

Листинг 127: plagiarism\ast_ast_features.h

```

#pragma once

#include <string>

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

#include <unordered_map>
#include <vector>

struct AstFeatures {
    bool parse_ok = false;

    int functions_cnt = 0;
    int ifs_cnt = 0;
    int fors_cnt = 0;
    int whiles_cnt = 0;
    int returns_cnt = 0;
    int calls_cnt = 0;

    int max_depth = 0;

    std::unordered_map<std::string, int> kind_freq;
    std::vector<std::string> preorder_kinds;
};

```

1.128. plagiarism\ast_ast_hashing.cpp

Листинг 128: plagiarism\ast_ast_hashing.cpp

```

#include "ast_hashing.h"

#include <boost/functional/hash.hpp>

#include <cstdint>

std::string AstNodeTypeToString(AstNodeType type) {
    if (type == AstNodeType::kTranslationUnit) return "TranslationUnit";
    if (type == AstNodeType::kFunctionDecl) return "FunctionDecl";
    if (type == AstNodeType::kParamDecl) return "ParamDecl";
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

if (type == AstNodeType::kVarDecl) return "VarDecl";
if (type == AstNodeType::kCompoundStmt) return "CompoundStmt";
if (type == AstNodeType::kIfStmt) return "IfStmt";
if (type == AstNodeType::kForStmt) return "ForStmt";
if (type == AstNodeType::kWhileStmt) return "WhileStmt";
if (type == AstNodeType::kReturnStmt) return "ReturnStmt";
if (type == AstNodeType::kCallExpr) return "CallExpr";
if (type == AstNodeType::kBinaryOp) return "BinaryOp";
if (type == AstNodeType::kUnaryOp) return "UnaryOp";
if (type == AstNodeType::kDeclRefExpr) return "DeclRefExpr";
if (type == AstNodeType::kIntegerLiteral) return "IntegerLiteral";
if (type == AstNodeType::kFloatLiteral) return "FloatLiteral";
if (type == AstNodeType::kStringLiteral) return "StringLiteral";
return "Unknown";
}

```

```

std::string AstNodeRoleToString(AstNodeRole role) {
    if (role == AstNodeRole::kNone) return "None";
    if (role == AstNodeRole::kCondition) return "Condition";
    if (role == AstNodeRole::kBody) return "Body";
    if (role == AstNodeRole::kInit) return "Init";
    if (role == AstNodeRole::kUpdate) return "Update";
    if (role == AstNodeRole::kCallee) return "Callee";
    if (role == AstNodeRole::kArgument) return "Argument";
    if (role == AstNodeRole::kLeft) return "Left";
    if (role == AstNodeRole::kRight) return "Right";
    return "Unknown";
}

```

```

static std::uint64_t HashNodeParams(const AstNode* node) {
    std::size_t seed = 0;
    boost::hash_combine(seed, static_cast<int>(node->type));
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

boost::hash_combine(seed, static_cast<int>(node->role));

if (node->type == AstNodeType::kIntegerLiteral ||
    node->type == AstNodeType::kFloatLiteral ||
    node->type == AstNodeType::kStringLiteral) {
    boost::hash_combine(seed, "LITERAL");
} else if (node->type == AstNodeType::kDeclRefExpr) {
    boost::hash_combine(seed, "REF");
} else if (node->type == AstNodeType::kVarDecl || node->type == AstNodeType::
    ↪ kParamDecl) {
    boost::hash_combine(seed, "DECL");
}

return static_cast<std::uint64_t>(seed);
}

std::uint64_t ComputeAstSubtreeHash(
    const AstNode* node,
    std::unordered_map<std::uint64_t, int>& subtree_hash_freq,
    int min_subtree_size
) {
    if (node == nullptr) {
        return 0;
    }

    std::uint64_t seed = HashNodeParams(node);
    for (const auto& child : node->children) {
        const std::uint64_t child_hash = ComputeAstSubtreeHash(child.get(),
            ↪ subtree_hash_freq, min_subtree_size);
        boost::hash_combine(seed, child_hash);
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    bool is_deep_and_small = (node->depth > 8 && node->subtree_size < 5);
    if (node->subtree_size >= min_subtree_size && !is_deep_and_small) {
        subtree_hash_freq[seed]++;
    }
    return seed;
}

```

```

std::unordered_map<std::uint64_t, int> BuildAstSubtreeHashFreq(
    const AstTree& tree,
    int min_subtree_size
) {
    std::unordered_map<std::uint64_t, int> freq;
    if (!tree.parse_ok || tree.root == nullptr) {
        return freq;
    }
    ComputeAstSubtreeHash(tree.root.get(), freq, min_subtree_size);
    return freq;
}

```

```

static void CollectAstNormalizedSequencePreorder(
    const AstNode* node,
    std::vector<std::string>& ret,
    int min_subtree_size = 2
) {
    if (node == nullptr) {
        return;
    }

    if (node->subtree_size < min_subtree_size) {
        return;
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

ret.push_back(AstNodeTypeToString(node->type));

for (auto& child : node->children) {
    CollectAstNormalizedSequencePreorder(child.get(), ret, min_subtree_size);
}
}

std::vector<std::string> BuildAstNormalizedSequencePreorder(const AstTree& tree) {
    std::vector<std::string> ret;
    if (!tree.parse_ok || tree.root == nullptr) {
        return ret;
    }
    CollectAstNormalizedSequencePreorder(tree.root.get(), ret, 3);
    return ret;
}

```

1.129. plagiarism\{}ast\{}ast_hashing.h

Листинг 129: plagiarism\{}ast\{}ast_hashing.h

```

#pragma once

#include <cstdint>
#include <string>
#include <unordered_map>
#include <vector>

#include "ast_tree.h"

std::string AstNodeTypeToString(AstNodeType type);
std::string AstNodeRoleToString(AstNodeRole role);

std::uint64_t ComputeAstSubtreeHash(

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    const AstNode* node,
    std::unordered_map<std::uint64_t, int>& subtree_hash_freq,
    int min_subtree_size = 2
);

std::unordered_map<std::uint64_t, int> BuildAstSubtreeHashFreq(
    const AstTree& tree,
    int min_subtree_size = 2
);

std::vector<std::string> BuildAstNormalizedSequencePreorder(const AstTree& tree);

void CollectAstNormalizedSequencePreorder(
    const AstNode* node,
    std::vector<std::string>& ret
);

```

1.130. plagiarism\{}ast\{}ast_tree.h

Листинг 130: plagiarism\{}ast\{}ast_tree.h

```

#pragma once

#include <cstdint>
#include <memory>
#include <string>
#include <vector>

enum class AstNodeType {
    kTranslationUnit,
    kFunctionDecl,
    kParamDecl,
    kVarDecl,

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    kCompoundStmt,
    kIfStmt,
    kForStmt,
    kWhileStmt,
    kReturnStmt,
    kCallExpr,
    kBinaryOp,
    kUnaryOp,
    kDeclRefExpr,
    kIntegerLiteral,
    kFloatLiteral,
    kStringLiteral,
    kUnknown,
};

```

```

enum class AstNodeRole {
    kNone,
    kCondition,
    kBody,
    kInit,
    kUpdate,
    kCallee,
    kArgument,
    kLeft,
    kRight,
    kUnknown,
};

```

```

struct LineRange {
    int start_line = 0;
    int end_line = 0;
};

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
struct AstNode {  
    AstNodeType type = AstNodeType::kUnknown;  
    AstNodeRole role = AstNodeRole::kNone;  
  
    std::string raw_name;  
    std::string normalized_value;  
    std::string usr;  
  
    LineRange line_range;  
  
    AstNode* parent = nullptr;  
    std::vector<std::unique_ptr<AstNode>> children;  
  
    int depth = 0;  
    int subtree_size = 1;  
};  
  
struct AstTree {  
    bool parse_ok = false;  
    std::unique_ptr<AstNode> root;  
};
```

1.131. plagiarism\ast\call_graph.cpp

Листинг 131: plagiarism\ast\call_graph.cpp

```
#include "call_graph.h"  
  
#include <algorithm>  
#include <functional>  
  
namespace {
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

void CollectDeclaredFunctions(const AstNode* node, std::unordered_set<std::string
    ↪ >& declared) {
    if (node == nullptr) {
        return;
    }

    if (node->type == AstNodeType::kFunctionDecl && !node->raw_name.empty()) {
        declared.insert(node->raw_name);
    }

    for (const auto& child : node->children) {
        CollectDeclaredFunctions(child.get(), declared);
    }
}

void CollectCallsInSubtree(const AstNode* node, std::vector<std::string>& calls) {
    if (node == nullptr) {
        return;
    }

    if (node->type == AstNodeType::kCallExpr && !node->raw_name.empty()) {
        calls.push_back(node->raw_name);
    }

    for (const auto& child : node->children) {
        CollectCallsInSubtree(child.get(), calls);
    }
}

void CollectEdges(
    const AstNode* node,

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

const std::unordered_set<std::string>& declared,
std::unordered_map<std::string, int>& in_degree,
std::unordered_map<std::string, int>& out_degree,
std::unordered_map<std::uint64_t, int>& edge_hash_freq,
int& edges_cnt
) {
    if (node == nullptr) {
        return;
    }

    if (node->type == AstNodeType::kFunctionDecl && !node->raw_name.empty()) {
        const std::string& caller = node->raw_name;
        std::vector<std::string> calls;
        CollectCallsInSubtree(node, calls);

        for (const std::string& callee : calls) {
            if (declared.find(callee) == declared.end()) {
                continue;
            }

            ++edges_cnt;
            ++out_degree[caller];
            ++in_degree[callee];

            const std::uint64_t edge_hash = static_cast<std::uint64_t>(std::hash<std::
                ↳ ::string>{}(caller + "->" + callee));
            ++edge_hash_freq[edge_hash];
        }

        if (in_degree.find(caller) == in_degree.end()) {
            in_degree[caller] = 0;
        }
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        if (out_degree.find(caller) == out_degree.end()) {
            out_degree[caller] = 0;
        }
    }

    for (const auto& child : node->children) {
        CollectEdges(child.get(), declared, in_degree, out_degree, edge_hash_freq,
            ↪ edges_cnt);
    }
}

std::unordered_map<int, int> BuildDegreeHistogram(const std::unordered_map<std::
    ↪ string, int>& degree_by_function) {
    std::unordered_map<int, int> hist;
    for (const auto& [name, degree] : degree_by_function) {
        (void)name;
        ++hist[degree];
    }
    return hist;
}

} // namespace

CallGraphFeatures BuildCallGraphFeatures(const AstTree& tree) {
    CallGraphFeatures ret;
    if (!tree.parse_ok || tree.root == nullptr) {
        return ret;
    }

    std::unordered_set<std::string> declared;
    CollectDeclaredFunctions(tree.root.get(), declared);

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

ret.parse_ok = true;
ret.functions_cnt = static_cast<int>(declared.size());

std::unordered_map<std::string, int> in_degree;
std::unordered_map<std::string, int> out_degree;
CollectEdges(tree.root.get(), declared, in_degree, out_degree, ret.
    ↪ edge_hash_freq, ret.edges_cnt);

ret.in_degree_hist = BuildDegreeHistogram(in_degree);
ret.out_degree_hist = BuildDegreeHistogram(out_degree);
return ret;
}

```

1.132. plagiarism\ast\call_graph.h

Листинг 132: plagiarism\ast\call_graph.h

```

#pragma once

#include <cstdint>
#include <string>
#include <unordered_map>
#include <unordered_set>

#include "ast_tree.h"

struct CallGraphFeatures {
    bool parse_ok = false;
    int functions_cnt = 0;
    int edges_cnt = 0;

    std::unordered_map<std::uint64_t, int> edge_hash_freq;
    std::unordered_map<int, int> in_degree_hist;

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    std::unordered_map<int, int> out_degree_hist;
};

CallGraphFeatures BuildCallGraphFeatures(const AstTree& tree);

```

1.133. plagiarism\{}\cmake-build-debug\{}\CMakeFiles\{}\4.1.2\{}\CompilerIdCXX\{}\CMakeCXXCompilerId.cxx

Листинг 133: plagiarism\{}\cmake-build-debug\{}\CMakeFiles\{}\4.1.2\{}\CompilerIdCXX\{}\CMakeCXXCompilerId.cxx

```

/* This source file must have a .cpp extension so that all C++ compilers
   recognize the extension without flags. Borland does not know .cxx for
   example. */

#ifndef __cplusplus
# error "A C compiler has been selected for C++."
#endif

#if !defined(__has_include)
/* If the compiler does not have __has_include, pretend the answer is
   always no. */
# define __has_include(x) 0
#endif

/* Version number components: V=Version, R=Revision, P=Patch
   Version date components: YYYY=Year, MM=Month, DD=Day */

#if defined(__INTEL_COMPILER) || defined(__ICC)
# define COMPILER_ID "Intel"
# if defined(_MSC_VER)
# define SIMULATE_ID "MSVC"
# endif
# endif

# if defined(__GNUC__)
# define SIMULATE_ID "GNU"

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# endif

/* __INTEL_COMPILER = VRR prior to 2021, and then VVVV for 2021 and later,
   except that a few beta releases use the old format with V=2021. */
# if __INTEL_COMPILER < 2021 || __INTEL_COMPILER == 202110 || __INTEL_COMPILER ==
    ↪ 202111
# define COMPILER_VERSION_MAJOR DEC(__INTEL_COMPILER/100)
# define COMPILER_VERSION_MINOR DEC(__INTEL_COMPILER/10 % 10)
# if defined(__INTEL_COMPILER_UPDATE)
# define COMPILER_VERSION_PATCH DEC(__INTEL_COMPILER_UPDATE)
# else
# define COMPILER_VERSION_PATCH DEC(__INTEL_COMPILER % 10)
# endif
# else
# define COMPILER_VERSION_MAJOR DEC(__INTEL_COMPILER)
# define COMPILER_VERSION_MINOR DEC(__INTEL_COMPILER_UPDATE)
/* The third version component from --version is an update index,
   but no macro is provided for it. */
# define COMPILER_VERSION_PATCH DEC(0)
# endif
# if defined(__INTEL_COMPILER_BUILD_DATE)
/* __INTEL_COMPILER_BUILD_DATE = YYYYMMDD */
# define COMPILER_VERSION_TWEAK DEC(__INTEL_COMPILER_BUILD_DATE)
# endif
# if defined(_MSC_VER)
/* _MSC_VER = VVRR */
# define SIMULATE_VERSION_MAJOR DEC(_MSC_VER / 100)
# define SIMULATE_VERSION_MINOR DEC(_MSC_VER % 100)
# endif
# if defined(__GNUC__)
# define SIMULATE_VERSION_MAJOR DEC(__GNUC__)
# elif defined(__GNUG__)
# define SIMULATE_VERSION_MAJOR DEC(__GNUG__)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# endif
# if defined(__GNUC_MINOR__)
# define SIMULATE_VERSION_MINOR DEC(__GNUC_MINOR__)
# endif
# if defined(__GNUC_PATCHLEVEL__)
# define SIMULATE_VERSION_PATCH DEC(__GNUC_PATCHLEVEL__)
# endif

#elif (defined(__clang__) && defined(__INTEL_CLANG_COMPILER)) || defined(
    ↪ __INTEL_LLVM_COMPILER)
# define COMPILER_ID "IntelLLVM"
#if defined(_MSC_VER)
# define SIMULATE_ID "MSVC"
#endif
#if defined(__GNUC__)
# define SIMULATE_ID "GNU"
#endif
/* __INTEL_LLVM_COMPILER = VVVVRP prior to 2021.2.0, VVVVRRPP for 2021.2.0 and
 * later. Look for 6 digit vs. 8 digit version number to decide encoding.
 * VVVV is no smaller than the current year when a version is released.
 */
#if __INTEL_LLVM_COMPILER < 1000000L
# define COMPILER_VERSION_MAJOR DEC(__INTEL_LLVM_COMPILER/100)
# define COMPILER_VERSION_MINOR DEC(__INTEL_LLVM_COMPILER/10 % 10)
# define COMPILER_VERSION_PATCH DEC(__INTEL_LLVM_COMPILER % 10)
#else
# define COMPILER_VERSION_MAJOR DEC(__INTEL_LLVM_COMPILER/10000)
# define COMPILER_VERSION_MINOR DEC(__INTEL_LLVM_COMPILER/100 % 100)
# define COMPILER_VERSION_PATCH DEC(__INTEL_LLVM_COMPILER % 100)
#endif
#if defined(_MSC_VER)
    /* _MSC_VER = VVRR */

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# define SIMULATE_VERSION_MAJOR DEC(_MSC_VER / 100)
# define SIMULATE_VERSION_MINOR DEC(_MSC_VER % 100)
#endif

#if defined(__GNUC__)
# define SIMULATE_VERSION_MAJOR DEC(__GNUC__)
#elif defined(__GNUG__)
# define SIMULATE_VERSION_MAJOR DEC(__GNUG__)
#endif

#if defined(__GNUC_MINOR__)
# define SIMULATE_VERSION_MINOR DEC(__GNUC_MINOR__)
#endif

#if defined(__GNUC_PATCHLEVEL__)
# define SIMULATE_VERSION_PATCH DEC(__GNUC_PATCHLEVEL__)
#endif

#elif defined(__PATHCC__)
# define COMPILER_ID "PathScale"
# define COMPILER_VERSION_MAJOR DEC(__PATHCC__)
# define COMPILER_VERSION_MINOR DEC(__PATHCC_MINOR__)
# if defined(__PATHCC_PATCHLEVEL__)
# define COMPILER_VERSION_PATCH DEC(__PATHCC_PATCHLEVEL__)
# endif

#elif defined(__BORLANDC__) && defined(__CODEGEARC_VERSION__)
# define COMPILER_ID "Embarcadero"
# define COMPILER_VERSION_MAJOR HEX(__CODEGEARC_VERSION__>>24 & 0x00FF)
# define COMPILER_VERSION_MINOR HEX(__CODEGEARC_VERSION__>>16 & 0x00FF)
# define COMPILER_VERSION_PATCH DEC(__CODEGEARC_VERSION__ & 0xFFFF)

#elif defined(__BORLANDC__)
# define COMPILER_ID "Borland"

/* __BORLANDC__ = 0xVRR */

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# define COMPILER_VERSION_MAJOR HEX(__BORLANDC__>>8)
# define COMPILER_VERSION_MINOR HEX(__BORLANDC__ & 0xFF)

#elif defined(__WATCOMC__) && __WATCOMC__ < 1200
# define COMPILER_ID "Watcom"
    /* __WATCOMC__ = VVRR */
# define COMPILER_VERSION_MAJOR DEC(__WATCOMC__ / 100)
# define COMPILER_VERSION_MINOR DEC((__WATCOMC__ / 10) % 10)
# if (__WATCOMC__ % 10) > 0
# define COMPILER_VERSION_PATCH DEC(__WATCOMC__ % 10)
# endif

#elif defined(__WATCOMC__)
# define COMPILER_ID "OpenWatcom"
    /* __WATCOMC__ = VVRP + 1100 */
# define COMPILER_VERSION_MAJOR DEC((__WATCOMC__ - 1100) / 100)
# define COMPILER_VERSION_MINOR DEC((__WATCOMC__ / 10) % 10)
# if (__WATCOMC__ % 10) > 0
# define COMPILER_VERSION_PATCH DEC(__WATCOMC__ % 10)
# endif

#elif defined(__SUNPRO_CC)
# define COMPILER_ID "SunPro"
# if __SUNPRO_CC >= 0x5100
    /* __SUNPRO_CC = 0xVRRP */
# define COMPILER_VERSION_MAJOR HEX(__SUNPRO_CC>>12)
# define COMPILER_VERSION_MINOR HEX(__SUNPRO_CC>>4 & 0xFF)
# define COMPILER_VERSION_PATCH HEX(__SUNPRO_CC & 0xF)
# else
    /* __SUNPRO_CC = 0xVRP */
# define COMPILER_VERSION_MAJOR HEX(__SUNPRO_CC>>8)
# define COMPILER_VERSION_MINOR HEX(__SUNPRO_CC>>4 & 0xF)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

# define COMPILER_VERSION_PATCH HEX(__SUNPRO_CC & 0xF)
# endif

#elif defined(__HP_aCC)
# define COMPILER_ID "HP"
    /* __HP_aCC = VVRRPP */
# define COMPILER_VERSION_MAJOR DEC(__HP_aCC/10000)
# define COMPILER_VERSION_MINOR DEC(__HP_aCC/100 % 100)
# define COMPILER_VERSION_PATCH DEC(__HP_aCC % 100)

#elif defined(__DECCXX)
# define COMPILER_ID "Compaq"
    /* __DECCXX_VER = VVRRTPPPP */
# define COMPILER_VERSION_MAJOR DEC(__DECCXX_VER/10000000)
# define COMPILER_VERSION_MINOR DEC(__DECCXX_VER/100000 % 100)
# define COMPILER_VERSION_PATCH DEC(__DECCXX_VER % 10000)

#elif defined(__IBMCPP__) && defined(__COMPILER_VER__)
# define COMPILER_ID "zOS"
    /* __IBMCPP__ = VRP */
# define COMPILER_VERSION_MAJOR DEC(__IBMCPP__/100)
# define COMPILER_VERSION_MINOR DEC(__IBMCPP__/10 % 10)
# define COMPILER_VERSION_PATCH DEC(__IBMCPP__ % 10)

#elif defined(__open_xl__) && defined(__clang__)
# define COMPILER_ID "IBMClang"
# define COMPILER_VERSION_MAJOR DEC(__open_xl_version__)
# define COMPILER_VERSION_MINOR DEC(__open_xl_release__)
# define COMPILER_VERSION_PATCH DEC(__open_xl_modification__)
# define COMPILER_VERSION_TWEAK DEC(__open_xl_ptf_fix_level__)
# define COMPILER_VERSION_INTERNAL_STR __clang_version__

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

#elif defined(__ibmxl__) && defined(__clang__)
# define COMPILER_ID "XLClang"
# define COMPILER_VERSION_MAJOR DEC(__ibmxl_version__)
# define COMPILER_VERSION_MINOR DEC(__ibmxl_release__)
# define COMPILER_VERSION_PATCH DEC(__ibmxl_modification__)
# define COMPILER_VERSION_TWEAK DEC(__ibmxl_ptf_fix_level__)

#elif defined(__IBMCPP__) && !defined(__COMPILER_VER__) && __IBMCPP__ >= 800
# define COMPILER_ID "XL"
/* __IBMCPP__ = VRP */
# define COMPILER_VERSION_MAJOR DEC(__IBMCPP__/100)
# define COMPILER_VERSION_MINOR DEC(__IBMCPP__/10 % 10)
# define COMPILER_VERSION_PATCH DEC(__IBMCPP__ % 10)

#elif defined(__IBMCPP__) && !defined(__COMPILER_VER__) && __IBMCPP__ < 800
# define COMPILER_ID "VisualAge"
/* __IBMCPP__ = VRP */
# define COMPILER_VERSION_MAJOR DEC(__IBMCPP__/100)
# define COMPILER_VERSION_MINOR DEC(__IBMCPP__/10 % 10)
# define COMPILER_VERSION_PATCH DEC(__IBMCPP__ % 10)

#elif defined(__NVCOMPILER)
# define COMPILER_ID "NVHPC"
# define COMPILER_VERSION_MAJOR DEC(__NVCOMPILER_MAJOR__)
# define COMPILER_VERSION_MINOR DEC(__NVCOMPILER_MINOR__)
# if defined(__NVCOMPILER_PATCHLEVEL__)
# define COMPILER_VERSION_PATCH DEC(__NVCOMPILER_PATCHLEVEL__)
# endif

#elif defined(__PGI)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# define COMPILER_ID "PGI"
# define COMPILER_VERSION_MAJOR DEC(__PGIC__)
# define COMPILER_VERSION_MINOR DEC(__PGIC_MINOR__)
# if defined(__PGIC_PATCHLEVEL__)
# define COMPILER_VERSION_PATCH DEC(__PGIC_PATCHLEVEL__)
# endif

#elif defined(__clang__) && defined(__cray__)
# define COMPILER_ID "CrayClang"
# define COMPILER_VERSION_MAJOR DEC(__cray_major__)
# define COMPILER_VERSION_MINOR DEC(__cray_minor__)
# define COMPILER_VERSION_PATCH DEC(__cray_patchlevel__)
# define COMPILER_VERSION_INTERNAL_STR __clang_version__

#elif defined(_CRAYC)
# define COMPILER_ID "Cray"
# define COMPILER_VERSION_MAJOR DEC(_RELEASE_MAJOR)
# define COMPILER_VERSION_MINOR DEC(_RELEASE_MINOR)

#elif defined(__TI_COMPILER_VERSION__)
# define COMPILER_ID "TI"
/* __TI_COMPILER_VERSION__ = VVVRRRRPPP */
# define COMPILER_VERSION_MAJOR DEC(__TI_COMPILER_VERSION__/1000000)
# define COMPILER_VERSION_MINOR DEC(__TI_COMPILER_VERSION__/1000 % 1000)
# define COMPILER_VERSION_PATCH DEC(__TI_COMPILER_VERSION__ % 1000)

#elif defined(__CLANG_FUJITSU)
# define COMPILER_ID "FujitsuClang"
# define COMPILER_VERSION_MAJOR DEC(__FCC_major__)
# define COMPILER_VERSION_MINOR DEC(__FCC_minor__)
# define COMPILER_VERSION_PATCH DEC(__FCC_patchlevel__)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# define COMPILER_VERSION_INTERNAL_STR __clang_version__

#elif defined(__FUJITSU)
# define COMPILER_ID "Fujitsu"
# if defined(__FCC_version__)
# define COMPILER_VERSION __FCC_version__
# elif defined(__FCC_major__)
# define COMPILER_VERSION_MAJOR DEC(__FCC_major__)
# define COMPILER_VERSION_MINOR DEC(__FCC_minor__)
# define COMPILER_VERSION_PATCH DEC(__FCC_patchlevel__)
# endif
# if defined(__fcc_version)
# define COMPILER_VERSION_INTERNAL DEC(__fcc_version)
# elif defined(__FCC_VERSION)
# define COMPILER_VERSION_INTERNAL DEC(__FCC_VERSION)
# endif

#elif defined(__ghs__)
# define COMPILER_ID "GHS"
/* __GHS_VERSION_NUMBER = VVVVRP */
# ifdef __GHS_VERSION_NUMBER
# define COMPILER_VERSION_MAJOR DEC(__GHS_VERSION_NUMBER / 100)
# define COMPILER_VERSION_MINOR DEC(__GHS_VERSION_NUMBER / 10 % 10)
# define COMPILER_VERSION_PATCH DEC(__GHS_VERSION_NUMBER % 10)
# endif

#elif defined(__TASKING__)
# define COMPILER_ID "Tasking"
# define COMPILER_VERSION_MAJOR DEC(__VERSION__/1000)
# define COMPILER_VERSION_MINOR DEC(__VERSION__ % 100)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# define COMPILER_VERSION_INTERNAL DEC(__VERSION__)

#elif defined(__ORANGEC__)
# define COMPILER_ID "OrangeC"
# define COMPILER_VERSION_MAJOR DEC(__ORANGEC_MAJOR__)
# define COMPILER_VERSION_MINOR DEC(__ORANGEC_MINOR__)
# define COMPILER_VERSION_PATCH DEC(__ORANGEC_PATCHLEVEL__)

#elif defined(__RENESAS__)
# define COMPILER_ID "Renesas"
/* __RENESAS_VERSION__ = 0xVVRPP00 */
# define COMPILER_VERSION_MAJOR HEX(__RENESAS_VERSION__ >> 24 & 0xFF)
# define COMPILER_VERSION_MINOR HEX(__RENESAS_VERSION__ >> 16 & 0xFF)
# define COMPILER_VERSION_PATCH HEX(__RENESAS_VERSION__ >> 8 & 0xFF)

#elif defined(__SCO_VERSION__)
# define COMPILER_ID "SCO"

#elif defined(__ARMCC_VERSION) && !defined(__clang__)
# define COMPILER_ID "ARMCC"
#if __ARMCC_VERSION >= 1000000
/* __ARMCC_VERSION = VRRPPPP */
# define COMPILER_VERSION_MAJOR DEC(__ARMCC_VERSION/1000000)
# define COMPILER_VERSION_MINOR DEC(__ARMCC_VERSION/10000 % 100)
# define COMPILER_VERSION_PATCH DEC(__ARMCC_VERSION % 10000)
#else
/* __ARMCC_VERSION = VRPPPP */
# define COMPILER_VERSION_MAJOR DEC(__ARMCC_VERSION/100000)
# define COMPILER_VERSION_MINOR DEC(__ARMCC_VERSION/10000 % 10)
# define COMPILER_VERSION_PATCH DEC(__ARMCC_VERSION % 10000)
#endif
#endif

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
#elif defined(__clang__) && defined(__apple_build_version__)
# define COMPILER_ID "AppleClang"
# if defined(_MSC_VER)
# define SIMULATE_ID "MSVC"
# endif
# define COMPILER_VERSION_MAJOR DEC(__clang_major__)
# define COMPILER_VERSION_MINOR DEC(__clang_minor__)
# define COMPILER_VERSION_PATCH DEC(__clang_patchlevel__)
# if defined(_MSC_VER)
    /* _MSC_VER = VVRR */
# define SIMULATE_VERSION_MAJOR DEC(_MSC_VER / 100)
# define SIMULATE_VERSION_MINOR DEC(_MSC_VER % 100)
# endif
# define COMPILER_VERSION_TWEAK DEC(__apple_build_version__)

#elif defined(__clang__) && defined(__ARMCOMPILER_VERSION)
# define COMPILER_ID "ARMClang"
# define COMPILER_VERSION_MAJOR DEC(__ARMCOMPILER_VERSION/1000000)
# define COMPILER_VERSION_MINOR DEC(__ARMCOMPILER_VERSION/10000 % 100)
# define COMPILER_VERSION_PATCH DEC(__ARMCOMPILER_VERSION/100 % 100)
# define COMPILER_VERSION_INTERNAL DEC(__ARMCOMPILER_VERSION)

#elif defined(__clang__) && defined(__ti__)
# define COMPILER_ID "TIClang"
# define COMPILER_VERSION_MAJOR DEC(__ti_major__)
# define COMPILER_VERSION_MINOR DEC(__ti_minor__)
# define COMPILER_VERSION_PATCH DEC(__ti_patchlevel__)
# define COMPILER_VERSION_INTERNAL DEC(__ti_version__)

#elif defined(__clang__)
# define COMPILER_ID "Clang"
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# if defined(_MSC_VER)
# define SIMULATE_ID "MSVC"
# endif

# define COMPILER_VERSION_MAJOR DEC(__clang_major__)
# define COMPILER_VERSION_MINOR DEC(__clang_minor__)
# define COMPILER_VERSION_PATCH DEC(__clang_patchlevel__)
# if defined(_MSC_VER)
    /* _MSC_VER = VVRR */
# define SIMULATE_VERSION_MAJOR DEC(_MSC_VER / 100)
# define SIMULATE_VERSION_MINOR DEC(_MSC_VER % 100)
# endif

#elif defined(__LCC__) && (defined(__GNUC__) || defined(__GNUG__) || defined(
    ↪ __MCST__))
# define COMPILER_ID "LCC"
# define COMPILER_VERSION_MAJOR DEC(__LCC__ / 100)
# define COMPILER_VERSION_MINOR DEC(__LCC__ % 100)
# if defined(__LCC_MINOR__)
# define COMPILER_VERSION_PATCH DEC(__LCC_MINOR__)
# endif
# if defined(__GNUC__) && defined(__GNUC_MINOR__)
# define SIMULATE_ID "GNU"
# define SIMULATE_VERSION_MAJOR DEC(__GNUC__)
# define SIMULATE_VERSION_MINOR DEC(__GNUC_MINOR__)
# if defined(__GNUC_PATCHLEVEL__)
# define SIMULATE_VERSION_PATCH DEC(__GNUC_PATCHLEVEL__)
# endif
# endif

#elif defined(__GNUC__) || defined(__GNUG__)
# define COMPILER_ID "GNU"
# if defined(__GNUC__)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# define COMPILER_VERSION_MAJOR DEC(__GNUC__)
# else
# define COMPILER_VERSION_MAJOR DEC(__GNUG__)
# endif

# if defined(__GNUC_MINOR__)
# define COMPILER_VERSION_MINOR DEC(__GNUC_MINOR__)
# endif

# if defined(__GNUC_PATCHLEVEL__)
# define COMPILER_VERSION_PATCH DEC(__GNUC_PATCHLEVEL__)
# endif

#elif defined(_MSC_VER)
# define COMPILER_ID "MSVC"
    /* _MSC_VER = VVRR */
# define COMPILER_VERSION_MAJOR DEC(_MSC_VER / 100)
# define COMPILER_VERSION_MINOR DEC(_MSC_VER % 100)
# if defined(_MSC_FULL_VER)
# if _MSC_VER >= 1400
    /* _MSC_FULL_VER = VVRRPPPP */
# define COMPILER_VERSION_PATCH DEC(_MSC_FULL_VER % 100000)
# else
    /* _MSC_FULL_VER = VVRRPPPP */
# define COMPILER_VERSION_PATCH DEC(_MSC_FULL_VER % 10000)
# endif
# endif
# endif

# if defined(_MSC_BUILD)
# define COMPILER_VERSION_TWEAK DEC(_MSC_BUILD)
# endif

#elif defined(_ADI_COMPILER)
# define COMPILER_ID "ADSP"
#endif

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

/* __VERSIONNUM__ = 0xVRRRPPTT */
# define COMPILER_VERSION_MAJOR DEC(__VERSIONNUM__ >> 24 & 0xFF)
# define COMPILER_VERSION_MINOR DEC(__VERSIONNUM__ >> 16 & 0xFF)
# define COMPILER_VERSION_PATCH DEC(__VERSIONNUM__ >> 8 & 0xFF)
# define COMPILER_VERSION_TWEAK DEC(__VERSIONNUM__ & 0xFF)
#endif

#elif defined(__IAR_SYSTEMS_ICC__) || defined(__IAR_SYSTEMS_ICC)
# define COMPILER_ID "IAR"
# if defined(__VER__) && defined(__ICCARM__)
# define COMPILER_VERSION_MAJOR DEC((__VER__) / 1000000)
# define COMPILER_VERSION_MINOR DEC(((__VER__) / 1000) % 1000)
# define COMPILER_VERSION_PATCH DEC((__VER__) % 1000)
# define COMPILER_VERSION_INTERNAL DEC(__IAR_SYSTEMS_ICC__)
# elif defined(__VER__) && (defined(__ICCAVR__) || defined(__ICCRX__) || defined(
    ↪ __ICCRH850__) || defined(__ICCRL78__) || defined(__ICC430__) || defined(
    ↪ __ICCRISCV__) || defined(__ICCV850__) || defined(__ICC8051__) || defined(
    ↪ __ICCSTM8__))
# define COMPILER_VERSION_MAJOR DEC((__VER__) / 100)
# define COMPILER_VERSION_MINOR DEC((__VER__) - (((__VER__) / 100)*100))
# define COMPILER_VERSION_PATCH DEC(__SUBVERSION__)
# define COMPILER_VERSION_INTERNAL DEC(__IAR_SYSTEMS_ICC__)
# endif

#elif defined(__DCC__) && defined(_DIAB_TOOL)
# define COMPILER_ID "Diab"
# define COMPILER_VERSION_MAJOR DEC(__VERSION_MAJOR_NUMBER__)
# define COMPILER_VERSION_MINOR DEC(__VERSION_MINOR_NUMBER__)
# define COMPILER_VERSION_PATCH DEC(__VERSION_ARCH_FEATURE_NUMBER__)
# define COMPILER_VERSION_TWEAK DEC(__VERSION_BUG_FIX_NUMBER__)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

/* These compilers are either not known or too old to define an
   identification macro. Try to identify the platform and guess that
   it is the native compiler. */
#elif defined(__hpux) || defined(__hpua)
# define COMPILER_ID "HP"

#else /* unknown compiler */
# define COMPILER_ID ""
#endif

/* Construct the string literal in pieces to prevent the source from
   getting matched. Store it in a pointer rather than an array
   because some compilers will just produce instructions to fill the
   array rather than assigning a pointer to a static array. */
char const* info_compiler = "INFO" ":" "compiler[" COMPILER_ID "];
#ifdef SIMULATE_ID
char const* info_simulate = "INFO" ":" "simulate[" SIMULATE_ID "];
#endif

#ifdef __QNXNTO__
char const* qnxnto = "INFO" ":" "qnxnto[]";
#endif

#ifdef __CRAYXT_COMPUTE_LINUX_TARGET
char const *info_cray = "INFO" ":" "compiler_wrapper[CrayPrgEnv]";
#endif

#define STRINGIFY_HELPER(X) #X
#define STRINGIFY(X) STRINGIFY_HELPER(X)

/* Identify known platforms by name. */

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

#if defined(__linux) || defined(__linux__) || defined(linux)
# define PLATFORM_ID "Linux"

#elif defined(__MSYS__)
# define PLATFORM_ID "MSYS"

#elif defined(__CYGWIN__)
# define PLATFORM_ID "Cygwin"

#elif defined(__MINGW32__)
# define PLATFORM_ID "MinGW"

#elif defined(__APPLE__)
# define PLATFORM_ID "Darwin"

#elif defined(_WIN32) || defined(__WIN32__) || defined(WIN32)
# define PLATFORM_ID "Windows"

#elif defined(__FreeBSD__) || defined(__FreeBSD)
# define PLATFORM_ID "FreeBSD"

#elif defined(__NetBSD__) || defined(__NetBSD)
# define PLATFORM_ID "NetBSD"

#elif defined(__OpenBSD__) || defined(__OPENBSD)
# define PLATFORM_ID "OpenBSD"

#elif defined(__sun) || defined(sun)
# define PLATFORM_ID "SunOS"

#elif defined(_AIX) || defined(__AIX) || defined(__AIX__) || defined(__aix) ||
    ↪ defined(__aix__)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# define PLATFORM_ID "AIX"

#elif defined(__hpux) || defined(__hpux__)
# define PLATFORM_ID "HP-UX"

#elif defined(__HAIKU__)
# define PLATFORM_ID "Haiku"

#elif defined(__BeOS) || defined(__BEOS__) || defined(_BEOS)
# define PLATFORM_ID "BeOS"

#elif defined(__QNX__) || defined(__QNXNTO__)
# define PLATFORM_ID "QNX"

#elif defined(__tru64) || defined(_tru64) || defined(__TRU64__)
# define PLATFORM_ID "Tru64"

#elif defined(__riscos) || defined(__riscos__)
# define PLATFORM_ID "RISCos"

#elif defined(__sinix) || defined(__sinix__) || defined(__SINIX__)
# define PLATFORM_ID "SINIX"

#elif defined(__UNIX_SV__)
# define PLATFORM_ID "UNIX_SV"

#elif defined(__bsdos__)
# define PLATFORM_ID "BSDOS"

#elif defined(_MPRAS) || defined(MPRAS)
# define PLATFORM_ID "MP-RAS"

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

#elif defined(__osf) || defined(__osf__)
# define PLATFORM_ID "OSF1"

#elif defined(_SCO_SV) || defined(SCO_SV) || defined(sco_sv)
# define PLATFORM_ID "SCO_SV"

#elif defined(__ultrix) || defined(__ultrix__) || defined(_ULTRIX)
# define PLATFORM_ID "ULTRIX"

#elif defined(__XENIX__) || defined(_XENIX) || defined(XENIX)
# define PLATFORM_ID "Xenix"

#elif defined(__WATCOMC__)
# if defined(__LINUX__)
# define PLATFORM_ID "Linux"

# elif defined(__DOS__)
# define PLATFORM_ID "DOS"

# elif defined(__OS2__)
# define PLATFORM_ID "OS2"

# elif defined(__WINDOWS__)
# define PLATFORM_ID "Windows3x"

# elif defined(__VXWORKS__)
# define PLATFORM_ID "VxWorks"

# else /* unknown platform */
# define PLATFORM_ID
# endif

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

#elif defined(__INTEGRITY)
# if defined(INT_178B)
# define PLATFORM_ID "Integrity178"

# else /* regular Integrity */
# define PLATFORM_ID "Integrity"
# endif

# elif defined(_ADI_COMPILER)
# define PLATFORM_ID "ADSP"

#else /* unknown platform */
# define PLATFORM_ID

#endif

/* For windows compilers MSVC and Intel we can determine
the architecture of the compiler being used. This is because
the compilers do not have flags that can change the architecture,
but rather depend on which compiler is being used
*/
#if defined(_WIN32) && defined(_MSC_VER)
# if defined(_M_IA64)
# define ARCHITECTURE_ID "IA64"

# elif defined(_M_ARM64EC)
# define ARCHITECTURE_ID "ARM64EC"

# elif defined(_M_X64) || defined(_M_AMD64)
# define ARCHITECTURE_ID "x64"

# elif defined(_M_IX86)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# define ARCHITECTURE_ID "X86"

# elif defined(_M_ARM64)
# define ARCHITECTURE_ID "ARM64"

# elif defined(_M_ARM)
# if _M_ARM == 4
# define ARCHITECTURE_ID "ARMV4I"
# elif _M_ARM == 5
# define ARCHITECTURE_ID "ARMV5I"
# else
# define ARCHITECTURE_ID "ARMV" STRINGIFY(_M_ARM)
# endif

# elif defined(_M_MIPS)
# define ARCHITECTURE_ID "MIPS"

# elif defined(_M_SH)
# define ARCHITECTURE_ID "SHx"

# else /* unknown architecture */
# define ARCHITECTURE_ID ""
# endif

#elif defined(__WATCOMC__)
# if defined(_M_I86)
# define ARCHITECTURE_ID "I86"

# elif defined(_M_I86)
# define ARCHITECTURE_ID "X86"

# else /* unknown architecture */

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# define ARCHITECTURE_ID ""
# endif

#elif defined(__IAR_SYSTEMS_ICC__) || defined(__IAR_SYSTEMS_ICC)
# if defined(__ICCARM__)
# define ARCHITECTURE_ID "ARM"

# elif defined(__ICCRX__)
# define ARCHITECTURE_ID "RX"

# elif defined(__ICCRH850__)
# define ARCHITECTURE_ID "RH850"

# elif defined(__ICCRL78__)
# define ARCHITECTURE_ID "RL78"

# elif defined(__ICCRISCV__)
# define ARCHITECTURE_ID "RISCV"

# elif defined(__ICCAVR__)
# define ARCHITECTURE_ID "AVR"

# elif defined(__ICC430__)
# define ARCHITECTURE_ID "MSP430"

# elif defined(__ICCV850__)
# define ARCHITECTURE_ID "V850"

# elif defined(__ICC8051__)
# define ARCHITECTURE_ID "8051"

# elif defined(__ICCSTM8__)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

# define ARCHITECTURE_ID "STM8"

# else /* unknown architecture */
# define ARCHITECTURE_ID ""
# endif

#elif defined(__ghs__)
# if defined(__PPC64__)
# define ARCHITECTURE_ID "PPC64"

# elif defined(__ppc__)
# define ARCHITECTURE_ID "PPC"

# elif defined(__ARM__)
# define ARCHITECTURE_ID "ARM"

# elif defined(__x86_64__)
# define ARCHITECTURE_ID "x64"

# elif defined(__i386__)
# define ARCHITECTURE_ID "X86"

# else /* unknown architecture */
# define ARCHITECTURE_ID ""
# endif

#elif defined(__clang__) && defined(__ti__)
# if defined(__ARM_ARCH)
# define ARCHITECTURE_ID "ARM"

# else /* unknown architecture */
# define ARCHITECTURE_ID ""

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# endif

#elif defined(__TI_COMPILER_VERSION__)
# if defined(__TI_ARM__)
# define ARCHITECTURE_ID "ARM"

# elif defined(__MSP430__)
# define ARCHITECTURE_ID "MSP430"

# elif defined(__TMS320C28XX__)
# define ARCHITECTURE_ID "TMS320C28x"

# elif defined(__TMS320C6X__) || defined(_TMS320C6X)
# define ARCHITECTURE_ID "TMS320C6x"

# else /* unknown architecture */
# define ARCHITECTURE_ID ""
# endif

# elif defined(__ADSPSHARC__)
# define ARCHITECTURE_ID "SHARC"

# elif defined(__ADSPBLACKFIN__)
# define ARCHITECTURE_ID "Blackfin"

#elif defined(__TASKING__)

# if defined(__CTC__) || defined(__CPTC__)
# define ARCHITECTURE_ID "TriCore"

# elif defined(__CMCS__)
# define ARCHITECTURE_ID "MCS"

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
# elif defined(__CARM__) || defined(__CPARM__)
```

```
# define ARCHITECTURE_ID "ARM"
```

```
# elif defined(__CARC__)
```

```
# define ARCHITECTURE_ID "ARC"
```

```
# elif defined(__C51__)
```

```
# define ARCHITECTURE_ID "8051"
```

```
# elif defined(__CPCP__)
```

```
# define ARCHITECTURE_ID "PCP"
```

```
# else
```

```
# define ARCHITECTURE_ID ""
```

```
# endif
```

```
#elif defined(__RENESAS__)
```

```
# if defined(__CCR__)
```

```
# define ARCHITECTURE_ID "RX"
```

```
# elif defined(__CCRL__)
```

```
# define ARCHITECTURE_ID "RL78"
```

```
# elif defined(__CCR__)
```

```
# define ARCHITECTURE_ID "RH850"
```

```
# else
```

```
# define ARCHITECTURE_ID ""
```

```
# endif
```

```
#else
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# define ARCHITECTURE_ID
#endif

/* Convert integer to decimal digit literals. */
#define DEC(n) \
    ('0' + (((n) / 10000000)%10)), \
    ('0' + (((n) / 1000000)%10)), \
    ('0' + (((n) / 100000)%10)), \
    ('0' + (((n) / 10000)%10)), \
    ('0' + (((n) / 1000)%10)), \
    ('0' + (((n) / 100)%10)), \
    ('0' + (((n) / 10)%10)), \
    ('0' + ((n) % 10))

/* Convert integer to hex digit literals. */
#define HEX(n) \
    ('0' + ((n)>>28 & 0xF)), \
    ('0' + ((n)>>24 & 0xF)), \
    ('0' + ((n)>>20 & 0xF)), \
    ('0' + ((n)>>16 & 0xF)), \
    ('0' + ((n)>>12 & 0xF)), \
    ('0' + ((n)>>8 & 0xF)), \
    ('0' + ((n)>>4 & 0xF)), \
    ('0' + ((n) & 0xF))

/* Construct a string literal encoding the version number. */
#ifdef COMPILER_VERSION
char const* info_version = "INFO" ":" "compiler_version[" COMPILER_VERSION "];

/* Construct a string literal encoding the version number components. */
#elif defined(COMPILER_VERSION_MAJOR)
char const info_version[] = {

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

'I', 'N', 'F', 'O', ':',
'c', 'o', 'm', 'p', 'i', 'l', 'e', 'r', '_', 'v', 'e', 'r', 's', 'i', 'o', 'n', '[',
COMPILER_VERSION_MAJOR,
# ifdef COMPILER_VERSION_MINOR
'.', COMPILER_VERSION_MINOR,
# ifdef COMPILER_VERSION_PATCH
'.', COMPILER_VERSION_PATCH,
# ifdef COMPILER_VERSION_TWEAK
'.', COMPILER_VERSION_TWEAK,
# endif
# endif
# endif
']', '\0'};
#endif

/* Construct a string literal encoding the internal version number. */
#ifdef COMPILER_VERSION_INTERNAL
char const info_version_internal[] = {
'I', 'N', 'F', 'O', ':',
'c', 'o', 'm', 'p', 'i', 'l', 'e', 'r', '_', 'v', 'e', 'r', 's', 'i', 'o', 'n', '_',
'i', 'n', 't', 'e', 'r', 'n', 'a', 'l', '[',
COMPILER_VERSION_INTERNAL, ']', '\0'};
#elif defined(COMPILER_VERSION_INTERNAL_STR)
char const* info_version_internal = "INFO" ":" "compiler_version_internal["
↪ COMPILER_VERSION_INTERNAL_STR " "];
#endif

/* Construct a string literal encoding the version number components. */
#ifdef SIMULATE_VERSION_MAJOR
char const info_simulate_version[] = {
'I', 'N', 'F', 'O', ':',
's', 'i', 'm', 'u', 'l', 'a', 't', 'e', '_', 'v', 'e', 'r', 's', 'i', 'o', 'n', '[',

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

SIMULATE_VERSION_MAJOR,
# ifdef SIMULATE_VERSION_MINOR
    '.', SIMULATE_VERSION_MINOR,
# ifdef SIMULATE_VERSION_PATCH
    '.', SIMULATE_VERSION_PATCH,
# ifdef SIMULATE_VERSION_TWEAK
    '.', SIMULATE_VERSION_TWEAK,
# endif
# endif
# endif
    ']', '\0'};
#endif

/* Construct the string literal in pieces to prevent the source from
   getting matched. Store it in a pointer rather than an array
   because some compilers will just produce instructions to fill the
   array rather than assigning a pointer to a static array. */
char const* info_platform = "INFO" ":" "platform[" PLATFORM_ID "];"
char const* info_arch = "INFO" ":" "arch[" ARCHITECTURE_ID "];

#define CXX_STD_98 199711L
#define CXX_STD_11 201103L
#define CXX_STD_14 201402L
#define CXX_STD_17 201703L
#define CXX_STD_20 202002L
#define CXX_STD_23 202302L

#if defined(__INTEL_COMPILER) && defined(_MSVC_LANG)
# if _MSVC_LANG > CXX_STD_17
# define CXX_STD _MSVC_LANG

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# elif _MSVC_LANG == CXX_STD_17 && defined(__cpp_aggregate_paren_init)
# define CXX_STD CXX_STD_20
# elif _MSVC_LANG > CXX_STD_14 && __cplusplus > CXX_STD_17
# define CXX_STD CXX_STD_20
# elif _MSVC_LANG > CXX_STD_14
# define CXX_STD CXX_STD_17
# elif defined(__INTEL_CXX11_MODE__) && defined(__cpp_aggregate_nsdmi)
# define CXX_STD CXX_STD_14
# elif defined(__INTEL_CXX11_MODE__)
# define CXX_STD CXX_STD_11
# else
# define CXX_STD CXX_STD_98
# endif
#elif defined(_MSC_VER) && defined(_MSVC_LANG)
# if _MSVC_LANG > __cplusplus
# define CXX_STD _MSVC_LANG
# else
# define CXX_STD __cplusplus
# endif
#elif defined(__NVCOMPILER)
# if __cplusplus == CXX_STD_17 && defined(__cpp_aggregate_paren_init)
# define CXX_STD CXX_STD_20
# else
# define CXX_STD __cplusplus
# endif
#elif defined(__INTEL_COMPILER) || defined(__PGI)
# if __cplusplus == CXX_STD_11 && defined(__cpp_namespace_attributes)
# define CXX_STD CXX_STD_17
# elif __cplusplus == CXX_STD_11 && defined(__cpp_aggregate_nsdmi)
# define CXX_STD CXX_STD_14
# else
# define CXX_STD __cplusplus

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# endif

#elif (defined(__IBMCPP__) || defined(__ibmxl__)) && defined(__linux__)
# if __cplusplus == CXX_STD_11 && defined(__cpp_aggregate_nsdmi)
# define CXX_STD CXX_STD_14
# else
# define CXX_STD __cplusplus
# endif

#elif __cplusplus == 1 && defined(__GXX_EXPERIMENTAL_CXX0X__)
# define CXX_STD CXX_STD_11
#else
# define CXX_STD __cplusplus
#endif

const char* info_language_standard_default = "INFO" ":" "standard_default["
#if CXX_STD > CXX_STD_23
    "26"
#elif CXX_STD > CXX_STD_20
    "23"
#elif CXX_STD > CXX_STD_17
    "20"
#elif CXX_STD > CXX_STD_14
    "17"
#elif CXX_STD > CXX_STD_11
    "14"
#elif CXX_STD >= CXX_STD_11
    "11"
#else
    "98"
#endif
"];

const char* info_language_extensions_default = "INFO" ":" "extensions_default["

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

#if (defined(__clang__) || defined(__GNUC__) || defined(__xlc__) || \
    defined(__TI_COMPILER_VERSION__) || defined(__RENASAS__)) && \
    !defined(__STRICT_ANSI__)
    "ON"
#else
    "OFF"
#endif
"];

/*-----*/

int main(int argc, char* argv[])
{
    int require = 0;
    require += info_compiler[argc];
    require += info_platform[argc];
    require += info_arch[argc];
#ifdef COMPILER_VERSION_MAJOR
    require += info_version[argc];
#endif
#if defined(COMPILER_VERSION_INTERNAL) || defined(COMPILER_VERSION_INTERNAL_STR)
    require += info_version_internal[argc];
#endif
#ifdef SIMULATE_ID
    require += info_simulate[argc];
#endif
#ifdef SIMULATE_VERSION_MAJOR
    require += info_simulate_version[argc];
#endif
#if defined(__CRAYXT_COMPUTE_LINUX_TARGET)
    require += info_cray[argc];
#endif

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
require += info_language_standard_default[argc];  
require += info_language_extensions_default[argc];  
(void)argv;  
return require;  
}
```

1.134. plagiarism\{}\models\{}\metrics.h

Листинг 134: plagiarism\{}\models\{}\metrics.h

```
#pragma once  
  
struct Metrics {  
    double token_jaccard = 0.0;  
    double normalized_levenshtein = 0.0;  
    double overall_score = 0.0;  
};
```

1.135. plagiarism\{}\models\{}\programming_language.h

Листинг 135: plagiarism\{}\models\{}\programming_language.h

```
#pragma once  
  
enum class ProgrammingLanguage {  
    Cpp,  
    Python,  
    Unknown  
};
```

1.136. plagiarism\{}\models\{}\similar_submission_pair.h

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Листинг 136: plagiarism\{}models\{}similar_submission_pair.h

```
#pragma once

#include <string>

struct SimilarSubmissionPair {
    std::string first_submission_id;
    std::string second_submission_id;
    double plagiarism_percent = 0.0;

    std::string first_participant;
    std::string second_participant;

    std::string first_problem;
    std::string second_problem;

    std::string first_file_name;
    std::string second_file_name;

    std::string first_source_path;
    std::string second_source_path;
};
```

1.137. plagiarism\{}models\{}similarity_result.h

Листинг 137: plagiarism\{}models\{}similarity_result.h

```
#pragma once

struct SimilarityResult {
    double token_score = 0.0;
    double ast_score = 0.0;
    double overall_score = 0.0;
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
};
```

1.138. plagiarism\{}\models\{}\submission.h

Листинг 138: plagiarism\{}\models\{}\submission.h

```
#pragma once
#include <string>
#include "programming_language.h"

struct Submission {
    std::string id;
    ProgrammingLanguage language = ProgrammingLanguage::Unknown;
    std::string rawCode;
    std::string participant;
    std::string problem;
    std::string file_name;
    std::string source_path;
};
```

1.139. plagiarism\{}\models\{}\submission_data.h

Листинг 139: plagiarism\{}\models\{}\submission_data.h

```
#pragma once

#include <cstdint>
#include <unordered_map>
#include <string>
#include <vector>

#include "token.h"
#include "token_features.h"
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

#include "winnowing_features.h"
#include "../ast/ast_features.h"

struct SubmissionData {
    std::string submission_id;

    std::string raw_code;
    std::string ast_code;
    std::string token_code;
    std::string ir_code;
    bool ir_parse_ok = false;

    std::vector<Token> tokens;
    std::vector<Token> normalized_tokens;
    std::vector<std::string> normalized_token_texts;
    WinnowingFeatures winnowing_features;
    TokenFeatures token_features;
    AstFeatures ast_features;

    std::unordered_map<std::uint64_t, int> ast_subtree_hash_freq;
    std::vector<std::string> ast_normalized_sequence;
};

```

1.140. plagiarism\{}models\{}submission_features.h

Листинг 140: plagiarism\{}models\{}submission_features.h

```

#pragma once

#include "token_features.h"
#include "../ast/ast_features.h"
#include "preprocessor_features.h"

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
struct SubmissionFeatures {
    TokenFeatures token;
    AstFeatures ast;
    PreprocessorFeatures pp;
};
```

1.141. plagiarism\{}\models\{}\token.h

Листинг 141: plagiarism\{}\models\{}\token.h

```
#pragma once

#include <cstddef>
#include <string>

enum class TokenType {
    Punctuation,
    Keyword,
    Identifier,
    Literal,
    Unknown
};

struct Token {
    TokenType type;
    std::string text;
    std::size_t offset;
};
```

1.142. plagiarism\{}\models\{}\token_features.h

Листинг 142: plagiarism\{}\models\{}\token_features.h

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
#pragma once

#include <string>
#include <unordered_map>
#include <vector>

struct TokenFeatures {
    int tokens_cnt = 0;
    int literals_cnt = 0;
    int punctuations_cnt = 0;
    int keywords_cnt = 0;
    int identifiers_cnt = 0;

    int unique_tokens = 0;

    std::unordered_map<std::string, int> freq;
    std::vector<std::string> grams3;
};
```

1.143. plagiarism\{}models\{}winnowing_features.h

Листинг 143: plagiarism\{}models\{}winnowing_features.h

```
#pragma once

#include <cstdint>
#include <vector>

struct WinnowingFeatures {
    std::vector<std::uint64_t> fingerprints;
    std::vector<std::uint64_t> minhash_signature;
};
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.144. plagiarism\{}outdated_metrics\{}levenshtein_similarity.cpp

Листинг 144: plagiarism\{}outdated_metrics\{}levenshtein_similarity.cpp

```

#include "levenshtein_similarity.h"

#include <algorithm>
#include <vector>

std::size_t CalculateLevenshteinDistance(const std::string& left,
                                         const std::string& right) {
    if (left.empty()) {
        return right.size();
    }

    if (right.empty()) {
        return left.size();
    }

    std::vector<std::size_t> previous(right.size() + 1);
    std::vector<std::size_t> current(right.size() + 1);

    for (std::size_t j = 0; j <= right.size(); ++j) {
        previous[j] = j;
    }

    for (std::size_t i = 1; i <= left.size(); ++i) {
        current[0] = i;

        for (std::size_t j = 1; j <= right.size(); ++j) {
            std::size_t cost = (left[i - 1] == right[j - 1]) ? 0 : 1;

            current[j] = std::min({

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
        previous[j] + 1,
        current[j - 1] + 1,
        previous[j - 1] + cost
    });

}

    std::swap(previous, current);
}

return previous[right.size()];
}

double CalculateLevenshteinSimilarity(const std::string& left,
                                     const std::string& right) {
    std::size_t max_length = std::max(left.size(), right.size());

    if (max_length == 0) {
        return 1.0;
    }

    std::size_t distance = CalculateLevenshteinDistance(left, right);

    return 1.0 - static_cast<double>(distance) / static_cast<double>(max_length);
}
```

1.145. plagiarism\{}outdated_metrics\{}levenshtein_similarity.h

Листинг 145: plagiarism\{}outdated_metrics\{}levenshtein_similarity.h

```
#pragma once
```

```
#include <string>
```

```
#include <cstdint>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
std::size_t CalculateLevenshteinDistance(const std::string& left,
                                         const std::string& right);

double CalculateLevenshteinSimilarity(const std::string& left,
                                      const std::string& right);
```

1.146. plagiarism\{}\preprocessing\{}\comments.cpp

Листинг 146: plagiarism\{}\preprocessing\{}\comments.cpp

```
#include "comments.h"

DeleteComments::DeleteComments(const std::string& code)
    : code_(code) {}

void DeleteComments::ProcessNormal() {
    if (code_[i_] == '"') {
        result_ += code_[i_];
        now_ = State::StringLiteral;
        flag_ = false;
    } else if (code_[i_] == '/') {
        now_ = State::Slash;
    } else if (code_[i_] == '\\') {
        result_ += code_[i_];
        now_ = State::CharLiteral;
        flag_ = false;
    } else {
        result_ += code_[i_];
    }
}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

void DeleteComments::ProcessSlash() {
    if (code_[i_] == '/') {
        now_ = State::LineComment;
    } else if (code_[i_] == '*') {
        now_ = State::BlockComment;
    } else {
        result_ += '/';
        result_ += code_[i_];
        now_ = State::Normal;
    }
}

void DeleteComments::ProcessBlockComment() {
    if (code_[i_] == '*' && (i_ != code_.size() - 1 && code_[i_ + 1] == '/')) {
        i_++;
        now_ = State::Normal;
    }
}

void DeleteComments::ProcessLineComment() {
    if (code_[i_] == '\n') {
        now_ = State::Normal;
    }
}

void DeleteComments::ProcessStringLiteral() {
    result_ += code_[i_];

    if (flag_) {
        flag_ = false;
    } else if (code_[i_] == '\\') {
        flag_ = true;
    } else if (code_[i_] == '"') {
        now_ = State::Normal;
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    }
}

void DeleteComments::ProcessCharLiteral() {
    result_ += code_[i_];

    if (flag_) {
        flag_ = false;
    } else if (code_[i_] == '\\') {
        flag_ = true;
    } else if (code_[i_] == '\') {
        now_ = State::Normal;
    }
}

std::string DeleteComments::Process() {
    for (i_ = 0; i_ < code_.size(); ++i_) {
        if (now_ == State::Normal) {
            ProcessNormal();
        } else if (now_ == State::Slash) {
            ProcessSlash();
        } else if (now_ == State::LineComment) {
            ProcessLineComment();
        } else if (now_ == State::BlockComment) {
            ProcessBlockComment();
        } else if (now_ == State::StringLiteral) {
            ProcessStringLiteral();
        } else if (now_ == State::CharLiteral) {
            ProcessCharLiteral();
        }
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    if (now_ == State::Slash) {
        result_ += '/';
    }
    return result_;
}

std::string RemoveComments(const std::string& code) {
    return DeleteComments(code).Process();
}

```

1.147. plagiarism\{}\preprocessing\{}\comments.h

Листинг 147: plagiarism\{}\preprocessing\{}\comments.h

```

#pragma once

#include <string>

class DeleteComments {
public:
    DeleteComments(const std::string& code);
    std::string Process();

private:

    enum class State {
        Normal,
        Slash,
        LineComment,
        BlockComment,
        StringLiteral,
        CharLiteral
    };
};

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

//
void ProcessNormal();
void ProcessSlash();
void ProcessLineComment();
void ProcessBlockComment();
void ProcessStringLiteral();
void ProcessCharLiteral();

const std::string& code_;
std::string result_;
std::size_t i_ = 0;
State now_ = State::Normal;

// , , \ "
//

bool flag_ = false;

};

std::string RemoveComments(const std::string& code);

```

1.148. plagiarism\{}\preprocessing\{}\cpp_preprocessor.cpp

Листинг 148: plagiarism\{}\preprocessing\{}\cpp_preprocessor.cpp

```

#include "cpp_preprocessor.h"

bool WriteAll(int fd, const std::string& text) {
    size_t pos = 0;

    while (pos < text.size()) {

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    ssize_t written = write(fd, text.data() + pos, text.size() - pos);
    if (written < 0) {
        if (errno == EINTR) {
            continue;
        }
        //TODO
        return false;
    }
    pos += static_cast<size_t>(written);
}
return true;
}

```

```

bool ReadAll(int fd, std::string& text) {
    text.clear();
    char buffer[4096];

    while (true) {
        ssize_t r = read(fd, buffer, sizeof(buffer));
        if (r == 0) {
            break;
        }
        if (r < 0) {
            if (errno == EINTR) {
                continue;
            }
            //TODO
            return false;
        }
        text.append(buffer, r);
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    return true;
}

bool WriteFile(const std::string& path, const std::string& text) {
    int fd = open(path.c_str(), O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        return false;
    }

    bool ok = WriteAll(fd, text);
    close(fd);
    return ok;
}

bool ReadFile(const std::string& path, std::string& text) {
    int fd = open(path.c_str(), O_RDONLY);
    if (fd < 0) {
        return false;
    }

    bool ok = ReadAll(fd, text);
    close(fd);
    return ok;
}

std::string MakeFile(const std::string& prefix, const std::string& ext) {
    long long t = std::chrono::steady_clock::now().time_since_epoch().count();
    return "." + prefix + "_" + std::to_string(getpid()) + "_" + std::to_string(t)
        + ext;
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

bool RunPreprocessor(const std::string& inputPath, const std::string& outputPath)
    ⇨ {
    pid_t pid = fork();

    if (pid < 0) {
        return false;
    }

    if (pid == 0) {
        char* argv[] = {
            (char*)"g++",
            (char*)"-E",
            (char*)"-P",
            (char*)"-x",
            (char*)"c++",
            (char*)inputPath.c_str(),
            (char*)"-o",
            (char*)outputPath.c_str(),
            nullptr
        };

        execvp("g++", argv);

        _exit(127);
    }

    int status = 0;

    while (waitpid(pid, &status, 0) < 0) {
        if (errno == EINTR) {
            continue;
        }
    }

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        return false;
    }

    if (!WIFEXITED(status)) {
        return false;
    }

    return WEXITSTATUS(status) == 0;
}

std::string PreprocessCode(const std::string& code) {
    std::string inputPath = MakeFile("pp_input", ".cpp");
    std::string outputPath = MakeFile("pp_output", ".ii");

    if (!WriteFile(inputPath, code)) {
        return "";
    }

    if (!RunPreprocessor(inputPath, outputPath)) {
        unlink(inputPath.c_str());
        unlink(outputPath.c_str());
        return "";
    }

    std::string result;
    if (!ReadFile(outputPath, result)) {
        unlink(inputPath.c_str());
        unlink(outputPath.c_str());
        return "";
    }

    unlink(inputPath.c_str());
    unlink(outputPath.c_str());

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    return result;  
}
```

1.149. plagiarism\{}\preprocessing\{}\cpp_preprocessor.h

Листинг 149: plagiarism\{}\preprocessing\{}\cpp_preprocessor.h

```
#pragma once  
  
#include <unistd.h>  
#include <fcntl.h>  
#include <sys/wait.h>  
#include <chrono>  
#include <string>  
#include <vector>  
#include <errno.h>  
  
bool WriteAll(int fd, const std::string& text);  
bool ReadAll(int fd, std::string& text);  
  
bool WriteFile(const std::string& path, const std::string& text);  
bool ReadFile(const std::string& path, std::string& text);  
  
std::string MakeFile(const std::string& prefix, const std::string& ext);  
bool RunPreprocessor(const std::string& inputPath, const std::string& outputPath);  
  
std::string PreprocessCode(const std::string& code);
```

1.150. plagiarism\{}\preprocessing\{}\line_endings.cpp

Листинг 150: plagiarism\{}\preprocessing\{}\line_endings.cpp

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

#include "line_endings.h"

#include <string>

std::string NormalizeLineEndings(const std::string& code) {
    std::string result;

    for(std::size_t i = 0; i < code.size(); i++) {
        if (code[i] == '\r') {
            if ((i != code.size() - 1 && code[i + 1] == '\n')) {
                continue;
            }
            result.push_back('\n');
        } else {
            result.push_back(code[i]);
        }
    }
    return result;
}

```

1.151. plagiarism\preprocessing\line_endings.h

Листинг 151: plagiarism\preprocessing\line_endings.h

```

#pragma once

#include <string>

// Converts all line endings to '\n'
std::string NormalizeLineEndings(const std::string& code);

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.152. plagiarism\{}\preprocessing\{}\lines.cpp

Листинг 152: plagiarism\{}\preprocessing\{}\lines.cpp

```
#include "lines.h"
#include "string_normalizer.h"

LinesNormalizer::LinesNormalizer(const std::string& code)
    : code_(code) {}

bool LinesNormalizer::IsEmptyLine(const std::string& line) const {
    return line.empty();
}

void LinesNormalizer::SplitLines() {
    std::string current;

    for (char c : code_) {
        if (c == '\\n') {
            lines_.push_back(current);
            current = "";
        } else {
            current += c;
        }
    }
    lines_.push_back(current);
}

void LinesNormalizer::NormalizeEachLine() {
    for (std::string& line : lines_) {
        line = NormalizeString(line);
    }
}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

}

void LinesNormalizer::JoinLines() {
    code_ = "";
    for (std::string& line : lines_) {
        code_ += line;
        code_ += '\n';
    }
}

void LinesNormalizer::RemoveEmptyLines() {
    std::vector < std::string > result;
    for (std::string& line : lines_) {
        if (!IsEmptyLine(line)) {
            result.push_back(line);
        }
    }
    lines_ = result;
}

void LinesNormalizer::RemoveIncludesAndPragmas() {
    std::vector < std::string > result;
    for (std::string& line : lines_) {
        if (!(line.size() >= 8 && line.substr(0, 8) == "#include") &&
            !(line.size() >= 7 && line.substr(0, 7) == "#pragma")) {
            result.push_back(line);
        }
    }
    lines_ = result;
}

std::string LinesNormalizer::Transform() {
    SplitLines();
    NormalizeEachLine();
    RemoveEmptyLines();
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    RemoveIncludesAndPragmas();  
    JoinLines();  
    return code_;  
};  
  
std::string NormalizeLines(const std::string& code) {  
    return LinesNormalizer(code).Transform();  
}
```

1.153. plagiarism\{}\preprocessing\{}\lines.h

Листинг 153: plagiarism\{}\preprocessing\{}\lines.h

```
#pragma once  
  
#include <string>  
#include <vector>  
  
class LinesNormalizer {  
public:  
    explicit LinesNormalizer(const std::string& code);  
    std::string Transform();  
  
private:  
    void SplitLines();  
    void NormalizeEachLine();  
    void RemoveEmptyLines();  
    void JoinLines();  
    void RemoveIncludesAndPragmas();  
  
    bool IsEmptyLine(const std::string& line) const;  
  
    std::string code_;
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    std::vector<std::string> lines_;
};

std::string NormalizeLines(const std::string& code);

```

1.154. plagiarism\{}\preprocessing\{}\llvm_ir_pipeline.cpp

Листинг 154: plagiarism\{}\preprocessing\{}\llvm_ir_pipeline.cpp

```

#include "llvm_ir_pipeline.h"

#include "cpp_preprocessor.h"

#include <sys/wait.h>
#include <unistd.h>

#include <cerrno>
#include <string>

static bool RunCommand(char* const argv[]) {
    const pid_t pid = fork();
    if (pid < 0) {
        return false;
    }

    if (pid == 0) {
        execvp(argv[0], argv);
        _exit(127);
    }

    int status = 0;
    while (waitpid(pid, &status, 0) < 0) {
        if (errno == EINTR) {

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
        continue;
    }
    return false;
}

return WIFEXITED(status) && WEXITSTATUS(status) == 0;
}

static bool EmitLlvmIr(const std::string& input_cpp, const std::string& output_ll)
    ↪ {
    char* argv[] = {
        (char*)"clang++",
        (char*)"-std=c++17",
        (char*)"-O0",
        (char*)"-Xclang",
        (char*)"-disable-O0-optnone",
        (char*)"-S",
        (char*)"-emit-llvm",
        (char*)input_cpp.c_str(),
        (char*)"-o",
        (char*)output_ll.c_str(),
        nullptr
    };
    return RunCommand(argv);
}

static bool OptimizeLlvmIr(const std::string& input_ll, const std::string&
    ↪ output_ll) {
    char* argv[] = {
        (char*)"opt",
        (char*)"-S",
        (char*)"-passes=mem2reg,instcombine,simplifcfg,adce",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        (char*)input_ll.c_str(),
        (char*)"-o",
        (char*)output_ll.c_str(),
        nullptr
    };
    return RunCommand(argv);
}

std::string BuildLlvmIrFromCode(const std::string& code) {
    const std::string cpp_path = MakeFile("ir_input", ".cpp");
    const std::string raw_ll_path = MakeFile("ir_raw", ".ll");
    const std::string opt_ll_path = MakeFile("ir_opt", ".ll");

    std::string result;
    if (!WriteFile(cpp_path, code)) {
        return "";
    }

    if (!EmitLlvmIr(cpp_path, raw_ll_path) || !OptimizeLlvmIr(raw_ll_path,
        ↪ opt_ll_path)) {
        unlink(cpp_path.c_str());
        unlink(raw_ll_path.c_str());
        unlink(opt_ll_path.c_str());
        return "";
    }

    if (!ReadFile(opt_ll_path, result)) {
        unlink(cpp_path.c_str());
        unlink(raw_ll_path.c_str());
        unlink(opt_ll_path.c_str());
        return "";
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    unlink(cpp_path.c_str());
    unlink(raw_ll_path.c_str());
    unlink(opt_ll_path.c_str());
    return result;
}

```

1.155. plagiarism\{}\preprocessing\{}\llvm_ir_pipeline.h

Листинг 155: plagiarism\{}\preprocessing\{}\llvm_ir_pipeline.h

```

#pragma once

#include <string>

// Generates optimized LLVM IR (.ll text) from C++ source code.
// Returns empty string on any tooling or IO error.
std::string BuildLlvmIrFromCode(const std::string& code);

```

1.156. plagiarism\{}\preprocessing\{}\normalize.cpp

Листинг 156: plagiarism\{}\preprocessing\{}\normalize.cpp

```

#include "normalize.h"

#include "cpp_preprocessor.h"
#include "comments.h"
#include "line_endings.h"
#include "lines.h"

std::string NormalizeForTokenizer(const std::string& code) {
    std::string result = NormalizeLineEndings(code);
    result = RemoveComments(result);
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    result = NormalizeLines(result);
    result = PreprocessCode(result);
    return result;
}

std::string NormalizeForAST(const std::string& code) {
    std::string result = NormalizeLineEndings(code);
    result = RemoveComments(result);
    result = NormalizeLines(result);
    result = PreprocessCode(result);
    return result;
}
```

1.157. plagiarism\{}\preprocessing\{}\normalize.h

Листинг 157: plagiarism\{}\preprocessing\{}\normalize.h

```
#pragma once

#include <string>

std::string NormalizeForTokenizer(const std::string& code);
std::string NormalizeForAST(const std::string& code);
```

1.158. plagiarism\{}\preprocessing\{}\string_normalizer.cpp

Листинг 158: plagiarism\{}\preprocessing\{}\string_normalizer.cpp

```
#include "string_normalizer.h"

StringNormalizer::StringNormalizer(const std::string& line) : line_(line) {}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

void StringNormalizer::RemoveLeadingAndTrailingSpaces() {
    std::size_t left = 0;
    while (left < line_.size() && (line_[left] == ' ' || line_[left] == '\t')) {
        left++;
    }
    std::size_t right = line_.size();
    while (right > left && (line_[right - 1] == ' ' || line_[right - 1] == '\t')) {
        --right;
    }

    line_ = line_.substr(left, right - left);
}

void StringNormalizer::ProcessNormal() {
    if (line_[i_] == '"') {
        now_ = State::StringLiteral;
        result_ += line_[i_];
        flag_ = false;
    } else if (line_[i_] == '\\') {
        now_ = State::CharLiteral;
        result_ += line_[i_];
        flag_ = false;
    } else if (line_[i_] == ' ' || line_[i_] == '\t') {
        if (result_.empty() || result_.back() != ' ') {
            result_ += ' ';
        }
    } else {
        result_ += line_[i_];
    }
}

void StringNormalizer::ProcessStringLiteral() {

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    result_ += line_[i_];
    if (flag_) {
        flag_ = false;
    } else if (line_[i_] == '\\') {
        flag_ = true;
    } else if (line_[i_] == '"') {
        now_ = State::Normal;
    }
}

void StringNormalizer::ProcessCharLiteral() {
    char c = line_[i_];
    result_ += c;

    if (flag_) {
        flag_ = false;
    } else if (c == '\\') {
        flag_ = true;
    } else if (c == '\') {
        now_ = State::Normal;
    }
}

void StringNormalizer::RemoveDoubleSpaces() {
    for (i_ = 0; i_ < line_.size(); ++i_) {
        if (now_ == State::Normal) {
            ProcessNormal();
        } else if (now_ == State::CharLiteral) {
            ProcessCharLiteral();
        } else if (now_ == State::StringLiteral) {
            ProcessStringLiteral();
        }
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    }
    line_ = result_;
}

std::string StringNormalizer::Transform() {
    RemoveLeadingAndTrailingSpaces();
    RemoveDoubleSpaces();
    return line_;
}

std::string NormalizeString(const std::string& line) {
    return StringNormalizer(line).Transform();
}

```

1.159. plagiarism\{}\preprocessing\{}\string_normalizer.h

Листинг 159: plagiarism\{}\preprocessing\{}\string_normalizer.h

```

#pragma once

#include <string>

class StringNormalizer {
public:
    explicit StringNormalizer(const std::string& line);
    std::string Transform();
private:
    enum class State {
        Normal,
        StringLiteral,
        CharLiteral
    };
};

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
void ProcessNormal();
void ProcessStringLiteral();
void ProcessCharLiteral();

void RemoveLeadingAndTrailingSpaces();
void RemoveDoubleSpaces();

std::string line_;

State now_ = State::Normal;
std::size_t i_ = 0;
std::string result_;
bool flag_ = false;

};

std::string NormalizeString(const std::string& line);
```

1.160. plagiarism\{}\similarity\{}\bindings.cpp

Листинг 160: plagiarism\{}\similarity\{}\bindings.cpp

```
#include <pybind11/pybind11.h>
#include <pybind11/stl.h>

#include "compare_submissions.h"
#include <models/submission.h>
#include <models/similar_submission_pair.h>

namespace py = pybind11;

PYBIND11_MODULE(plagiarism_cpp, m) {
    py::enum_<ProgrammingLanguage>(m, "ProgrammingLanguage")
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
.value("Cpp", ProgrammingLanguage::Cpp)
.value("Python", ProgrammingLanguage::Python)
.value("Unknown", ProgrammingLanguage::Unknown)
.export_values();

py::class_<Submission>(m, "Submission")
    .def(py::init<>())
    .def_readwrite("id", &Submission::id)
    .def_readwrite("language", &Submission::language)
    .def_readwrite("rawCode", &Submission::rawCode)
    .def_readwrite("participant", &Submission::participant)
    .def_readwrite("problem", &Submission::problem)
    .def_readwrite("file_name", &Submission::file_name)
    .def_readwrite("source_path", &Submission::source_path);

py::class_<SimilarSubmissionPair>(m, "SimilarSubmissionPair")
    .def_readonly("first_submission_id", &SimilarSubmissionPair::
        ↪ first_submission_id)
    .def_readonly("second_submission_id", &SimilarSubmissionPair::
        ↪ second_submission_id)
    .def_readonly("plagiarism_percent", &SimilarSubmissionPair::
        ↪ plagiarism_percent)
    .def_readonly("first_participant", &SimilarSubmissionPair::
        ↪ first_participant)
    .def_readonly("second_participant", &SimilarSubmissionPair::
        ↪ second_participant)
    .def_readonly("first_problem", &SimilarSubmissionPair::first_problem)
    .def_readonly("second_problem", &SimilarSubmissionPair::second_problem)
    .def_readonly("first_file_name", &SimilarSubmissionPair::first_file_name)
    .def_readonly("second_file_name", &SimilarSubmissionPair::second_file_name)
    .def_readonly("first_source_path", &SimilarSubmissionPair::
        ↪ first_source_path)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        .def_readonly("second_source_path", &SimilarSubmissionPair::
            ↪ second_source_path);

    m.def(
        "compute_similarity_pairs",
        &compute_similarity_pairs,
        py::arg("submissions"),
        py::arg("threshold")
    );
}

```

1.161. plagiarism\{}\similarity\{}\compare_submissions.cpp

Листинг 161: plagiarism\{}\similarity\{}\compare_submissions.cpp

```

#include "compare_submissions.h"

#include <algorithm>
#include <unordered_map>

#include "lsh.h"
#include "metrics.h"
#include "../submission_builder/submission_builder.h"

static double compute_similarity_impl(const SubmissionData& lft, const
    ↪ SubmissionData& rht) {
    double token_similarity = compute_token_similarity(lft, rht);
    double ast_similarity = compute_ast_similarity(lft, rht);
    double ir_similarity = compute_ir_similarity(lft, rht);

    bool has_ast = lft.ast_features.parse_ok && rht.ast_features.parse_ok;
    bool has_ir = lft.ir_parse_ok && rht.ir_parse_ok;
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

return compute_overall_similarity(token_similarity, ast_similarity, has_ast,
    ↪ ir_similarity, has_ir);
}

std::vector<SimilarSubmissionPair> compute_similarity_pairs(
    const std::vector<Submission>& submissions,
    double threshold
) {
    std::vector<SubmissionData> prepared;
    prepared.reserve(submissions.size());

    for (const auto& submission : submissions) {
        prepared.push_back(BuildSubmissionData(submission));
    }

    std::vector<SimilarSubmissionPair> result;

    std::unordered_map<std::string, std::size_t> id_to_index;
    id_to_index.reserve(submissions.size());
    for (std::size_t i = 0; i < submissions.size(); ++i) {
        id_to_index[submissions[i].id] = i;
    }

    const auto candidate_pairs = generate_lsh_candidate_pairs(prepared);
    for (const auto& [left_id, right_id] : candidate_pairs)
    {
        const auto left_it = id_to_index.find(left_id);
        const auto right_it = id_to_index.find(right_id);
        if (left_it == id_to_index.end() || right_it == id_to_index.end()) {
            continue;
        }
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

const std::size_t i = left_it->second;
const std::size_t j = right_it->second;
if (submissions[i].participant == submissions[j].participant) {
    continue;
}

if (!submissions[i].problem.empty() &&
    !submissions[j].problem.empty() &&
    submissions[i].problem != submissions[j].problem) {
    continue;
}

double similarity = compute_similarity_impl(prepared[i], prepared[j]);
if (similarity < threshold) {
    continue;
}

result.push_back(SimilarSubmissionPair{
    submissions[i].id,
    submissions[j].id,
    similarity * 100.0,

    submissions[i].participant,
    submissions[j].participant,

    submissions[i].problem,
    submissions[j].problem,

    submissions[i].file_name,
    submissions[j].file_name,

    submissions[i].source_path,

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        submissions[j].source_path
    });

}

std::sort(result.begin(), result.end(), [](const SimilarSubmissionPair& left,
    ↪ const SimilarSubmissionPair& right) {
    if (left.plagiarism_percent != right.plagiarism_percent) {
        return left.plagiarism_percent > right.plagiarism_percent;
    }

    if (left.first_submission_id != right.first_submission_id) {
        return left.first_submission_id < right.first_submission_id;
    }

    return left.second_submission_id < right.second_submission_id;
});

return result;
}

```

1.162. plagiarism\{}\similarity\{}\compare_submissions.h

Листинг 162: plagiarism\{}\similarity\{}\compare_submissions.h

```

#pragma once

#include <vector>

#include "../models/submission.h"
#include "../models/similar_submission_pair.h"

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
std::vector<SimilarSubmissionPair> compute_similarity_pairs(  
    const std::vector<Submission>& submissions,  
    double threshold = 0.0  
);
```

1.163. plagiarism\{}\similarity\{}\lsh.cpp

Листинг 163: plagiarism\{}\similarity\{}\lsh.cpp

```
#include "lsh.h"  
#include <boost/functional/hash.hpp>  
#include <cstdint>  
#include <utility>  
#include <unordered_map>  
#include <iostream>  
  
std::uint64_t build_lsh_bucket_key(  
    const std::vector<std::uint64_t>& signature,  
    std::size_t start,  
    int rows_per_band  
) {  
    std::uint64_t ret = 0;  
  
    for (size_t i = 0; i < rows_per_band; i++) {  
        ret = splitmix64(ret ^ signature[start + i]);  
    }  
    return ret;  
}  
  
struct PairHash {  
    std::size_t operator()(const std::pair<std::string, std::string>& p) const  
        ↪ noexcept {  
        std::size_t seed = 0;
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        boost::hash_combine(seed, p.first);
        boost::hash_combine(seed, p.second);
        return seed;
    }
};

std::vector<std::pair < std::string, std::string > > generate_lsh_candidate_pairs(
    const std::vector<SubmissionData>& prepared
) {
    std::vector < std::pair < std::string, std::string > > candidates;

    std::unordered_map<std::uint64_t, std::vector<std::string>> buckets;
    std::unordered_map<std::pair < std::string, std::string >, int, PairHash >
        ↪ in_one_bucket_cnt;
    for (auto& to : prepared) {
        for (size_t band = 0; band < BANDS; band++) {
            std::uint64_t bucket_key = build_lsh_bucket_key(
                to.winnowing_features.minhash_signature,
                band * ROWS_PER_BAND
            );
            buckets[bucket_key].push_back(to.submission_id);
        }
    }

    std::cout << "\n---LSH_BUCKET_STATS---\n";
    std::cout << "Total_buckets:" << buckets.size() << "\n";
    int buckets_size_1 = 0;
    int buckets_size_small = 0;
    int buckets_size_huge = 0;
    int max_bucket_size = 0;

    for (const auto& [hashh, indexes] : buckets) {
        max_bucket_size = std::max(max_bucket_size, (int)indexes.size());
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    if (indexes.size() == 1) buckets_size_1++;
    else if (indexes.size() <= MAX_BUCKET_SIZE) buckets_size_small++;
    else buckets_size_huge++;
}

std::cout << "Buckets with 1 item: " << buckets_size_1 << "\n";
std::cout << "Buckets with 2 to " << MAX_BUCKET_SIZE << " items (processed): "
    << buckets_size_small << "\n";
std::cout << "Buckets with > " << MAX_BUCKET_SIZE << " items (ignored): " <<
    << buckets_size_huge << "\n";
std::cout << "Max bucket size: " << max_bucket_size << "\n";
std::cout << "-----\n";

for (auto& [hashh, indexes] : buckets) {
    if (indexes.size() > MAX_BUCKET_SIZE) {
        continue;
    }
    for (size_t i = 0; i < indexes.size(); i++) {
        for (size_t j = i + 1; j < indexes.size(); j++) {
            std::pair < std::string, std::string > p = {std::min(indexes[i],
                <=> indexes[j]), std::max(indexes[i], indexes[j])};
            in_one_bucket_cnt[p]++;
        }
    }
}

for (auto& [p, cnt] : in_one_bucket_cnt) {
    if (cnt >= MIN_COMMON_BAND) {
        candidates.push_back(p);
    }
}

return candidates;

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

}

1.164. plagiarism\{}\similarity\{}\lsh.h

Листинг 164: plagiarism\{}\similarity\{}\lsh.h

```
#pragma once

#include <cstdint>
#include <cstdint>
#include <utility>
#include <vector>

#include "../models/submission_data.h"
#include "winnowing.h"

const int BANDS = 32;
const int ROWS_PER_BAND = 4;
const int MIN_COMMON_BAND = 1;
const std::size_t MAX_BUCKET_SIZE = 2000;

std::uint64_t build_lsh_bucket_key(
    const std::vector<std::uint64_t>& signature,
    std::size_t start,
    int rows_per_band = ROWS_PER_BAND
);

std::vector<std::pair< std::string, std::string >> generate_lsh_candidate_pairs
    ⇨ (
    const std::vector<SubmissionData>& prepared
);
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.165. plagiarism\{}\similarity\{}\metrics.cpp

Листинг 165: plagiarism\{}\similarity\{}\metrics.cpp

```

#include "metrics.h"

#include <unordered_set>
#include <cmath>
#include <sstream>

static std::vector<std::string> extract_ir_opcodes(const std::string& ir_code) {
    std::vector<std::string> ops;
    std::istringstream in(ir_code);
    std::string line;

    while (std::getline(in, line)) {
        std::size_t first = line.find_first_not_of("\t");
        if (first == std::string::npos) {
            continue;
        }

        char lead = line[first];
        if (lead == ';' || lead == '!' || lead == '}') {
            continue;
        }

        if (line.compare(first, 6, "define") == 0 ||
            line.compare(first, 7, "declare") == 0 ||
            line.compare(first, 5, "entry") == 0) {
            continue;
        }

        std::size_t pos = first;

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    if (line[pos] == '%') {
        std::size_t eq = line.find('=', pos);
        if (eq == std::string::npos) {
            continue;
        }
        pos = line.find_first_not_of("\t", eq + 1);
        if (pos == std::string::npos) {
            continue;
        }
    }

    std::size_t end = line.find_first_of("\t(", pos);
    if (end == std::string::npos) {
        end = line.size();
    }

    std::string op = line.substr(pos, end - pos);
    if (!op.empty() && op != "label") {
        ops.push_back(op);
    }
}

return ops;
}

static std::vector<std::string> build_grams3(const std::vector<std::string>& items
↪ ) {
    if (items.size() < 3) {
        return items;
    }

    std::vector<std::string> grams;

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

grams.reserve(items.size() - 2);
for (std::size_t i = 0; i + 2 < items.size(); ++i) {
    grams.push_back(items[i] + "|" + items[i + 1] + "|" + items[i + 2]);
}
return grams;
}

double jaccard_score(
    const std::vector<std::string>& lft,
    const std::vector<std::string>& rht
) {
    std::unordered_set<std::string> lft_set(lft.begin(), lft.end());
    std::unordered_set<std::string> rht_set(rht.begin(), rht.end());

    if (lft_set.empty() && rht_set.empty()) {
        return 1.0;
    }

    int intersec = 0;
    for (const auto& token : lft_set) {
        if (rht_set.find(token) != rht_set.end()) {
            ++intersec;
        }
    }

    std::size_t union_size = lft_set.size();
    for (const auto& token : rht_set) {
        if (lft_set.find(token) == lft_set.end()) {
            ++union_size;
        }
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    return static_cast<double>(intersec) / static_cast<double>(union_size);
}

double cosine_similarity_score(
    const std::unordered_map<std::string, int>& lft,
    const std::unordered_map<std::string, int>& rht
) {
    double lft_norm = 0, rht_norm = 0;
    double scalar = 0.0;

    for (const auto& to : lft) {
        lft_norm += static_cast<double>(to.second) * static_cast<double>(to.second)
            ↪ ;

        auto it = rht.find(to.first);
        if (it != rht.end()) {
            scalar += static_cast<double>(to.second) * static_cast<double>(it->
                ↪ second);
        }
    }

    for (const auto& to : rht) {
        rht_norm += static_cast<double>(to.second) * static_cast<double>(to.second)
            ↪ ;
    }

    if (lft_norm == 0.0 && rht_norm == 0.0) {
        return 1.0;
    }

    if (lft_norm == 0.0 || rht_norm == 0.0 ) {
        return 0.0;
    }

    return scalar / (std::sqrt(lft_norm) * std::sqrt(rht_norm));
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

}

double compute_token_similarity(
    const SubmissionData& lft,
    const SubmissionData& rht
) {
    double grams3 = jaccard_score(lft.token_features.grams3, rht.token_features.
        ↪ grams3);
    double freq_cos = cosine_similarity_score(lft.token_features.freq, rht.
        ↪ token_features.freq);
    return 0.75 * grams3 + 0.25 * freq_cos;
}

double compute_ast_counts_similarity(
    const AstFeatures& left,
    const AstFeatures& right
) {
    int diff_sum = 0;
    int max_sum = 0;

    diff_sum += std::abs(left.functions_cnt - right.functions_cnt);
    diff_sum += std::abs(left.ifs_cnt - right.ifs_cnt);
    diff_sum += std::abs(left.fors_cnt - right.fors_cnt);
    diff_sum += std::abs(left.whiles_cnt - right.whiles_cnt);
    diff_sum += std::abs(left.returns_cnt - right.returns_cnt);
    diff_sum += std::abs(left.calls_cnt - right.calls_cnt);

    max_sum += std::max(left.functions_cnt, right.functions_cnt);
    max_sum += std::max(left.ifs_cnt, right.ifs_cnt);
    max_sum += std::max(left.fors_cnt, right.fors_cnt);
    max_sum += std::max(left.whiles_cnt, right.whiles_cnt);
    max_sum += std::max(left.returns_cnt, right.returns_cnt);
    max_sum += std::max(left.calls_cnt, right.calls_cnt);
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    if (max_sum == 0) {
        return 1.0;
    }

    return 1.0 - static_cast<double>(diff_sum) / static_cast<double>(max_sum);
}

double weighted_jaccard_hash_freq(
    const std::unordered_map<std::uint64_t, int>& lft,
    const std::unordered_map<std::uint64_t, int>& rht
) {
    if (lft.empty() && rht.empty()) {
        return 1.0;
    }

    double intersec = 0.0;
    double each = 0.0;

    for (const auto& to : lft) {
        auto it = rht.find(to.first);
        if (it != rht.end()) {
            intersec += std::min(static_cast<double>(to.second), static_cast<double>(it->second));
            each += std::max(static_cast<double>(to.second), static_cast<double>(it->second));
        } else {
            each += static_cast<double>(to.second);
        }
    }

    for (const auto& to : rht) {
        if (lft.find(to.first) == lft.end()) {
            each += static_cast<double>(to.second);
        }
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    }
}
if (each == 0.0) {
    return 0.0;
}
return intersec / each;
}

double compute_ast_subtree_similarity(
    const SubmissionData& lft,
    const SubmissionData& rht
) {
    return weighted_jaccard_hash_freq(lft.ast_subtree_hash_freq, rht.
        ↪ ast_subtree_hash_freq);
}

double compute_ast_sequence_similarity(
    const SubmissionData& lft,
    const SubmissionData& rht
) {
    std::vector<std::string> lft_grams = build_ast_grams3(lft.
        ↪ ast_normalized_sequence);
    std::vector<std::string> rht_grams = build_ast_grams3(rht.
        ↪ ast_normalized_sequence);

    return jaccard_score(lft_grams, rht_grams);
}

std::vector<std::string> build_ast_grams3(const std::vector<std::string>&
    ↪ preorder_kinds) {
    std::vector<std::string> grams3;

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

    for (size_t i = 0; i + 2 < preorder_kinds.size(); ++i) {
        grams3.push_back(preorder_kinds[i] + "|" + preorder_kinds[i + 1] + "|" +
            ⇨ preorder_kinds[i + 2]);
    }

    return grams3;
}

double compute_ast_similarity(const SubmissionData& lft, const SubmissionData& rht
    ⇨ ) {
    if (!lft.ast_features.parse_ok || !rht.ast_features.parse_ok) {
        return 0.0;
    }

    const double count_score = compute_ast_counts_similarity(lft.ast_features, rht.
        ⇨ ast_features);
    const double subtree_score = compute_ast_subtree_similarity(lft, rht);
    const double sequence_score = compute_ast_sequence_similarity(lft, rht);
    return 0.20 * count_score + 0.65 * subtree_score + 0.15 * sequence_score;
}

double compute_ir_similarity(
    const SubmissionData& lft,
    const SubmissionData& rht
) {
    if (!lft.ir_parse_ok || !rht.ir_parse_ok) {
        return 0.0;
    }

    std::vector<std::string> lft_ops = extract_ir_opcodes(lft.ir_code);
    std::vector<std::string> rht_ops = extract_ir_opcodes(rht.ir_code);

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

std::vector<std::string> lft_grams = build_grams3(lft_ops);
std::vector<std::string> rht_grams = build_grams3(rht_ops);
return jaccard_score(lft_grams, rht_grams);
}

double compute_overall_similarity(
    double token_score,
    double ast_score,
    bool has_ast,
    double ir_score,
    bool has_ir
) {
    if (!has_ast && !has_ir) {
        return std::pow(token_score, 1.15);
    }

    if (has_ast && !has_ir) {
        const double linear = 0.65 * token_score + 0.35 * ast_score;
        return std::pow(linear, 1.15);
    }

    if (!has_ast && has_ir) {
        const double linear = 0.85 * token_score + 0.15 * ir_score;
        return std::pow(linear, 1.15);
    }

    const double linear = 0.58 * token_score + 0.30 * ast_score + 0.12 * ir_score;
    return std::pow(linear, 1.15);
}

```

1.166. plagiarism\{}\similarity\{}\metrics.h

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Листинг 166: plagiarism\{}\similarity\{}\metrics.h

```
#pragma once

#include <cstdint>
#include <string>
#include <unordered_map>
#include <vector>

#include "../models/submission_data.h"

double jaccard_score(
    const std::vector<std::string>& lft,
    const std::vector<std::string>& rht
);

double cosine_similarity_score(
    const std::unordered_map<std::string, int>& lft,
    const std::unordered_map<std::string, int>& rht
);

double weighted_jaccard_hash_freq(
    const std::unordered_map<std::uint64_t, int>& lft,
    const std::unordered_map<std::uint64_t, int>& rht
);

double compute_token_similarity(
    const SubmissionData& lft,
    const SubmissionData& rht
);

double compute_ast_counts_similarity(
    const AstFeatures& left,
    const AstFeatures& right
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
);

double compute_ast_subtree_similarity(
    const SubmissionData& lft,
    const SubmissionData& rht
);

double compute_ast_sequence_similarity(
    const SubmissionData& lft,
    const SubmissionData& rht
);

std::vector<std::string> build_ast_grams3(const std::vector<std::string>&
    ⇔ preorder_kinds);

double compute_ast_similarity(
    const SubmissionData& lft,
    const SubmissionData& rht
);

double compute_ir_similarity(
    const SubmissionData& lft,
    const SubmissionData& rht
);

double compute_overall_similarity(
    double token_score,
    double ast_score,
    bool has_ast,
    double ir_score,
    bool has_ir
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

);

1.167. plagiarism\{}\similarity\{}\similarity_params.h

Листинг 167: plagiarism\{}\similarity\{}\similarity_params.h

1.168. plagiarism\{}\similarity\{}\winnowing.cpp

Листинг 168: plagiarism\{}\similarity\{}\winnowing.cpp

```
#include "winnowing.h"

#include <algorithm>
#include <limits>

std::uint64_t splitmix64(std::uint64_t x) {
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

inline std::uint64_t get_kgram_hash(const std::string& str) {
    std::uint64_t hash = kFnv1aOffsetBasis;
    for (unsigned char c : str) {
        hash ^= static_cast<std::uint64_t>(c);
        hash *= kFnv1aPrime;
    }
    return hash;
}

std::vector<std::uint64_t> build_kgram_hashes(
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    const std::vector<std::string>& token_texts,
    int k
) {
    if (token_texts.size() < k) {
        std::string ret;
        for (size_t j = 0; j < token_texts.size(); j++) {
            if (j > 0) {
                ret += "|";
            }
            ret += token_texts[j];
        }
        return {get_kgram_hash(ret)};
    }

    std::vector<std::uint64_t> hashes;
    for (size_t i = 0; i + k - 1 < token_texts.size(); i++) {
        std::string kgram;
        for (size_t j = 0; j < k; j++) {
            if (j > 0) {
                kgram += "|";
            }
            kgram += token_texts[i + j];
        }
        hashes.push_back(get_kgram_hash(kgram));
    }
    return hashes;
}

std::vector<std::uint64_t> run_winnowing(
    const std::vector<std::uint64_t>& hashes,
    int window
) {
    std::vector<std::uint64_t> fingerprints;

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

if (hashes.size() < static_cast<size_t>(window)) {
    return {*std::min_element(hashes.begin(), hashes.end())};
}

for (size_t i = 0; i + window - 1 < hashes.size(); i++) {
    std::uint64_t min_hash = hashes[i];
    for (size_t j = 1; j < window; j++) {
        min_hash = std::min(min_hash, hashes[i + j]);
    }
    fingerprints.push_back(min_hash);
}

std::vector<std::uint64_t> compressed;
for (auto h : fingerprints) {
    if (compressed.empty() || compressed.back() != h) {
        compressed.push_back(h);
    }
}

return compressed;
}

std::vector<std::uint64_t> build_minhash_signature(const std::vector<std::uint64_t
↪ >& fingerprints) {
    std::vector<std::uint64_t> ret;
    for (size_t i = 0; i < MINHASH_SIZE; i++) {
        std::uint64_t min_hash = std::numeric_limits<std::uint64_t>::max();

        for (auto h : fingerprints) {
            min_hash = std::min(min_hash, splitmix64(h ^ i));
        }
        ret.push_back(min_hash);
    }

    return ret;
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1.169. plagiarism\{}\similarity\{}\winnowing.h

Листинг 169: plagiarism\{}\similarity\{}\winnowing.h

```
#pragma once
#include <cstdint>
#include <string>
#include <vector>

inline constexpr std::uint64_t kFnv1aOffsetBasis = 1469598103934665603ULL;
inline constexpr std::uint64_t kFnv1aPrime = 1099511628211ULL;

std::uint64_t splitmix64(std::uint64_t x);

inline constexpr std::size_t MINHASH_SIZE = 128;

std::uint64_t get_kgram_hash(const std::string& str);

const size_t KGRAM = 5;

std::vector<std::uint64_t> build_kgram_hashes(
    const std::vector<std::string>& token_texts,
    int k = KGRAM
);

std::vector<std::uint64_t> run_winnowing(
    const std::vector<std::uint64_t>& hashes,
    int window
);
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
std::vector < uint64_t > build_minhash_signature(const std::vector<std::uint64_t>&
    ↪ fingerprints);
```

1.170. plagiarism\{}submission_builder\{}submission_builder.cpp

Листинг 170: plagiarism\{}submission_builder\{}submission_builder.cpp

```
#include "submission_builder.h"

#include "submission_stages.h"
#include "../similarity/winnowing.h"

SubmissionData BuildSubmissionData(const Submission& submission) {
    SubmissionData dat;

    dat.submission_id = submission.id;
    dat.raw_code = submission.rawCode;
    dat.ast_code = BuildSubmissionAstCode(submission);
    dat.token_code = BuildSubmissionTokenCode(submission);
    // if (submission.language == ProgrammingLanguage::Cpp) {
    // dat.ir_code = BuildSubmissionIrCode(submission.rawCode);
    // dat.ir_parse_ok = !dat.ir_code.empty();
    // } else {
        dat.ir_code.clear();
        dat.ir_parse_ok = false;
    // }

    dat.ir_parse_ok = !dat.ir_code.empty();

    dat.tokens = BuildSubmissionTokens(dat.token_code);
    dat.normalized_tokens = BuildNormalizedSubmissionTokens(dat.tokens);
    dat.normalized_token_texts = BuildNormalizedTokenTexts(dat.normalized_tokens);

    auto hashes = build_kgram_hashes(dat.normalized_token_texts, KGRAM);
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

auto fingerprints = run_winning(hashes, 5);
dat.winning_features.fingerprints = fingerprints;
dat.winning_features.minhash_signature = build_minhash_signature(fingerprints
    ↪ );

dat.token_features = BuildSubmissionTokenFeatures(dat.tokens, dat.
    ↪ normalized_tokens);

auto ast_pair = BuildSubmissionAstAndFeatures(dat.ast_code);
dat.ast_features = std::move(ast_pair.first);
const AstTree ast_tree = std::move(ast_pair.second);
dat.ast_subtree_hash_freq = BuildSubmissionAstSubtreeHashFreq(ast_tree);
dat.ast_normalized_sequence = BuildSubmissionAstNormalizedSequence(ast_tree);

return dat;
}

```

1.171. plagiarism\{}submission_builder\{}submission_builder.h

Листинг 171: plagiarism\{}submission_builder\{}submission_builder.h

```

#pragma once

#include "../models/submission.h"
#include "../models/submission_data.h"

SubmissionData BuildSubmissionData(const Submission& submission);

```

1.172. plagiarism\{}submission_builder\{}submission_stages.cpp

Листинг 172: plagiarism\{}submission_builder\{}submission_stages.cpp

```

#include "submission_stages.h"

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

#include "../ast/ast_hashing.h"
#include "../ast/ast_analyzer.h"
#include "../preprocessing/normalize.h"
#include "../preprocessing/llvm_ir_pipeline.h"
#include "../tokenizer/normalize_tokens.h"
#include "../tokenizer/token_features_builder.h"
#include "../tokenizer/tokenizer.h"

std::string BuildSubmissionAstCode(const Submission& submission) {
    return NormalizeForAST(submission.rawCode);
}

std::string BuildSubmissionTokenCode(const Submission& submission) {
    return NormalizeForTokenizer(submission.rawCode);
}

std::string BuildSubmissionIrCode(const std::string& normalized_code) {
    if (normalized_code.empty()) {
        return "";
    }

    const std::string prelude =
        "#include<bits/stdc++.h>\n"
        "using namespace std;\n";

    return BuildLlvmIrFromCode(prelude + normalized_code);
}

std::vector<Token> BuildSubmissionTokens(const std::string& token_code) {
    return TokenizerWithClang(token_code);
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

std::vector<Token> BuildNormalizedSubmissionTokens(const std::vector<Token>&
    ↪ tokens) {
    return NormalizeTokens(tokens);
}

std::vector<std::string> BuildNormalizedTokenTexts(
    const std::vector<Token>& normalized_tokens
) {
    return TextTokens(normalized_tokens);
}

TokenFeatures BuildSubmissionTokenFeatures(
    const std::vector<Token>& raw_tokens,
    const std::vector<Token>& normalized_tokens
) {
    return BuildTokenFeatures(raw_tokens, normalized_tokens);
}

AstFeatures BuildSubmissionAstFeatures(const std::string& ast_code) {
    return analyze_ast(ast_code);
}

AstTree BuildSubmissionAstTree(const std::string& ast_code) {
    return build_ast_tree(ast_code);
}

std::unordered_map<std::uint64_t, int> BuildSubmissionAstSubtreeHashFreq(
    const AstTree& tree,
    int min_subtree_size
) {

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    return BuildAstSubtreeHashFreq(tree, min_subtree_size);
}

std::vector<std::string> BuildSubmissionAstNormalizedSequence(const AstTree& tree)
    ↪ {
    return BuildAstNormalizedSequencePreorder(tree);
}

std::pair<AstFeatures, AstTree> BuildSubmissionAstAndFeatures(const std::string&
    ↪ ast_code) {
    return analyze_and_build_ast(ast_code);
}

```

1.173. plagiarism\{}submission_builder\{}submission_stages.h

ЛИСТИНГ 173: plagiarism\{}submission_builder\{}submission_stages.h

```

#pragma once

#include <unordered_map>
#include <string>
#include <vector>

#include "../ast/ast_features.h"
#include "../ast/ast_hashing.h"
#include "../models/submission.h"
#include "../models/token.h"
#include "../models/token_features.h"

std::string BuildSubmissionAstCode(const Submission& submission);
std::string BuildSubmissionTokenCode(const Submission& submission);
std::string BuildSubmissionIrCode(const std::string& normalized_code);

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
std::vector<Token> BuildSubmissionTokens(const std::string& token_code);
std::vector<Token> BuildNormalizedSubmissionTokens(const std::vector<Token>&
    ↪ tokens);
std::vector<std::string> BuildNormalizedTokenTexts(
    const std::vector<Token>& normalized_tokens
);

TokenFeatures BuildSubmissionTokenFeatures(
    const std::vector<Token>& raw_tokens,
    const std::vector<Token>& normalized_tokens
);

AstFeatures BuildSubmissionAstFeatures(const std::string& ast_code);
AstTree BuildSubmissionAstTree(const std::string& ast_code);
std::pair<AstFeatures, AstTree> BuildSubmissionAstAndFeatures(const std::string&
    ↪ ast_code);
std::unordered_map<std::uint64_t, int> BuildSubmissionAstSubtreeHashFreq(
    const AstTree& tree,
    int min_subtree_size = 2
);
std::vector<std::string> BuildSubmissionAstNormalizedSequence(const AstTree& tree)
    ↪ ;
```

1.174. plagiarism\{}\submission_loader\{}\submission_loader.cpp

Листинг 174: plagiarism\{}\submission_loader\{}\submission_loader.cpp

```
#include "submission_loader.h"

#include <filesystem>
#include <fstream>
#include <sstream>
#include <iostream>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

namespace fs = std::filesystem;

namespace {

bool is_supported_source_file(const fs::path& path) {
    const std::string ext = path.extension().string();
    return ext == ".cpp" || ext == ".cc" || ext == ".cxx" || ext == ".c++" || ext
        ↪ == ".py";
}

void load_submissions_from_directory(
    const fs::path& dir,
    const std::string& participant,
    const std::string& problem,
    std::vector<Submission>& submissions,
    int& submission_id
) {
    if (!fs::exists(dir) || !fs::is_directory(dir)) {
        return;
    }

    for (const auto& file_entry : fs::directory_iterator(dir)) {
        if (!file_entry.is_regular_file()) continue;

        if (!is_supported_source_file(file_entry.path())) continue;

        Submission sub = SubmissionLoader::load_submission(std::to_string(
            ↪ submission_id), file_entry.path().string());
        sub.participant = participant;
        sub.problem = problem;
        sub.file_name = file_entry.path().filename().string();
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        sub.source_path = file_entry.path().string();
        submissions.push_back(sub);
        ++submission_id;
    }
}

}

ProgrammingLanguage SubmissionLoader::detect_language(const std::string& file_path
↪ ) {
    std::string ext = fs::path(file_path).extension().string();

    if (ext == ".cpp" || ext == ".cc" || ext == ".cxx" || ext == ".c++") {
        return ProgrammingLanguage::Cpp;
    }

    if (ext == ".py") {
        return ProgrammingLanguage::Python;
    }

    return ProgrammingLanguage::Unknown;
}

std::string SubmissionLoader::read_file_content(const std::string& file_path) {
    std::ifstream file(file_path);
    if (!file.is_open()) {
        std::cerr << "Error: Cannot open file" << file_path << std::endl;
        return "";
    }

    std::stringstream buffer;
    buffer << file.rdbuf();
    return buffer.str();
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

}

```
Submission SubmissionLoader::load_submission(const std::string& id, const std::
    ↳ string& file_path) {
    Submission sub;
    sub.id = id;
    sub.language = detect_language(file_path);
    sub.rawCode = read_file_content(file_path);
    return sub;
}
```

```
std::vector<Submission> SubmissionLoader::load_from_directory(const std::string&
    ↳ root_dir) {
    std::vector<Submission> submissions;

    if (!fs::exists(root_dir)) {
        std::cerr << "Error: Directory " << root_dir << " does not exist" << std::
            ↳ endl;
        return submissions;
    }

    int submission_id = 1;

    for (const auto& user_entry : fs::directory_iterator(root_dir)) {
        if (!user_entry.is_directory()) continue;

        const std::string participant = user_entry.path().filename().string();
        load_submissions_from_directory(user_entry.path(), participant, "",
            ↳ submissions, submission_id);

        for (const auto& entry : fs::directory_iterator(user_entry.path())) {
            if (!entry.is_directory()) continue;
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        const std::string problem = entry.path().filename().string();
        load_submissions_from_directory(entry.path(), participant, problem,
            ↪ submissions, submission_id);
    }
}

return submissions;
}

std::vector<Submission> SubmissionLoader::load_problem_submissions(
    const std::string& root_dir,
    const std::vector<std::string>& problem_letters
) {
    std::vector<Submission> submissions;

    if (!fs::exists(root_dir)) {
        std::cerr << "Error: Directory " << root_dir << " does not exist" << std::
            ↪ endl;
        return submissions;
    }

    int submission_id = 1;

    for (const auto& user_entry : fs::directory_iterator(root_dir)) {
        if (!user_entry.is_directory()) continue;

        const std::string participant = user_entry.path().filename().string();

        for (const auto& file_entry : fs::directory_iterator(user_entry.path())) {
            if (!file_entry.is_regular_file()) continue;

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

std::string filename = file_entry.path().filename().string();

for (const std::string& problem_letter : problem_letters) {
    //    "B-"    "C-"    ..
    if (filename.rfind(problem_letter + "-", 0) == 0) {
        if (!is_supported_source_file(file_entry.path())) break;

        Submission sub = SubmissionLoader::load_submission(std::
            ↪ to_string(submission_id), file_entry.path().string());
        sub.participant = participant;
        sub.problem = problem_letter;
        sub.file_name = filename;
        sub.source_path = file_entry.path().string();
        submissions.push_back(sub);
        ++submission_id;
        break;
    }
}

return submissions;
}

```

1.175. plagiarism\{}submission_loader\{}submission_loader.h

Листинг 175: plagiarism\{}submission_loader\{}submission_loader.h

```
#pragma once
```

```
#include <string>
```

```
#include <vector>
```

```
#include "../models/submission.h"
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

#include "../models/programming_language.h"

class SubmissionLoader {
public:
    static std::vector<Submission> load_from_directory(const std::string& root_dir)
        ↪ ;

    static std::vector<Submission> load_problem_submissions(
        const std::string& root_dir,
        const std::vector<std::string>& problem_letters
    );

    static Submission load_submission(const std::string& id, const std::string&
        ↪ file_path);

private:
    static ProgrammingLanguage detect_language(const std::string& file_path);

    static std::string read_file_content(const std::string& file_path);
};

```

1.176. plagiarism\{}tokenizer\{}normalize_tokens.cpp

Листинг 176: plagiarism\{}tokenizer\{}normalize_tokens.cpp

```

#include <unordered_set>

#include "normalize_tokens.h"

const std::unordered_set<std::string> allowed_ids = {
    "std",
    "vector",
    "string",

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

"map",
"set",
"unordered_map",
"unordered_set",
"queue",
"stack",
"deque",
"pair",
"tuple",
"sort",
"lower_bound",
"upper_bound",
"push_back",
"emplace_back",
"begin",
"end",
"cin",
"cout",
"endl",
"min",
"max",
"swap",
"abs"
};

bool IsString(const std::string& str) {
    return (!str.empty() && str[0] == '"');
}

bool IsChar(const std::string& str) {
    return (!str.empty() && str[0] == '\\');
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

Token NormalizeToken(const Token& token) {
    Token returned_token = token;

    if (token.type == TokenType::Identifier) {
        if (allowed_ids.find(token.text) == allowed_ids.end()) {
            returned_token.text = "ID";
        }

    } else if (token.type == TokenType::Literal) {
        if (IsString(token.text)) {
            returned_token.text = "STR";
        } else if (IsChar(token.text)) {
            returned_token.text = "CHAR";
        } else {
            returned_token.text = "NUM";
        }
    }

    return returned_token;
}

std::vector<Token> NormalizeTokens(const std::vector<Token>& tokens) {
    std::vector<Token> return_tokens;
    return_tokens.reserve(tokens.size());

    for (const auto& token : tokens) {
        return_tokens.push_back(NormalizeToken(token));
    }

    return return_tokens;
}

std::vector<std::string> TextTokens(const std::vector<Token>& tokens) {
    std::vector < std::string > return_texts;

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    return_texts.reserve(tokens.size());
    for (const auto& token : tokens) {
        return_texts.push_back(token.text);
    }
    return return_texts;
}

```

1.177. plagiarism\{}tokenizer\{}normalize_tokens.h

Листинг 177: plagiarism\{}tokenizer\{}normalize_tokens.h

```

#pragma once

#include <string>
#include <vector>

#include "../models/token.h"

Token NormalizeToken(const Token& token);
std::vector<Token> NormalizeTokens(const std::vector<Token>& tokens);

std::vector<std::string> TextTokens(const std::vector<Token>& tokens);

```

1.178. plagiarism\{}tokenizer\{}token_features_builder.cpp

Листинг 178: plagiarism\{}tokenizer\{}token_features_builder.cpp

```

#include "token_features_builder.h"

#include <unordered_set>

static bool IsParenPunctuation(const Token& token) {
    return token.type == TokenType::Punctuation && (token.text == "(" || token.text
        ↪ == ")");
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

}

```
TokenFeatures BuildTokenFeatures(const std::vector<Token>& raw_tokens,
                                const std::vector<Token>& normalized_tokens) {
    TokenFeatures features;

    for (const auto& token : normalized_tokens) {
        features.tokens_cnt++;
        if (token.type == TokenType::Identifier) {
            features.identifiers_cnt++;
        } else if (token.type == TokenType::Literal) {
            features.literals_cnt++;
        } else if (token.type == TokenType::Punctuation) {
            features.punctuations_cnt++;
        } else if (token.type == TokenType::Keyword) {
            features.keywords_cnt++;
        }
        features.freq[token.text]++;
    }

    std::unordered_set<std::string> uniq;
    for (const auto& token : raw_tokens) {
        uniq.insert(token.text);
    }

    std::vector<std::string> grams_source;
    grams_source.reserve(normalized_tokens.size());
    for (const auto& token : normalized_tokens) {
        if (IsParenPunctuation(token)) {
            continue;
        }
        grams_source.push_back(token.text);
    }
}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

}

for (size_t i = 0; i + 2 < grams_source.size(); i++) {
    features.grams3.push_back(
        grams_source[i] + "|" + grams_source[i + 1] + "|" + grams_source[i + 2]
    );
}

features.unique_tokens = static_cast<int>(uniq.size());
return features;
}

```

1.179. plagiarism\{}tokenizer\{}token_features_builder.h

Листинг 179: plagiarism\{}tokenizer\{}token_features_builder.h

```

#pragma once

#include <vector>

#include "../models/token.h"
#include "../models/token_features.h"

TokenFeatures BuildTokenFeatures(const std::vector<Token>& raw_tokens,
    const std::vector<Token>& normalized_tokens);

```

1.180. plagiarism\{}tokenizer\{}tokenizer.cpp

Листинг 180: plagiarism\{}tokenizer\{}tokenizer.cpp

```

#include "tokenizer.h"
#include <clang-c/Index.h>

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
#include <string>
#include <vector>

static std::string ConvertCxStringToStdString(CXString s) {
    const char* cstr = clang_getCString(s);
    std::string result = cstr ? cstr : "";
    clang_disposeString(s);
    return result;
}

static TokenType ConvertClangTokenKindToTokenType(CXTokenKind kind) {
    if (kind == CXToken_Punctuation) {
        return TokenType::Punctuation;
    }
    if (kind == CXToken_Keyword) {
        return TokenType::Keyword;
    }
    if (kind == CXToken_Identifier) {
        return TokenType::Identifier;
    }
    if (kind == CXToken_Literal) {
        return TokenType::Literal;
    }
    return TokenType::Unknown;
}

static std::size_t GetTokenStartOffset(CXTranslationUnit tu, CXToken token) {
    CXSourceLocation location = clang_getTokenLocation(tu, token);

    unsigned offset = 0;
    clang_getSpellingLocation(location, nullptr, nullptr, nullptr, &offset);
}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    return static_cast<std::size_t>(offset);
}

std::vector<Token> TokenizerWithClang(const std::string& code) {
    std::vector<Token> result;

    CXIndex index = clang_createIndex(0, 0);

    const char* filename = "input.cpp";

    CXUnsavedFile unsaved;
    unsaved.Filename = filename;
    unsaved.Contents = code.c_str();
    unsaved.Length = code.size();

    const char* args[] = {
        "-x", "c++",
        "-std=c++20"
    };

    CXTranslationUnit tu = nullptr;
    CXErrorCode err = clang_parseTranslationUnit2(
        index,
        filename,
        args, 3,
        &unsaved, 1,
        CXTranslationUnit_None,
        &tu
    );

    if (err != CXError_Success || tu == nullptr) {
        clang_disposeIndex(index);
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    return result;
}

CXFile file = clang_getFile(tu, filename);
if (file == nullptr) {
    clang_disposeTranslationUnit(tu);
    clang_disposeIndex(index);
    return result;
}

CXSourceLocation start = clang_getLocationForOffset(tu, file, 0);
CXSourceLocation end =
    clang_getLocationForOffset(tu, file, static_cast<unsigned>(code.size()));

CXSourceRange range = clang_getRange(start, end);

CXToken* tokens = nullptr;
unsigned numTokens = 0;
clang_tokenize(tu, range, &tokens, &numTokens);

result.reserve(numTokens);

for (unsigned i = 0; i < numTokens; ++i) {
    Token token;
    token.type = ConvertClangTokenKindToTokenType(clang_getTokenKind(tokens[i])
        ↪ );
    token.text = ConvertCxStringToStdString(clang_getTokenSpelling(tu, tokens[i]
        ↪ ));
    token.offset = GetTokenStartOffset(tu, tokens[i]);
    result.push_back(token);
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
clang_disposeTokens(tu, tokens, numTokens);  
clang_disposeTranslationUnit(tu);  
clang_disposeIndex(index);  
  
return result;  
}
```

1.181. plagiarism\{}tokenizer\{}tokenizer.h

Листинг 181: plagiarism\{}tokenizer\{}tokenizer.h

```
#pragma once  
  
#include <string>  
#include <vector>  
#include "../models/token.h"  
  
std::vector<Token> TokenizerWithClang(const std::string& code);
```

1.182. tests\{}__init__.py

Листинг 182: tests\{}__init__.py

1.183. tests\{}api_tests\{}__init__.py

Листинг 183: tests\{}api_tests\{}__init__.py

1.184. tests\{}api_tests\{}test_base.py

Листинг 184: tests\{}api_tests\{}test_base.py

```
from backend.app.server import app
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
from fastapi.testclient import TestClient

client = TestClient(app)
```

1.185. tests\api_tests\test_health.py

Листинг 185: tests\api_tests\test_health.py

1.186. tests\api_tests\user\test_change_password.py

Листинг 186: tests\api_tests\user\test_change_password.py

```
from backend.tests.api_tests.test_base import client

# Basic successful password change
def test_change_password_success():
    client.post(
        "/register",
        json={
            "login": "change_password_success_user",
            "password": "Aa1!aaaa",
            "email": "change_password_success_user@example.com",
        },
    )

    login_response = client.post(
        "/login",
        json={
            "login": "change_password_success_user",
            "password": "Aa1!aaaa",
        },
    )
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
)

refresh_token = login_response.json()["refresh_token"]

response = client.post(
    "/change_password",
    json={
        "payload": {
            "old_password": "Aa1!aaaa",
            "new_password": "Bb2@bbbb",
            "confirm_password": "Bb2@bbbb",
        },
        "token": {
            "refresh_token": refresh_token,
        },
    },
)

assert response.status_code == 200
assert response.json() == {
    "message": "Password changed successfully. Please login again."
}
```

Refresh token does not exist

```
def test_change_password_invalid_token():
```

```
    response = client.post(
        "/change_password",
        json={
            "payload": {
                "old_password": "Aa1!aaaa",
                "new_password": "Bb2@bbbb",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        "confirm_password": "Bb2@bbbb",
    },
    "token": {
        "refresh_token": "invalid_refresh_token",
    },
},
)

assert response.status_code == 401
assert response.json() == {"detail": "Invalid_token"}
```

Old password is wrong

```
def test_change_password_invalid_old_password():
    client.post(
        "/register",
        json={
            "login": "change_password_wrong_old_user",
            "password": "Aa1!aaaa",
            "email": "change_password_wrong_old_user@example.com",
        },
    )

    login_response = client.post(
        "/login",
        json={
            "login": "change_password_wrong_old_user",
            "password": "Aa1!aaaa",
        },
    )

    refresh_token = login_response.json()["refresh_token"]
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
response = client.post(
    "/change_password",
    json={
        "payload": {
            "old_password": "WrongPassword123!",
            "new_password": "Bb2@bbbb",
            "confirm_password": "Bb2@bbbb",
        },
        "token": {
            "refresh_token": refresh_token,
        },
    },
)

assert response.status_code == 400
assert response.json() == {"detail": "Invalid_old_password"}

# New password and confirm password do not match
def test_change_password_confirmation_mismatch():
    client.post(
        "/register",
        json={
            "login": "change_password_mismatch_user",
            "password": "Aa1!aaaa",
            "email": "change_password_mismatch_user@example.com",
        },
    )

    login_response = client.post(
        "/login",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        json={
            "login": "change_password_mismatch_user",
            "password": "Aa1!aaaa",
        },
    )

    refresh_token = login_response.json()["refresh_token"]

    response = client.post(
        "/change_password",
        json={
            "payload": {
                "old_password": "Aa1!aaaa",
                "new_password": "Bb2@bbbb",
                "confirm_password": "Cc3#cccc",
            },
            "token": {
                "refresh_token": refresh_token,
            },
        },
    )

    assert response.status_code == 400
    assert response.json() == {"detail": "New_password_and_confirmation_do_not_
        ↪ match"}

# New password is too short
def test_change_password_too_short_new_password():
    client.post(
        "/register",
        json={
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        "login": "change_password_short_user",
        "password": "Aa1!aaaa",
        "email": "change_password_short_user@example.com",
    },
)

login_response = client.post(
    "/login",
    json={
        "login": "change_password_short_user",
        "password": "Aa1!aaaa",
    },
)

refresh_token = login_response.json()["refresh_token"]

response = client.post(
    "/change_password",
    json={
        "payload": {
            "old_password": "Aa1!aaaa",
            "new_password": "Bb2@bbb",
            "confirm_password": "Bb2@bbb",
        },
        "token": {
            "refresh_token": refresh_token,
        },
    },
)

assert response.status_code == 400
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
assert response.json() == {"detail": "Password must be at least 8 characters_
↳ long"}
```

Old password should not work after password change

```
def test_change_password_old_password_is_invalid_after_change():
```

```
    client.post(
        "/register",
        json={
            "login": "change_password_old_invalid_user",
            "password": "Aa1!aaaa",
            "email": "change_password_old_invalid_user@example.com",
        },
    )
```

```
login_response = client.post(
    "/login",
    json={
        "login": "change_password_old_invalid_user",
        "password": "Aa1!aaaa",
    },
)
```

```
refresh_token = login_response.json()["refresh_token"]
```

```
change_response = client.post(
    "/change_password",
    json={
        "payload": {
            "old_password": "Aa1!aaaa",
            "new_password": "Bb2@bbbb",
            "confirm_password": "Bb2@bbbb",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    },
    "token": {
        "refresh_token": refresh_token,
    },
},
)

old_login_response = client.post(
    "/login",
    json={
        "login": "change_password_old_invalid_user",
        "password": "Aa1!aaaa",
    },
)

assert change_response.status_code == 200
assert old_login_response.status_code == 401
assert old_login_response.json() == {"detail": "Invalid_login_or_password"}
```

New password should work after password change

```
def test_change_password_new_password_works_after_change():
```

```
    client.post(
        "/register",
        json={
            "login": "change_password_new_valid_user",
            "password": "Aa1!aaaa",
            "email": "change_password_new_valid_user@example.com",
        },
    )

    login_response = client.post(
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
"/login",
json={
    "login": "change_password_new_valid_user",
    "password": "Aa1!aaaa",
},
)

refresh_token = login_response.json()["refresh_token"]

change_response = client.post(
    "/change_password",
    json={
        "payload": {
            "old_password": "Aa1!aaaa",
            "new_password": "Bb2@bbbb",
            "confirm_password": "Bb2@bbbb",
        },
        "token": {
            "refresh_token": refresh_token,
        },
    },
)

new_login_response = client.post(
    "/login",
    json={
        "login": "change_password_new_valid_user",
        "password": "Bb2@bbbb",
    },
)

assert change_response.status_code == 200
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
assert new_login_response.status_code == 200
```

All sessions should be closed after password change

```
def test_change_password_logs_out_all_sessions():
```

```
    client.post(
        "/register",
        json={
            "login": "change_password_logout_all_user",
            "password": "Aa1!aaaa",
            "email": "change_password_logout_all_user@example.com",
        },
    )
```

```
login_response_1 = client.post(
    "/login",
    json={
        "login": "change_password_logout_all_user",
        "password": "Aa1!aaaa",
    },
)
```

```
login_response_2 = client.post(
    "/login",
    json={
        "login": "change_password_logout_all_user",
        "password": "Aa1!aaaa",
    },
)
```

```
refresh_token_1 = login_response_1.json()["refresh_token"]
```

```
refresh_token_2 = login_response_2.json()["refresh_token"]
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
change_response = client.post(
    "/change_password",
    json={
        "payload": {
            "old_password": "Aa1!aaaa",
            "new_password": "Bb2@bbbb",
            "confirm_password": "Bb2@bbbb",
        },
        "token": {
            "refresh_token": refresh_token_1,
        },
    },
)
```

```
logout_response_1 = client.post(
    "/logout",
    json={
        "refresh_token": refresh_token_1,
    },
)
```

```
logout_response_2 = client.post(
    "/logout",
    json={
        "refresh_token": refresh_token_2,
    },
)
```

```
assert change_response.status_code == 200
assert logout_response_1.status_code == 400
assert logout_response_1.json() == {"detail": "Invalid refresh token"}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
assert logout_response_2.status_code == 400
assert logout_response_2.json() == {"detail": "Invalid_refresh_token"}
```

Missing payload field

```
def test_change_password_without_payload():
```

```
    response = client.post(
        "/change_password",
        json={
            "token": {
                "refresh_token": "some_token",
            },
        },
    )
```

```
    assert response.status_code == 422
```

Missing token field

```
def test_change_password_without_token():
```

```
    response = client.post(
        "/change_password",
        json={
            "payload": {
                "old_password": "Aa1!aaaa",
                "new_password": "Bb2@bbbb",
                "confirm_password": "Bb2@bbbb",
            },
        },
    )
```

```
    assert response.status_code == 422
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
# New password has no digit
def test_change_password_without_digit():
    client.post(
        "/register",
        json={
            "login": "change_password_without_digit_user",
            "password": "Aa1!aaaa",
            "email": "change_password_without_digit_user@example.com",
        },
    )

    login_response = client.post(
        "/login",
        json={
            "login": "change_password_without_digit_user",
            "password": "Aa1!aaaa",
        },
    )

    refresh_token = login_response.json()["refresh_token"]

    response = client.post(
        "/change_password",
        json={
            "payload": {
                "old_password": "Aa1!aaaa",
                "new_password": "Aa!aaaaa",
                "confirm_password": "Aa!aaaaa",
            },
            "token": {
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        "refresh_token": refresh_token,
    },
},
)

assert response.status_code == 400
assert response.json() == {"detail": "Password must contain at least one digit"
    ↪ }
```

New password has no lowercase latin letter

```
def test_change_password_without_lowercase_latin_letter():
```

```
    client.post(
        "/register",
        json={
            "login": "change_password_without_lowercase_user",
            "password": "Aa1!aaaa",
            "email": "change_password_without_lowercase_user@example.com",
        },
    )
```

```
login_response = client.post(
    "/login",
    json={
        "login": "change_password_without_lowercase_user",
        "password": "Aa1!aaaa",
    },
)
```

```
refresh_token = login_response.json()["refresh_token"]
```

```
response = client.post(
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
"/change_password",
json={
    "payload": {
        "old_password": "Aa1!aaaa",
        "new_password": "AA1!AAAA",
        "confirm_password": "AA1!AAAA",
    },
    "token": {
        "refresh_token": refresh_token,
    },
},
)

assert response.status_code == 400
assert response.json() == {
    "detail": "Password must contain at least one lowercase Latin letter"
}

# New password has no uppercase latin letter
def test_change_password_without_uppercase_latin_letter():
    client.post(
        "/register",
        json={
            "login": "change_password_without_uppercase_user",
            "password": "Aa1!aaaa",
            "email": "change_password_without_uppercase_user@example.com",
        },
    )

    login_response = client.post(
        "/login",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    json={
        "login": "change_password_without_uppercase_user",
        "password": "Aa1!aaaa",
    },
)

refresh_token = login_response.json()["refresh_token"]

response = client.post(
    "/change_password",
    json={
        "payload": {
            "old_password": "Aa1!aaaa",
            "new_password": "aa1!aaaa",
            "confirm_password": "aa1!aaaa",
        },
        "token": {
            "refresh_token": refresh_token,
        },
    },
)

assert response.status_code == 400
assert response.json() == {
    "detail": "Password must contain at least one uppercase Latin letter"
}

# New password has no special symbol
def test_change_password_without_special_symbol():
    client.post(
        "/register",

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    json={
        "login": "change_password_without_special_user",
        "password": "Aa1!aaaa",
        "email": "change_password_without_special_user@example.com",
    },
)

login_response = client.post(
    "/login",
    json={
        "login": "change_password_without_special_user",
        "password": "Aa1!aaaa",
    },
)

refresh_token = login_response.json()["refresh_token"]

response = client.post(
    "/change_password",
    json={
        "payload": {
            "old_password": "Aa1!aaaa",
            "new_password": "Aa1aaaaa",
            "confirm_password": "Aa1aaaaa",
        },
        "token": {
            "refresh_token": refresh_token,
        },
    },
)

assert response.status_code == 400
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
assert response.json() == {
    "detail": "Password must contain at least one special symbol"
}

# New password has only lowercase latin letters
def test_change_password_only_lowercase_latin_letters():
    client.post(
        "/register",
        json={
            "login": "change_password_only_lowercase_user",
            "password": "Aa1!aaaa",
            "email": "change_password_only_lowercase_user@example.com",
        },
    )

    login_response = client.post(
        "/login",
        json={
            "login": "change_password_only_lowercase_user",
            "password": "Aa1!aaaa",
        },
    )

    refresh_token = login_response.json()["refresh_token"]

    response = client.post(
        "/change_password",
        json={
            "payload": {
                "old_password": "Aa1!aaaa",
                "new_password": "aaabbbccc",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        "confirm_password": "aaabbbccc",
    },
    "token": {
        "refresh_token": refresh_token,
    },
},
)

assert response.status_code == 400
assert response.json() == {"detail": "Only lowercase letters!"}
```

New password has only uppercase latin letters

```
def test_change_password_only_uppercase_latin_letters():
    client.post(
        "/register",
        json={
            "login": "change_password_only_uppercase_user",
            "password": "Aa1!aaaa",
            "email": "change_password_only_uppercase_user@example.com",
        },
    )

    login_response = client.post(
        "/login",
        json={
            "login": "change_password_only_uppercase_user",
            "password": "Aa1!aaaa",
        },
    )

    refresh_token = login_response.json()["refresh_token"]
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
response = client.post(
    "/change_password",
    json={
        "payload": {
            "old_password": "Aa1!aaaa",
            "new_password": "AAABBBCCC",
            "confirm_password": "AAABBBCCC",
        },
        "token": {
            "refresh_token": refresh_token,
        },
    },
)

assert response.status_code == 400
assert response.json() == {"detail": "Only uppercase letters!"}

# New password has cyrillic letters
def test_change_password_with_cyrillic_letters():
    client.post(
        "/register",
        json={
            "login": "change_password_cyrillic_user",
            "password": "Aa1!aaaa",
            "email": "change_password_cyrillic_user@example.com",
        },
    )

    login_response = client.post(
        "/login",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    json={
        "login": "change_password_cyrillic_user",
        "password": "Aa1!aaaa",
    },
)

refresh_token = login_response.json()["refresh_token"]

response = client.post(
    "/change_password",
    json={
        "payload": {
            "old_password": "Aa1!aaaa",
            "new_password": "1!",
            "confirm_password": "1!",
        },
        "token": {
            "refresh_token": refresh_token,
        },
    },
)

assert response.status_code == 400
assert response.json() == {"detail": "Password must contain only Latin letters"}
↪ }

```

1.187. tests\{ }api_tests\{ }user\{ }test_login.py

Листинг 187: tests\{ }api_tests\{ }user\{ }test_login.py

```
from backend.tests.api_tests.test_base import client
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
def assert_login_response_json(response):
    data = response.json()

    assert response.status_code == 200
    assert "access_token" in data
    assert "refresh_token" in data
    assert "token_type" in data
    assert data["access_token"] != ""
    assert data["refresh_token"] != ""
    assert data["token_type"] == "bearer"
    assert data["access_token"] != data["refresh_token"]

# Basic successful login
def test_login_success():
    client.post(
        "/register",
        json={
            "login": "login_success_user",
            "password": "Aa1!aaaa",
            "email": "login_success_user@example.com",
        },
    )

    response = client.post(
        "/login",
        json={
            "login": "login_success_user",
            "password": "Aa1!aaaa",
        },
    )
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
assert response.status_code == 200
```

```
# User does not exist
```

```
def test_login_user_not_found():
```

```
    response = client.post(
        "/login",
        json={
            "login": "missing_user",
            "password": "Aa1!aaaa",
        },
    )
```

```
    assert response.status_code == 401
```

```
    assert response.json() == {"detail": "Invalid_login_or_password"}
```

```
# Wrong password for existing user
```

```
def test_login_wrong_password():
```

```
    client.post(
        "/register",
        json={
            "login": "wrong_password_user",
            "password": "Aa1!aaaa",
            "email": "wrong_password_user@example.com",
        },
    )
```

```
    response = client.post(
```

```
        "/login",
```

```
        json={
```

```
            "login": "wrong_password_user",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        "password": "WrongPassword123!",
    },
)

assert response.status_code == 401
assert response.json() == {"detail": "Invalid_login_or_password"}

# Check full json in normal successful login
def test_login_returns_correct_json():
    client.post(
        "/register",
        json={
            "login": "json_user",
            "password": "Aa1!aaaa",
            "email": "json_user@example.com",
        },
    )

    response = client.post(
        "/login",
        json={
            "login": "json_user",
            "password": "Aa1!aaaa",
        },
    )

    assert_login_response_json(response)

# Check full json after repeated login
def test_login_twice_returns_correct_json():
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
client.post(
    "/register",
    json={
        "login": "login_twice_user",
        "password": "Aa1!aaaa",
        "email": "login_twice_user@example.com",
    },
)

response_1 = client.post(
    "/login",
    json={
        "login": "login_twice_user",
        "password": "Aa1!aaaa",
    },
)

response_2 = client.post(
    "/login",
    json={
        "login": "login_twice_user",
        "password": "Aa1!aaaa",
    },
)

assert_login_response_json(response_1)
assert_login_response_json(response_2)
assert response_1.json()["refresh_token"] != response_2.json()["refresh_token"]
```

Check full json for another user too

`def test_login_another_user_returns_correct_json():`

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
client.post(
    "/register",
    json={
        "login": "another_login_user",
        "password": "Bb2@bbbb",
        "email": "another_login_user@example.com",
    },
)
```

```
response = client.post(
    "/login",
    json={
        "login": "another_login_user",
        "password": "Bb2@bbbb",
    },
)
```

```
assert_login_response_json(response)
```

Two different users should both get correct json

```
def test_login_two_users_return_correct_json():
```

```
    client.post(
        "/register",
        json={
            "login": "first_json_user",
            "password": "Aa1!aaaa",
            "email": "first_json_user@example.com",
        },
    )
```

```
    client.post(
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
"/register",
json={
    "login": "second_json_user",
    "password": "Bb2@bbbb",
    "email": "second_json_user@example.com",
},
)

response_1 = client.post(
    "/login",
    json={
        "login": "first_json_user",
        "password": "Aa1!aaaa",
    },
)

response_2 = client.post(
    "/login",
    json={
        "login": "second_json_user",
        "password": "Bb2@bbbb",
    },
)

assert_login_response_json(response_1)
assert_login_response_json(response_2)
assert response_1.json()["access_token"] != response_2.json()["access_token"]
assert response_1.json()["refresh_token"] != response_2.json()["refresh_token"]

# Missing login field
def test_login_without_login():
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

response = client.post(
    "/login",
    json={
        "password": "Aa1!aaaa",
    },
)

assert response.status_code == 422

# Missing password field
def test_login_without_password():
    response = client.post(
        "/login",
        json={
            "login": "some_user",
        },
    )

    assert response.status_code == 422

```

1.188. tests\api_tests\user\test_logout.py

Листинг 188: tests\api_tests\user\test_logout.py

```

from backend.tests.api_tests.test_base import client

# Basic successful logout
def test_logout_success():
    client.post(
        "/register",
        json={

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        "login": "logout_success_user",
        "password": "Aa1!aaaa",
        "email": "logout_success_user@example.com",
    },
)

login_response = client.post(
    "/login",
    json={
        "login": "logout_success_user",
        "password": "Aa1!aaaa",
    },
)

refresh_token = login_response.json()["refresh_token"]

response = client.post(
    "/logout",
    json={
        "refresh_token": refresh_token,
    },
)

assert response.status_code == 200
assert response.json() == {"message": "Logout successful!"}

# Refresh token does not exist
def test_logout_invalid_refresh_token():
    response = client.post(
        "/logout",
        json={
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        "refresh_token": "invalid_refresh_token",
    },
)

assert response.status_code == 400
assert response.json() == {"detail": "Invalid_refresh_token"}

# Same refresh token is used again
def test_logout_twice_with_same_refresh_token():
    client.post(
        "/register",
        json={
            "login": "logout_twice_user",
            "password": "Aa1!aaaa",
            "email": "logout_twice_user@example.com",
        },
    )

    login_response = client.post(
        "/login",
        json={
            "login": "logout_twice_user",
            "password": "Aa1!aaaa",
        },
    )

    refresh_token = login_response.json()["refresh_token"]

    response_1 = client.post(
        "/logout",
        json={
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        "refresh_token": refresh_token,
    },
)
```

```
response_2 = client.post(
    "/logout",
    json={
        "refresh_token": refresh_token,
    },
)
```

```
assert response_1.status_code == 200
assert response_1.json() == {"message": "Logout successful!"}

assert response_2.status_code == 400
assert response_2.json() == {"detail": "Invalid refresh token"}
```

Missing refresh token field

```
def test_logout_without_refresh_token():
    response = client.post(
        "/logout",
        json={},
    )

    assert response.status_code == 422
```

User logs in two times and logs out from both sessions

```
def test_logout_from_two_sessions():
    client.post(
        "/register",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    json={
        "login": "logout_two_sessions_user",
        "password": "Aa1!aaaa",
        "email": "logout_two_sessions_user@example.com",
    },
)

login_response_1 = client.post(
    "/login",
    json={
        "login": "logout_two_sessions_user",
        "password": "Aa1!aaaa",
    },
)

login_response_2 = client.post(
    "/login",
    json={
        "login": "logout_two_sessions_user",
        "password": "Aa1!aaaa",
    },
)

refresh_token_1 = login_response_1.json()["refresh_token"]
refresh_token_2 = login_response_2.json()["refresh_token"]

response_1 = client.post(
    "/logout",
    json={
        "refresh_token": refresh_token_1,
    },
)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
response_2 = client.post(
    "/logout",
    json={
        "refresh_token": refresh_token_2,
    },
)

assert login_response_1.status_code == 200
assert login_response_2.status_code == 200
assert refresh_token_1 != refresh_token_2

assert response_1.status_code == 200
assert response_1.json() == {"message": "Logout successful!"}

assert response_2.status_code == 200
assert response_2.json() == {"message": "Logout successful!"}
```

1.189. tests\api_tests\user\test_logout_all.py

Листинг 189: tests\api_tests\user\test_logout_all.py

```
from backend.tests.api_tests.test_base import client

# Basic successful logout all
def test_logout_all_success():
    client.post(
        "/register",
        json={
            "login": "logout_all_success_user",
            "password": "Aa1!aaaa",
            "email": "logout_all_success_user@example.com",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    },
)

login_response = client.post(
    "/login",
    json={
        "login": "logout_all_success_user",
        "password": "Aa1!aaaa",
    },
)

refresh_token = login_response.json()["refresh_token"]

response = client.post(
    "/logout_all",
    json={
        "refresh_token": refresh_token,
    },
)

assert response.status_code == 200
assert response.json() == {"message": "Logout_all_successful!"}

# Refresh token does not exist
def test_logout_all_invalid_refresh_token():
    response = client.post(
        "/logout_all",
        json={
            "refresh_token": "invalid_refresh_token",
        },
    )
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
assert response.status_code == 400  
assert response.json() == {"detail": "Invalid_refresh_token"}
```

User logs in two times and logs out from all sessions

```
def test_logout_all_removes_all_user_sessions():  
    client.post(  
        "/register",  
        json={  
            "login": "logout_all_two_sessions_user",  
            "password": "Aa1!aaaa",  
            "email": "logout_all_two_sessions_user@example.com",  
        },  
    )  
  
    login_response_1 = client.post(  
        "/login",  
        json={  
            "login": "logout_all_two_sessions_user",  
            "password": "Aa1!aaaa",  
        },  
    )  
  
    login_response_2 = client.post(  
        "/login",  
        json={  
            "login": "logout_all_two_sessions_user",  
            "password": "Aa1!aaaa",  
        },  
    )
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

refresh_token_1 = login_response_1.json()["refresh_token"]
refresh_token_2 = login_response_2.json()["refresh_token"]

logout_all_response = client.post(
    "/logout_all",
    json={
        "refresh_token": refresh_token_1,
    },
)

logout_response_1 = client.post(
    "/logout",
    json={
        "refresh_token": refresh_token_1,
    },
)

logout_response_2 = client.post(
    "/logout",
    json={
        "refresh_token": refresh_token_2,
    },
)

assert logout_all_response.status_code == 200
assert logout_all_response.json() == {"message": "Logout_all_successful!"}

assert logout_response_1.status_code == 400
assert logout_response_1.json() == {"detail": "Invalid_refresh_token"}

assert logout_response_2.status_code == 400
assert logout_response_2.json() == {"detail": "Invalid_refresh_token"}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
# Logout all should not affect another user
def test_logout_all_does_not_affect_another_user():
    client.post(
        "/register",
        json={
            "login": "first_logout_all_user",
            "password": "Aa1!aaaa",
            "email": "first_logout_all_user@example.com",
        },
    )

    client.post(
        "/register",
        json={
            "login": "second_logout_all_user",
            "password": "Bb2@bbbb",
            "email": "second_logout_all_user@example.com",
        },
    )

    first_login_response_1 = client.post(
        "/login",
        json={
            "login": "first_logout_all_user",
            "password": "Aa1!aaaa",
        },
    )

    first_login_response_2 = client.post(
        "/login",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        json={
            "login": "first_logout_all_user",
            "password": "Aa1!aaaa",
        },
    )

second_login_response = client.post(
    "/login",
    json={
        "login": "second_logout_all_user",
        "password": "Bb2@bbbb",
    },
)

first_refresh_token_1 = first_login_response_1.json()["refresh_token"]
first_refresh_token_2 = first_login_response_2.json()["refresh_token"]
second_refresh_token = second_login_response.json()["refresh_token"]

logout_all_response = client.post(
    "/logout_all",
    json={
        "refresh_token": first_refresh_token_1,
    },
)

first_logout_response_1 = client.post(
    "/logout",
    json={
        "refresh_token": first_refresh_token_1,
    },
)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
first_logout_response_2 = client.post(
    "/logout",
    json={
        "refresh_token": first_refresh_token_2,
    },
)

second_logout_response = client.post(
    "/logout",
    json={
        "refresh_token": second_refresh_token,
    },
)

assert logout_all_response.status_code == 200
assert logout_all_response.json() == {"message": "Logout_all_successful!"}

assert first_logout_response_1.status_code == 400
assert first_logout_response_1.json() == {"detail": "Invalid_refresh_token"}

assert first_logout_response_2.status_code == 400
assert first_logout_response_2.json() == {"detail": "Invalid_refresh_token"}

assert second_logout_response.status_code == 200
assert second_logout_response.json() == {"message": "Logout_successful!"}

# Same refresh token is used again after logout all
def test_logout_all_twice_with_same_refresh_token():
    client.post(
        "/register",
        json={
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        "login": "logout_all_twice_user",
        "password": "Aa1!aaaa",
        "email": "logout_all_twice_user@example.com",
    },
)

login_response = client.post(
    "/login",
    json={
        "login": "logout_all_twice_user",
        "password": "Aa1!aaaa",
    },
)

refresh_token = login_response.json()["refresh_token"]

response_1 = client.post(
    "/logout_all",
    json={
        "refresh_token": refresh_token,
    },
)

response_2 = client.post(
    "/logout_all",
    json={
        "refresh_token": refresh_token,
    },
)

assert response_1.status_code == 200
assert response_1.json() == {"message": "Logout_all_successful!"}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
assert response_2.status_code == 400
assert response_2.json() == {"detail": "Invalid_refresh_token"}
```

Missing refresh token field

```
def test_logout_all_without_refresh_token():
    response = client.post(
        "/logout_all",
        json={},
    )

    assert response.status_code == 422
```

1.190. tests\api_tests\user\test_refresh.py

ЛИСТИНГ 190: tests\api_tests\user\test_refresh.py

```
from backend.tests.api_tests.test_base import client
```

```
def assert_refresh_response_json(response):
    data = response.json()

    assert response.status_code == 200
    assert "access_token" in data
    assert "refresh_token" in data
    assert "token_type" in data
    assert data["token_type"] == "bearer"
```

Basic successful refresh

```
def test_refresh_success():
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
client.post(
    "/register",
    json={
        "login": "refresh_success_user",
        "password": "Aa1!aaaa",
        "email": "refresh_success_user@example.com",
    },
)

login_response = client.post(
    "/login",
    json={
        "login": "refresh_success_user",
        "password": "Aa1!aaaa",
    },
)

old_refresh_token = login_response.json()["refresh_token"]

response = client.post(
    "/refresh",
    json={
        "refresh_token": old_refresh_token,
    },
)

assert response.status_code == 200

# Check full json after refresh
def test_refresh_returns_correct_json():
    client.post(
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
"/register",
json={
    "login": "refresh_json_user",
    "password": "Aa1!aaaa",
    "email": "refresh_json_user@example.com",
},
)

login_response = client.post(
    "/login",
    json={
        "login": "refresh_json_user",
        "password": "Aa1!aaaa",
    },
)

old_refresh_token = login_response.json()["refresh_token"]

response = client.post(
    "/refresh",
    json={
        "refresh_token": old_refresh_token,
    },
)

assert_refresh_response_json(response)

# New refresh token should be different from old one
def test_refresh_returns_new_refresh_token():
    client.post(
        "/register",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
    json={
        "login": "refresh_new_token_user",
        "password": "Aa1!aaaa",
        "email": "refresh_new_token_user@example.com",
    },
)

login_response = client.post(
    "/login",
    json={
        "login": "refresh_new_token_user",
        "password": "Aa1!aaaa",
    },
)

old_refresh_token = login_response.json()["refresh_token"]

refresh_response = client.post(
    "/refresh",
    json={
        "refresh_token": old_refresh_token,
    },
)

new_refresh_token = refresh_response.json()["refresh_token"]

assert refresh_response.status_code == 200
assert new_refresh_token != old_refresh_token

# Old refresh token should not work after refresh
def test_refresh_old_token_is_invalid_after_refresh():
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
client.post(
    "/register",
    json={
        "login": "refresh_old_token_user",
        "password": "Aa1!aaaa",
        "email": "refresh_old_token_user@example.com",
    },
)

login_response = client.post(
    "/login",
    json={
        "login": "refresh_old_token_user",
        "password": "Aa1!aaaa",
    },
)

old_refresh_token = login_response.json()["refresh_token"]

first_refresh_response = client.post(
    "/refresh",
    json={
        "refresh_token": old_refresh_token,
    },
)

second_refresh_response = client.post(
    "/refresh",
    json={
        "refresh_token": old_refresh_token,
    },
)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
assert first_refresh_response.status_code == 200
assert second_refresh_response.status_code == 401
assert second_refresh_response.json() == {"detail": "Invalid_refresh_token"}
```

New refresh token should work too

```
def test_refresh_new_token_can_be_used_again():
```

```
    client.post(
        "/register",
        json={
            "login": "refresh_again_user",
            "password": "Aa1!aaaa",
            "email": "refresh_again_user@example.com",
        },
    )
```

```
login_response = client.post(
    "/login",
    json={
        "login": "refresh_again_user",
        "password": "Aa1!aaaa",
    },
)
```

```
first_refresh_token = login_response.json()["refresh_token"]
```

```
first_refresh_response = client.post(
    "/refresh",
    json={
        "refresh_token": first_refresh_token,
    },
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

)

```
second_refresh_token = first_refresh_response.json()["refresh_token"]
```

```
second_refresh_response = client.post(
    "/refresh",
    json={
        "refresh_token": second_refresh_token,
    },
)
```

```
assert first_refresh_response.status_code == 200
assert second_refresh_response.status_code == 200
assert_refresh_response_json(second_refresh_response)
```

Refresh token does not exist

```
def test_refresh_invalid_refresh_token():
    response = client.post(
        "/refresh",
        json={
            "refresh_token": "invalid_refresh_token",
        },
    )
```

```
assert response.status_code == 401
assert response.json() == {"detail": "Invalid refresh token"}
```

Missing refresh token field

```
def test_refresh_without_refresh_token():
    response = client.post(
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        "/refresh",
        json={},
    )

    assert response.status_code == 422
```

1.191. tests\{}\api_tests\{}\user\{}\test_register.py

Листинг 191: tests\{}\api_tests\{}\user\{}\test_register.py

```
from backend.tests.api_tests.test_base import client

# Basic successful case
def test_register_success():
    response = client.post(
        "/register",
        json={
            "login": "test_user_success",
            "password": "Aa1!aaaa",
            "email": "test_user_success@example.com",
        },
    )
    assert response.status_code == 200

# Same user tries to register again
def test_register_existing_user():
    response = client.post(
        "/register",
        json={
            "login": "test_user_duplicate",
            "password": "Aa1!aaaa",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        "email": "test_user_duplicate@example.com",
    },
)
assert response.status_code == 200

response = client.post(
    "/register",
    json={
        "login": "test_user_duplicate",
        "password": "Bb2@bbbb",
        "email": "test_user_duplicate_second@example.com",
    },
)
assert response.status_code == 400
assert response.json() == {"detail": "User with this login already exists"}
```

Password is shorter than 8 symbols

```
def test_register_password_too_short():
    response = client.post(
        "/register",
        json={
            "login": "short_password_user",
            "password": "Aa1!aaa",
            "email": "short_password_user@example.com",
        },
    )
    assert response.status_code == 400
    assert response.json() == {"detail": "Password must be at least 8 characters
    ↳ long"}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Password has exactly 8 symbols

```
def test_register_password_exactly_8_symbols():
```

```
    response = client.post(
```

```
        "/register",
```

```
        json={
```

```
            "login": "exactly_8_user",
```

```
            "password": "Aa1!aaaa",
```

```
            "email": "exactly_8_user@example.com",
```

```
        },
```

```
    )
```

```
    assert response.status_code == 200
```

No digit in password

```
def test_register_password_without_digit():
```

```
    response = client.post(
```

```
        "/register",
```

```
        json={
```

```
            "login": "without_digit_user",
```

```
            "password": "Aa!aaaaa",
```

```
            "email": "without_digit_user@example.com",
```

```
        },
```

```
    )
```

```
    assert response.status_code == 400
```

```
    assert response.json() == {"detail": "Password must contain at least one digit"
```

```
    } }
```

No lowercase latin letter

```
def test_register_password_without_lowercase_latin_letter():
```

```
    response = client.post(
```

```
        "/register",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    json={
        "login": "without_lowercase_user",
        "password": "AA1!AAAA",
        "email": "without_lowercase_user@example.com",
    },
)
assert response.status_code == 400
assert response.json() == {
    "detail": "Password must contain at least one lowercase Latin letter"
}
```

No uppercase latin letter

```
def test_register_password_without_uppercase_latin_letter():
    response = client.post(
        "/register",
        json={
            "login": "without_uppercase_user",
            "password": "aa1!aaaa",
            "email": "without_uppercase_user@example.com",
        },
    )
    assert response.status_code == 400
    assert response.json() == {
        "detail": "Password must contain at least one uppercase Latin letter"
    }
```

No special symbol

```
def test_register_password_without_special_symbol():
    response = client.post(
        "/register",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
        json={
            "login": "without_special_user",
            "password": "Aa1aaaaa",
            "email": "without_special_user@example.com",
        },
    )
    assert response.status_code == 400
    assert response.json() == {
        "detail": "Password must contain at least one special symbol"
    }

# Password has only digits
def test_register_password_only_digits():
    response = client.post(
        "/register",
        json={
            "login": "only_digits_user",
            "password": "12345678",
            "email": "only_digits_user@example.com",
        },
    )
    assert response.status_code == 400
    assert response.json() == {"detail": "Password format is invalid"}
```

```
# Password has only special symbols
def test_register_password_only_special_symbols():
    response = client.post(
        "/register",
        json={
            "login": "only_specials_user",
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        "password": "!@#$%^&*",
        "email": "only_specials_user@example.com",
    },
)

assert response.status_code == 400

assert response.json() == {"detail": "Password_format_is_invalid"}
```

Password has only lowercase latin letters

```
def test_register_password_only_lowercase_latin_letters():
```

```
    response = client.post(
        "/register",
        json={
            "login": "only_lowercase_user",
            "password": "aaabbbccc",
            "email": "only_lowercase_user@example.com",
        },
    )

    assert response.status_code == 400

    assert response.json() == {"detail": "Only_lowercase_letters!"}
```

Password has only uppercase latin letters

```
def test_register_password_only_uppercase_latin_letters():
```

```
    response = client.post(
        "/register",
        json={
            "login": "only_uppercase_user",
            "password": "AAABBBCCC",
            "email": "only_uppercase_user@example.com",
        },
    )
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
assert response.status_code == 400
assert response.json() == {"detail": "Only uppercase letters!"}
```

Password has cyrillic letters

```
def test_register_password_with_cyrillic_letters():
    response = client.post(
        "/register",
        json={
            "login": "cyrillic_password_user",
            "password": "1!",
            "email": "cyrillic_password_user@example.com",
        },
    )
    assert response.status_code == 400
    assert response.json() == {"detail": "Password must contain only Latin letters"}
    ↪ }
```

Password has both latin and cyrillic letters

```
def test_register_password_with_mixed_latin_and_cyrillic_letters():
    response = client.post(
        "/register",
        json={
            "login": "mixed_letters_user",
            "password": "Aa1!",
            "email": "mixed_letters_user@example.com",
        },
    )
    assert response.status_code == 400
    assert response.json() == {"detail": "Password must contain only Latin letters"}
    ↪ }
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
# Password has a space
def test_register_password_with_space():
    response = client.post(
        "/register",
        json={
            "login": "password_with_space_user",
            "password": "Aa1!_aaab",
            "email": "password_with_space_user@example.com",
        },
    )
    assert response.status_code == 400
    assert response.json() == {"detail": "Password must not contain spaces"}
```

```
# Long valid password
def test_register_long_valid_password():
    response = client.post(
        "/register",
        json={
            "login": "long_valid_password_user",
            "password": "VeryStrongPassword123!A",
            "email": "long_valid_password_user@example.com",
        },
    )
    assert response.status_code == 200
```

```
# Minimal valid password by rules
def test_register_password_minimal_valid_complexity():
    response = client.post(
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    "/register",
    json={
        "login": "minimal_valid_complexity_user",
        "password": "A1!bobau",
        "email": "minimal_valid_complexity_user@example.com",
    },
)
assert response.status_code == 200

# Email is not valid
def test_register_with_invalid_email():
    response = client.post(
        "/register",
        json={
            "login": "invalid_email_user",
            "password": "Aa1!aaaa",
            "email": "not-an-email",
        },
    )
    assert response.status_code == 422

```

1.192. tests\{ }api_tests\{ }user\{ }test_unlink_codeforces.py

Листинг 192: tests\{ }api_tests\{ }user\{ }test_unlink_codeforces.py

```

from datetime import datetime, timedelta, timezone

from backend.app.database import get_db
from backend.tests.api_tests.test_base import client

from models.user.refresh_token import RefreshToken
from models.user.user import User

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

# Codeforces keys should be removed from db
def test_unlink_codeforces_removes_keys_from_db():
    login = "unlink_cf_user_1"
    refresh_token_value = "refresh_1"

    db = get_db()
    try:
        user = User(
            login=login,
            password="fake_password_hash",
            email=f"{login}@example.com",
            codeforces_api_key="fake_cf_key",
            codeforces_api_secret="fake_cf_secret",
        )
        db.add(user)
        db.commit()

        db_refresh_token = RefreshToken(
            user_login=login,
            refresh_token=refresh_token_value,
            created_at=datetime.now(timezone.utc),
            expires_in=datetime.now(timezone.utc) + timedelta(days=7),
        )
        db.add(db_refresh_token)
        db.commit()
    finally:
        db.close()

    response = client.post(
        "/unlink_codeforces",

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        json={
            "refresh_token": refresh_token_value,
        },
    )

db = get_db()
try:
    updated_user = db.query(User).filter(User.login == login).first()
finally:
    db.close()

assert response.status_code == 200
assert response.json() == {"message": "Codeforces_account_unlinked!"}
assert updated_user is not None
assert updated_user.codeforces_api_key is None
assert updated_user.codeforces_api_secret is None

```

1.193. tests\api_tests\user\test_unlink_yandex.py

Листинг 193: tests\api_tests\user\test_unlink_yandex.py

```

from datetime import datetime, timedelta, timezone

from backend.app.database import get_db
from backend.tests.api_tests.test_base import client

from models.user.refresh_token import RefreshToken
from models.user.user import User

# Yandex token should be removed from db
def test_unlink_yandex_removes_token_from_db():
    login = "unlink_yandex_user_2"

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
refresh_token_value = "refresh_2"

db = get_db()

try:
    user = User(
        login=login,
        password="fake_password_hash",
        email=f"{login}@example.com",
        yandex_access_token="fake_yandex_access_token",
    )
    db.add(user)
    db.commit()

    db_refresh_token = RefreshToken(
        user_login=login,
        refresh_token=refresh_token_value,
        created_at=datetime.now(timezone.utc),
        expires_in=datetime.now(timezone.utc) + timedelta(days=7),
    )
    db.add(db_refresh_token)
    db.commit()

finally:
    db.close()

response = client.post(
    "/unlink_yandex",
    json={
        "refresh_token": refresh_token_value,
    },
)

db = get_db()
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
try:
    updated_user = db.query(User).filter(User.login == login).first()
finally:
    db.close()

assert response.status_code == 200
assert response.json() == {"message": "Yandex_account_unlinked!"}
assert updated_user is not None
assert updated_user.yandex_access_token is None
```

1.194. tests\{ }api_tests\{ }yandex\{ }test_login.py

Листинг 194: tests\{ }api_tests\{ }yandex\{ }test_login.py

```
def test_trash():
    assert True
```

2. ТЕКСТ ПРОГРАММЫ (Frontend)

2.1. App.tsx

Листинг 195: App.tsx

```
import {BrowserRouter, Routes, Route, Navigate} from 'react-router-dom';
import {Landing} from './pages/Landing';
import {LoginPage} from './pages/LoginPage';
import {RegisterPage} from './pages/RegisterPage';
import {useAuthStore} from './store/useAuthStore';
import {ThemeInitializer} from './components/ThemeInitializer.tsx';
import {MainLayout} from './components/layout/MainLayout.tsx';
import {AuthInitializer} from './components/AuthInitializer.tsx';
import {Profile} from './pages/Profile.tsx';
import {YandexCallback} from './pages/YandexCallback.tsx';
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

import {CodeforcesCallback} from "../pages/CodeforcesCallback.tsx";
import {LoadContestPage} from "../pages/LoadContestPage.tsx";
import {ContestsPage} from "../pages/ContestsPage.tsx";
import {ContestViewPage} from "../pages/ContestViewPage.tsx";
import {NotFound} from "../pages/NotFound.tsx";
import {ContestLayout} from "../components/layout/ContestLayout.tsx";
import {ContestOverview} from "../pages/ContestOverview.tsx";
import {ContestSubmissions} from "../pages/ContestSubmissions.tsx";
import {ContestAnalytics} from "../pages/ContestAnalytics.tsx";
import {AITasksList} from "../pages/AITasksList.tsx";
import {AITaskSession} from "../pages/AITaskSession.tsx";
import {ImportSubmissions} from "../pages/ImportSubmissions.tsx";
import {SubmissionSource} from "../pages/SubmissionSource.tsx";
import {PlagiarismReport} from "../pages/PlagiarismReport.tsx";
import {PlagiarismComparison} from "../pages/PlagiarismComparison.tsx";
import {PlagiarismSetup} from "../pages/PlagiarismSetup.tsx";

function App() {
  const {isAuthenticated} = useAuthStore();

  return (
    <BrowserRouter>
      <AuthInitializer/>
      <ThemeInitializer/>
      <Routes>
        <Route path="/" element={isAuthenticated ? <MainLayout/> : <Landing
          ↪ />}/>
        <Route path="/login" element={isAuthenticated ? <MainLayout/> : <
          ↪ LoginPage/>}/>
        <Route path="/register" element={isAuthenticated ? <MainLayout/> : <
          ↪ RegisterPage/>}/>
      </Routes>
    </BrowserRouter>
  );
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<Route path="/" element={isAuthenticated ? <MainLayout/> : <Navigate
  ↪ to="/login"/>}>
  <Route path="contests" element={<ContestsPage/>}/>
  <Route path="contests/sync" element={<LoadContestPage/>}/>
  <Route path="contests/:id" element={<ContestLayout/>}>
    <Route index element={<ContestOverview/>}/>
    <Route path="table" element={<ContestViewPage/>}/>
    <Route path="submissions" element={<ContestSubmissions/>}/>
    <Route path="/contests/:id/submissions/:subId" element={<
      ↪ SubmissionSource />} />
    <Route path="import-submissions" element={<ImportSubmissions
      ↪ />}/>
    <Route path="analytics" element={<ContestAnalytics/>} />
    <Route path="analytics/check" element={<PlagiarismSetup />}
      ↪ />
    <Route path="analytics/reports/:reportId" element={<
      ↪ PlagiarismReport />} />
    <Route path="analytics/compare/:pairId" element={<
      ↪ PlagiarismComparison />} />
  </Route>
  <Route path="participants" element={<div> </div>}/>
  <Route path="ai-tasks" element={<AITasksList/>}/>
  <Route path="ai-tasks/:sessionId" element={<AITaskSession/>}/>
  <Route path="profile" element={<Profile/>}/>
  <Route path="yandex/callback" element={<YandexCallback/>}/>
  <Route path="codeforces/callback" element={<CodeforcesCallback
    ↪ />}/>
</Route>

<Route path="*" element={<NotFound/>}/>
</Routes>
</BrowserRouter>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    );  
}  
  
export default App;
```

2.2. index.css

Листинг 196: index.css

```
@import "tailwindcss";  
  
@variant dark (&:where(.dark, .dark *));  
  
html, body {  
    margin: 0;  
    padding: 0;  
    transition: background-color 0.3s ease, color 0.3s ease;  
}
```

2.3. main.tsx

Листинг 197: main.tsx

```
import { StrictMode } from 'react'  
import { createRoot } from 'react-dom/client'  
import './index.css'  
import App from './App.tsx'  
  
createRoot(document.getElementById('root')!).render(  
    <StrictMode>  
        <App />  
    </StrictMode>,  
)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2.4. api\{}\instance.ts

Листинг 198: api\{}\instance.ts

```
import axios from 'axios';
import {useAuthStore} from "../store/useAuthStore.ts";

const API_URL = import.meta.env.VITE_API_URL;

export const api = axios.create({
  baseURL: API_URL,
  headers: {
    'Content-Type': 'application/json',
  },
})

api.interceptors.request.use((config) => {
  const {accessToken, tokenType} = useAuthStore.getState();
  if (accessToken && config.headers) {
    config.headers.Authorization = `${tokenType} ${accessToken}`;
  }
  return config;
});

let isRefreshing = false;
let refreshSubscribers: any[] = [];

const subscribeTokenRefresh = (cb: any) => {
  refreshSubscribers.push(cb);
}

const onRefreshed = (token: any, type: any) => {
  refreshSubscribers.map((cb) => cb(token, type));
}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    refreshSubscribers = [];  
}  
  
api.interceptors.response.use((response) => response, async (error) => {  
    const {config, response} = error;  
    const request = config;  
  
    if (response?.status === 401 && !request._retry) {  
        if (isRefreshing) {  
            return new Promise(resolve => {  
                subscribeTokenRefresh((newToken: any, newType: any) => {  
                    request.headers.Authorization = `${newType} ${newToken}`;  
                    resolve(api(request));  
                })  
            });  
        }  
  
        request._retry = true;  
        isRefreshing = true;  
  
        const {accessToken, refreshToken, tokenType} = useAuthStore.getState();  
  
        if (refreshToken) {  
            try {  
                const response = await axios.post(`${import.meta.env.VITE_API_URL}/  
                    ↪ auth/refresh`, {  
                    access_token: accessToken,  
                    refresh_token: refreshToken,  
                    token_type: tokenType,  
                })  
  
                const {access_token, refresh_token, token_type} = response.data;
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        useAuthStore.getState().updateTokens(access_token, refresh_token,
            ↪ token_type);

        isRefreshing = false;
        onRefreshed(access_token, token_type);

        request.headers.Authorization = `${token_type} ${access_token}`;
        return api(request);
    } catch (refreshError) {
        isRefreshing = false;
        refreshSubscribers = [];

        useAuthStore.getState().logout();

        if (window.location.pathname !== '/login') {
            window.location.href = '/login';
        }

        return Promise.reject(refreshError);
    }
}

return Promise.reject(error);
});

```

2.5. components\{}\AuthInitializer.tsx

Листинг 199: components\{}\AuthInitializer.tsx

```

import { useEffect } from 'react';
import { useAuthStore } from '../store/useAuthStore';

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

import { api } from '../api/instance';

export const AuthInitializer = () => {
  const { isAuthenticated, user, setUser, logout } = useAuthStore();

  useEffect(() => {
    const fetchMe = async () => {
      if (isAuthenticated && !user) {
        try {
          const res = await api.get('/auth/me');
          setUser(res.data);
        } catch (e) {
          console.error("Session expired or invalid");
          logout();
        }
      }
    };

    fetchMe();
  }, [isAuthenticated, user]);

  return null;
};

```

2.6. components\{}\Pagination.tsx

Листинг 200: components\{}\Pagination.tsx

```

import { ChevronLeft, ChevronRight, ChevronsLeft, ChevronsRight } from 'lucide-
  ↩ react';

interface PaginationProps {
  page: number;
  totalPages: number;

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

total: number;
perPage: number;
onPageChange: (page: number) => void;
onPerPageChange: (perPage: number) => void;
}

export const Pagination = ({ page, totalPages, total, perPage, onPageChange,
  ↪ onPerPageChange }: PaginationProps) => {
  const getPageNumbers = () => {
    const pages: (number | string)[] = [];
    const delta = 2;

    if (totalPages <= 7) {
      for (let i = 1; i <= totalPages; i++) pages.push(i);
      return pages;
    }

    pages.push(1);
    if (page - delta > 2) pages.push('...');

    for (let i = Math.max(2, page - delta); i <= Math.min(totalPages - 1, page
      ↪ + delta); i++) {
      pages.push(i);
    }

    if (page + delta < totalPages - 1) pages.push('...');
    if (totalPages > 1) pages.push(totalPages);

    return pages;
  };

  const from = (page - 1) * perPage + 1;

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
const to = Math.min(page * perPage, total);

return (
  <div className="flex flex-col sm:flex-row items-center justify-between gap
    ↪ -4 px-2">
    {/*      */}
    <div className="flex items-center gap-4 text-sm text-slate-500">
      <span>
        <span className="font-bold text-slate-700 dark:text-slate
          ↪ -300">{from}{to}</span> {' '}
        <span className="font-bold text-slate-700 dark:text-slate-300">{
          ↪ total}</span>
      </span>
      <select
        value={perPage}
        onChange={(e) => onPerPageChange(Number(e.target.value))}
        className="bg-white dark:bg-slate-800 border border-slate-200
          ↪ dark:border-slate-700 rounded-lg px-2 py-1 text-sm outline
          ↪ -none focus:ring-2 focus:ring-blue-500 dark:text-white
          ↪ cursor-pointer"
      >
        {[10, 25, 50, 100, 200].map(n => (
          <option key={n} value={n}>{n} / .</option>
        ))}
      </select>
    </div>

    {/*      */}
    <div className="flex items-center gap-1">
      <button
        onClick={() => onPageChange(1)}
        disabled={page === 1}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
className="p-2 rounded-lg hover:bg-slate-100 dark:hover:bg-slate
  ↪ -800 disabled:opacity-30 disabled:cursor-not-allowed
  ↪ transition-colors"
title=" "
>
  <ChevronsLeft size={16} className="text-slate-500" />
</button>
<button
  onClick={() => onPageChange(page - 1)}
  disabled={page === 1}
  className="p-2 rounded-lg hover:bg-slate-100 dark:hover:bg-slate
    ↪ -800 disabled:opacity-30 disabled:cursor-not-allowed
    ↪ transition-colors"
  title=""
>
  <ChevronLeft size={16} className="text-slate-500" />
</button>

{getPageNumbers().map((p, idx) =>
  typeof p === 'string' ? (
    <span key={`dots-${idx}`} className="px-2 text-slate-400 text
      ↪ -sm"></span>
  ) : (
    <button
      key={p}
      onClick={() => onPageChange(p)}
      className={`min-w-9 h-9 rounded-lg text-sm font-medium
        ↪ transition-all ${
          p === page
            ? 'bg-blue-500 text-white shadow-md shadow-blue
              ↪ -500/25'

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        : 'text-slate-600 dark:text-slate-400 hover:bg-
        ↪ slate-100 dark:hover:bg-slate-800'

      }' }
    >
      {p}
    </button>
  )
)}

<button
  onClick={() => onPageChange(page + 1)}
  disabled={page === totalPages}
  className="p-2 rounded-lg hover:bg-slate-100 dark:hover:bg-slate
    ↪ -800 disabled:opacity-30 disabled:cursor-not-allowed
    ↪ transition-colors"
  title=""
>
  <ChevronRight size={16} className="text-slate-500" />
</button>
<button
  onClick={() => onPageChange(totalPages)}
  disabled={page === totalPages}
  className="p-2 rounded-lg hover:bg-slate-100 dark:hover:bg-slate
    ↪ -800 disabled:opacity-30 disabled:cursor-not-allowed
    ↪ transition-colors"
  title=" "
>
  <ChevrnsRight size={16} className="text-slate-500" />
</button>
</div>
</div>
);

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
};
```

2.7. components\ThemeInitializer.tsx

Листинг 201: components\ThemeInitializer.tsx

```
import { useEffect } from 'react';
import { useThemeStore } from '../store/useThemeStore';

export const ThemeInitializer = () => {
  const { theme } = useThemeStore();

  useEffect(() => {
    const root = window.document.documentElement;

    const isDark =
      theme === 'dark' ||
      (theme === 'system' && window.matchMedia('(prefers-color-scheme: dark)')
        .matches);

    if (isDark) {
      root.classList.add('dark');
    } else {
      root.classList.remove('dark');
    }
  }, [theme]);

  return null;
};
```

2.8. components\ThemeToggle.tsx

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Листинг 202: components\ThemeToggle.tsx

```

import { Moon, Sun, Monitor } from 'lucide-react';
import { useThemeStore } from '../store/useThemeStore';

export const ThemeToggle = () => {
  const { theme, setTheme } = useThemeStore();

  return (
    <div className="flex bg-slate-100 dark:bg-slate-800 p-1 rounded-full border
      ↪ border-slate-200 dark:border-slate-700">
      <button
        onClick={() => setTheme('light')}
        className={`p-2 rounded-full transition-all ${theme === 'light' ? '
          ↪ bg-white shadow-sm text-blue-600' : 'text-slate-500 hover:text
          ↪ -slate-700'}`}
      >
        <Sun size={18} />
      </button>
      <button
        onClick={() => setTheme('system')}
        className={`p-2 rounded-full transition-all ${theme === 'system' ? '
          ↪ bg-white shadow-sm text-blue-600' : 'text-slate-500 hover:text
          ↪ -slate-700'}`}
      >
        <Monitor size={18} />
      </button>
      <button
        onClick={() => setTheme('dark')}
        className={`p-2 rounded-full transition-all ${theme === 'dark' ? 'bg
          ↪ -white shadow-sm text-blue-600' : 'text-slate-500 hover:text-
          ↪ slate-700'}`}
      >

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        <Moon size={18} />
      </button>
    </div>
  );
};
```

2.9. components\layout\ContestLayout.tsx

Листинг 203: components\layout\ContestLayout.tsx

```
import { Outlet, Link, useParams, useLocation } from 'react-router-dom';
import { Table, FileCode, BarChart3, Info, ChevronLeft, Download } from 'lucide-
  ↪ react';

export const ContestLayout = () => {
  const { id } = useParams();
  const location = useLocation();

  const tabs = [
    { icon: Info, label: '', path: '/contests/${id}' },
    { icon: Table, label: ' ', path: '/contests/${id}/table' },
    { icon: Download, label: ' ', path: '/contests/${id}/import-submissions' },
    { icon: FileCode, label: ' ', path: '/contests/${id}/submissions' },
    { icon: BarChart3, label: ' ', path: '/contests/${id}/analytics' },
  ];

  return (
    <div className="flex flex-col h-full space-y-6">
      <div className="flex items-center gap-4">
        <Link to="/contests" className="p-2 hover:bg-slate-200 dark:hover:bg
          ↪ -slate-800 rounded-full transition-colors">
          <ChevronLeft size={20} className="text-slate-500" />
        </Link>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    <h2 className="text-xl font-bold dark:text-white">  #{id}</h2>
  </div>

  <div className="flex gap-6 h-full">
    <aside className="w-64 space-y-2 flex-shrink-0">
      {tabs.map((tab) => {
        const isActive = location.pathname === tab.path;
        return (
          <Link
            key={tab.path}
            to={tab.path}
            className={`flex items-center gap-3 px-4 py-3 rounded-
              ↪ xl transition-all font-medium ${
                isActive
                  ? 'bg-blue-600 text-white shadow-lg shadow-
                    ↪ blue-500/20'
                  : 'text-slate-500 hover:bg-white dark:hover:bg-
                    ↪ -slate-800 bg-transparent'
              }`}
          >
            <tab.icon size={18} />
            <span className="text-sm">{tab.label}</span>
          </Link>
        );
      })}
    </aside>

    <main className="flex-1 min-w-0">
      <Outlet />
    </main>
  </div>
</div>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
);  
};
```

2.10. components\layout\MainLayout.tsx

Листинг 204: components\layout\MainLayout.tsx

```
import { Outlet, Link, useLocation, useNavigate } from 'react-router-dom';  
import {  
    LayoutDashboard, Trophy, Users, User,  
    LogOut, ChevronRight, Sparkles, Settings, X,  
    Check  
} from 'lucide-react';  
// Settings, X, Check      AISettingsPopup ( )  
import { useAuthStore } from '../..store/useAuthStore';  
import { ThemeToggle } from '../ThemeToggle';  
import { api } from "../../api/instance.ts";  
import { useState, useRef, useEffect } from 'react';  
  
export const AI_MODELS = [  
    { id: 'anthropic/claude-opus-4.7', name: 'Claude 4.7 Opus' },  
    { id: 'anthropic/claude-opus-4.6-fast', name: 'Claude 4.6 Fast' },  
    { id: 'google/gemini-3.1-pro-preview', name: 'Gemini 3.1 Pro' },  
    { id: 'google/gemini-3-flash-preview', name: 'Gemini 3 Flash' },  
    { id: 'openai/gpt-5.5-pro', name: 'GPT-5.5 Pro' },  
];  
  
//  
//      localStorage      AITasks  
const AI_SETTINGS_KEY = 'ai_tasks_settings';  
  
export interface AISettings {  
    model: string;
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    systemPrompt: string;
}

const DEFAULT_SETTINGS: AISettings = {
  model: AI_MODELS[0].id,
  systemPrompt: '',
};

export const useAISettings = () => {
  const load = (): AISettings => {
    try {
      const raw = localStorage.getItem(AI_SETTINGS_KEY);
      return raw ? { ...DEFAULT_SETTINGS, ...JSON.parse(raw) } :
        ↪ DEFAULT_SETTINGS;
    } catch {
      return DEFAULT_SETTINGS;
    }
  };

  const save = (settings: AISettings) => {
    localStorage.setItem(AI_SETTINGS_KEY, JSON.stringify(settings));
  };

  return { load, save };
};

const AISettingsPopup = ({
  onClose,
}: {
  onClose: () => void;
}) => {
  const { load, save } = useAISettings();

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

const [settings, setSettings] = useState<AISettings>(load);
const popupRef = useRef<HTMLDivElement>(null);

useEffect(() => {
  const handleClick = (e: MouseEvent) => {
    if (popupRef.current && !popupRef.current.contains(e.target as Node)) {
      onClose();
    }
  };
  document.addEventListener('mousedown', handleClick);
  return () => document.removeEventListener('mousedown', handleClick);
}, [onClose]);

const handleSave = () => {
  save(settings);
  onClose();
};

return (
  <>
    {/*      () */}
    <div
      className="fixed inset-0 z-[998] bg-black/5"
      onClick={onClose}
    />

    {/*    */}
    <div
      ref={popupRef}
      className="
        fixed z-[999]
        top-14 left-[calc(16rem+2rem)]

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
w-80 bg-white dark:bg-slate-900
border border-slate-200 dark:border-slate-700
rounded-2xl shadow-2xl p-4
animate-in fade-in slide-in-from-top-2 duration-150
"
style={{
  /* 16rem = (w-64), + */
  /* */
}}
>
<div className="flex items-center justify-between mb-4">
  <h3 className="font-black text-sm dark:text-white"> </h3>
  <button
    onClick={onClose}
    className="text-slate-400 hover:text-slate-700 dark:hover:
      ↪ text-white transition-colors"
  >
    <X size={16} />
  </button>
</div>

{ /* */ }
<div className="mb-4">
  <label className="text-[10px] font-bold uppercase text-slate-400
    ↪ mb-1.5 block">

  </label>
  <select
    value={settings.model}
    onChange={e => setSettings(prev => ({ ...prev, model: e.
      ↪ target.value })))}
    className="
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

w-full bg-slate-50 dark:bg-slate-800
border border-slate-200 dark:border-slate-700
rounded-xl px-3 py-2 text-sm dark:text-white
outline-none focus:border-blue-500 transition-colors
"
>
{AI_MODELS.map(m => (
  <option key={m.id} value={m.id}>{m.name}</option>
))}
</select>
</div>

{ /* */ }
<div className="mb-4">
  <label className="text-[10px] font-bold uppercase text-slate-400
    ↪ mb-1.5 block">

</label>
<textarea
  value={settings.systemPrompt}
  onChange={e => setSettings(prev => ({ ...prev, systemPrompt:
    ↪ e.target.value })))}
  placeholder=":    ..."
  rows={5}
  className="
    w-full bg-slate-50 dark:bg-slate-800
    border border-slate-200 dark:border-slate-700
    rounded-xl px-3 py-2 text-xs dark:text-white
    outline-none focus:border-blue-500 transition-colors
    resize-none font-mono
    "
  />
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        </div>

        <button
            onClick={handleSave}
            className="
                w-full flex items-center justify-center gap-2
                bg-blue-600 hover:bg-blue-700 text-white
                px-4 py-2 rounded-xl text-sm font-bold
                transition-all shadow-lg shadow-blue-500/20
            "
        >
            <Check size={14} />
        </button>
    </div>
</>
);
};

// MainLayout

export const MainLayout = () => {
    const location = useLocation();
    const navigate = useNavigate();
    const { user, logout } = useAuthStore();

    const handleLogout = async () => {
        const { accessToken, refreshToken, tokenType } = useAuthStore.getState();
        try {
            await api.post('/auth/logout', {
                access_token: accessToken,
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        refresh_token: refreshToken,
        token_type: tokenType,
    });
} catch {}
logout();
navigate('/');
};

const menuItems = [
    { icon: LayoutDashboard, label: '', path: '/' },
    { icon: Trophy, label: '', path: '/contests' },
    { icon: Users, label: '', path: '/participants' },
    { icon: Sparkles, label: 'AI ', path: '/ai-tasks', isAi: true },
    { icon: User, label: '', path: '/profile' },
];

const currentLabel = menuItems.find(i => i.path === location.pathname)?.label
    <= || '';

return (
    <div className="flex h-screen bg-slate-50 dark:bg-slate-950 transition-
    <= colors duration-300">
        {/* */}
        <aside className="w-64 bg-white dark:bg-slate-900 border-r border-slate
        <= -200 dark:border-slate-800 flex flex-col">
            <div className="p-6 flex items-center gap-3 border-b border-slate
            <= -100 dark:border-slate-800">
                <div className="w-8 h-8 bg-blue-600 rounded-lg flex items-center
                <= justify-center text-white font-bold shadow-lg">
                    МС
                </div>

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    <span className="font-bold text-xl dark:text-white tracking-
      ↪ tight">
      JudgeSystem
    </span>
  </div>

  <nav className="flex-1 p-4 space-y-1 overflow-y-auto">
    {menuItems.map(item => {
      const isActive = location.pathname === item.path;
      return (
        <Link
          key={item.path}
          to={item.path}
          className={`
            flex items-center justify-between px-4 py-3
            ↪ rounded-xl
            transition-all group
            ${isActive
              ? 'bg-blue-600 text-white shadow-lg shadow-blue
                ↪ -500/30'
              : 'text-slate-500 dark:text-slate-400 hover:bg-
                ↪ slate-100 dark: hover:bg-slate-800 hover:text
                ↪ -slate-900 dark: hover:text-white'
            }
          `}
        >
          <div className="flex items-center gap-3">
            <item.icon size={20} />
            <span className="font-medium">{item.label}</span>
          </div>
          {isActive && <ChevronRight size={16} />}
        </Link>
      )
    })}
  </nav>

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
        );  
      }  
    }  
  </nav>  
  
  <div className="p-4 border-t border-slate-100 dark:border-slate  
    ↪ -800">  
    <button  
      onClick={handleLogout}  
      className="flex items-center gap-3 px-4 py-3 w-full text-  
        ↪ slate-500 hover:text-red-600 dark:hover:text-red-400  
        ↪ transition-colors"  
    >  
      <LogOut size={20} />  
      <span className="font-medium"></span>  
    </button>  
  </div>  
</aside>  
  
{ /* */}  
<div className="flex-1 flex flex-col overflow-hidden">  
  { /* */}  
  <header className="h-16 bg-white dark:bg-slate-900 border-b border-  
    ↪ slate-200 dark:border-slate-800 flex items-center justify-  
    ↪ between px-8">  
    <div className="flex items-center gap-3">  
      <span className="text-slate-500 dark:text-slate-400 font-  
        ↪ medium">  
        {currentLabel}  
      </span>  
    </div>  
  
    <div className="flex items-center gap-6">
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

<ThemeToggle />
<div className="flex items-center gap-3 pl-6 border-l border-
  ↪ slate-200 dark:border-slate-700">
  <div className="text-right hidden sm:block">
    <p className="text-sm font-bold dark:text-white">
      {user?.login || ''}
    </p>
    <p className="text-xs text-slate-500">Online</p>
  </div>
  <div className="w-10 h-10 bg-slate-100 dark:bg-slate-800
    ↪ rounded-full flex items-center justify-center
    ↪ border border-slate-200 dark:border-slate-700">
    <User size={20} className="text-slate-400" />
  </div>
</div>
</div>
</header>

<main className="flex-1 overflow-y-auto p-8 bg-slate-50 dark:bg-
  ↪ slate-950">
  <div className="max-w-6xl mx-auto">
    <Outlet />
  </div>
</main>
</div>
</div>
);
};

```

2.11. pages\{}\AITaskSession.tsx

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Листинг 205: pages\{}\AITaskSession.tsx

```
// pages/AITaskSession.tsx

import { useState, useEffect, useRef, useCallback } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import {
  Send, Sparkles, Loader2, CheckCircle, AlertCircle,
  FileText, UploadCloud, Code,
  Terminal, RefreshCw, Edit3, Check, X, Package,
  AlertTriangle, ChevronRight, ArrowLeft
} from 'lucide-react';
import ReactMarkdown from 'react-markdown';
import remarkMath from 'remark-math';
import rehypeKatex from 'rehype-katex';
import 'katex/dist/katex.min.css';
import { api } from '../api/instance';
import { useAISettings, AI_MODELS } from '../components/layout/MainLayout';

//

const FILE_LABELS: Record<string, string> = {
  validator: 'validator.cpp',
  generator: 'generator.cpp',
  checker: 'checker.cpp',
  solution_cpp: 'solution.cpp',
  solution_py: 'solution.py',
  wa_sol: 'wa.cpp',
  tl_sol: 'tl.cpp',
  re_sol: 're.cpp',
  ml_sol: 'ml.cpp',
  script: 'script.txt',
};
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
//

type PipelineStage =
    | 'statement'
    | 'files_review'
    | 'uploading'
    | 'fixing_errors'
    | 'building_package'
    | 'done'
    | 'failed';

interface Statement {
    name: string;
    legend: string;
    input: string;
    output: string;
    notes?: string;
    tutorial?: string;
}

interface TechnicalData {
    validator?: string;
    generator?: string;
    checker?: string;
    solution_cpp?: string;
    solution_py?: string;
    wa_sol?: string;
    tl_sol?: string;
    re_sol?: string;
    ml_sol?: string;
    script?: string;
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

}

interface UploadError {
  file_name: string;
  error: string;
  needs_manual_fix: boolean;
}

interface Progress {
  status: string;
  current_step?: string;
  error?: string;
  retries?: number;
}

//

const EDITABLE_STAGES: PipelineStage[] = [
  'files_review',
  'fixing_errors',
  'failed',
  'done',
];

const canEditFiles = (stage: PipelineStage) =>
  EDITABLE_STAGES.includes(stage);

// StepBadge

const StepBadge = ({ stage }: { stage: PipelineStage }) => {
  const steps: { key: PipelineStage; label: string }[] = [
    { key: 'statement', label: '' },

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
{ key: 'files_review', label: '' },
{ key: 'uploading', label: '' },
{ key: 'building_package', label: '' },
{ key: 'done', label: '' },
];

const stageOrder: Record<string, number> = {
  statement: 0,
  files_review: 1,
  uploading: 2,
  fixing_errors: 2,
  building_package: 3,
  done: 4,
  failed: -1,
};

const currentIdx = stageOrder[stage] ?? 0;

return (
  <div className="flex items-center gap-1">
    {steps.map((s, i) => {
      const idx = stageOrder[s.key];
      const done = idx < currentIdx;
      const active = idx === currentIdx;
      return (
        <div key={s.key} className="flex items-center gap-1">
          <div className={`
            flex items-center gap-1.5 px-2.5 py-1 rounded-full
            text-[10px] font-bold transition-all
            ${done ? 'bg-green-100 text-green-700 dark:bg-green
              ↩ -900/30 dark:text-green-400' : ''}`

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        ${active ? 'bg-blue-100 text-blue-700 dark:bg-blue-900/30
        ↪ dark:text-blue-400' : ''}

        ${!done && !active ? 'bg-slate-100 text-slate-400 dark:bg
        ↪ -slate-800 dark:text-slate-600' : ''}

    '>
        {done && <Check size={10} />}
        {s.label}
    </div>

    {i < steps.length - 1 && (
        <ChevronRight size={10} className="text-slate-300 dark:
        ↪ text-slate-700" />

    )}
</div>

);

    )}
</div>

);

};

//

type ChatMessage = { role: 'user' | 'assistant' | 'system'; content: string };

export const AITaskSession = () => {
    const { sessionId: urlSessionId } = useParams<{ sessionId: string }>();
    const navigate = useNavigate();
    const { load: loadSettings, save: saveSettings } = useAISettings();

    // ( localStorage, )
    const [sessionModel, setSessionModel] = useState<string>(() => loadSettings().
        ↪ model);

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

const [messages, setMessages] = useState<ChatMessage[]>([]);
const [inputValue, setInputValue] = useState('');
const [loading, setLoading] = useState(false);

const [sessionId, setSessionId] = useState<string | null>(urlSessionId || null)
  ↪ ;
const [stage, setStage] = useState<PipelineStage>('statement');
const [statement, setStatement] = useState<Statement | null>(null);
const [techData, setTechData] = useState<TechnicalData | null>(null);
const [uploadErrors, setUploadErrors] = useState<Record<string, UploadError> |
  ↪ null>(null);
const [polygonProblemId, setPolygonProblemId] = useState<number | null>(null);
const [progress, setProgress] = useState<Progress>({ status: 'idle' });
const [initialLoading, setInitialLoading] = useState(true);

const [viewMode, setViewMode] = useState<'statement' | 'files'>('statement');
const [selectedFile, setSelectedFile] = useState<keyof TechnicalData>('
  ↪ solution_cpp');
const [editingFile, setEditingFile] = useState<string | null>(null);
const [editContent, setEditContent] = useState('');
const [refiningFile, setRefiningFile] = useState<string | null>(null);

// Per-file chat state
const [fileChatHistories, setFileChatHistories] = useState<Record<string,
  ↪ ChatMessage[]>>({});
const [fileInput, setFileInput] = useState('');

const chatEndRef = useRef<HTMLDivElement>(null);
const pollingRef = useRef<ReturnType<typeof setInterval> | null>(null);

//

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

useEffect(() => { chatEndRef.current?.scrollIntoView({ behavior: 'smooth' });
  ↪ }, [messages]);
useEffect(() => { chatEndRef.current?.scrollIntoView({ behavior: 'smooth' });
  ↪ }, [fileChatHistories, selectedFile]);

//
useEffect(() => {
  if (!urlSessionId) { setInitialLoading(false); return; }

  api.get(`/ai/session/${urlSessionId}`)
    .then(res => {
      const d = res.data;
      setSessionId(d.session_id);
      setStage(d.stage);
      setStatement(d.statement);
      setTechData(d.technical_data);
      setSessionModel(d.model);
      setProgress(d.progress || { status: 'idle' });
      if (d.upload_errors && Object.keys(d.upload_errors).length)
        setUploadErrors(d.upload_errors);
      if (d.polygon_problem_id)
        setPolygonProblemId(d.polygon_problem_id);
      if (d.technical_data)
        setViewMode('files');

      //
      if (d.stage === 'uploading' || d.stage === 'building_package')
        startPolling(d.session_id);

      addSystemMessage(' . : ${stageLabel(d.stage)}');
    })
    .catch(() => {

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        addSystemMessage(' ');
        setTimeout(() => navigate('/ai-tasks'), 2000);
    })
    .finally(() => setInitialLoading(false));
}, [urlSessionId]);

//
const handleModelChange = async (newModel: string) => {
    setSessionModel(newModel);
    const settings = loadSettings();
    saveSettings({ ...settings, model: newModel });
    if (sessionId) {
        try {
            await api.patch(`/ai/session/${sessionId}/settings`, { model:
                ↪ newModel });
        } catch { /* non-critical */ }
    }
};

useEffect(() => () => { if (pollingRef.current) clearInterval(pollingRef.
    ↪ current); }, []);

// Helpers

const stageLabel = (s: PipelineStage) => ({
    statement: ' ',
    files_review: ' ',
    uploading: ' Polygon',
    fixing_errors: ' ',
    building_package: ' ',
    done: '',
    failed: '',
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
[s] || s);

const addSystemMessage = (content: string) =>
  setMessages(prev => [...prev, { role: 'system', content }]);

// failed files_review
const handleBackToFiles = () => {
  setStage('files_review');
  setViewMode('files');
  setProgress({ status: 'idle' });
  addSystemMessage('      .      .');
};

//

const startPolling = useCallback((sid: string) => {
  if (pollingRef.current) clearInterval(pollingRef.current);

  pollingRef.current = setInterval(async () => {
    try {
      const res = await api.get(`/ai/upload-progress/${sid}`);
      const d = res.data;

      setProgress({
        status: d.status,
        current_step: d.current_step,
        error: d.error,
        retries: d.retries,
      });

      if (d.stage) setStage(d.stage);
      if (d.technical_data) setTechData(d.technical_data);
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

if (d.upload_errors && Object.keys(d.upload_errors).length)
    setUploadErrors(d.upload_errors);
if (d.polygon_problem_id)
    setPolygonProblemId(d.polygon_problem_id);

//
if (d.status === 'done') {
    clearInterval(pollingRef.current!);
    addSystemMessage(' ! ID: ${d.polygon_problem_id}');
}

//
if (d.status === 'failed') {
    clearInterval(pollingRef.current!);

    //
    addSystemMessage(
        ' : ${d.error || ' '}'. ' +
        ' . '
    );
}

//
if (d.status === 'waiting_manual_fix') {
    clearInterval(pollingRef.current!);
    setStage('fixing_errors');
    addSystemMessage(
        ' . ' +
        ' . '
    );
}
} catch (e) {

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        console.error('Polling error', e);
    }
    }, 2000);
}, []);

//

const handleSendMessage = async () => {
    if (!inputValue.trim() || loading) return;

    const userMsg = inputValue.trim();
    setMessages(prev => [...prev, { role: 'user', content: userMsg }]);
    setInputValue('');
    setLoading(true);

    try {
        if (stage === 'statement' && sessionId) {
            const res = await api.post('/ai/refine-statement', {
                session_id: sessionId,
                feedback: userMsg,
            });
            setStatement(res.data.statement);
            setMessages(prev => [...prev, {
                role: 'assistant',
                content: ' . .',
            }]);
        }
    } catch {
        setMessages(prev => [...prev, {
            role: 'assistant',
            content: ' . .',
        }]);
    }
};
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    } finally {
      setLoading(false);
    }
  };

  //

const handleApproveStatement = async () => {
  if (!sessionId) return;
  setLoading(true);
  addSystemMessage('    ...');

  try {
    const res = await api.post('/ai/approve-statement', { session_id:
      ↪ sessionId });
    setTechData(res.data.technical_data);
    setStage('files_review');
    setViewMode('files');
    setMessages(prev => [...prev, {
      role: 'assistant',
      content: '    !    Polygon.',
    }]);
  } catch (e: any) {
    setMessages(prev => [...prev, {
      role: 'assistant',
      content: '    : ${e?.response?.data?.detail || e.message}',
    }]);
  } finally {
    setLoading(false);
  }
};
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
//

const addFileMessage = (fileKey: string, msg: ChatMessage) => {
  setFileChatHistories(prev => ({
    ...prev,
    [fileKey]: [...(prev[fileKey] || []), msg],
  }));
};

const handleFileChatSend = async () => {
  if (!sessionId || !fileInput.trim() || refiningFile) return;

  const msg = fileInput.trim();
  setFileInput('');
  const key = selectedFile as string;

  addFileMessage(key, { role: 'user', content: msg });
  setRefiningFile(key);

  try {
    const res = await api.post('/ai/refine-file', {
      session_id: sessionId,
      file_key: key,
      feedback: msg,
    });
    setTechData(prev => prev ? { ...prev, [key]: res.data.new_code } : prev)
    ↪ ;
    addFileMessage(key, {
      role: 'assistant',
      content: ` ${FILE_LABELS[key] || key} .`,
    });
  } catch {
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        addFileMessage(key, { role: 'assistant', content: '        .' }));
    } finally {
        setRefiningFile(null);
    }
};

const handleManualEdit = (fileKey: string) => {
    setEditingFile(fileKey);
    setEditContent(techData?.[fileKey as keyof TechnicalData] || '');
};

const handleSaveManualEdit = async () => {
    if (!editingFile || !techData || !sessionId) return;

    const newTechData = { ...techData, [editingFile]: editContent };
    setTechData(newTechData);

    //
    try {
        await api.post('/ai/manual-fix-file', {
            session_id: sessionId,
            file_key: editingFile,
            new_content: editContent,
        });

        //
        setUploadErrors(prev => {
            if (!prev) return prev;
            const updated = { ...prev };
            delete updated[editingFile!];
            return Object.keys(updated).length ? updated : null;
        });
    }
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
        addSystemMessage(' ${FILE_LABELS[editingFile] || editingFile} .');
    } catch {
        addSystemMessage('      .');
    }

    setEditingFile(null);
};

//      Polygon

const handleUploadToPolygon = async () => {
    if (!sessionId) return;

    try {
        setStage('uploading');
        setUploadErrors(null);
        setProgress({ status: 'uploading', current_step: ' ...' });

        await api.post('/ai/approve-files', { session_id: sessionId });
        startPolling(sessionId);
        addSystemMessage('      Polygon...');
    } catch (e: any) {
        setStage('failed');
        setProgress({
            status: 'failed',
            error: e?.response?.data?.detail || ' ',
        });
        addSystemMessage('      .');
    }
};
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
const handleRetryUpload = async () => {
  if (!sessionId) return;

  try {
    setStage('uploading');
    setUploadErrors(null);
    setProgress({ status: 'uploading', current_step: '...' });

    await api.post('/ai/retry-after-manual-fix', { session_id: sessionId });
    startPolling(sessionId);
    addSystemMessage('...');
  } catch {
    setStage('failed');
    addSystemMessage('...');
  }
};

//

const isUploading = stage === 'uploading' || stage === 'building_package';
const isEditable = canEditFiles(stage);

if (initialLoading) {
  return (
    <div className="flex items-center justify-center h-[calc(100vh-80px)] bg
      ↪ -slate-50 dark:bg-slate-950">
      <div className="text-center">
        <Loader2 size={40} className="animate-spin text-blue-500 mx-auto
          ↪ mb-4" />
        <p className="text-slate-400 font-bold"> ...</p>
      </div>
    </div>
  );
}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инов. № подл.	Подп. и дата	Взам. инв. №	Инов. № дубл.	Подп. и дата

```
);  
}  
  
return (  
  <div className="flex h-[calc(100vh-80px)] gap-4 p-4 bg-slate-50 dark:bg-  
    ↪ slate-950 overflow-hidden">  
  
    {/*      */}  
  
    <div className="flex-1 bg-white dark:bg-slate-900 rounded-3xl border  
      ↪ border-slate-200 dark:border-slate-800 flex flex-col overflow-  
      ↪ hidden shadow-xl">  
  
      {/*    */}  
  
      <div className="p-3 border-b border-slate-100 dark:border-slate-800  
        ↪ flex flex-wrap justify-between items-center gap-2 bg-slate  
        ↪ -50/50 dark:bg-slate-800/50">  
        <div className="flex items-center gap-3">  
          <button  
            onClick={() => navigate('/ai-tasks')}  
            className="flex items-center gap-1.5 px-3 py-1.5 rounded-  
              ↪ xl text-xs font-bold text-slate-500 hover:text-  
              ↪ slate-900 dark:hover:text-white hover:bg-slate-100  
              ↪ dark:hover:bg-slate-800 transition-all"  
          >  
            <ArrowLeft size={14} />  
          </button>  
  
          <div className="flex bg-slate-200/50 dark:bg-slate-800 p-1  
            ↪ rounded-xl">  
            <button  
              onClick={() => setViewMode('statement')}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        className={'flex items-center gap-2 px-4 py-1.5
        ↪ rounded-lg text-sm font-bold transition-all ${
        viewMode === 'statement'
        ? 'bg-white dark:bg-slate-700 shadow-sm text-
        ↪ blue-600'
        : 'text-slate-500'
        }'}
    >
    <FileText size={16} />
</button>
<button
    onClick={() => setViewMode('files')}
    className={'flex items-center gap-2 px-4 py-1.5
    ↪ rounded-lg text-sm font-bold transition-all ${
    viewMode === 'files'
    ? 'bg-white dark:bg-slate-700 shadow-sm text-
    ↪ blue-600'
    : 'text-slate-500'
    }'}
    >
    <Code size={16} />
    {techData && <span className="w-2 h-2 bg-green-500
    ↪ rounded-full animate-pulse" />}
</button>
</div>

{sessionId && <StepBadge stage={stage} />}
</div>

<div className="flex items-center gap-2">
    {/*    */}
    {statement && stage === 'statement' && (
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<button
  onClick={handleApproveStatement}
  disabled={loading}
  className="flex items-center gap-2 bg-emerald-600
    ↪ hover:bg-emerald-700 text-white px-5 py-2
    ↪ rounded-2xl text-sm font-black transition-all
    ↪ disabled:opacity-50 shadow-lg shadow-emerald
    ↪ -500/20"
>
  {loading ? <Loader2 size={16} className="animate-spin"
    ↪ /> : <Check size={16} />}

</button>
)}

{/* Polygon files_review */}
{techData && stage === 'files_review' && (
  <button
    onClick={handleUploadToPolygon}
    disabled={loading}
    className="flex items-center gap-2 bg-blue-600 hover:
      ↪ bg-blue-700 text-white px-5 py-2 rounded-2xl
      ↪ text-sm font-black transition-all disabled:
      ↪ opacity-50 shadow-lg shadow-blue-500/20"
  >
    <UploadCloud size={16} />
    Polygon
  </button>
)}

{/* failed */}
{stage === 'failed' && techData && (
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<button
  onClick={handleBackToFiles}
  className="flex items-center gap-2 bg-slate-600 hover:
    ↪ bg-slate-700 text-white px-4 py-2 rounded-2xl
    ↪ text-sm font-bold transition-all shadow-lg"
>

  <Edit3 size={16} />

</button>
)}}

{/*    fixing_errors    failed */}
{(stage === 'fixing_errors' || stage === 'failed') &&
  ↪ techData && (
    <button
      onClick={handleRetryUpload}
      className="flex items-center gap-2 bg-amber-500 hover:
        ↪ bg-amber-600 text-white px-5 py-2 rounded-2xl
        ↪ text-sm font-black transition-all shadow-lg
        ↪ shadow-amber-500/20"
    >

      <RefreshCw size={16} />

    </button>
  )}

{/* */}
{stage === 'done' && (
  <>

    <button
      onClick={handleRetryUpload}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        className="flex items-center gap-2 bg-blue-500
        ↪ hover:bg-blue-600 text-white px-4 py-2
        ↪ rounded-2xl text-sm font-bold transition-all
        ↪ shadow-lg"
    >

    <UploadCloud size={16} />

</button>
{polygonProblemId && (
    <a
        href={`https://polygon.codeforces.com/problems
        ↪ /${polygonProblemId}`}
        target="_blank"
        rel="noopener noreferrer"
        className="flex items-center gap-2 bg-green
        ↪ -600 hover:bg-green-700 text-white px-5
        ↪ py-2 rounded-2xl text-sm font-black
        ↪ transition-all shadow-lg"
    >

    <Package size={16} />

    Polygon

    </a>
)}
</>
)}
</div>
</div>

{ /* */ }

<div className="flex-1 overflow-y-auto">
    {viewMode === 'statement' ? (
        <div className="p-8 prose dark:prose-invert max-w-none">

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
{statement ? (  
  <div className="animate-in fade-in duration-500 dark:  
    ↪ text-white">  
    <h1 className="text-4xl font-black text-slate-800  
      ↪ dark:text-white mb-8">  
      {statement.name}  
    </h1>  
    <ReactMarkdown remarkPlugins={[remarkMath]}  
      ↪ rehypePlugins={[rehypeKatex]}>  
      {statement.legend}  
    </ReactMarkdown>  
    <h3 className="text-xl font-bold mt-8 border-b pb  
      ↪ -2"> </h3>  
    <ReactMarkdown remarkPlugins={[remarkMath]}  
      ↪ rehypePlugins={[rehypeKatex]}>  
      {statement.input}  
    </ReactMarkdown>  
    <h3 className="text-xl font-bold mt-6 border-b pb  
      ↪ -2"> </h3>  
    <ReactMarkdown remarkPlugins={[remarkMath]}  
      ↪ rehypePlugins={[rehypeKatex]}>  
      {statement.output}  
    </ReactMarkdown>  
    {statement.notes && (  
      <>  
        <h3 className="text-xl font-bold mt-6  
          ↪ border-b pb-2"></h3>  
        <ReactMarkdown remarkPlugins={[remarkMath]}  
          ↪ rehypePlugins={[rehypeKatex]}>  
          {statement.notes}  
        </ReactMarkdown>  
      </>  
    )  
  )  
}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
    )}
  </div>
) : (
  <div className="h-full flex flex-col items-center
    ↪ justify-center text-slate-400 py-20">
    <Sparkles size={64} className="mb-4 opacity-10" />
    <p className="font-bold text-lg"> ...</p>
  </div>
)}
</div>
) : (
  /*      */
  <div className="flex h-full">
    {/*    */}
    <div className="w-48 border-r dark:border-slate-800 p-2
      ↪ flex flex-col gap-1 bg-slate-50/30 dark:bg-slate
      ↪ -900/30 overflow-y-auto">
      {techData ? (
        Object.keys(techData).map(key => {
          const hasError = uploadErrors?.[key];
          return (
            <button
              key={key}
              onClick={() => { setSelectedFile(key as
                ↪ keyof TechnicalData);
                ↪ setEditingFile(null); }}
              className={`
                text-left px-3 py-2 rounded-lg text-
                  ↪ xs font-bold truncate
                transition-all flex items-center
                  ↪ justify-between
                ${selectedFile === key
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        ? 'bg-blue-50 dark:bg-blue-900/30
        ↪ text-blue-600'
        : 'text-slate-500 hover:bg-slate-100
        ↪ dark: hover: bg-slate-800'}
    '}'
  >
    <span>{FILE_LABELS[key] || key}</span>
    {hasError && <AlertTriangle size={12}
    ↪ className="text-amber-500 shrink
    ↪ -0" />}
  </button>
);
})
) : (
  <div className="p-4 text-center text-xs text-slate
  ↪ -400">

    </div>
  )}
</div>

{ /* / */}
<div className="flex-1 flex flex-col overflow-hidden">
  {techData && techData[selectedFile] ? (
    <>
      { /* */}
      <div className="flex items-center gap-2 px-4
      ↪ py-2 border-b dark: border-slate-700 bg-
      ↪ slate-900 text-slate-400">
        <span className="text-xs font-mono font-
        ↪ bold text-slate-300">
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        {FILE_LABELS[selectedFile] ||
          ↪ selectedFile}
    </span>

    {/*      */}
    {uploadErrors?.[selectedFile] && (
      <div className="ml-2 flex items-center
        ↪ gap-1.5 text-amber-400 text-xs bg
        ↪ -amber-900/20 px-2 py-1 rounded-
        ↪ lg">
        <AlertTriangle size={12} />
        <span className="truncate max-w-xs">
          {uploadErrors[selectedFile].
            ↪ error}
        </span>
      </div>
    )}

    <div className="ml-auto flex items-center
      ↪ gap-2">
      {isEditable && (
        <button
          onClick={() => handleManualEdit(
            ↪ selectedFile)}
          className="flex items-center gap
            ↪ -1 text-xs hover:text-
            ↪ white transition-colors"
          >
            <Edit3 size={14} />
          </button>
        )}
    </div>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
</div>

{ /*      */ }

{editingFile === selectedFile ? (
  <div className="flex-1 flex flex-col bg-
    ↪ slate-950">
    <textarea
      value={editContent}
      onChange={e => setEditContent(e.
        ↪ target.value)}
      className="flex-1 w-full bg-
        ↪ transparent text-slate-200
        ↪ font-mono text-sm p-4 outline
        ↪ -none resize-none"
      spellCheck={false}
    />
    <div className="flex gap-2 p-2 border-t
      ↪ dark:border-slate-700 bg-slate
      ↪ -900">
      <button
        onClick={handleSaveManualEdit}
        className="flex items-center gap
          ↪ -1 text-xs bg-blue-600
          ↪ hover:bg-blue-700 text-
          ↪ white px-3 py-1.5 rounded-
          ↪ lg transition-all"
        >
          <Check size={12} />
        </button>
        <button
          onClick={() => setEditingFile(
            ↪ null)}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
{/*      :      */}

<div className="w-95 flex flex-col gap-3">

  <div className="flex-1 bg-white dark:bg-slate-900 rounded-3xl border
    ↪ border-slate-200 dark:border-slate-800 flex flex-col overflow
    ↪ -hidden shadow-sm">

    {/*          */}

    <div className="p-3 border-b dark:border-slate-800 flex items-
      ↪ center justify-between gap-2">
      <div className="flex items-center gap-2">
        <Terminal size={16} className="text-blue-500 shrink-0" />
        <span className="font-black text-xs uppercase tracking-
          ↪ widest text-slate-700 dark:text-slate-200 truncate
          ↪ ">
          {stage === 'statement'
            ? ''
              : canEditFiles(stage) && viewMode === 'files'
                ? (FILE_LABELS[selectedFile as string] ||
                  ↪ selectedFile)
                  : 'AI Agent'}
        </span>
      </div>
    </div>
    <select
      value={sessionModel}
      onChange={e => handleModelChange(e.target.value)}
      className="text-[10px] bg-slate-100 dark:bg-slate-800
        ↪ border border-slate-200 dark:border-slate-700
        ↪ rounded-lg px-2 py-1 text-slate-600 dark:text-slate
        ↪ -300 outline-none cursor-pointer shrink-0"
    >
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
{AI_MODELS.map(m => (  
  <option key={m.id} value={m.id}>{m.name}</option>  
))}  
</select>  
</div>  
  
{stage === 'statement' ? (  
  /*      */  
  <>  
    <div className="flex-1 overflow-y-auto p-4 space-y-3">  
      {messages.length === 0 && (  
        <div className="text-center text-slate-400 text-xs  
          ↪ py-8">  
          <Sparkles size={32} className="mx-auto mb-2  
            ↪ opacity-20" />  
          {statement  
            ? ' ' ,  
            : ' ' , '}  
          </div>  
        )}  
      {messages.map((m, i) => (  
        <div key={i} className={`flex ${m.role === 'user'  
          ↪ ? 'justify-end' : 'justify-start'}{}`>  
          {m.role === 'system' ? (  
            <div className="w-full text-center">  
              <span className="text-[10px] text-slate  
                ↪ -400 bg-slate-100 dark:bg-slate  
                ↪ -800 px-3 py-1 rounded-full">  
                {m.content}  
              </span>  
            </div>  
          ) : (  
            </div>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<div className={`max-w-[90%] px-4 py-2.5
  ↪ rounded-2xl text-sm
  ${m.role === 'user'
    ? 'bg-blue-600 text-white shadow-md
      ↪ rounded-br-sm'
    : 'bg-slate-100 dark:bg-slate-800
      ↪ dark:text-slate-200 rounded-
      ↪ bl-sm'}`}>
  {m.content}
</div>
)}
</div>
)}}
{loading && (
  <div className="flex justify-start">
    <div className="bg-slate-100 dark:bg-slate-800
      ↪ p-3 rounded-2xl">
      <div className="flex gap-1">
        <div className="w-1.5 h-1.5 bg-blue-500
          ↪ rounded-full animate-bounce" />
        <div className="w-1.5 h-1.5 bg-blue-500
          ↪ rounded-full animate-bounce [
          ↪ animation-delay:0.2s]" />
        <div className="w-1.5 h-1.5 bg-blue-500
          ↪ rounded-full animate-bounce [
          ↪ animation-delay:0.4s]" />
      </div>
    </div>
  </div>
)}
<div ref={chatEndRef} />
</div>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
<div className="p-3 border-t dark:border-slate-800">
  <div className="relative">
    <textarea
      value={inputValue}
      onChange={e => setInputValue(e.target.value)}
      onKeyDown={e => {
        if (e.key === 'Enter' && !e.shiftKey) {
          e.preventDefault();
          handleSendMessage();
        }
      }}
      placeholder={statement ? '...' : '...'}
      className="w-full bg-slate-50 dark:bg-slate
        ↪ -800 border-2 border-transparent focus:
        ↪ border-blue-500 rounded-2xl p-3 pr-12
        ↪ text-sm outline-none dark:text-white
        ↪ transition-all resize-none h-20 shadow-
        ↪ inner"
    />
    <button
      onClick={handleSendMessage}
      disabled={loading || !inputValue.trim()}
      className="absolute right-2 bottom-2 p-2.5 bg-
        ↪ blue-600 text-white rounded-xl hover:bg-
        ↪ blue-700 disabled:opacity-50 transition-
        ↪ all shadow-lg"
    >
      <Send size={16} />
    </button>
  </div>
</div>
</>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

) : canEditFiles(stage) && viewMode === 'files' ? (
  /*      */
  <>

  <div className="flex-1 overflow-y-auto p-4 space-y-3">
    {(fileChatHistories[selectedFile as string] || []).
      ↪ length === 0 && (
        <div className="text-center text-slate-400 text-xs
          ↪ py-8">
            <Code size={32} className="mx-auto mb-2
              ↪ opacity-20" />
            {', '}
            <span className="font-bold">{FILE_LABELS[
              ↪ selectedFile as string] || selectedFile
              ↪ }</span>
          </div>
        )}
    {(fileChatHistories[selectedFile as string] || []).map
      ↪ ((m, i) => (
        <div key={i} className={`flex ${m.role === 'user'
          ↪ ? 'justify-end' : 'justify-start'}{`}>
          <div className={`max-w-[90%] px-4 py-2.5
            ↪ rounded-2xl text-sm
              ${m.role === 'user'
                ? 'bg-blue-600 text-white shadow-md
                  ↪ rounded-br-sm'
                : 'bg-slate-100 dark:bg-slate-800 dark:
                  ↪ text-slate-200 rounded-bl-sm'}{`}>
                {m.content}
              </div>
            </div>
          </div>
        )))}
    {refiningFile === (selectedFile as string) && (

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

<div className="flex justify-start">
  <div className="bg-slate-100 dark:bg-slate-800
    ↪ p-3 rounded-2xl">
    <div className="flex gap-1">
      <div className="w-1.5 h-1.5 bg-blue-500
        ↪ rounded-full animate-bounce" />
      <div className="w-1.5 h-1.5 bg-blue-500
        ↪ rounded-full animate-bounce [
        ↪ animation-delay:0.2s]" />
      <div className="w-1.5 h-1.5 bg-blue-500
        ↪ rounded-full animate-bounce [
        ↪ animation-delay:0.4s]" />
    </div>
  </div>
</div>
)}
</div>
<div className="p-3 border-t dark:border-slate-800">
  <div className="relative">
    <textarea
      value={fileInput}
      onChange={e => setFileInput(e.target.value)}
      onKeyDown={e => {
        if (e.key === 'Enter' && !e.shiftKey) {
          e.preventDefault();
          handleFileChatSend();
        }
      }}
      placeholder={` ${FILE_LABELS[selectedFile as
        ↪ string] || selectedFile}...`}
      className="w-full bg-slate-50 dark:bg-slate
        ↪ -800 border-2 border-transparent focus:

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        ↪ border-blue-500 rounded-2xl p-3 pr-12
        ↪ text-sm outline-none dark:text-white
        ↪ transition-all resize-none h-20 shadow-
        ↪ inner"
    />
    <button
      onClick={handleFileChatSend}
      disabled={!fileInput.trim() || !!refiningFile}
      className="absolute right-2 bottom-2 p-2.5 bg-
        ↪ blue-600 text-white rounded-xl hover:bg-
        ↪ blue-700 disabled:opacity-50 transition-
        ↪ all shadow-lg"
    >
      {refiningFile === (selectedFile as string)
        ? <Loader2 size={16} className="animate-
          ↪ spin" />
        : <Sparkles size={16} />}
    </button>
  </div>
</div>
</>
) : (
  /* : / */
  <>
    <div className="flex-1 overflow-y-auto p-4 space-y-3">
      {messages.map((m, i) => (
        <div key={i} className={`flex ${m.role === 'user'
          ↪ ? 'justify-end' : 'justify-start'}{`}>
          {m.role === 'system' ? (
            <div className="w-full text-center">
              <span className="text-[10px] text-slate
                ↪ -400 bg-slate-100 dark:bg-slate
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        ↪ -800 px-3 py-1 rounded-full">
        {m.content}
    </span>
</div>
) : (
    <div className={`max-w-[90%] px-4 py-2.5
        ↪ rounded-2xl text-sm
        ${m.role === 'user'
            ? 'bg-blue-600 text-white shadow-md
            ↪ rounded-br-sm'
            : 'bg-slate-100 dark:bg-slate-800
            ↪ dark:text-slate-200 rounded-
            ↪ bl-sm'}'}`>
        {m.content}
    </div>
    )}
</div>
)}}
<div ref={chatEndRef} />
</div>
<div className="p-3 border-t dark:border-slate-800 text-
    ↪ center text-xs text-slate-400">
    {isUploading && ' ...'}
    {stage === 'done' && ' !'}
</div>
</>
)}
</div>

{/* - */}
{progress.status !== 'idle' && (
    <div className={`
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

bg-white dark:bg-slate-900 p-4 rounded-3xl border-2 shadow-xl
animate-in slide-in-from-bottom-4
${progress.status === 'failed' || stage === 'fixing_errors'
? 'border-amber-200 dark:border-amber-900'
: progress.status === 'done'
  ? 'border-green-200 dark:border-green-900'
  : 'border-blue-100 dark:border-blue-900'}
'>
<div className="flex items-center justify-between mb-2">
  <h4 className="font-black text-xs uppercase dark:text-
    ↪ white tracking-tighter">
    Polygon Sync
  </h4>
  <div className="flex items-center gap-1.5">
    {isUploading && <Loader2 size={14} className="animate-
      ↪ spin text-blue-500" />}
    {progress.status === 'done' && <CheckCircle size={14}
      ↪ className="text-green-500" />}
    {(progress.status === 'failed' || stage === '
      ↪ fixing_errors') && <AlertCircle size={14}
      ↪ className="text-amber-500" />}
    {polygonProblemId && <span className="text-[10px] text
      ↪ -slate-400">ID: {polygonProblemId}</span>}
  </div>
</div>
<div className="w-full bg-slate-100 dark:bg-slate-800 h-1.5
  ↪ rounded-full mb-2 overflow-hidden">
  <div className={`h-full transition-all duration-700 ${
    progress.status === 'failed' ? 'bg-red-500 w-full' :
    stage === 'fixing_errors' ? 'bg-amber-500 w-3/4' :
    progress.status === 'done' ? 'bg-green-500 w-full' :
    'bg-blue-500 w-2/3 animate-pulse'
  }`>

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        }' } />
      </div>
      <p className="text-[10px] font-bold text-slate-500 uppercase
        ↪ truncate">
        {progress.current_step}
      </p>
      {progress.error && (
        <p className="text-[10px] text-red-400 mt-1 font-mono
          ↪ break-all">
          {progress.error}
        </p>
      )}
      {progress.retries !== undefined && progress.retries > 0 && (
        <p className="text-[10px] text-amber-400 mt-1">
          : {progress.retries}/3
        </p>
      )}
    </div>
  )}
</div>
</div>
);
};
```

2.12. pages\{}AITasksList.tsx

Листинг 206: pages\{}AITasksList.tsx

```
// pages/AITasksList.tsx

import { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import {
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    Plus, Sparkles, Loader2, Trash2, ExternalLink,
    FileText, Code, UploadCloud, Package, CheckCircle,
    AlertCircle, Clock, ChevronRight
} from 'lucide-react';
import { api } from '../api/instance';
import { useAISettings } from '../components/layout/MainLayout';

interface SessionSummary {
    session_id: string;
    stage: string;
    name: string;
    model: string;
    polygon_problem_id: number | null;
    created_at: string | null;
    updated_at: string | null;
}

const STAGE_CONFIG: Record<string, {
    label: string;
    icon: React.ReactNode;
    color: string;
    bg: string;
}> = {
    statement: {
        label: '',
        icon: <FileText size={14} />,
        color: 'text-blue-600 dark:text-blue-400',
        bg: 'bg-blue-50 dark:bg-blue-900/20',
    },
    files_review: {
        label: ' ',
        icon: <Code size={14} />,

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
    color: 'text-purple-600 dark:text-purple-400',
    bg: 'bg-purple-50 dark:bg-purple-900/20',
  },
  uploading: {
    label: '',
    icon: <UploadCloud size={14} />,
    color: 'text-amber-600 dark:text-amber-400',
    bg: 'bg-amber-50 dark:bg-amber-900/20',
  },
  fixing_errors: {
    label: ' ',
    icon: <AlertCircle size={14} />,
    color: 'text-red-600 dark:text-red-400',
    bg: 'bg-red-50 dark:bg-red-900/20',
  },
  building_package: {
    label: ' ',
    icon: <Package size={14} />,
    color: 'text-orange-600 dark:text-orange-400',
    bg: 'bg-orange-50 dark:bg-orange-900/20',
  },
  done: {
    label: '',
    icon: <CheckCircle size={14} />,
    color: 'text-green-600 dark:text-green-400',
    bg: 'bg-green-50 dark:bg-green-900/20',
  },
  failed: {
    label: '',
    icon: <AlertCircle size={14} />,
    color: 'text-red-600 dark:text-red-400',
    bg: 'bg-red-50 dark:bg-red-900/20',
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    },
};

export const AITasksList = () => {
  const navigate = useNavigate();
  const { load: loadSettings } = useAISettings();

  const [sessions, setSessions] = useState<SessionSummary[]>([]);
  const [loading, setLoading] = useState(true);
  const [creating, setCreating] = useState(false);
  const [deletingId, setDeletingId] = useState<string | null>(null);

  //
  const fetchSessions = async () => {
    try {
      setLoading(true);
      const res = await api.get('/ai/sessions');
      setSessions(res.data);
    } catch (e) {
      console.error('Failed to load sessions', e);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchSessions();
  }, []);

  //
  const handleCreate = async () => {
    if (creating) return;
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
setCreating(true);
try {
  const settings = loadSettings();
  const res = await api.post('/ai/create-session', {
    idea: '',
    model: settings.model,
    user_prompt: settings.systemPrompt,
  });
  navigate(`/ai-tasks/${res.data.session_id}`);
} catch (e) {
  console.error('Failed to create session', e);
} finally {
  setCreating(false);
}
};

//
const handleDelete = async (sessionId: string, e: React.MouseEvent) => {
  e.stopPropagation();
  if (!confirm(' ?')) return;

  setDeletingId(sessionId);
  try {
    await api.delete(`/ai/session/${sessionId}`);
    setSessions(prev => prev.filter(s => s.session_id !== sessionId));
  } catch (err) {
    console.error('Failed to delete session', err);
  } finally {
    setDeletingId(null);
  }
};
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
//
const formatDate = (dateStr: string | null) => {
  if (!dateStr) return '';
  const date = new Date(dateStr);
  const now = new Date();
  const diff = now.getTime() - date.getTime();

  if (diff < 60000) return ' ';
  if (diff < 3600000) return `${Math.floor(diff / 60000)} `;
  if (diff < 86400000) return `${Math.floor(diff / 3600000)} `;

  return date.toLocaleDateString('ru-RU', {
    day: 'numeric',
    month: 'short',
    hour: '2-digit',
    minute: '2-digit',
  });
};

return (
  <div className="min-h-[calc(100vh-80px)] bg-slate-50 dark:bg-slate-950 p
    ↪ -6">
    <div className="max-w-4xl mx-auto">

      {/* */}
      <div className="flex items-center justify-between mb-8">
        <div>
          <h1 className="text-3xl font-black text-slate-800 dark:text-
            ↪ white flex items-center gap-3">
            <Sparkles className="text-blue-500" size={32} />
            AI
          </h1>

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    <p className="text-slate-500 mt-1 text-sm">
      Polygon
    </p>
  </div>

  <button
    onClick={handleCreate}
    disabled={creating}
    className="flex items-center gap-2 bg-blue-600 hover:bg-blue
      ↪ -700
      text-white px-6 py-3 rounded-2xl font-bold
      transition-all shadow-lg shadow-blue-500/20
      hover:shadow-xl hover:shadow-blue-500/30
      active:scale-95 disabled:opacity-60"
  >
    {creating
      ? <Loader2 size={20} className="animate-spin" />
      : <Plus size={20} />
    }

  </button>
</div>

{ /* */ }
{loading ? (
  <div className="flex flex-col items-center justify-center py
    ↪ -20">
    <Loader2 size={32} className="animate-spin text-blue-500 mb
      ↪ -4" />
    <p className="text-slate-400 text-sm"> ...</p>
  </div>
) : sessions.length === 0 ? (

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<div className="flex flex-col items-center justify-center py-20
      text-slate-400">

  <Sparkles size={64} className="mb-4 opacity-10" />

  <p className="font-bold text-lg">  </p>

  <p className="text-sm mt-2">

    ,

  </p>
</div>

) : (

<div className="space-y-3">
  {sessions.map(session => {
    const stageConf = STAGE_CONFIG[session.stage]
      || STAGE_CONFIG.statement;

    return (
      <div
        key={session.session_id}
        onClick={() =>
          navigate(`/ai-tasks/${session.session_id}`)
        }
        className="group bg-white dark:bg-slate-900
          ↪ rounded-2xl
            border border-slate-200 dark:border-
              ↪ slate-800
            p-5 cursor-pointer transition-all
            hover:border-blue-300 dark:hover:border-
              ↪ blue-700
            hover:shadow-lg hover:shadow-blue-500/5
            active:scale-[0.99]"
          >

        <div className="flex items-center justify-between
          ↪ ">
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<div className="flex-1 min-w-0">
  {/* */}
  <h3 className="font-bold text-slate-800
    dark:text-white text-lg
    ↪ truncate
    group-hover:text-blue-600
    dark:group-hover:text-blue
    ↪ -400
    transition-colors">
    {session.name}
  </h3>

  {/* - */}
  <div className="flex items-center gap-3 mt
    ↪ -2
    flex-wrap">
    {/* */}
    <span className={`
      flex items-center gap-1.5 px-2.5 py
      ↪ -1
      rounded-full text-[11px] font-bold
      ${stageConf.color} ${stageConf.bg}
    `}>
      {stageConf.icon}
      {stageConf.label}
    </span>

    {/* */}
    <span className="text-[11px] text-slate
      ↪ -400
      bg-slate-100
      dark:bg-slate-800
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
px-2 py-1 rounded-full
font-mono">
{session.model}
</span>

{/* Polygon ID */}
{session.polygon_problem_id && (
  <a
    href={'https://polygon.
      ↪ codeforces.com/problems/${
      ↪ session.polygon_problem_id
      ↪ }'}
    target="_blank"
    rel="noopener noreferrer"
    onClick={e => e.stopPropagation
      ↪ ()}
    className="flex items-center gap
      ↪ -1
      text-[11px] text-green
        ↪ -600
        dark:text-green-400
        bg-green-50
        dark:bg-green-900/20
        px-2 py-1 rounded-full
        font-bold
        hover:bg-green-100
        dark:hover:bg-green
          ↪ -900/40
          transition-colors"
    >
    <ExternalLink size={10} />
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

        Polygon #{session.
          ↪ polygon_problem_id}
      </a>
    })

    {/* */}
    <span className="flex items-center gap
      ↪ -1
        text-[11px] text-slate
          ↪ -400">
      <Clock size={10} />
      {formatDate(
        session.updated_at
        || session.created_at
      )}
    </span>
  </div>
</div>

{/* : */}
<div className="flex items-center gap-2 ml-4">
  <button
    onClick={e => handleDelete(
      session.session_id, e
    )}
    disabled={deletingId === session.
      ↪ session_id}
    className="p-2 text-slate-400
      hover:text-red-500
      hover:bg-red-50
      dark:hover:bg-red-900/20
      rounded-xl transition-all
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        opacity-0
        group-hover:opacity-100"
    >
        {deletingId === session.session_id ? (
            <Loader2
                size={16}
                className="animate-spin"
            />
        ) : (
            <Trash2 size={16} />
        )}
    </button>
    <ChevronRight
        size={20}
        className="text-slate-300
            dark:text-slate-700
            group-hover:text-blue-500
            transition-colors"
    />
</div>
</div>
</div>
);
    )}
</div>
    )}
</div>
</div>
);
};
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2.13. pages\{}\CodeforcesCallback.tsx

Листинг 207: pages\{}\CodeforcesCallback.tsx

```
import { useEffect, useRef } from 'react';
import { useNavigate, useSearchParams } from 'react-router-dom';
import { api } from '../api/instance';
import { useAuthStore } from "../store/useAuthStore.ts";

export const CodeforcesCallback = () => {
  const [searchParams] = useSearchParams();
  const navigate = useNavigate();
  const processed = useRef(false);

  useEffect(() => {
    if (processed.current) return;
    processed.current = true;

    const code = searchParams.get('code');
    const verifier = sessionStorage.getItem('cf_verifier');
    if (!code || !verifier) {
      navigate('/profile');
      return;
    }

    api.post('/codeforces/callback', {
      code: code,
      code_verifier: verifier,
    })
      .then(async () => {
        await useAuthStore.getState().fetchUser();
        navigate('/profile');
      })
  });
}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        .catch((e) => {
            console.error(e);
            alert(' : ' + (e.response?.data?.detail || e.message));
            navigate('/profile');
        });
    }, []);

    return (
        <div className="flex items-center justify-center min-h-screen">
            <div className="text-center">
                <div className="animate-spin rounded-full h-12 w-12 border-b-2
                    ↪ border-blue-600 mx-auto mb-4"></div>
                <p className="text-slate-500"> Codeforces ...</p>
            </div>
        </div>
    );
};

```

2.14. pages\{}\ContestAnalytics.tsx

Листинг 208: pages\{}\ContestAnalytics.tsx

```

import { useState, useEffect } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import {
    ShieldAlert, Play, BarChart, AlertCircle,
    History, ChevronRight, Clock, CheckCircle2, Loader2
} from 'lucide-react';
import { api } from '../api/instance';

export const ContestAnalytics = () => {
    const { id } = useParams();
    const navigate = useNavigate();

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

const [reports, setReports] = useState<any[]>([]);
const [isLoading, setIsLoading] = useState(true);

//
useEffect(() => {
  fetchReports();
}, [id]);

const fetchReports = async () => {
  try {
    //
    const res = await api.get(`/analytics/contests/${id}/reports`);
    setReports(res.data);
  } catch (e) {
    console.error(" ");
  } finally {
    setIsLoading(false);
  }
};

const getStatusStyle = (status: string) => {
  switch (status) {
    case 'completed': return 'text-green-500 bg-green-50 dark:bg-green
      ↪ -900/20';
    case 'failed': return 'text-red-500 bg-red-50 dark:bg-red-900/20';
    default: return 'text-blue-500 bg-blue-50 dark:bg-blue-900/20';
  }
};

return (
  <div className="space-y-8 animate-in fade-in duration-500">
    {/* */}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

<div className="grid grid-cols-1 md:grid-cols-2 gap-6">
  {/* */}
  <div className="bg-white dark:bg-slate-900 p-8 rounded-3xl border
    ↪ border-slate-100 dark:border-slate-800 shadow-sm flex flex-col
    ↪ items-center text-center space-y-4">
    <div className="p-4 bg-amber-100 dark:bg-amber-900/30 text-amber
      ↪ -600 rounded-2xl">
        <ShieldAlert size={32} />
      </div>
    <div className="space-y-1">
      <h3 className="text-xl font-bold dark:text-white"> </h3>
      <p className="text-slate-500 text-sm"> C++ LSH.</p>
    </div>
    <button
      onClick={() => navigate(`/contests/${id}/analytics/check`)}
      className="w-full bg-slate-900 dark:bg-blue-600 text-white py
        ↪ -3.5 rounded-2xl font-bold flex justify-center items-
        ↪ center gap-2 hover:opacity-90 transition-all shadow-lg
        ↪ shadow-blue-500/10"
    >
      <Play size={18} />
      <span> </span>
    </button>
  </div>

  {/* */}
  <div className="bg-white dark:bg-slate-900 p-8 rounded-3xl border
    ↪ border-slate-100 dark:border-slate-800 shadow-sm flex flex-col
    ↪ items-center text-center space-y-4 opacity-60">
    <div className="p-4 bg-blue-100 dark:bg-blue-900/30 text-blue
      ↪ -600 rounded-2xl">
      <BarChart size={32} />
    </div>
  </div>

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    </div>
    <div className="space-y-1">
      <h3 className="text-xl font-bold dark:text-white"> </h3>
      <p className="text-slate-500 text-sm">      .</p>
    </div>
    <button disabled className="w-full border-2 border-slate-100
      ↪ dark:border-slate-800 dark:text-white py-3 rounded-2xl
      ↪ font-bold cursor-not-allowed">

      </button>
    </div>
  </div>

  {/*      */}
  <div className="space-y-4">
    <div className="flex items-center gap-2 px-2">
      <History size={20} className="text-slate-400" />
      <h2 className="text-lg font-bold dark:text-white"> </h2>
    </div>

    {isLoading ? (
      <div className="py-10 flex justify-center"><Loader2 className="
        ↪ animate-spin text-slate-300" /></div>
    ) : reports.length > 0 ? (
      <div className="grid grid-cols-1 gap-3">
        {reports.map((report) => (
          <button
            key={report.id}
            onClick={() => navigate(`/contests/${id}/analytics/
              ↪ reports/${report.id}`)}
            className="w-full bg-white dark:bg-slate-900 p-5
              ↪ rounded-2xl border border-slate-100 dark:border
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    ↪ -slate-800 flex items-center justify-between
    ↪ hover:border-blue-500/50 transition-all group"
  >
  <div className="flex items-center gap-6">
    <div className={`p-2.5 rounded-xl ${getStatusStyle
      ↪ (report.status)}{}`>
      {report.status === 'completed' ? <CheckCircle2
        ↪ size={20} /> :
        report.status === 'failed' ? <AlertCircle
          ↪ size={20} /> :
          <Loader2 size={20} className="animate-
            ↪ spin" />}
    </div>

    <div className="text-left">
      <p className="font-bold text-sm dark:text-
        ↪ white"> #{report.id}</p>
      <div className="flex items-center gap-3 text-
        ↪ xs text-slate-500 mt-0.5">
        <span className="flex items-center gap-1"><
          ↪ Clock size={12}/> {new Date(report.
            ↪ created_at).toLocaleString()}</span>
        <span></span>
        <span>: {Math.round(report.threshold * 100)
          ↪ }%</span>
      </div>
    </div>
  </div>

  <div className="flex items-center gap-4">
    {report.status === 'completed' && (

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

        <span className="text-xs font-bold text-blue
          ↪ -600 bg-blue-50 dark:bg-blue-900/30 px-3
          ↪ py-1 rounded-lg">
            : {report.pairs_count}
        </span>
      )}
      <ChevronRight size={20} className="text-slate-300
        ↪ group-hover:text-blue-600 group-hover:
        ↪ translate-x-1 transition-all" />
    </div>
  </button>
  )})
</div>
) : (
  <div className="flex items-center gap-3 p-6 bg-slate-50 dark:bg-
    ↪ slate-800/50 text-slate-500 rounded-3xl border border-
    ↪ dashed border-slate-200 dark:border-slate-700">
    <AlertCircle size={20} />
    <p className="text-sm font-medium">      .</p>
  </div>
  )}
</div>
</div>
);
};

```

2.15. pages\{}\ContestOverview.tsx

Листинг 209: pages\{}\ContestOverview.tsx

```

import { useState, useEffect } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import {

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    Users, Trophy, FileCode, Calendar,
    Hash, ArrowRight, BarChart3
} from 'lucide-react';
import { api } from '../api/instance';

const StatCard = ({ title, value, icon: Icon, color }: any) => (
  <div className="bg-white dark:bg-slate-900 p-6 rounded-2xl border border-slate
    ↪ -100 dark:border-slate-800 shadow-sm">
    <div className="flex justify-between items-start mb-4">
      <div className={`p-3 rounded-xl ${color}`}>
        <Icon size={20} className="text-white" />
      </div>
    </div>
    <p className="text-sm text-slate-500 dark:text-slate-400 font-medium">{
      ↪ title}</p>
    <h3 className="text-2xl font-bold mt-1 dark:text-white">{value}</h3>
  </div>
);

export const ContestOverview = () => {
  const { id } = useParams();
  const navigate = useNavigate();
  const [data, setData] = useState<any>(null);

  useEffect(() => {
    api.get(`/contests/${id}/overview`).then(res => setData(res.data));
  }, [id]);

  if (!data) return null;

  const actionCards = [
    { label: ' ', path: 'table', icon: Trophy, desc: ' ', },

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
{ label: ' ', path: 'submissions', icon: FileCode, desc: ' ', },
{ label: ' ', path: 'analytics', icon: BarChart3, desc: ' ', },
];

return (
  <div className="space-y-8 animate-in fade-in duration-500">
    {/* */}
    <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
      <StatCard title="" value={data.stats.tasks} icon={Trophy} color="bg-blue-600" />
      <StatCard title="" value={data.stats.participants} icon={Users}
        ↪ color="bg-purple-600" />
      <StatCard title="" value={data.stats.submissions} icon={FileCode}
        ↪ color="bg-emerald-600" />
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">
      {/* : */}
      <div className="lg:col-span-1 space-y-6">
        <div className="bg-white dark:bg-slate-900 p-6 rounded-3xl
          ↪ border border-slate-100 dark:border-slate-800">
          <h3 className="font-bold dark:text-white mb-6"></h3>
          <div className="space-y-4">
            <div className="flex items-center justify-between text-sm
              ↪ ">
              <div className="flex items-center gap-2 text-slate
                ↪ -500"><Calendar size={16} /> </div>
              <span className="font-medium dark:text-slate-200">{new
                ↪ Date(data.start_time).toLocaleString()}</span>
            </div>
            <div className="flex items-center justify-between text-sm
              ↪ ">
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    <div className="flex items-center gap-2 text-slate
      ↪ -500"><Hash size={16} /> External ID</div>
    <span className="font-mono font-medium dark:text-slate
      ↪ -200">{data.external_id}</span>
  </div>
  <div className="flex items-center justify-between text-sm
    ↪ ">
    <div className="flex items-center gap-2 text-slate
      ↪ -500"><Trophy size={16} /> </div>
    <span className="px-2 py-0.5 bg-blue-50 dark:bg-blue
      ↪ -900/30 text-blue-600 rounded text-xs font-bold
      ↪ ">{data.type}</span>
  </div>
</div>
</div>
</div>

{ /*   :   */ }

<div className="lg:col-span-2 space-y-4">
  <h3 className="font-bold dark:text-white px-2"> </h3>
  {actionCards.map((card) => (
    <button
      key={card.path}
      onClick={() => navigate(card.path)}
      className="w-full flex items-center justify-between p-6
        ↪ bg-white dark:bg-slate-900 rounded-3xl border
        ↪ border-slate-100 dark:border-slate-800 hover:border
        ↪ -blue-500/50 hover:shadow-lg hover:shadow-blue
        ↪ -500/5 transition-all group"
    >

    <div className="flex items-center gap-4">
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        <div className="p-3 bg-slate-50 dark:bg-slate-800
            ↪ rounded-2xl text-slate-400 group-hover:text-
            ↪ blue-600 transition-colors">
            <card.icon size={24} />
        </div>
        <div className="text-left">
            <p className="font-bold dark:text-white">{card.
                ↪ label}</p>
            <p className="text-xs text-slate-500">{card.desc
                ↪ }</p>
        </div>
    </div>
    <ArrowRight size={20} className="text-slate-300 group-
        ↪ hover:text-blue-600 group-hover:translate-x-1
        ↪ transition-all" />
</button>
    )})
</div>
</div>
</div>
);
};
```

2.16. pages\{}ContestSubmissions.tsx

Листинг 210: pages\{}ContestSubmissions.tsx

```
import { useState, useEffect, useCallback } from 'react';
import { useParams, useNavigate, Link } from 'react-router-dom';
import { Search, ArrowLeft, RefreshCw } from 'lucide-react';
import { api } from '../api/instance';
import { Pagination } from '../components/Pagination';
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
export const ContestSubmissions = () => {
  const { id } = useParams();
  const navigate = useNavigate();
  const [subs, setSubs] = useState<any[]>([]);
  const [search, setSearch] = useState('');
  const [debouncedSearch, setDebouncedSearch] = useState('');
  const [page, setPage] = useState(1);
  const [perPage, setPerPage] = useState(50);
  const [pagination, setPagination] = useState({ page: 1, per_page: 50, total: 0,
    ↪ total_pages: 1 });
  const [isLoading, setIsLoading] = useState(false);

  useEffect(() => {
    const timer = setTimeout(() => {
      setDebouncedSearch(search);
      setPage(1);
    }, 400);
    return () => clearTimeout(timer);
  }, [search]);

  const fetchSubmissions = useCallback(() => {
    setIsLoading(true);
    const params = new URLSearchParams({
      page: String(page),
      per_page: String(perPage),
    });
    if (debouncedSearch.trim()) {
      params.set('search', debouncedSearch.trim());
    }

    api.get(`/contests/${id}/submissions_list?${params}`)
      .then(res => {
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        setSubs(res.data.items);
        setPagination(res.data.pagination);
    })
    .finally(() => setIsLoading(false));
}, [id, page, perPage, debouncedSearch]);

useEffect(() => {
    fetchSubmissions();
}, [fetchSubmissions]);

const handlePerPageChange = (newPerPage: number) => {
    setPerPage(newPerPage);
    setPage(1);
};

const getVerdictStyle = (verdict: string) => {
    switch (verdict) {
        case 'OK': return 'text-green-500';
        case 'PARTIAL': return 'text-yellow-500 dark:text-yellow-400';
        case 'COMPILATION_ERROR':
        case 'CompilationError': return 'text-blue-500 dark:text-blue-400';
        default: return 'text-red-500';
    }
};

return (
    <div className="space-y-6 animate-in fade-in duration-500">
        <div className="flex justify-between items-center">
            <button onClick={() => navigate(`/contests/${id}`)}
                className="flex items-center gap-2 text-slate-500 hover:text-
                    ↩ blue-600 font-medium">
                <ArrowLeft size={20} />
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    </button>

    <h1 className="text-xl font-bold dark:text-white">  #{id}</h1>
</div>

<div className="relative">
  <Search className="absolute left-4 top-1/2 -translate-y-1/2 text-
    ↪ slate-400" size={18} />
  <input
    type="text"
    placeholder="  ,  ID..."
    className="w-full bg-white dark:bg-slate-900 dark:text-white
      ↪ rounded-2xl py-3 pl-12 pr-4 outline-none border border-
      ↪ slate-100 dark:border-slate-800 focus:ring-2 focus:ring-
      ↪ blue-500 text-sm"
    value={search}
    onChange={(e) => setSearch(e.target.value)}
  />
  {isLoading && (
    <RefreshCw className="absolute right-4 top-1/2 -translate-y-1/2
      ↪ animate-spin text-blue-500" size={16} />
  )}
</div>

<div className="bg-white dark:bg-slate-900 rounded-3xl border border-
  ↪ slate-100 dark:border-slate-800 overflow-hidden shadow-sm">
  <table className="w-full text-sm text-left">
    <thead className="bg-slate-50 dark:bg-slate-800/50 border-b dark
      ↪ :border-slate-800">
      <tr>
        <th className="px-6 py-4 font-bold dark:text-white">ID</th>
        <th className="px-6 py-4 font-bold dark:text-white"> </th>
        <th className="px-6 py-4 font-bold dark:text-white"></th>

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
<th className="px-6 py-4 font-bold dark:text-white"></th>
<th className="px-6 py-4 font-bold dark:text-white"></th>
<th className="px-6 py-4 font-bold dark:text-white"></th>
<th className="px-6 py-4 font-bold dark:text-white"></th>
</tr>
</thead>
<tbody className={isLoading ? 'opacity-50 transition-opacity' :
  ↪ 'transition-opacity'}>
{subs.map(s => (
  <tr key={s.id} className="border-b border-slate-50 dark:
    ↪ border-slate-800/50 hover:bg-slate-50 dark:hover:bg-
    ↪ slate-800/30 transition-colors">
    <td className="px-6 py-4">
      <Link
        to={`/${contests}/${id}/submissions/${s.id}`}
        className="font-mono text-[10px] text-blue-500
          ↪ hover:underline bg-blue-50 dark:bg-blue
          ↪ -900/20 px-2 py-1 rounded"
      >
        #{s.id.split('_').pop()}
      </Link>
    </td>
    <td className="px-6 py-4 text-slate-400 text-[11px]">
      {new Date(s.send_time).toLocaleString()}
    </td>
    <td className="px-6 py-4 font-bold dark:text-white">{s.
      ↪ participant_login}</td>
    <td className="px-6 py-4 text-slate-500">{s.task_name}</
      ↪ td>
    <td className="px-6 py-4 text-xs text-slate-400">{s.
      ↪ language}</td>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        <td className="px-6 py-4 font-mono text-blue-600 font-
            ↪ bold">{s.score}</td>

        <td className={`px-6 py-4 font-bold ${getVerdictStyle(s.
            ↪ verdict)}'`>

            {s.verdict}

        </td>

    </tr>

    )})

</tbody>

</table>

{!isLoading && subs.length === 0 && (
    <div className="p-12 text-center text-slate-500"> </div>
)}

</div>

{pagination.total > 0 && (
    <Pagination
        page={pagination.page}
        totalPages={pagination.total_pages}
        total={pagination.total}
        perPage={perPage}
        onPageChange={setPage}
        onPerPageChange={handlePerPageChange}
    />
    )}

</div>

);

};
```

2.17. pages\{}ContestViewPage.tsx

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Листинг 211: pages\{}\ContestViewPage.tsx

```
import { useState, useEffect, useCallback } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import { ArrowLeft, RefreshCw, Clock, Search } from 'lucide-react';
import { api } from '../api/instance';
import { Pagination } from '../components/Pagination';

export const ContestViewPage = () => {
  const { id } = useParams();
  const navigate = useNavigate();
  const [data, setData] = useState<any>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [page, setPage] = useState(1);
  const [perPage, setPerPage] = useState(50);
  const [search, setSearch] = useState('');
  const [debouncedSearch, setDebouncedSearch] = useState('');
  const [pagination, setPagination] = useState({ page: 1, per_page: 50, total: 0,
    ↪ total_pages: 1 });

  // Debounce
  useEffect(() => {
    const timer = setTimeout(() => {
      setDebouncedSearch(search);
      setPage(1);
    }, 400);
    return () => clearTimeout(timer);
  }, [search]);

  const fetchData = useCallback(() => {
    setIsLoading(true);
    const params = new URLSearchParams({
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        page: String(page),
        per_page: String(perPage),
    });
    if (debouncedSearch.trim()) {
        params.set('search', debouncedSearch.trim());
    }

    api.get(`/contests/${id}/table?${params}`)
        .then(res => {
            setData(res.data);
            setPagination(res.data.pagination);
        })
        .catch(err => setError(err.response?.data?.detail || " "))
        .finally(() => setIsLoading(false));
}, [id, page, perPage, debouncedSearch]);

useEffect(() => {
    fetchData();
}, [fetchData]);

const handlePerPageChange = (newPerPage: number) => {
    setPerPage(newPerPage);
    setPage(1);
};

const getTextColor = (res: any) => {
    if (res.verdict === 'OK') return 'text-green-500';
    if (res.verdict === 'PARTIAL') return 'text-yellow-500';
    if (res.verdict === 'WA' || (res.tries > 0 && res.score === 0)) return '
        ↪ text-red-500';
    return 'text-slate-300 dark:text-slate-700';
};
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
const formatSubmissionTime = (timeStr: string | null) => {
  if (!timeStr) return null;
  const date = new Date(timeStr);
  return date.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });
};

const renderCellContent = (res: any, contestType: string) => {
  if (res.tries === 0 && res.score === 0 && res.verdict === 'NULL') return
    '↔ ';;

  if (contestType === 'ICPC') {
    if (res.verdict === 'OK') {
      return res.tries === 0 ? '+' : `+${res.tries}`;
    }
    return res.tries > 0 ? `-${res.tries}` : '';
  }

  return res.score;
};

if (error) return <div className="p-8 text-center text-red-500">{error}</div>;
if (!data && isLoading) return (
  <div className="flex justify-center py-20">
    <RefreshCw className="animate-spin text-blue-600" size={32} />
  </div>
);
if (!data) return null;

return (
  <div className="space-y-6 animate-in fade-in duration-500 h-full flex flex-
    ↔ col">
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<div className="flex items-center gap-4">
  <button onClick={() => navigate('/contests')}
    className="p-2 hover:bg-slate-100 dark:hover:bg-slate-800
      ↪ rounded-xl transition-colors">
    <ArrowLeft size={24} className="text-slate-500" />
  </button>
  <div className="flex-1">
    <h1 className="text-2xl font-bold dark:text-white">{data.
      ↪ contest_name}</h1>
    <span className="text-[10px] px-2 py-0.5 bg-blue-100 dark:bg-
      ↪ blue-900/30 text-blue-600 dark:text-blue-400 rounded font-
      ↪ bold uppercase">
      {data.contest_type}
    </span>
  </div>
</div>

{ /* */ }

<div className="relative">
  <Search className="absolute left-4 top-1/2 -translate-y-1/2 text-
    ↪ slate-400" size={18} />
  <input
    type="text"
    placeholder="    ..."
    className="w-full bg-white dark:bg-slate-900 dark:text-white
      ↪ rounded-2xl py-3 pl-12 pr-4 outline-none border border-
      ↪ slate-100 dark:border-slate-800 focus:ring-2 focus:ring-
      ↪ blue-500 text-sm"
    value={search}
    onChange={(e) => setSearch(e.target.value)}
  />

  {isLoading && (
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<RefreshCw className="absolute right-4 top-1/2 -translate-y-1/2
  ↪ animate-spin text-blue-500" size={16} />

  })
</div>

<div className="bg-white dark:bg-slate-900 rounded-3xl border border-
  ↪ slate-100 dark:border-slate-800 shadow-sm overflow-hidden flex
  ↪ -1">

  <div className="overflow-x-auto w-full custom-scrollbar">
    <table className={`w-full text-left border-collapse min-w-max
      ↪ transition-opacity ${isLoading ? 'opacity-50' : ''}`>
      <thead>
        <tr className="bg-slate-50 dark:bg-slate-800/50 border-b
          ↪ border-slate-100 dark:border-slate-800">
          <th className="px-6 py-4 font-bold dark:text-white w-16
            ↪ text-center">#</th>
          <th className="px-6 py-4 font-bold dark:text-white min-w
            ↪ -50"></th>
          <th className="px-6 py-4 font-bold dark:text-white text-
            ↪ center w-20 border-l border-slate-100 dark:border-
            ↪ slate-800"></th>
          {data.tasks.map((task: any, i: number) => (
            <th key={i} title={task.full_name}
              className="px-2 py-4 font-bold text-center dark:
                ↪ text-white border-l border-slate-100 dark:
                ↪ border-slate-800 min-w-20">
                {task.short_name}
            </th>
          ))}
        </tr>
      </thead>
      <tbody>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
{data.rows.map((row: any) => (  
  <tr key={row.id}  
    className="border-b border-slate-50 dark:border-slate  
      ↪ -800/50 hover:bg-slate-50/50 dark:hover:bg-  
      ↪ slate-800/30">  
    <td className="px-6 py-4 text-center font-bold text-  
      ↪ slate-400 text-xs">{row.rank}</td>  
    <td className="px-6 py-4">  
      <div className="font-bold dark:text-white text-sm  
        ↪ ">{row.name}</div>  
      <div className="text-[10px] text-slate-400 font-  
        ↪ mono">@{row.login}</div>  
    </td>  
    <td className="px-4 py-4 text-center font-black text-  
      ↪ slate-700 dark:text-slate-300 border-l border-  
      ↪ slate-100 dark:border-slate-800">  
      {row.total_score}  
    </td>  
    {row.results.map((res: any, idx: number) => {  
      const cellText = renderCellContent(res, data.  
        ↪ contest_type);  
      return (  
        <td key={idx}  
          className="px-1 py-2 border-l border-slate  
            ↪ -50 dark:border-slate-800/20">  
          <div className="flex flex-col items-center  
            ↪ justify-center min-h-11">  
            <span className={'font-black text-base  
              ↪ leading-tight ${getTextColor(res)  
              ↪ }}>  
              {cellText}  
            </span>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

{data.contest_type !== 'ICPC' && res.
  ↪ tries > 0 && (
    <span className="text-[8px] text-
      ↪ slate-400 uppercase font-bold
      ↪ ">
        {res.tries} {res.tries === 1 ? '
          ↪ try' : 'tries'}
    </span>
  )}
{res.time && (
  <div className="flex items-center
    ↪ gap-0.5 text-[9px] text-slate
    ↪ -400 opacity-60 mt-0.5 font-
    ↪ mono">
    <Clock size={8} />
    {formatSubmissionTime(res.time)}
  </div>
  )}
</div>
</td>
);
)}}
</tr>
)}}
</tbody>
</table>
</div>
{!isLoading && data.rows.length === 0 && (
  <div className="p-12 text-center text-slate-500"> </div>
)}
</div>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
{pagination.total > 0 && (  
  <Pagination  
    page={pagination.page}  
    totalPages={pagination.total_pages}  
    total={pagination.total}  
    perPage={perPage}  
    onPageChange={setPage}  
    onPerPageChange={handlePerPageChange}  
  />  
  )}  
</div>  
);  
};
```

2.18. pages\{}\ContestsPage.tsx

Листинг 212: pages\{}\ContestsPage.tsx

```
import { useState, useEffect } from 'react';  
import { useNavigate } from 'react-router-dom';  
import {  
  Trophy, Plus, ExternalLink,  
  RefreshCw, Loader2, Trash2  
} from 'lucide-react';  
import { api } from '../api/instance';  
  
export const ContestsPage = () => {  
  const navigate = useNavigate();  
  const [contests, setContests] = useState<any[]>([]);  
  const [isLoading, setIsLoading] = useState(true);  
  const [syncingId, setSyncingId] = useState<number | null>(null);
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
const [unofficialMap, setUnofficialMap] = useState<Record<number, boolean>>({})  
  ↪ ;  
  
useEffect(() => {  
  fetchContests();  
}, []);  
  
const fetchContests = async () => {  
  try {  
    const res = await api.get('/contests/list');  
    const data = res.data;  
    setContests(data);  
  
    const initialMap: Record<number, boolean> = {};  
    data.forEach((contest: any) => {  
      initialMap[contest.id] = contest.unofficial ?? true;  
    });  
    setUnofficialMap(initialMap);  
  } catch (e) {  
    console.error(e);  
  } finally {  
    setIsLoading(false);  
  }  
};  
  
const handleSync = async (contest: any) => {  
  setSyncingId(contest.id);  
  const isUnofficial = unofficialMap[contest.id] ?? true;  
  
  try {  
    const endpoint = contest.platform === 'cf' ? '/codeforces/standings' :  
      ↪ '/yandex/standings';
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    await api.post(endpoint, {
      contest_id: contest.external_id,
      as_manager: true,
      from_pos: 1,
      count: 100,
      show_unofficial: isUnofficial
    });
    alert('  "${contest.name}"  (${isUnofficial ? ' ' : ' '})');
    fetchContests();
  } catch (e) {
    alert(' ');
  } finally {
    setSyncingId(null);
  }
};

const handleToggleAndSync = async (contest: any) => {
  const currentVal = unofficialMap[contest.id] ?? contest.unofficial ?? true;
  const newVal = !currentVal;

  setUnofficialMap(prev => ({ ...prev, [contest.id]: newVal }));

  setSyncingId(contest.id);
  try {
    const endpoint = contest.platform === 'cf' ? '/codeforces/standings' :
      ↪ '/yandex/standings';
    await api.post(endpoint, {
      contest_id: contest.external_id,
      as_manager: true,
      from_pos: 1,
      count: 100,
      show_unofficial: newVal

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
});

    await fetchContests();
  } catch (e) {
    console.error(e);
    alert(' ');
    setUnofficialMap(prev => ({ ...prev, [contest.id]: currentVal }));
  } finally {
    setSyncingId(null);
  }
};

const handleDelete = async (contestId: number, name: string) => {
  if (!confirm(' ', `${name}`? .')) return;

  try {
    await api.delete(`/contests/${contestId}`);
    setContests(contests.filter(c => c.id !== contestId));
  } catch (e) {
    alert(' ');
  }
};

return (
  <div className="space-y-8 animate-in fade-in duration-500">
    <div className="flex justify-between items-center">
      <h1 className="text-2xl font-bold text-slate-900 dark:text-slate-900"> </h1>
      <button onClick={() => navigate('/contests/sync')} className="bg-blue-600 text-slate-200 px-5 py-2.5 rounded-xl font-bold flex items-center gap-2 hover:bg-blue-700 transition-all">
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        <Plus size={20} />
      </button>
    </div>

    {isLoading ? (
      <div className="py-20 text-center"><RefreshCw className="animate-
        ↳ spin inline text-blue-600" /></div>
    ) : (
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap
        ↳ -6">
        {contests.map((contest) => (
          <div key={contest.id} className="bg-slate-200 dark:bg-slate
            ↳ -900 rounded-3xl border border-slate-100 dark:border-
            ↳ slate-800 p-6 flex flex-col group relative">
            <button
              onClick={() => handleDelete(contest.id, contest.name)}
              className="absolute top-4 right-4 p-2 text-slate-300
                ↳ hover:text-red-500 hover:bg-red-50 dark:hover:
                ↳ bg-red-900/20 rounded-lg transition-all opacity
                ↳ -0 group-hover:opacity-100"
            >
              <Trash2 size={18} />
            </button>

            <div className="flex justify-between items-start mb-4">
              <div className={`p-3 rounded-2xl ${contest.platform
                ↳ === 'cf' ? 'bg-orange-100 text-orange-600' : '
                ↳ bg-red-100 text-red-600'}`}>
                <Trophy size={24} />
              </div>

              {syncingId === contest.id && (

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    <div className="flex items-center gap-2 text-blue
      ↪ -600 animate-pulse">
      <Loader2 size={16} className="animate-spin" />
      <span className="text-[10px] font-bold
        ↪ uppercase">...</span>
    </div>
  )}
</div>

<h3 className="font-bold text-lg dark:text-slate-200 line
  ↪ -clamp-2 mb-4 pr-8">{contest.name}</h3>

<div className="mt-auto pt-4 border-t border-slate-50
  ↪ dark:border-slate-800">
  <div className="flex items-center justify-between mb
    ↪ -4">
    <label className={`flex items-center gap-2 cursor-
      ↪ pointer group/label ${syncingId === contest.
      ↪ id ? 'opacity-50 pointer-events-none' :
      ↪ ''}`>
    <input
      type="checkbox"
      checked={unofficialMap[contest.id] ??
        ↪ contest.unofficial ?? true}
      onChange={() => handleToggleAndSync(contest
        ↪ )}
      className="w-4 h-4 rounded border-slate-300
        ↪ text-blue-600 focus:ring-blue-500
        ↪ transition-all"
    />
    <span className="text-xs text-slate-500 dark:
      ↪ text-slate-400 group-hover/label:text-
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        ↪ blue-600 transition-colors font-medium">

    </span>
    </label>

    <button
      onClick={() => handleSync(contest)}
      disabled={syncingId === contest.id}
      className="p-1.5 text-slate-400 hover:text-
        ↪ blue-600 rounded-lg transition-colors"
    >
      <RefreshCw size={14} className={syncingId ===
        ↪ contest.id ? "animate-spin" : ""} />
    </button>
  </div>

  <button onClick={() => navigate(`/contests/${contest.
    ↪ id}`)} className="w-full flex items-center
    ↪ justify-center gap-2 py-3 rounded-xl bg-slate
    ↪ -600 dark:bg-slate-200 text-slate-200 dark:text
    ↪ -slate-900 font-bold hover:opacity-90
    ↪ transition-all">
    <span> </span>
    <ExternalLink size={16} />
  </button>
</div>
</div>
  )})
</div>
})
</div>
);
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

};

2.19. pages\{}DashboardHome.tsx

Листинг 213: pages\{}DashboardHome.tsx

```
import { Trophy, Users, FileCheck, AlertTriangle, TrendingUp } from 'lucide-react
  ↳ '

const StatCard = ({ title, value, icon: Icon, color, trend }: any) => (
  <div className="bg-white dark:bg-slate-900 p-6 rounded-2xl border border-slate
    ↳ -100 dark:border-slate-800 shadow-sm">
    <div className="flex justify-between items-start mb-4">
      <div className={`p-3 rounded-xl ${color}`}>
        <Icon size={24} className="text-white" />
      </div>
      {trend && (
        <span className="flex items-center gap-1 text-green-500 text-xs font
          ↳ -bold">
            <TrendingUp size={14} /> {trend}
          </span>
        )}
    </div>
    <p className="text-sm text-slate-500 dark:text-slate-400 font-medium">{
      ↳ title}</p>
    <h3 className="text-2xl font-bold mt-1 dark:text-white">{value}</h3>
  </div>
);

export const DashboardHome = () => {
  return (
    <div className="space-y-8 animate-in fade-in duration-500">
      <div>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<h1 className="text-2xl font-bold text-slate-900 dark:text-white
  ↪ tracking-tight"> </h1>

<p className="text-slate-500 dark:text-slate-400">   .</p>
</div>

<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6">
  <StatCard title="" value="0" icon={Trophy} color="bg-blue-600" trend
    ↪ ="+0%" />

  <StatCard title="" value="0" icon={Users} color="bg-purple-600"
    ↪ trend="+0%" />

  <StatCard title="" value="0" icon={FileCheck} color="bg-emerald-600"
    ↪ trend="+0%" />

  <StatCard title="" value="0" icon={AlertTriangle} color="bg-amber
    ↪ -600" />
</div>

<div className="bg-white dark:bg-slate-900 rounded-3xl border border-
  ↪ slate-100 dark:border-slate-800 p-8 text-center">

  <div className="max-w-sm mx-auto space-y-4">

    <div className="w-16 h-16 bg-slate-100 dark:bg-slate-800 rounded
      ↪ -full flex items-center justify-center mx-auto">

      <Trophy size={32} className="text-slate-300" />

    </div>

    <h3 className="text-lg font-bold dark:text-white">   </h3>

    <p className="text-slate-500 text-sm">           Codeforces   Yandex
      ↪ Contest.</p>

    <button className="bg-blue-600 text-white px-6 py-2 rounded-xl
      ↪ font-bold hover:bg-blue-700 transition-all">

      </button>

    </div>

  </div>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        </div>
    );
};

```

2.20. pages\{}\ImportSubmissions.tsx

Листинг 214: pages\{}\ImportSubmissions.tsx

```

import { useState, useEffect } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import {
    Download, ArrowLeft, Loader2, Info,
    UserCog, Layers, Code2, Database, Hash
} from 'lucide-react';
import { api } from '../api/instance';

export const ImportSubmissions = () => {
    const { id } = useParams();
    const navigate = useNavigate();
    const [loading, setLoading] = useState(false);
    const [contestInfo, setContestInfo] = useState<any>(null);

    //
    const [formData, setFormData] = useState({
        from_pos: 1,
        count: 100,
        as_manager: true, // CF
        include_source: true // CF
    });

    useEffect(() => {
        api.get(`/contests/${id}/overview`).then(res => {
            setContestInfo(res.data);

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
});  
, [id]);  
  
const handleImport = async (e: React.FormEvent) => {  
  e.preventDefault();  
  setLoading(true);  
  
  const platform = contestInfo.type; // 'cf' 'yandex'  
  const externalId = parseInt(contestInfo.external_id);  
  
  try {  
    let endpoint = '';  
    let payload = {};  
  
    if (platform === 'cf') {  
      endpoint = '/codeforces/submissions';  
      payload = {  
        contest_id: externalId,  
        as_manager: formData.as_manager,  
        from_pos: formData.from_pos,  
        count: formData.count,  
        include_source: formData.include_source  
      };  
    } else {  
      endpoint = '/yandex/submissions';  
      payload = {  
        contest_id: externalId,  
        from_pos: formData.from_pos,  
        count: formData.count  
      };  
    }  
  }  
}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    await api.post(endpoint, payload);
    alert('  !');
    navigate(`/contests/${id}/submissions`);
  } catch (err: any) {
    alert(' : ' + (err.response?.data?.detail || '  API'));
  } finally {
    setLoading(false);
  }
};

if (!contestInfo) return null;

const isCF = contestInfo.type === 'cf';

return (
  <div className="max-w-2xl mx-auto space-y-6 animate-in slide-in-from-bottom
    ↪ -4 duration-500">
    <button onClick={() => navigate(-1)} className="flex items-center gap-2
      ↪ text-slate-500 hover:text-blue-600 font-medium">
      <ArrowLeft size={20} />
    </button>

    <div className="bg-white dark:bg-slate-900 p-8 rounded-3xl border border
      ↪ -slate-100 dark:border-slate-800 shadow-sm">
      <div className="flex items-center gap-4 mb-8">
        <div className="p-3 bg-blue-600 rounded-2xl text-white">
          <Download size={28} />
        </div>
        <div>
          <h1 className="text-2xl font-bold dark:text-white"> </h1>
          <p className="text-slate-500 text-sm">    <span className="
            ↪ text-blue-600 font-bold">"{contestInfo.name}"</span></p>
        </div>
      </div>
    </div>
  </div>
);
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    ↪ >

  </div>
</div>

<form onSubmit={handleImport} className="space-y-6">
  {/* (    ) */}
  <div className="grid grid-cols-2 gap-4">
    <div className={`p-4 rounded-2xl border-2 text-center
      ↪ transition-all ${isCF ? 'border-orange-500 bg-orange
      ↪ -50/50 dark:bg-orange-900/10' : 'border-transparent bg-
      ↪ slate-50 dark:bg-slate-800 opacity-50'}`}>
      <span className={`block font-bold ${isCF ? 'text-orange
        ↪ -600' : 'dark:text-white'}`}>Codeforces</span>
      <span className="text-[10px] text-slate-500 uppercase">
        ↪ Platform Active</span>
    </div>
    <div className={`p-4 rounded-2xl border-2 text-center
      ↪ transition-all ${!isCF ? 'border-red-500 bg-red-50/50
      ↪ dark:bg-red-900/10' : 'border-transparent bg-slate-50
      ↪ dark:bg-slate-800 opacity-50'}`}>
      <span className={`block font-bold ${!isCF ? 'text-red
        ↪ -600' : 'dark:text-white'}`}>Yandex</span>
      <span className="text-[10px] text-slate-500 uppercase">
        ↪ Platform Active</span>
    </div>
  </div>

  {/* - ID */}
  <div className="p-4 bg-slate-50 dark:bg-slate-800 rounded-2xl
    ↪ flex items-center justify-between border border-dashed
    ↪ border-slate-200 dark:border-slate-700">

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

<div className="flex items-center gap-3 text-slate-500 text-
  ↪ sm">
  <Database size={18} /> <span> ID </span>
</div>

<span className="font-mono font-bold text-blue-600">{
  ↪ contestInfo.external_id}</span>
</div>

{ /*    */ }

<div className="grid grid-cols-2 gap-6">
  <div className="space-y-2">
    <label className="text-sm font-bold text-slate-700 dark:
      ↪ text-slate-300 ml-1 flex items-center gap-2">
      <Layers size={14} className="text-blue-600" />
    </label>
    <input
      type="number"
      min="1"
      className="w-full p-4 bg-slate-50 dark:bg-slate-800
        ↪ rounded-2xl dark:text-white outline-none focus:
        ↪ ring-2 focus:ring-blue-500 transition-all"
      value={formData.from_pos}
      onChange={e => setFormData({...formData, from_pos:
        ↪ parseInt(e.target.value) || 1})}
    />
    <p className="text-[10px] text-slate-400 ml-1">    </p>
  </div>
  <div className="space-y-2">
    <label className="text-sm font-bold text-slate-700 dark:
      ↪ text-slate-300 ml-1 flex items-center gap-2">
      <Hash size={14} className="text-blue-600" />
    </label>

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<input
  type="number"
  min="1"
  max="10000"
  className="w-full p-4 bg-slate-50 dark:bg-slate-800
    ↪ rounded-2xl dark:text-white outline-none focus:
    ↪ ring-2 focus:ring-blue-500 transition-all"
  value={formData.count}
  onChange={e => setFormData({...formData, count:
    ↪ parseInt(e.target.value) || 1})}
/>
<p className="text-[10px] text-slate-400 ml-1"> ( 10000)
  ↪ </p>
</div>
</div>

{/* ( Codeforces) */}
{isCF && (
  <div className="space-y-3 animate-in fade-in slide-in-from-
    ↪ top-2">
    <h3 className="text-xs font-bold text-slate-400 uppercase
      ↪ tracking-wider ml-1"> Codeforces</h3>

    <label className="flex items-center justify-between p-4
      ↪ bg-slate-50 dark:bg-slate-800 rounded-2xl cursor-
      ↪ pointer hover:bg-slate-100 dark:hover:bg-slate
      ↪ -700/50 transition-colors">
      <div className="flex items-center gap-3">
        <UserCog size={20} className="text-blue-600" />
        <div>
          <p className="font-bold text-sm dark:text-
            ↪ white"> </p>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

        <p className="text-[10px] text-slate-500">
            ↪ </p>
    </div>
</div>
<input
    type="checkbox"
    className="w-5 h-5 rounded border-none bg-slate
        ↪ -200 dark:bg-slate-600 text-blue-600 focus:
        ↪ ring-0"
    checked={formData.as_manager}
    onChange={(e) => setFormData({...formData,
        ↪ as_manager: e.target.checked})}
/>
</label>

<label className="flex items-center justify-between p-4
    ↪ bg-slate-50 dark:bg-slate-800 rounded-2xl cursor-
    ↪ pointer hover:bg-slate-100 dark: hover:bg-slate
    ↪ -700/50 transition-colors">
    <div className="flex items-center gap-3">
        <Code2 size={20} className="text-emerald-600" />
        <div>
            <p className="font-bold text-sm dark:text-
                ↪ white"> </p>
            <p className="text-[10px] text-slate-500"> (
                ↪ Base64)</p>
        </div>
    </div>
</label>
<input
    type="checkbox"
    className="w-5 h-5 rounded border-none bg-slate
        ↪ -200 dark:bg-slate-600 text-emerald-600
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        ↪ focus:ring-0"
        checked={formData.include_source}
        onChange={(e) => setFormData({...formData,
        ↪ include_source: e.target.checked})}
      />
    </label>
  </div>
)}

<div className="bg-blue-50 dark:bg-blue-900/20 p-4 rounded-2xl
  ↪ flex gap-3 text-blue-700 dark:text-blue-400 text-sm">
  <Info className="shrink-0" size={20} />
  <p>          /.</p>
</div>

<button
  type="submit"
  disabled={loading}
  className="w-full bg-blue-600 text-white py-4 rounded-2xl
    ↪ font-bold hover:bg-blue-700 disabled:opacity-50
    ↪ transition-all flex justify-center items-center gap-2
    ↪ shadow-lg shadow-blue-500/20"
  >
  {loading ? <Loader2 className="animate-spin" /> : <Download
    ↪ size={20} />}
  <span>{loading ? ' ...' : ' '}</span>
</button>
</form>
</div>
</div>
);
};
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2.21. pages\{}\Landing.tsx

Листинг 215: pages\{}\Landing.tsx

```
import { Link } from 'react-router-dom';
import { ShieldCheck, BarChart3, Globe, Cpu, FileSpreadsheet, Sparkles } from '
  ↳ lucide-react';
import { ThemeToggle } from "../components/ThemeToggle.tsx";

const FeatureCard = ({ icon: Icon, title, description }: any) => (
  <div className="bg-white dark:bg-slate-400 p-8 md:p-8 rounded-2xl border border
    ↳ -slate-100 dark:border-slate-300 shadow-sm
    hover:shadow-md transition-all">
    <div className="w-12 h-12 bg-blue-50 dark: bg-blue-100 text-blue-600
      ↳ rounded-xl flex items-center justify-center mb-6">
      <Icon size={24} />
    </div>
    <h3 className="text-xl font-bold text-slate-900 mb-3">{title}</h3>
    <p className="text-slate-600 leading-relaxed text-sm md:text-base">{
      ↳ description}</p>
  </div>
);

const Step = ({ number, title, description }: any) => (
  <div className="flex flex-col items-center text-center px-4">
    <div className="w-10 h-10 bg-blue-600 text-white rounded-full flex items-
      ↳ center justify-center font-bold mb-4 shadow-lg">
      {number}
    </div>
    <h4 className="font-bold text-lg mb-2 dark:text-white">{title}</h4>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    <p className="text-slate-500 dark:text-slate-400 text-sm leading-relaxed">{
      ↪ description}</p>
  </div>
);

export const Landing = () => {
  return (
    <div className="min-h-screen w-full bg-slate-100 dark:bg-gray-900
      ↪ transition-colors duration-300">
      <nav className="w-full border-b border-slate-100 dark:border-slate-800
        ↪ sticky top-0 bg-slate-100/80 dark:bg-slate-900/80 backdrop-blur-
        ↪ md z-50">
        <div className="max-w-7xl mx-auto px-6 py-6 flex justify-between
          ↪ items-center">
          <div className="flex items-center gap-2 font-bold text-2xl text-
            ↪ slate-900 dark:text-white">
            <div className="w-12 h-12 bg-blue-600 dark:bg-blue-500
              ↪ rounded-lg flex items-center justify-center text-white
              ↪ shadow-lg">MC</div>

          </div>
          <div className="flex items-center gap-8">
            <ThemeToggle />
            <Link to="/login" className="font-medium text-slate-600 dark:
              ↪ text-slate-400 hover:text-blue-600"></Link>
            <Link to="/register" className="bg-slate-900 dark:bg-white
              ↪ dark:text-slate-900 text-white px-6 py-2.5 rounded-full
              ↪ font-medium hover:opacity-90"></Link>
          </div>
        </div>
      </div>
    </nav>
  )
}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<header className="w-full px-6 pt-20 pb-32">
  <div className="max-w-4xl mx-auto text-center">
    <h1 className="text-5xl md:text-7xl font-extrabold text-slate
      ↪ -900 dark:text-white tracking-tight mb-8">
      <span className="text-blue-600 dark:text-blue-500"></span>
    </h1>
    <p className="text-xl text-slate-600 dark:text-slate-400 leading
      ↪ -relaxed">
      Codeforces Yandex,
      .
    </p>
  </div>
</header>

<section className="w-full bg-slate-50 dark:bg-slate-800/50 py-32 border
  ↪ -y border-slate-100 dark:border-slate-800">
  <div className="max-w-7xl mx-auto px-6">
    <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
      <FeatureCard
        icon={Globe}
        title=""
        description=" API Codeforces Yandex Contest ."
      />
      <FeatureCard
        icon={ShieldCheck}
        title=""
        description=" ."
      />
      <FeatureCard
        icon={BarChart3}
        title=""
        description=" , , , ."
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    />
    <FeatureCard
      icon={Sparkles}
      title="AI "
      description=" , Polygon."
    />
  </div>
</div>
</section>

<section className="w-full py-32 bg-white dark:bg-slate-900">
  <div className="max-w-7xl mx-auto px-6 text-center">
    <h2 className="text-3xl md:text-4xl font-bold text-slate-900
      ↪ dark:text-white mb-16">    </h2>
    <div className="grid grid-cols-1 md:grid-cols-3 gap-12 relative
      ↪ ">
      <div className="hidden md:block absolute top-5 left-1/4 right
        ↪ -1/4 h-0.5 bg-slate-100 dark:bg-slate-800 z-0"></div>

      <div className="relative z-10"><Step number="1" title=""
        ↪ description=" ID Codeforces Yandex ." icon={Globe}
        ↪ /></div>
      <div className="relative z-10"><Step number="2" title=""
        ↪ description=" , ." icon={Cpu} /></div>
      <div className="relative z-10"><Step number="3" title=""
        ↪ description=" ." icon={FileSpreadsheet} /></div>
        ↪ div>
    </div>
  </div>
</section>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    <footer className="w-full py-12 border-t border-slate-100 dark:border-
      ↪ slate-800 text-center">
      <p className="text-slate-400 dark:text-slate-500 text-sm mb-4">
        {new Date().getFullYear()} Contest Manager System.
      </p>
      <div className="flex justify-center gap-6 text-slate-400 text-sm">
        <a href="/docs" className="hover:text-blue-600 transition-colors
          ↪ "></a>
        <a href="https://github.com/TryKuhn/YandexCodeforcesAnalyzer"
          ↪ className="hover:text-blue-600 transition-colors">GitHub</
          ↪ a>
      </div>
    </footer>
  </div>
);
};

```

2.22. pages\{}\LoadContestPage.tsx

ЛИСТИНГ 216: pages\{}\LoadContestPage.tsx

```

import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import {
  CloudDownload, ArrowLeft, Loader2, Info,
  UserCog, Users, Hash, Layers
} from 'lucide-react';
import { api } from '../api/instance';

export const LoadContestPage = () => {
  const navigate = useNavigate();
  const [platform, setPlatform] = useState<'cf' | 'yandex'>('cf');
  const [isLoading, setIsLoading] = useState(false);

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
const [formData, setFormData] = useState({
  contest_id: '',
  as_manager: true,
  from_pos: 1,
  count: 100,
  show_unofficial: true
});

const handleSync = async (e: React.FormEvent) => {
  e.preventDefault();
  setIsLoading(true);
  try {
    const endpoint = platform === 'cf' ? '/codeforces/standings' : '/yandex/
    ↪ standings';

    await api.post(endpoint, {
      contest_id: parseInt(formData.contest_id),
      as_manager: formData.as_manager,
      from_pos: formData.from_pos,
      count: formData.count,
      show_unofficial: formData.show_unofficial
    });

    alert(' ');
    navigate('/contests');
  } catch (err: any) {
    alert(' : ' + (err.response?.data?.detail || ' '));
  } finally {
    setIsLoading(false);
  }
};
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
return (  
  <div className="max-w-2xl mx-auto space-y-8 animate-in slide-in-from-bottom  
    ↪ -4 duration-500 pb-12">  
    <button  
      onClick={() => navigate('/contests')}  
      className="flex items-center gap-2 text-slate-500 hover:text-blue  
        ↪ -600 transition-colors font-medium"  
    >  
      <ArrowLeft size={20} />  
      <span> </span>  
    </button>  
  
    <div className="bg-white dark:bg-slate-900 rounded-3xl border border-  
      ↪ slate-100 dark:border-slate-800 p-8 shadow-sm">  
      <div className="flex items-center gap-4 mb-8">  
        <div className="p-3 bg-blue-600 rounded-2xl text-white">  
          <CloudDownload size={28} />  
        </div>  
        <div>  
          <h1 className="text-2xl font-bold dark:text-white"> </h1>  
          <p className="text-slate-500 text-sm"> .</p>  
        </div>  
      </div>  
  
      <form onSubmit={handleSync} className="space-y-6">  
        <div className="grid grid-cols-2 gap-4">  
          <button  
            type="button"  
            onClick={() => setPlatform('cf')}  
            className={`p-4 rounded-2xl border-2 transition-all text-  
              ↪ center ${platform === 'cf' ? 'border-blue-500 bg-
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        ↪ blue-50/50 dark:bg-blue-900/20' : 'border-
        ↪ transparent bg-slate-50 dark:bg-slate-800'}{'}
    >
    <span className="block font-bold dark:text-white">
        ↪ Codeforces</span>
    <span className="text-xs text-slate-500 italic">API Key /
        ↪ Secret</span>
</button>
<button
    type="button"
    onClick={() => setPlatform('yandex')}
    className={`p-4 rounded-2xl border-2 transition-all text-
        ↪ center ${platform === 'yandex' ? 'border-blue-500
        ↪ bg-blue-50/50 dark:bg-blue-900/20' : 'border-
        ↪ transparent bg-slate-50 dark:bg-slate-800'}{'}
    >
    <span className="block font-bold dark:text-white">Yandex
        ↪ </span>
    <span className="text-xs text-slate-500 italic">OAuth
        ↪ Token</span>
</button>
</div>

<div>
    <label className="flex items-center gap-2 text-sm font-bold
        ↪ text-slate-700 dark:text-slate-300 mb-2 ml-1">
        <Hash size={16} className="text-blue-600" />
        ID
    </label>
    <input
        type="text"
        required

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
placeholder=": 123456"
className="w-full bg-slate-50 dark:bg-slate-800 border-
  ↪ none rounded-2xl p-4 outline-none focus:ring-2
  ↪ focus:ring-blue-500 dark:text-white transition-all"
value={formData.contest_id}
onChange={(e) => setFormData({...formData, contest_id: e.
  ↪ target.value})}
/>
</div>

<div className="grid grid-cols-2 gap-6">
  <div>
    <label className="flex items-center gap-2 text-sm font-
      ↪ bold text-slate-700 dark:text-slate-300 mb-2 ml-1">
      <Layers size={16} className="text-blue-600" />

    </label>
    <input
      type="number"
      min="1"
      className="w-full bg-slate-50 dark:bg-slate-800 border
        ↪ -none rounded-2xl p-4 outline-none focus:ring-2
        ↪ focus:ring-blue-500 dark:text-white transition
        ↪ -all"
      value={formData.from_pos}
      onChange={(e) => setFormData({...formData, from_pos:
        ↪ parseInt(e.target.value)})}
    />
  </div>
  <div>
    <label className="flex items-center gap-2 text-sm font-
      ↪ bold text-slate-700 dark:text-slate-300 mb-2 ml-1">
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        <Users size={16} className="text-blue-600" />

    </label>
    <input
      type="number"
      min="1"
      max="10000"
      className="w-full bg-slate-50 dark:bg-slate-800 border
        ↪ -none rounded-2xl p-4 outline-none focus:ring-2
        ↪ focus:ring-blue-500 dark:text-white transition
        ↪ -all"
      value={formData.count}
      onChange={(e) => setFormData({...formData, count:
        ↪ parseInt(e.target.value)}})
    />
  </div>
</div>

<div className="grid grid-cols-1 gap-4">
  <label className="flex items-center justify-between p-4 bg-
    ↪ slate-50 dark:bg-slate-800 rounded-2xl cursor-pointer
    ↪ hover:bg-slate-100 dark:hover:bg-slate-700/50
    ↪ transition-colors">
    <div className="flex items-center gap-3">
      <UserCog size={20} className="text-blue-600" />
      <div>
        <p className="font-bold text-sm dark:text-white">
          ↪ </p>
        <p className="text-xs text-slate-500">      </p>
      </div>
    </div>
  </div>
  <input
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        type="checkbox"
        className="w-5 h-5 rounded border-none bg-slate-200
        ↪ dark:bg-slate-600 text-blue-600 focus:ring-0"
        checked={formData.as_manager}
        onChange={(e) => setFormData({...formData, as_manager:
        ↪ e.target.checked})}
    />
</label>

<label className="flex items-center justify-between p-4 bg-
    ↪ slate-50 dark:bg-slate-800 rounded-2xl cursor-pointer
    ↪ hover:bg-slate-100 dark:hover:bg-slate-700/50
    ↪ transition-colors">
    <div className="flex items-center gap-3">
        <Users size={20} className="text-blue-600" />
        <div>
            <p className="font-bold text-sm dark:text-white">
                ↪ </p>
            <p className="text-xs text-slate-500"> </p>
        </div>
    </div>
</label>
<input
    type="checkbox"
    className="w-5 h-5 rounded border-none bg-slate-200
    ↪ dark:bg-slate-600 text-blue-600 focus:ring-0"
    checked={formData.show_unofficial}
    onChange={(e) => setFormData({...formData,
    ↪ show_unofficial: e.target.checked})}
    />
</label>
</div>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

<div className="bg-amber-50 dark:bg-amber-900/20 p-4 rounded-2xl
  ↪ flex gap-3 text-amber-700 dark:text-amber-500 text-sm">
  <Info className="shrink-0" size={20} />
  <p>      Codeforces <b></b>      .</p>
</div>

<button
  type="submit"
  disabled={isLoading || !formData.contest_id}
  className="w-full bg-blue-600 text-white py-4 rounded-2xl
    ↪ font-bold hover:bg-blue-700 disabled:opacity-50
    ↪ transition-all flex justify-center items-center gap-2
    ↪ shadow-lg shadow-blue-500/20"
  >
  {isLoading ? <Loader2 className="animate-spin" /> : <
    ↪ CloudDownload size={20} />}
  <span>{isLoading ? ' ...' : ' '}</span>
</button>
</form>
</div>
</div>
);
};

```

2.23. pages\{}\LoginPage.tsx

Листинг 217: pages\{}\LoginPage.tsx

```

import {useState} from 'react';
import {Link, useNavigate} from 'react-router-dom';
import { api } from '../api/instance';
import { useAuthStore } from '../store/useAuthStore';
import {ThemeToggle} from "../components/ThemeToggle.tsx";

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
import {ArrowLeft} from "lucide-react";

export const LoginPage = () => {
  const [login, setLogin] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const navigate = useNavigate();
  const setAuth = useAuthStore((state) => state.setAuthenticated);
  const setUser = useAuthStore((state) => state.setUser);

  const handleSubmit = async (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    setError('');

    try {
      const response = await api.post('/auth/login', { login, password });
      const {access_token, refresh_token, token_type} = response.data;

      setAuth(access_token, refresh_token, token_type);

      const user_response = await api.get('/auth/me');
      const user = user_response.data;

      setUser(user);

      navigate('/');
    } catch (err: any) {
      setError(err.response?.data?.detail || ' ');
    }
  };

  return (
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<div className="min-h-screen w-full flex flex-col bg-white dark:bg-slate
  ↳ -900 transition-colors duration-300">

  <nav className="w-full border-b border-slate-100 dark:border-slate-800
    ↳ sticky top-0 bg-white/80 dark:bg-slate-900/80 backdrop-blur-md z
    ↳ -50">

    <div className="max-w-7xl mx-auto px-6 py-4 flex justify-between
      ↳ items-center">

      <div className="flex items-center gap-2 font-bold text-2xl text-
        ↳ slate-900 dark:text-white">

        <div className="w-10 h-10 bg-blue-600 rounded-lg flex items-
          ↳ center justify-center text-white shadow-lg">MC</div>

        <span className="hidden sm:inline"> </span>

      </div>

      <div className="flex items-center gap-6">

        <ThemeToggle />

        <Link
          to="/"
          className="flex items-center gap-2 font-medium text-slate
            ↳ -600 dark:text-slate-400 hover:text-blue-600
            ↳ transition-colors"
        >

          <ArrowLeft size={18} />

          <span> </span>

        </Link>

      </div>

    </nav>

    <div className="flex-1 flex items-center justify-center p-6">
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
<div className="bg-slate-50 dark:bg-slate-800 p-8 rounded-2xl shadow
  ↪ -xl border border-slate-100 dark:border-slate-700 w-full max-w
  ↪ -md animate-in fade-in zoom-in duration-300">

  <h2 className="text-2xl text-slate-900 dark:text-slate-100 font-
    ↪ bold mb-6 text-center">

</h2>

  {error && (
    <div className="bg-red-100 dark:bg-red-900/30 text-red-700
      ↪ dark:text-red-400 p-3 rounded-lg mb-4 text-sm border
      ↪ border-red-200 dark:border-red-800">

        {error}

      </div>
    )}

  <form onSubmit={handleSubmit} className="space-y-4">
    <div>
      <label className="block text-sm dark:text-slate-300 font-
        ↪ medium mb-1 ml-1">

</label>

      <input
        type="text"
        className="w-full border border-slate-200 dark:border-
          ↪ slate-600 dark:bg-slate-700 dark:text-white
          ↪ rounded-lg p-2.5 outline-none focus:ring-2
          ↪ focus:ring-blue-500 transition-all"
        placeholder=" "
        value={login}
        onChange={(e) => setLogin(e.target.value)}
        required
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        />
    </div>
    <div>
        <label className="block text-sm dark:text-slate-300 font-
            ↪ medium mb-1 ml-1">

        </label>
        <input
            type="password"
            className="w-full border border-slate-200 dark:border-
                ↪ slate-600 dark:bg-slate-700 dark:text-white
                ↪ rounded-lg p-2.5 outline-none focus:ring-2
                ↪ focus:ring-blue-500 transition-all"
            placeholder=""
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            required
        />
    </div>
    <button
        type="submit"
        className="w-full bg-blue-600 dark:bg-blue-500 text-white
            ↪ py-3 rounded-lg font-bold hover:bg-blue-700 dark:
            ↪ hover:bg-blue-600 transition-all shadow-lg shadow-
            ↪ blue-500/20 mt-2"
    >

    </button>
</form>

<p className="mt-6 text-center text-sm text-slate-500 dark:text-
    ↪ slate-400">
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        ? <Link to="/register" className="text-blue-600 hover:
          ↪ underline font-medium"> </Link>

      </p>
    </div>
  </div>
</div>
);
};
```

2.24. pages\{}\NotFound.tsx

Листинг 218: pages\{}\NotFound.tsx

```
import { useNavigate } from 'react-router-dom';
import { Ghost, Home } from 'lucide-react';

export const NotFound = () => {
  const navigate = useNavigate();

  return (
    <div className="min-h-screen bg-slate-50 dark:bg-slate-950 flex items-
      ↪ center justify-center p-4">
      <div className="text-center space-y-6">
        <div className="relative inline-block">
          <div className="absolute inset-0 bg-blue-500 blur-3xl opacity-20
            ↪ rounded-full"></div>
          <Ghost size={80} className="text-blue-600 relative mx-auto
            ↪ animate-bounce" />
        </div>
        <div className="space-y-2">
          <h1 className="text-4xl font-black text-slate-900 dark:text-
            ↪ white">404</h1>

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        <p className="text-slate-500 dark:text-slate-400 max-w-xs mx-
            ↪ auto">
            ! , .
        </p>
    </div>
    <button
        onClick={() => navigate('/')}>
        className="flex items-center gap-2 mx-auto bg-blue-600 text-
            ↪ white px-6 py-3 rounded-2xl font-bold hover:bg-blue-700
            ↪ transition-all shadow-xl shadow-blue-500/20"
    >
        <Home size={20} />
        <span> </span>
    </button>
</div>
</div>
);
};
```

2.25. pages\PlagiarismComparation.tsx

Листинг 219: pages\PlagiarismComparation.tsx

```
import {useState, useEffect} from 'react';
import {useParams, useNavigate} from 'react-router-dom';
import {ArrowLeft, Copy} from 'lucide-react';
import {api} from '../api/instance';

export const PlagiarismComparison = () => {
    const {pairId} = useParams();
    const navigate = useNavigate();
    const [data, setData] = useState<any>(null);
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
// (base64)
const decodeCode = (str: string) => {
  try {
    return new TextDecoder('utf-8').decode(Uint8Array.from(atob(str), c => c
      ↪ .charCodeAt(0)));
  } catch (e) {
    return " ";
  }
};

useEffect(() => {
  api.get(`/analytics/pairs/${pairId}`).then(res => {
    const d = res.data;
    d.code1 = decodeCode(d.code1);
    d.code2 = decodeCode(d.code2);
    setData(d);
  });
}, [pairId]);

if (!data) return null;

const CodeHeader = ({user, subId}: any) => (
  <div className="p-4 bg-white dark:bg-slate-900 border-b dark:border-slate
    ↪ -800 flex items-center justify-between">
    <div className="flex items-center gap-3">
      <div className="w-10 h-10 bg-blue-100 dark:bg-blue-900/30 text-blue
        ↪ -600 rounded-full flex items-center justify-center font-bold">
        {user ? user[0].toUpperCase() : ''}
      </div>
      <div>
        <p className="font-bold text-sm dark:text-white">{user || ''}</p>
        ↪ >
      </div>
    </div>
  </div>
);
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<div className="grid grid-cols-2 gap-4 flex-1 min-h-0 overflow-hidden">
  <div
    className="flex flex-col rounded-3xl border border-slate-100
      ↪ dark:border-slate-800 overflow-hidden bg-slate-950">
    <CodeHeader user={data.user1} subId={data.sub1_id}/>
    <div className="flex-1 p-6 overflow-auto font-mono text-[11px]
      ↪ leading-relaxed text-slate-300">
      <pre><code>{data.code1}</code></pre>
    </div>
  </div>

  <div
    className="flex flex-col rounded-3xl border border-slate-100
      ↪ dark:border-slate-800 overflow-hidden bg-slate-950">
    <CodeHeader user={data.user2} subId={data.sub2_id}/>
    <div className="flex-1 p-6 overflow-auto font-mono text-[11px]
      ↪ leading-relaxed text-slate-300">
      <pre><code>{data.code2}</code></pre>
    </div>
  </div>
</div>

</div>
);
};
```

2.26. pages\{}\PlagiarismReport.tsx

Листинг 220: pages\{}\PlagiarismReport.tsx

```
import { useState, useEffect, useCallback } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import {
  Loader2,
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    AlertTriangle,  
    User,  
    ChevronRight,  
    CheckCircle2  
} from 'lucide-react';  
import { api } from '../api/instance';  
import { Pagination } from '../components/Pagination';  
  
type ReportStatus = 'processing' | 'completed' | 'failed';  
  
interface Pair {  
    id: number;  
    sub1: string;  
    sub2: string;  
    user1: string;  
    user2: string;  
    task_name: string;  
    percent: number;  
}  
  
interface PaginationData {  
    page: number;  
    per_page: number;  
    total: number;  
    total_pages: number;  
}  
  
export const PlagiarismReport = () => {  
    const { id, reportId } = useParams();  
    const navigate = useNavigate();  
  
    const [status, setStatus] = useState<ReportStatus>('processing');
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
const [pairs, setPairs] = useState<Pair[]>([]);
const [isLoading, setIsLoading] = useState(true);

const [page, setPage] = useState(1);
const [perPage, setPerPage] = useState(20);
const [pagination, setPagination] = useState<PaginationData>({
  page: 1,
  per_page: 20,
  total: 0,
  total_pages: 1,
});

const [availableTasks, setAvailableTasks] = useState<string[]>([]);
const [selectedTask, setSelectedTask] = useState<string | null>(null);

const fetchReport = useCallback(async (silent = false) => {
  if (!reportId) return;
  if (!silent) setIsLoading(true);

  try {
    const res = await api.get(`/analytics/reports/${reportId}`, {
      params: {
        page,
        per_page: perPage,
        task_name: selectedTask,
      },
    });

    setStatus(res.data.status);
    setPairs(res.data.pairs || []);
    setAvailableTasks(res.data.tasks || []);
    setPagination(res.data.pagination || {
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        page: 1,
        per_page: 20,
        total: 0,
        total_pages: 1,
      });
    } catch {
      setStatus('failed');
    } finally {
      if (!silent) setIsLoading(false);
    }
  }, [reportId, page, perPage, selectedTask]);

useEffect(() => {
  setPage(1);
}, [selectedTask]);

useEffect(() => {
  if (status !== 'processing') return;

  const interval = setInterval(() => {
    fetchReport(true);
  }, 2000);

  return () => clearInterval(interval);
}, [status, fetchReport]);

const handlePerPageChange = (newPerPage: number) => {
  setPerPage(newPerPage);
  setPage(1);
};

if (status === 'processing') {
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
return (  
  <div className="h-100 flex flex-col items-center justify-center space-y  
    ↪ -4 bg-white dark:bg-slate-900 rounded-3xl border border-dashed  
    ↪ border-slate-200 dark:border-slate-800">  
    <Loader2 size={48} className="animate-spin text-blue-600" />  
    <div className="text-center">  
      <h2 className="text-xl font-bold dark:text-white"> ...</h2>  
      <p className="text-slate-500 text-sm mt-1">  
        AST-  
      </p>  
    </div>  
  </div>  
);  
}  
  
if (status === 'failed') {  
  return (  
    <div className="h-100 flex flex-col items-center justify-center space-y  
      ↪ -4 bg-white dark:bg-slate-900 rounded-3xl border border-dashed  
      ↪ border-red-200 dark:border-red-900/30">  
      <AlertTriangle size={48} className="text-red-500" />  
      <div className="text-center">  
        <h2 className="text-xl font-bold dark:text-white"> </h2>  
        <p className="text-slate-500 text-sm mt-1">  
  
        </p>  
      </div>  
    </div>  
  );  
}  
  
return (  

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<div className="space-y-6 animate-in fade-in duration-500">
  <div className="flex justify-between items-center">
    <h1 className="text-2xl font-bold dark:text-white flex items-center
      ↪ gap-3">
      <AlertTriangle className="text-amber-500" />

      {isLoading && <Loader2 size={18} className="animate-spin text-
        ↪ blue-500" />}
    </h1>

    <div className="flex items-center gap-2 text-xs font-bold text-slate
      ↪ -400 bg-slate-100 dark:bg-slate-800 px-3 py-1.5 rounded-full">
      <CheckCircle2 size={14} className="text-green-500" />
    </div>
  </div>

  {availableTasks.length > 0 && (
    <div className="flex flex-wrap gap-2">
      <button
        onClick={() => setSelectedTask(null)}
        className={`px-4 py-2 rounded-2xl text-xs font-bold
          ↪ transition-all ${
            selectedTask === null
              ? 'bg-blue-600 text-white shadow-lg shadow-blue
                ↪ -500/30'
              : 'bg-white dark:bg-slate-900 text-slate-500 border
                ↪ border-slate-100 dark:border-slate-800 hover:
                ↪ border-blue-500/50'
            }`}
      >
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

</button>
{availableTasks.map(task => (
  <button
    key={task}
    onClick={() => setSelectedTask(task)}
    className={`px-4 py-2 rounded-2xl text-xs font-bold
      ↪ transition-all ${
        selectedTask === task
          ? 'bg-blue-600 text-white shadow-lg shadow-blue
            ↪ -500/30'
          : 'bg-white dark:bg-slate-900 text-slate-500
            ↪ border border-slate-100 dark:border-slate
            ↪ -800 hover:border-blue-500/50'
      }}
  >
    {task}
  </button>
)}}
</div>
)}

<div className={`grid grid-cols-1 gap-3 transition-opacity ${isLoading ?
  ↪ 'opacity-50' : ''}`}>
  {pairs.map((pair) => (
    <button
      key={pair.id}
      onClick={() => navigate(`/contests/${id}/analytics/compare/${
        ↪ pair.id`)}
      className="w-full bg-white dark:bg-slate-900 p-5 rounded-3xl
        ↪ border border-slate-100 dark:border-slate-800 flex
        ↪ items-center justify-between hover:border-blue-500/50

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
↪ hover:shadow-lg transition-all group"
>
<div className="flex items-center gap-8">
  <div className="text-center w-16">
    <div className={`text-2xl font-black ${pair.percent} >
      ↪ 90 ? 'text-red-600' : 'text-orange-500'}`>
      {pair.percent}%
    </div>
    <div className="text-[10px] font-bold text-slate-400
      ↪ uppercase">Match</div>
  </div>

  <div className="flex items-center gap-6">
    <div className="text-left">
      <p className="text-xs text-slate-400 font-bold mb
        ↪ -1"> 1</p>
      <div className="flex items-center gap-2 text-sm
        ↪ font-bold dark:text-white">
        <User size={14} className="text-blue-500" />
        {pair.user1}
      </div>
    </div>

    <div className="h-10 w-px bg-slate-100 dark:bg-slate
      ↪ -800"></div>

    <div className="text-left">
      <p className="text-xs text-slate-400 font-bold mb
        ↪ -1"> 2</p>
      <div className="flex items-center gap-2 text-sm
        ↪ font-bold dark:text-white">
        <User size={14} className="text-blue-500" />
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        {pair.user2}
      </div>
    </div>
  </div>

  <div className="hidden lg:block px-4 py-2 bg-slate-50
    ↪ dark:bg-slate-800 rounded-2xl">
    <p className="text-[10px] text-slate-400 font-bold mb
      ↪ -1 uppercase text-left">

    </p>
    <p className="text-xs font-bold dark:text-slate-200">
      {pair.task_name}
    </p>
  </div>
</div>

<div className="p-2 bg-slate-50 dark:bg-slate-800 rounded-
  ↪ full text-slate-300 group-hover:text-blue-600
  ↪ transition-colors">
  <ChevronRight size={20} />
</div>
</button>
  )})
</div>

{!isLoading && pairs.length === 0 && (
  <div className="bg-white dark:bg-slate-900 rounded-3xl border border
    ↪ -slate-100 dark:border-slate-800 p-12 text-center text-slate
    ↪ -500">

  </div>

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    )}

    {pagination.total > 0 && (
      <Pagination
        page={pagination.page}
        totalPages={pagination.total_pages}
        total={pagination.total}
        perPage={perPage}
        onPageChange={setPage}
        onPerPageChange={handlePerPageChange}
      />
    )}
  </div>
);
};
```

2.27. pages\{}\PlagiarismSetup.tsx

Листинг 221: pages\{}\PlagiarismSetup.tsx

```
import { useState } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import { ShieldCheck, ArrowLeft, Settings2, Zap, AlertCircle } from 'lucide-react
  ⇨ ';
import { api } from '../api/instance';

export const PlagiarismSetup = () => {
  const { id } = useParams();
  const navigate = useNavigate();
  const [loading, setLoading] = useState(false);

  const [config, setConfig] = useState({
    threshold: 70,
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

    onlyOk: true
  });

const startCheck = async () => {
  setLoading(true);
  try {
    //      0.7  C++,   70
    const res = await api.post(`/analytics/contests/${id}/check`, {
      threshold: config.threshold / 100,
      onlyOk: config.onlyOk
    });
    //      (report_id    )
    navigate(`/contests/${id}/analytics/reports/${res.data.reportId}`);
  } catch (e) {
    alert("    ");
  } finally {
    setLoading(false);
  }
};

return (
  <div className="max-w-2xl mx-auto space-y-6 animate-in slide-in-from-bottom
    ↪ -4">

    <button onClick={() => navigate(-1)} className="flex items-center gap-2
      ↪ text-slate-500 hover:text-blue-600 transition-colors font-medium
      ↪ ">

      <ArrowLeft size={20} />
    </button>

    <div className="bg-white dark:bg-slate-900 p-8 rounded-3xl border border
      ↪ -slate-100 dark:border-slate-800 shadow-sm">

      <div className="flex items-center gap-4 mb-8">
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<div className="p-3 bg-blue-600 rounded-2xl text-white shadow-lg
  ↪ shadow-blue-500/20">
  <Settings2 size={24} />
</div>
<div>
  <h1 className="text-2xl font-bold dark:text-white"> </h1>
  <p className="text-slate-500 text-sm font-medium"> </p>
</div>
</div>

<div className="space-y-8">
  {/* */}
  <div className="space-y-4">
    <div className="flex justify-between items-center">
      <label className="font-bold dark:text-slate-300"> : {
        ↪ config.threshold}%</label>
      <span className="text-[10px] uppercase font-black text-
        ↪ blue-500 bg-blue-50 dark:bg-blue-900/20 px-2 py-1
        ↪ rounded"> 70%+</span>
    </div>
    <input
      type="range" min="10" max="100" step="5"
      className="w-full h-2 bg-slate-100 dark:bg-slate-800
        ↪ rounded-lg appearance-none cursor-pointer accent-
        ↪ blue-600"
      value={config.threshold}
      onChange={(e) => setConfig({...config, threshold:
        ↪ parseInt(e.target.value)}})
    />
  </div>

  {/* Only OK */}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<div className="space-y-3">
  <label className="flex items-center justify-between p-4 bg-
    ↪ slate-50 dark:bg-slate-800/50 rounded-2xl cursor-
    ↪ pointer hover:bg-slate-100 dark: hover: bg-slate-800
    ↪ transition-all border border-transparent hover: border-
    ↪ slate-200 dark: hover: border-slate-700">
    <div className="flex items-center gap-3">
      <div className="p-2 bg-green-100 dark: bg-green-900/30
        ↪ text-green-600 rounded-xl">
        <ShieldCheck size={20} />
      </div>
      <div>
        <p className="font-bold text-sm dark: text-white">
          ↪ </p>
        <p className="text-xs text-slate-500">  WA, TL,
          ↪ RE  ..</p>
      </div>
    </div>
  </div>
  <input
    type="checkbox"
    className="w-6 h-6 rounded-lg border-none bg-slate-200
      ↪ dark: bg-slate-700 text-blue-600 focus: ring-0"
    checked={config.onlyOk}
    onChange={(e) => setConfig({...config, onlyOk: e.
      ↪ target.checked})}
  />
</label>
</div>

<div className="p-4 bg-amber-50 dark: bg-amber-900/20 rounded-2xl
  ↪ border border-amber-100 dark: border-amber-800/30 flex gap
  ↪ -3">
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    <AlertCircle className="text-amber-600 shrink-0" size={20} />
    <p className="text-xs text-amber-700 dark:text-amber-500
      ↳ leading-relaxed">
      C++      LSH-.
      .
    </p>
  </div>

  <button
    onClick={startCheck}
    disabled={loading}
    className="w-full bg-slate-900 dark:bg-blue-600 text-white py
      ↳ -4 rounded-2xl font-bold flex justify-center items-
      ↳ center gap-3 hover:opacity-90 transition-all shadow-xl
      ↳ shadow-blue-500/10 disabled:opacity-50"
  >
    {loading ? <div className="animate-spin rounded-full h-5 w-5
      ↳ border-2 border-white/30 border-t-white" /> : <Zap size
      ↳ ={20} />}
    <span>   </span>
  </button>
</div>
</div>
);
};
```

2.28. pages\{}Profile.tsx

Листинг 222: pages\{}Profile.tsx

```
import {useState, useEffect} from 'react';
import {useAuthStore} from '../store/useAuthStore';
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

import {api} from '../api/instance';
import {
  User, Key, Shield, Monitor, CheckCircle2, AlertCircle, Loader2, Globe, Unlink,
  ↪ Clock
} from 'lucide-react';
import {useNavigate} from "react-router-dom";

export const Profile = () => {
  const {user, logout} = useAuthStore();
  const navigate = useNavigate();
  const [sessions, setSessions] = useState<any[]>([]);
  const [isChangingPass, setIsChangingPass] = useState(false);
  const [passData, setPassPassData] = useState({old: '', new: '', confirm: ''});
  const [status, setStatus] = useState({type: '', msg: ''});

  {/* CF Legacy */
}

const [cfData, setCfData] = useState({key: '', secret: ''});
const [showCfModal, setShowCfModal] = useState(false);
const [isLinkingCf, setIsLinkingCf] = useState(false);

const [polyData, setPolyData] = useState({key: '', secret: ''});
const [showPolyModal, setShowPolyModal] = useState(false);
const [isLinkingPoly, setIsLinkingPoly] = useState(false);

const handleCFLink = async () => {
  setIsLinkingCf(true);
  try {
    await api.post('/codeforces/link', {
      api_key: cfData.key,
      api_secret: cfData.secret
    });
  }

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        await useAuthStore.getState().fetchUser();
        setShowCfModal(false);
        setCfData({key: '', secret: ''});
        alert('Codeforces !');
    } catch (e) {
        alert(' . .');
    } finally {
        setIsLinkingCf(false);
    }
};

const handleCFUnlink = async () => {
    if (confirm(' Codeforces?')) {
        try {
            await api.post('/codeforces/unlink');
            await useAuthStore.getState().fetchUser();
        } catch (e) {
            console.error(e);
        }
    }
};

const handlePolyLink = async () => {
    setIsLinkingPoly(true);
    try {
        await api.post('/polygon/link', {
            api_key: polyData.key,
            api_secret: polyData.secret
        });
        await useAuthStore.getState().fetchUser();
        setShowPolyModal(false);
        setPolyData({key: '', secret: ''});
    }
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        alert('Polygon  !');
    } catch (e) {
        alert(' Polygon.  .');
    } finally {
        setIsLinkingPoly(false);
    }
};

const handlePolyUnlink = async () => {
    if (confirm(' Polygon?')) {
        try {
            await api.post('/polygon/unlink');
            await useAuthStore.getState().fetchUser();
        } catch (e) {
            console.error(e);
        }
    }
};

// const handleCFStart = async () => {
// try {
// const res = await api.get('/codeforces/auth_url');
// const {url, code_verifier} = res.data;
//
// sessionStorage.setItem('cf_verifier', code_verifier);
//
// window.location.href = url;
// } catch (e) {
// console.error(e);
// }
// };

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

// const handleCFUnlink = async () => {
// try {
// await api.post('/codeforces/unlink');
// await useAuthStore.getState().fetchUser();
// alert('Codeforces !');
// window.location.reload();
// } catch (e) {
// alert(' ');
// }
// }

const handleYandexStart = async () => {
  try {
    const res = await api.get('/yandex/auth_url');
    window.location.href = res.data.url;
  } catch (e) {
    console.error(e);
  }
};

const handleYandexUnlink = async () => {
  try {
    await api.post('/yandex/logout');
    await useAuthStore.getState().fetchUser();
    alert('Yandex ');
  } catch (e) {
    alert(' ');
  }
}

useEffect(() => {
  fetchSessions();

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
}, []);

const fetchSessions = async () => {
  try {
    const res = await api.get('/auth/sessions');
    setSessions(res.data);
  } catch (e) {
    console.error(e);
  }
};

const performLogoutAll = async () => {
  try {
    await api.post('/auth/logout_all');
  } catch (e) {
    console.error("Logout all failed", e);
  }
};

const handleLogoutAllManual = async () => {
  if (confirm('  ?')) {
    await performLogoutAll();
    useAuthStore.getState().logout()

    await fetchSessions();
    alert('  .');
  }
};

const handleChangePassword = async (e: React.FormEvent) => {
  e.preventDefault();
  setIsChangingPass(true);
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
setStatus({type: '', msg: ''});

try {
  await api.post('/auth/change_password', {
    old_password: passData.old,
    new_password: passData.new,
    confirm_password: passData.confirm
  });

  await performLogoutAll();
  useAuthStore.getState().logout()

  setStatus({type: 'success', msg: ' ! ...'});
  setTimeout(() => {
    logout();
    navigate('/login');
  }, 2000);

  setPassPassData({old: '', new: '', confirm: ''});
} catch (err: any) {
  let errorMessage = ' ';

  const detail = err.response?.data?.detail;

  if (typeof detail === 'string') {
    errorMessage = detail;
  } else if (Array.isArray(detail)) {
    errorMessage = detail[0]?.msg || ' ';
  } else if (err.response?.status === 422) {
    errorMessage = " ";
  }
}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        setStatus({type: 'error', msg: errorMessage});
    } finally {
        setIsChangingPass(false);
    }
};

const formatSessionInfo = (rawUa: string) => {
    if (!rawUa) return {location: '', browser: ' '};
    const [location, ...uaParts] = rawUa.split(' | ');
    const ua = uaParts.join(' ');

    const formatUa = (ua: string) => {
        if (!ua) return {device: 'Unknown', browser: 'Browser'};
        const browser = ua.includes('Edg/') ? 'Edge' : ua.includes('OPR') ? '
            ↪ Opera' : ua.includes('Chrome') ? 'Chrome' : ua.includes('Firefox
            ↪ ') ? 'Firefox' : 'Safari';

        let os: string;
        if (ua.includes('Android')) {
            os = 'Android';
        } else if (ua.includes('iPhone') || ua.includes('iPad')) {
            os = 'iOS';
        } else if (ua.includes('Windows')) {
            os = 'Windows';
        } else if (ua.includes('Macintosh') || ua.includes('Mac OS X')) {
            os = 'macOS';
        } else if (ua.includes('Linux')) {
            os = 'Linux';
        } else {
            os = ' ';
        }
    }
}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    return {device: os, browser: browser};
};

const {device, browser} = formatUa(ua);

return {location, device: `${browser} ${device}`};
};

const formatLastSeen = (dateStr: string) => {
    const date = new Date(dateStr);
    const now = new Date();
    const diff = Math.floor((now.getTime() - date.getTime()) / 1000);

    if (diff < 60) return ' ';
    if (diff < 3600) return `${Math.floor(diff / 60)} . `;
    if (diff < 86400) return `${Math.floor(diff / 3600)} . `;

    return date.toLocaleString([], {
        day: '2-digit',
        month: '2-digit',
        hour: '2-digit',
        minute: '2-digit'
    });
};

return (
    <div className="max-w-4xl mx-auto space-y-8 animate-in fade-in slide-in-
    ↪ from-bottom-4 duration-500">
        <div className="flex items-center gap-4">
            <div
                className="w-16 h-16 bg-blue-600 rounded-2xl flex items-center
                ↪ justify-center text-white shadow-xl shadow-blue-500/20">
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    <User size={32}/>
  </div>
  <div>
    <h1 className="text-3xl font-bold text-slate-900 dark:text-white
      ↪ ">{user?.login}</h1>
    <p className="text-slate-500 dark:text-slate-400">    </p>
  </div>
</div>

<div className="grid grid-cols-1 md:grid-cols-3 gap-8">
  <div className="md:col-span-1 space-y-6">
    <div
      className="bg-white dark:bg-slate-900 p-6 rounded-3xl border
        ↪ border-slate-100 dark:border-slate-800 shadow-sm">
      <div className="flex items-center gap-2 mb-6 text-slate-900
        ↪ dark:text-white font-bold">
        <Key size={20} className="text-blue-600"/>
        <span> </span>
      </div>

      <form onSubmit={handleChangePassword} className="space-y-4">
        <input
          type="password" placeholder=" "
          className="w-full bg-slate-50 dark:bg-slate-800 border
            ↪ -none rounded-xl p-3 text-sm focus:ring-2 focus
            ↪ :ring-blue-500 outline-none dark:text-white"
          value={passData.old} onChange={e => setPassPassData
            ↪ ({...passData, old: e.target.value}})
        />
        <input
          type="password" placeholder=" "
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        className="w-full bg-slate-50 dark:bg-slate-800 border
        ↪ -none rounded-xl p-3 text-sm focus:ring-2 focus
        ↪ :ring-blue-500 outline-none dark:text-white"
        value={passData.new} onChange={e => setPassPassData
        ↪ ({...passData, new: e.target.value})}
    />
    <input
        type="password" placeholder=" "
        className="w-full bg-slate-50 dark:bg-slate-800 border
        ↪ -none rounded-xl p-3 text-sm focus:ring-2 focus
        ↪ :ring-blue-500 outline-none dark:text-white"
        value={passData.confirm}
        onChange={e => setPassPassData({...passData, confirm:
        ↪ e.target.value})}
    />

    {status.msg && (
        <div
            className={`text-xs p-3 rounded-lg flex items-
            ↪ center gap-2 ${status.type === 'success' ? '
            ↪ bg-green-50 text-green-600' : 'bg-red-50
            ↪ text-red-600'}`}>
            {status.type === 'success' ? <CheckCircle2 size
            ↪ ={14}/> : <AlertCircle size={14}/>}
            {status.msg}
        </div>
    )}

    <button
        disabled={isChangingPass}
        className="w-full bg-slate-900 dark:bg-white dark:text
        ↪ -slate-900 text-white py-3 rounded-xl font-bold

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        ↪ hover:opacity-90 transition-all flex justify-
        ↪ center"
    >
        {isChangingPass ? <Loader2 className="animate-spin"
        ↪ size={20}/> : ' '}
    </button>
</form>
</div>
</div>

{ /* : */ }
<div className="md:col-span-2 space-y-6">
    <div
        className="bg-white dark:bg-slate-900 p-6 rounded-3xl border
        ↪ border-slate-100 dark:border-slate-800 shadow-sm">
        <div className="flex justify-between items-center mb-6">
            <div className="flex items-center gap-2 text-slate-900
            ↪ dark:text-white font-bold">
                <Shield size={20} className="text-blue-600"/>
                <span> </span>
            </div>
            <button
                onClick={handleLogoutAllManual}
                className="text-xs font-bold text-red-500 hover:bg-red
                ↪ -50 dark:hover:bg-red-900/20 px-3 py-1.5
                ↪ rounded-lg transition-colors"
            >
                </button>
        </div>
    </div>

    <div className="space-y-4">

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
{sessions.map((s) => {
  const {location, device} = formatSessionInfo(s.
    ↪ user_agent);
  const isOnline = new Date().getTime() - new Date(s.
    ↪ last_seen).getTime() < 300000;
  return (
    <div key={s.id} className={`p-5 rounded-2xl border
      ↪ transition-all ${
        s.is_current ? 'border-blue-500 bg-blue-50/30'
        ↪ : 'border-slate-100 dark:border-slate
        ↪ -800'
      }`>
      <div className="flex justify-between items-
        ↪ start">
        <div className="flex gap-4">
          <div className="relative">
            <div
              className={`p-3 rounded-xl ${s.
                ↪ is_current ? 'bg-blue-600
                ↪ text-white' : 'bg-slate
                ↪ -100 dark:bg-slate-800
                ↪ text-slate-400'}`>
              <Monitor size={24}/>
            </div>
            {isOnline && (
              <div
                className="absolute -top-1 -
                  ↪ right-1 w-3 h-3 bg-
                  ↪ green-500 border-2
                  ↪ border-white dark:
                  ↪ border-slate-900
                  ↪ rounded-full"></div>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
    )}
  </div>
  <div>
    <p className="font-bold dark:text-
      ↪ white">{location}</p>
    <p className="text-sm text-slate
      ↪ -500">{device}</p>

    <div
      className="flex items-center gap
        ↪ -1.5 mt-2 text-[11px] font
        ↪ -medium">
      <Clock size={12} className="text
        ↪ -slate-400"/>
      <span
        className={isOnline ? "text-
          ↪ green-600" : "text-
          ↪ slate-400"}>
        {isOnline ? ' ' : ' ': ${
          ↪ formatLastSeen(s.
          ↪ last_seen)}}' }
      </span>
    </div>
  </div>
</div>

{s.is_current && (
  <span
    className="text-[10px] font-bold bg-
      ↪ blue-100 text-blue-700 px-2
      ↪ py-1 rounded-lg uppercase">
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    </span>

    })

  </div>

</div>

);

}}

</div>

</div>

</div>

</div>

<div
  className="bg-white dark:bg-slate-900 p-6 rounded-3xl border border-
    ↪ slate-100 dark:border-slate-800 shadow-sm mt-8">
  <h2 className="text-xl font-bold mb-6 dark:text-white flex items-
    ↪ center gap-2">
    <Globe size={22} className="text-blue-600"/>
  </h2>

  <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
    {/* Legacy version Codeforces */}

    <div
      className="p-4 rounded-2xl bg-slate-50 dark:bg-slate-800/50
        ↪ border border-slate-100 dark:border-slate-800">
      <div className="flex justify-between items-center mb-4">
        <span className="font-bold dark:text-white">Codeforces</
          ↪ span>
        {user?.is_codeforces_linked ? (
          <span
            className="text-[10px] bg-green-100 text-green-700
              ↪ px-2 py-1 rounded-full uppercase"></span>

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    ) : (
      <span
        className="text-[10px] bg-slate-200 text-slate-500
          ↪ px-2 py-1 rounded-full uppercase"> </span>
      )}
    </div>
    {user?.is_codeforces_linked ? (
      <button onClick={handleCFUnlink}
        className="text-xs text-red-500 flex items-center
          ↪ gap-1 hover:underline">
        <Unlink size={14}/>
      </button>
    ) : (
      <button onClick={() => setShowCfModal(true)}
        className="w-full bg-blue-600 text-white py-2
          ↪ rounded-xl text-sm font-bold hover:bg-blue
          ↪ -700 transition-colors">
        API
      </button>
    )}
  </div>

  {/*<div className="p-4 rounded-2xl bg-slate-50 dark:bg-slate
    ↪ -800/50 border border-slate-100 dark:border-slate-800">*/}
  {/* <div className="flex justify-between items-center mb-4">*/}
  {/* <span className="font-bold dark:text-white">Codeforces</span
    ↪ >*/}
  {/* {user?.is_codeforces_linked ? (*/}
  {/* <div className="flex flex-col items-end">*/}
  {/* <span className="text-[10px] bg-green-100 text-green-700 px
    ↪ -2 py-1 rounded-full uppercase"></span>*/}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

{ /* <span className="text-xs font-bold text-blue-600 mt-1">@{
    ↪ user.is_codeforces_linked}</span>*/}

{ /* </div>*/}

{ /* ) : (*/}

{ /* <span className="text-[10px] bg-slate-200 text-slate-500 px
    ↪ -2 py-1 rounded-full uppercase"> </span>*/}

{ /* )}*/}

{ /* </div>*/}

{ /* {user?.is_codeforces_linked ? (*/}

{ /* <button onClick={handleCFUnlink} className="text-xs text-red
    ↪ -500 flex items-center gap-1 hover:underline">*/}

{ /* <Unlink size={14}/> */}

{ /* </button>*/}

{ /* ) : (*/}

{ /* <button onClick={handleCFStart} className="w-full bg-blue
    ↪ -600 text-white py-2 rounded-xl text-sm font-bold">*/}

{ /*   Codeforces*/}

{ /* </button>*/}

{ /* )}*/}

{ /*</div>*/}

<div
  className="p-4 rounded-2xl bg-slate-50 dark:bg-slate-800/50
    ↪ border border-slate-100 dark:border-slate-800">
  <div className="flex justify-between items-center mb-4">
    <span className="font-bold dark:text-white italic">Yandex
      ↪ <span
        className="text-red-500">Contest</span></span>
    {user?.is_yandex_linked ? (
      <span
        className="text-[10px] bg-green-100 text-green-700
          ↪ px-2 py-1 rounded-full uppercase"></span>

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    ) : (
      <span
        className="text-[10px] bg-slate-200 text-slate-500
          ↪ px-2 py-1 rounded-full uppercase"> </span>
      )}
    </div>
    {user?.is_yandex_linked ? (
      <button onClick={handleYandexUnlink}
        className="text-xs text-red-500 flex items-center
          ↪ gap-1 hover:underline">
        <Unlink size={14}/>
      </button>
    ) : (
      <button onClick={handleYandexStart}
        className="w-full bg-red-600 text-white py-2
          ↪ rounded-xl text-sm font-bold">

      </button>
    )}
  </div>

  <div
    className="p-4 rounded-2xl bg-slate-50 dark:bg-slate-800/50
      ↪ border border-slate-100 dark:border-slate-800">
    <div className="flex justify-between items-center mb-4">
      <span className="font-bold dark:text-white text-orange
        ↪ -600">Polygon</span>
      {user?.is_polygon_linked ? (
        <span
          className="text-[10px] bg-green-100 text-green-700
            ↪ px-2 py-1 rounded-full uppercase"></span>
        ) : (

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

      <span
        className="text-[10px] bg-slate-200 text-slate-500
          ↪ px-2 py-1 rounded-full uppercase"> </span>
    )}
  </div>
  {user?.is_polygon_linked ? (
    <button onClick={handlePolyUnlink}
      className="text-xs text-red-500 flex items-center
        ↪ gap-1 hover:underline">
      <Unlink size={14}/>
    </button>
  ) : (
    <button onClick={() => setShowPolyModal(true)}
      className="w-full bg-orange-600 text-white py-2
        ↪ rounded-xl text-sm font-bold hover:bg-orange
        ↪ -700 transition-colors">
      Polygon API
    </button>
  )}
</div>
</div>
</div>

{showCfModal && (
  <div
    className="fixed inset-0 bg-black/50 backdrop-blur-sm flex items
      ↪ -center justify-center z-[100] p-4">
    <div
      className="bg-white dark:bg-slate-900 p-8 rounded-3xl w-full
        ↪ max-w-sm shadow-2xl animate-in zoom-in-95 duration
        ↪ -200">

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<h3 className="text-xl font-bold mb-2 dark:text-white">API
  ↪ Codeforces</h3>

<p className="text-sm text-slate-500 mb-6">      Codeforces
  ( API)</p>

<div className="space-y-4">
  <div>
    <label className="text-xs font-bold text-slate-400 ml
      ↪ -1">API KEY</label>

    <input
      type="text"
      className="w-full bg-slate-50 dark:bg-slate-800
        ↪ border-none rounded-xl p-3 mt-1 outline-none
        ↪ focus:ring-2 focus:ring-blue-500 dark:text-
        ↪ white"
      value={cfData.key}
      onChange={e => setCfData({...cfData, key: e.target
        ↪ .value}})}

    />
  </div>
  <div>
    <label className="text-xs font-bold text-slate-400 ml
      ↪ -1">API SECRET</label>

    <input
      type="password"
      className="w-full bg-slate-50 dark:bg-slate-800
        ↪ border-none rounded-xl p-3 mt-1 outline-none
        ↪ focus:ring-2 focus:ring-blue-500 dark:text-
        ↪ white"
      value={cfData.secret}
      onChange={e => setCfData({...cfData, secret: e.
        ↪ target.value}})}
  </div>
</div>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        />
    </div>

    <button
      onClick={handleCFLink}
      disabled={isLinkingCf || !cfData.key || !cfData.secret
        ↪ }
      className="w-full bg-blue-600 text-white py-3 rounded-
        ↪ xl font-bold hover:bg-blue-700 disabled:opacity
        ↪ -50 transition-all flex justify-center"
    >
      {isLinkingCf ? <Loader2 className="animate-spin" size
        ↪ ={20}/> : ' '}
    </button>

    <button
      onClick={() => setShowCfModal(false)}
      className="w-full text-slate-500 text-sm font-medium
        ↪ hover:text-slate-700 dark:hover:text-slate-300"
    >

    </button>
  </div>
</div>
</div>
)}
{showPolyModal && (
  <div
    className="fixed inset-0 bg-black/50 backdrop-blur-sm flex items
      ↪ -center justify-center z-[100] p-4">
    <div
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```

className="bg-white dark:bg-slate-900 p-8 rounded-3xl w-full
  ↳ max-w-sm shadow-2xl animate-in zoom-in-95 duration
  ↳ -200">

<h3 className="text-xl font-bold mb-2 dark:text-white text-
  ↳ orange-600">Polygon API </h3>

<p className="text-sm text-slate-500 mb-6">      Polygon</p>

<div className="space-y-4">
  <div>
    <label className="text-xs font-bold text-slate-400 ml
      ↳ -1">API KEY</label>
    <input
      type="text"
      className="w-full bg-slate-50 dark:bg-slate-800
        ↳ border-none rounded-xl p-3 mt-1 outline-none
        ↳ focus:ring-2 focus:ring-orange-500 dark:
        ↳ text-white"
      value={polyData.key}
      onChange={e => setPolyData({...polyData, key: e.
        ↳ target.value})}
    />
  </div>
  <div>
    <label className="text-xs font-bold text-slate-400 ml
      ↳ -1">API SECRET</label>
    <input
      type="password"
      className="w-full bg-slate-50 dark:bg-slate-800
        ↳ border-none rounded-xl p-3 mt-1 outline-none
        ↳ focus:ring-2 focus:ring-orange-500 dark:
        ↳ text-white"
      value={polyData.secret}

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

} ;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2.29. pages\{}\RegisterPage.tsx

Листинг 223: pages\{}\RegisterPage.tsx

```
import { useState } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import { ArrowLeft, Loader2, CheckCircle2, XCircle } from 'lucide-react';
import { api } from '../api/instance';
import { ThemeToggle } from "../components/ThemeToggle.tsx";
import {useAuthStore} from "../store/useAuthStore.ts";

export const RegisterPage = () => {
  const [login, setLogin] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [confirmPassword, setConfirmPassword] = useState('');

  const [error, setError] = useState('');
  const [isLoading, setIsLoading] = useState(false);
  const [isSuccess, setIsSuccess] = useState(false);

  const navigate = useNavigate();

  const validatePassword = (pass: string) => {
    return {
      length: pass.length >= 8 && pass.length <= 30,
      hasUpper: /[A-Z]/.test(pass),
      hasLower: /[a-z]/.test(pass),
      hasNumber: /[0-9]/.test(pass),
      hasSpecial: /[.,<>_?!@#$$%^&*()]/.test(pass)
    };
  };
};
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
};

const passStatus = validatePassword(password);

const handleSubmit = async (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault();
  setError('');

  if (password !== confirmPassword) {
    setError(' ');
    return;
  }

  setIsLoading(true);

  try {
    const response = await api.post('/auth/register', {
      login,
      email,
      password
    });

    const { access_token, refresh_token, token_type } = response.data;

    const setAuth = useAuthStore.getState().setAuthenticated;
    setAuth(access_token, refresh_token, token_type);

    const user_response = await api.get('/auth/me');
    const user = user_response.data;

    const setUser = useAuthStore.getState().setUser;
    setUser(user);
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    </Link>
  </div>
</div>
</nav>

<div className="flex-1 flex items-center justify-center p-6 py-12">
  <div className="bg-slate-50 dark:bg-slate-800 p-8 rounded-2xl shadow
    ↪ -xl border border-slate-100 dark:border-slate-700 w-full max-w
    ↪ -md transition-all">
    {isSuccess ? (
      <div className="text-center space-y-4 py-8">
        <div className="flex justify-center"><CheckCircle2 size
          ↪ ={64} className="text-green-500 animate-bounce"
          ↪ /></div>
        <h2 className="text-2xl font-bold dark:text-white"> !</h2
          ↪ >
        <p className="text-slate-500"> ...</p>
      </div>
    ) : (
      <>
        <h2 className="text-2xl text-slate-900 dark:text-slate
          ↪ -100 font-bold mb-2 text-center italic"></h2>
        <p className="text-center text-slate-500 dark:text-slate
          ↪ -400 text-sm mb-6"> </p>

        {error && (
          <div className="bg-red-100 dark:bg-red-900/30 text-red
            ↪ -700 dark:text-red-400 p-3 rounded-lg mb-4 text
            ↪ -sm border border-red-200 dark:border-red-800">
            {error}
          </div>
        )}
      </>
    )}
  </div>
</div>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<form onSubmit={handleSubmit} className="space-y-4">
  <div>
    <label className="block text-sm dark:text-slate
      ↪ -300 font-medium mb-1 ml-1"></label>
    <input
      type="text"
      className="w-full border border-slate-200 dark
        ↪ :border-slate-600 dark:bg-slate-700 dark
        ↪ :text-white rounded-lg p-2.5 outline-
        ↪ none focus:ring-2 focus:ring-blue-500
        ↪ transition-all"
      placeholder=" 5 30 "
      value={login}
      onChange={(e) => setLogin(e.target.value)}
      required
    />
  </div>
  <div>
    <label className="block text-sm dark:text-slate
      ↪ -300 font-medium mb-1 ml-1">Email</label>
    <input
      type="email"
      className="w-full border border-slate-200 dark
        ↪ :border-slate-600 dark:bg-slate-700 dark
        ↪ :text-white rounded-lg p-2.5 outline-
        ↪ none focus:ring-2 focus:ring-blue-500
        ↪ transition-all"
      placeholder="example@mail.com"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
      required
    />
  </div>
</form>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
    />
  </div>
  <div>
    <label className="block text-sm dark:text-slate
      ↪ -300 font-medium mb-1 ml-1"></label>
    <input
      type="password"
      className="w-full border border-slate-200 dark
        ↪ :border-slate-600 dark:bg-slate-700 dark
        ↪ :text-white rounded-lg p-2.5 outline-
        ↪ none focus:ring-2 focus:ring-blue-500
        ↪ transition-all"
      placeholder=""
      value={password}
      onChange={(e) => setPassword(e.target.value)}
      required
    />
    <div className="mt-2 grid grid-cols-2 gap-1 text
      ↪ -[10px]">
      <PasswordReq label="8-30 " met={passStatus.
        ↪ length} />
      <PasswordReq label="A-Z ()" met={passStatus.
        ↪ hasUpper} />
      <PasswordReq label="a-z ()" met={passStatus.
        ↪ hasLower} />
      <PasswordReq label="0-9 ()" met={passStatus.
        ↪ hasNumber} />
      <PasswordReq label="" met={passStatus.
        ↪ hasSpecial} />
    </div>
  </div>
</div>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
<label className="block text-sm dark:text-slate
  ↪ -300 font-medium mb-1 ml-1"> </label>

<input
  type="password"
  className="w-full border border-slate-200 dark
    ↪ :border-slate-600 dark:bg-slate-700 dark
    ↪ :text-white rounded-lg p-2.5 outline-
    ↪ none focus:ring-2 focus:ring-blue-500
    ↪ transition-all"
  placeholder=""
  value={confirmPassword}
  onChange={(e) => setConfirmPassword(e.target.
    ↪ value)}
  required
/>
</div>

<button
  type="submit"
  disabled={isLoading}
  className="w-full bg-blue-600 dark:bg-blue-500
    ↪ text-white py-3 rounded-lg font-bold hover:
    ↪ bg-blue-700 dark:hover:bg-blue-600
    ↪ transition-all shadow-lg shadow-blue-500/20
    ↪ mt-4 flex justify-center items-center gap-2"
  >
    {isLoading ? <Loader2 className="animate-spin"
      ↪ size={20} /> : " "}
</button>
</form>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

        <p className="mt-6 text-center text-sm text-slate-500
          ↪ dark:text-slate-400">
          ? <Link to="/login" className="text-blue-600 hover:
            ↪ underline font-medium"></Link>
        </p>
      </>
    )}
  </div>
</div>
</div>
);
};

const PasswordReq = ({ label, met }: { label: string, met: boolean }) => (
  <div className={`flex items-center gap-1.5 ${met ? 'text-green-500' : 'text-
    ↪ slate-400'}>
    {met ? <CheckCircle2 size={12} /> : <XCircle size={12} />}
    <span>{label}</span>
  </div>
);

```

2.30. pages\SubmissionSource.tsx

Листинг 224: pages\SubmissionSource.tsx

```

import { useState, useEffect } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import { ArrowLeft, Code2, User, Clock, CheckCircle2, AlertCircle, Zap, HardDrive }
  ↪ from 'lucide-react';
import { api } from '../api/instance';

export const SubmissionSource = () => {
  const { id, subId } = useParams();

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

const navigate = useNavigate();
const [sub, setSub] = useState<any>(null);

useEffect(() => {
    api.get(`/contests/submissions/${subId}/source`).then(res => setSub(res.
        ↪ data));
}, [subId]);

if (!sub) return null;

const formatRunTime = (timeStr: string) => {
    if (!timeStr) return '0 ms';
    // timedelta    0:00:00.050000
    const parts = timeStr.split(':');
    if (parts.length === 3) {
        const seconds = parseFloat(parts[2]);
        return `${Math.round(seconds * 1000)} ms`;
    }
    return timeStr;
};

const formatMemory = (bytesStr: string) => {
    const bytes = parseInt(bytesStr);
    if (isNaN(bytes) || bytes === 0) return '0 KB';
    if (bytes < 1024) return `${bytes} B`;
    if (bytes < 1048576) return `${(bytes / 1024).toFixed(1)} KB`;
    return `${(bytes / 1048576).toFixed(1)} MB`;
};

return (
    <div className="space-y-6 animate-in slide-in-from-right-4 duration-300 pb
        ↪ -12">

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<button onClick={() => navigate(`/contests/${id}/submissions`)}  
  ↪ className="flex items-center gap-2 text-slate-500 hover:text-blue  
  ↪ -600 font-medium">  
    <ArrowLeft size={20} />  
</button>  
  
{/* */}  
  
<div className="bg-white dark:bg-slate-900 p-6 rounded-3xl border border  
  ↪ -slate-100 dark:border-slate-800 shadow-sm flex flex-wrap gap-8  
  ↪ items-center">  
  <div className={`p-4 rounded-2xl ${sub.verdict === 'OK' ? 'bg-green  
    ↪ -100 text-green-600' : 'bg-red-100 text-red-600'}{`}  
    {sub.verdict === 'OK' ? <CheckCircle2 size={32}/> : <AlertCircle  
      ↪ size={32}/>}  
  </div>  
  
  <div className="space-y-1">  
    <p className="text-xs text-slate-400 uppercase font-bold  
      ↪ tracking-wider"></p>  
    <h2 className="text-xl font-bold dark:text-white">{sub.task_name  
      ↪ }</h2>  
  </div>  
  
  <div className="space-y-1">  
    <p className="text-xs text-slate-400 uppercase font-bold  
      ↪ tracking-wider"></p>  
    <div className="flex items-center gap-2 dark:text-white font-  
      ↪ medium">  
      <User size={16} className="text-blue-500" /> {sub.  
        ↪ participant_login}  
    </div>  
  </div>  
</div>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
<div className="space-y-1">
  <p className="text-xs text-slate-400 uppercase font-bold
    ↪ tracking-wider"> / </p>
  <div className="flex items-center gap-2 text-slate-500 text-sm">
    <Code2 size={16} /> {sub.language}
    <span className="mx-2">|</span>
    <Clock size={16} /> {new Date(sub.send_time).toLocaleString()
      ↪ }
  </div>
</div>

<div className="ml-auto flex items-center gap-4 bg-slate-50 dark:bg-
  ↪ slate-800 p-4 rounded-2xl">
  <div className="text-center">
    <p className="text-[10px] text-slate-400 uppercase font-bold
      ↪ "></p>
    <p className="text-lg font-bold text-blue-600">{sub.score}</p>
    ↪ >
  </div>
  <div className="text-center border-1 dark:border-slate-700 pl-6
    ↪ pr-2">
    <p className="text-[10px] text-slate-400 uppercase font-bold
      ↪ mb-1"></p>
    <p className="text-sm font-bold dark:text-white flex items-
      ↪ center justify-center gap-1">
      <Zap size={14} className="text-amber-500"/>
      {formatRunTime(sub.run_time)}
    </p>
  </div>
</div>
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

<div className="text-center border-1 dark:border-slate-700 pl-6
  ↪ pr-2">
  <p className="text-[10px] text-slate-400 uppercase font-bold
    ↪ mb-1"></p>
  <p className="text-sm font-bold dark:text-white flex items-
    ↪ center justify-center gap-1">
    <HardDrive size={14} className="text-blue-400"/>
    {formatMemory(sub.memory_bytes)}
  </p>
</div>
</div>
</div>

{/* */}

<div className="bg-slate-950 rounded-3xl border border-slate-800
  ↪ overflow-hidden shadow-2xl">
  <div className="bg-slate-900 px-6 py-3 border-b border-slate-800
    ↪ flex justify-between items-center">
    <span className="text-xs font-mono text-slate-500 uppercase">
      ↪ source_code.{sub.language.includes('C++') ? 'cpp' : 'py
      ↪ '}</span>
    <div className="flex gap-1.5">
      <div className="w-2.5 h-2.5 rounded-full bg-red-500/20"></div>
      ↪ >
      <div className="w-2.5 h-2.5 rounded-full bg-amber-500/20"></
      ↪ div>
      <div className="w-2.5 h-2.5 rounded-full bg-green-500/20"></
      ↪ div>
    </div>
  </div>
</div>

<div className="p-8 overflow-x-auto">

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
        <pre className="font-mono text-sm text-slate-300 leading-relaxed
          ↪  selection:bg-blue-500/30">
            <code>{sub.source}</code>
        </pre>
    </div>
</div>
</div>
);
};
```

2.31. pages\{}\YandexCallback.tsx

Листинг 225: pages\{}\YandexCallback.tsx

```
import { useEffect, useRef } from 'react';
import { useNavigate, useSearchParams } from 'react-router-dom';
import { api } from '../api/instance';
import {useAuthStore} from "../store/useAuthStore.ts";

export const YandexCallback = () => {
  const [searchParams] = useSearchParams();
  const navigate = useNavigate();
  const processed = useRef(false);

  useEffect(() => {
    if (processed.current) return;
    processed.current = true;

    const code = searchParams.get('code');
    if (!code) {
      navigate('/profile');
      return;
    }
  })
}
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

api.post('/yandex/callback', { code })
  .then(async () => {
    await useAuthStore.getState().fetchUser();
    navigate('/profile');
  })
  .catch((e) => {
    console.error(e);
    alert(' : ' + (e.response?.data?.detail || e.message));
    navigate('/profile');
  });
}, []);

return (
  <div className="flex items-center justify-center min-h-screen">
    <div className="text-center">
      <div className="animate-spin rounded-full h-12 w-12 border-b-2
        ↪ border-blue-600 mx-auto mb-4"></div>
      <p className="text-slate-500"> ...</p>
    </div>
  </div>
);
};

```

2.32. store\useAuthStore.ts

Листинг 226: store\useAuthStore.ts

```

import { create } from 'zustand';
import { persist } from "zustand/middleware";
import { api } from "../api/instance.ts";

interface User {

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата


```
id: string;
login: string;
email: string;
role_id: string;
is_yandex_linked: boolean;
is_codeforces_linked: boolean;
is_polygon_linked: boolean;
}

interface AuthState {
  user: User | null;
  accessToken: string | null;
  refreshToken: string | null;
  tokenType: string | null;
  isAuthenticated: boolean;
  setAuthenticated: (accessToken: string, refreshToken: string, tokenType: string
    ↪ ) => void;
  setUser: (user: User) => void;
  updateTokens: (accessToken: string, refreshToken: string, tokenType: string) =>
    ↪ void;
  fetchUser: () => Promise<void>;
  logout: () => void;
}

export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      user: null,
      accessToken: null,
      refreshToken: null,
      tokenType: null,
      isAuthenticated: false,
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

    setAuthenticated: (accessToken, refreshToken, tokenType) => {
      set({accessToken, refreshToken, tokenType, isAuthenticated: true});
    },
    setUser: (user) => set({user}),
    updateTokens: (accessToken, refreshToken, tokenType) => {
      set({accessToken, refreshToken, tokenType});
    },
    fetchUser: async () => {
      try {
        const res = await api.get('/auth/me');
        set({ user: res.data });
      } catch (e) {
        console.error("Failed to fetch user", e);
      }
    },
    logout: () => {
      set({user: null, accessToken: null, refreshToken: null, tokenType:
        ↪ null, isAuthenticated: false});
    },
  }),
  {
    name: 'auth-storage'
  }
)
);

```

2.33. store\useThemeStore.ts

ЛІСТИНГ 227: store\useThemeStore.ts

```

import { create } from 'zustand';
import { persist } from 'zustand/middleware';

```

ІЗМ.	ЛІСТ	№ ДОКУМ.	ПОДП.	ДАТА
RU.17701729.05.05-01 12 01-1				
ІНВ. № ПОДЛ.	ПОДП. І ДАТА	ВЗАМ. ІНВ. №	ІНВ. № ДУБЛ.	ПОДП. І ДАТА

```
interface ThemeState {
  theme: 'light' | 'dark' | 'system';
  setTheme: (theme: 'light' | 'dark' | 'system') => void;
}

export const useThemeStore = create<ThemeState>()(
  persist(
    (set) => ({
      theme: 'system',
      setTheme: (theme) => set({ theme }),
    }),
    { name: 'theme-storage' }
  )
);
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

Список литературы

- [1] ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
- [2] ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
- [3] ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
- [4] ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
- [5] ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
- [6] ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
- [7] ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
- [8] ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
- [9] ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

[illegible]

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.05-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата